

# Appunti di Computer Architecture

AA 2023 - 2024

Saverio Mosti

Gabriele Pianigiani

# Sommario

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Il trend nelle micro-architetture.....</b>        | <b>8</b>  |
| [1986 - 2003].....                                   | 8         |
| [2003 - 2011].....                                   | 9         |
| Trend delle tecnologie.....                          | 11        |
| Come colmare il divario tra CPU e RAM?.....          | 12        |
| Categorie di calcolatori.....                        | 13        |
| <b>Benchmark.....</b>                                | <b>15</b> |
| Indici di performance.....                           | 15        |
| Come utilizzare il benchmark.....                    | 16        |
| Quale benchmark usare?.....                          | 17        |
| Benchmark ULP.....                                   | 17        |
| Benchmark per telefoni e tablet.....                 | 17        |
| Come calcolare il punteggio del benchmark.....       | 18        |
| Elenco di alcuni benchmark.....                      | 18        |
| Elaborazione di transazioni.....                     | 18        |
| <b>Progettazione di un calcolatore.....</b>          | <b>21</b> |
| Instruction set.....                                 | 22        |
| Tipologia di applicazione.....                       | 22        |
| Linguaggi di programmazione.....                     | 22        |
| Binary code e Object code.....                       | 23        |
| Sistema Operativo.....                               | 23        |
| Standard.....                                        | 24        |
| Le mode del momento.....                             | 24        |
| Tassonomia di Flynn.....                             | 25        |
| Gestione degli Errori.....                           | 26        |
| Error detection.....                                 | 26        |
| System reconfiguration.....                          | 26        |
| System recovery.....                                 | 26        |
| Metriche per l'analisi dei guasti.....               | 27        |
| Ridondanza.....                                      | 28        |
| Valutazione delle performance di un calcolatore..... | 30        |
| Come migliorare le performance.....                  | 30        |
| <b>Processore.....</b>                               | <b>33</b> |
| Componenti.....                                      | 33        |
| Register File.....                                   | 33        |
| Fetch dell'istruzione ed incremento del PC.....      | 34        |
| Metodologia del segnale del fronte del clock.....    | 35        |
| Clock positive/negative edge-triggered.....          | 35        |
| <b>Pipeline.....</b>                                 | <b>36</b> |

|                                                 |           |
|-------------------------------------------------|-----------|
| Pipeline con metodo asincrono.....              | 37        |
| Pipeline con metodo sincrono.....               | 37        |
| Hardware del processore e della pipeline.....   | 40        |
| Multi-port register file.....                   | 40        |
| ALU.....                                        | 42        |
| Formato delle istruzioni.....                   | 44        |
| Formato ADD, SUB, AND e OR.....                 | 44        |
| Formato LOAD e STORE.....                       | 44        |
| Istruzione di salto CBZ Addr, Rt.....           | 44        |
| Decodificare gli OPCODE.....                    | 45        |
| Divisione in stage delle istruzioni.....        | 46        |
| Registri intermedi.....                         | 49        |
| Pipeline hazards.....                           | 51        |
| Hazard strutturale.....                         | 51        |
| Control hazards.....                            | 52        |
| Data Hazards.....                               | 53        |
| Load-use data hazard.....                       | 54        |
| Pipeline e la gestione delle eccezioni.....     | 55        |
| Tipologie di Pipeline.....                      | 56        |
| Super-pipeline.....                             | 56        |
| Super-scalare.....                              | 56        |
| VLIW (Very Large Instruction Word).....         | 57        |
| Operazioni vettoriali.....                      | 57        |
| Processori a doppia emissione.....              | 58        |
| Prestazioni.....                                | 59        |
| Compilazione e pianificazione hardware.....     | 59        |
| Static two-issue datapath.....                  | 60        |
| Esecuzione out-of-order.....                    | 61        |
| RAW - Read After Write Hazard.....              | 62        |
| WAW - Write After Write Hazard.....             | 62        |
| WAR - Write After Read Hazard.....              | 62        |
| Renaming dei registri.....                      | 63        |
| Branch prediction - Esecuzione speculativa..... | 68        |
| <b>Multithreading.....</b>                      | <b>70</b> |
| Fine MT - Fine-grained MultiThreading.....      | 71        |
| Coarse MT - Coarse-grained MultiThreading.....  | 71        |
| SMT - Simultaneous MultiThreading.....          | 72        |
| <b>Processore Intel.....</b>                    | <b>73</b> |
| CISC - Complex Instruction Set Computing.....   | 73        |
| Intel i7.....                                   | 73        |
| Fase di fetch.....                              | 74        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| Pre-decodifica delle istruzioni.....                       | 75        |
| Decodifica delle micro-operazioni.....                     | 75        |
| Rilevamento del loop stream.....                           | 76        |
| Microfusione.....                                          | 76        |
| Emissione dell'istruzione base.....                        | 76        |
| Reservation station centralizzata.....                     | 77        |
| Esecuzione delle micro-operazioni.....                     | 78        |
| Commit.....                                                | 79        |
| Fase di rename.....                                        | 79        |
| <b>Processori ARM.....</b>                                 | <b>81</b> |
| Fase 1.....                                                | 81        |
| Fase 2.....                                                | 82        |
| Fase 3.....                                                | 83        |
| Core IP.....                                               | 84        |
| <b>Cache.....</b>                                          | <b>86</b> |
| Principio di località dei dati.....                        | 86        |
| Indici di prestazione delle cache.....                     | 87        |
| Cache ad accesso diretto.....                              | 89        |
| Accesso ad un blocco - Cache ad accesso diretto.....       | 91        |
| Aggiornare un blocco - Cache ad accesso diretto.....       | 92        |
| Cache ad accesso diretto con molteplici blocchi.....       | 92        |
| Cache e pipeline.....                                      | 93        |
| Tipologie di cache-miss e come migliorarle.....            | 94        |
| Compulsory miss (cold start o migrazione di processo)..... | 94        |
| Capacity miss.....                                         | 95        |
| Conflict miss.....                                         | 95        |
| Distribuzione e conseguenze delle Miss.....                | 96        |
| Algoritmi di rimpiazzamento.....                           | 99        |
| Cache Fully Associative.....                               | 99        |
| Cache Set Associative.....                                 | 100       |
| Funzionamento di una cache set associative.....            | 100       |
| Way Prediction.....                                        | 101       |
| Statistiche di miss e numero di vie.....                   | 101       |
| Cache Multilivello.....                                    | 102       |
| Strategie per hit in scrittura.....                        | 103       |
| Write-Through.....                                         | 103       |
| Write-Back.....                                            | 103       |
| Strategie per miss in scrittura.....                       | 104       |
| Write allocate.....                                        | 104       |
| No-Write allocate.....                                     | 104       |
| Strategie per il caricamento dei blocchi in cache.....     | 104       |



|                                                              |            |
|--------------------------------------------------------------|------------|
| Blocking.....                                                | 104        |
| Non-blocking.....                                            | 104        |
| No-Caching.....                                              | 104        |
| Prestazioni di una cache.....                                | 104        |
| Sintesi sul miglioramento delle performance della cache..... | 106        |
| Cache multibanked.....                                       | 107        |
| Ottimizzazione usando la cache.....                          | 107        |
| Migliorare le miss di conflitto.....                         | 108        |
| Soluzione Hardware.....                                      | 109        |
| Soluzione Software.....                                      | 109        |
| <b>Memoria virtuale.....</b>                                 | <b>110</b> |
| Indirizzi virtuali.....                                      | 110        |
| La pagina virtuale.....                                      | 111        |
| Page fault.....                                              | 111        |
| Working-set.....                                             | 111        |
| La dimensione della pagina.....                              | 112        |
| Segmentazione.....                                           | 113        |
| TLB.....                                                     | 113        |
| Cache e TLB.....                                             | 114        |
| Physically Indexed Physically Tagged.....                    | 114        |
| Virtually Indexed Virtually Tagged.....                      | 115        |
| TLB Multilivello.....                                        | 117        |
| TLB L1 e L2.....                                             | 117        |
| TLB Dati e Istruzioni.....                                   | 117        |
| <b>MIMD - Multiple Instruction Multiple Data.....</b>        | <b>118</b> |
| Tipologie di parallelismo di istruzioni.....                 | 118        |
| Instruction-Level Parallelism (ILP).....                     | 118        |
| Vector-Level Architecture.....                               | 118        |
| Thread-Level Parallelism.....                                | 118        |
| Request-level parallelism (RLP).....                         | 119        |
| Multiprocessori e Multicomputer.....                         | 119        |
| Multiprocessori - Accoppiamento stretto.....                 | 119        |
| Multicomputer - Accoppiamento lasco.....                     | 121        |
| Macchine Vettoriali.....                                     | 122        |
| Istruzioni vettoriali.....                                   | 122        |
| <b>Multiprocessori.....</b>                                  | <b>127</b> |
| SMP - Symmetric MultiProcessors.....                         | 127        |
| UMA - Uniform Memory Access.....                             | 128        |
| Accesso alla memoria condivisa e prestazioni.....            | 128        |
| DSM - Distributed Shared Memory.....                         | 129        |
| NUMA - NonUniform Memory Access.....                         | 130        |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| Cache + Memoria condivisa.....                                                 | 131        |
| Vantaggi di avere una cache in questo contesto.....                            | 131        |
| Svantaggi di avere una cache in questo contesto.....                           | 132        |
| Gestione della coerenza dei dati nelle cache.....                              | 132        |
| Approccio snooping based - SMP.....                                            | 133        |
| Approccio directory based - DSM e SMP.....                                     | 134        |
| MESI (Modified Exclusive Shared Invalid).....                                  | 135        |
| Stati di una cacheline.....                                                    | 135        |
| Funzionamento di MESI.....                                                     | 135        |
| In caso di cache-miss.....                                                     | 138        |
| Gestione della coerenza in cache multi-livello.....                            | 138        |
| Consultazione dei tag.....                                                     | 138        |
| Duplicazione dei Tag.....                                                      | 138        |
| Utilizzo della directory.....                                                  | 139        |
| Gestione della presenza dei dati nelle cache e la proprietà di inclusione..... | 140        |
| Il rimpiazzamento e il problema della duplicazione.....                        | 141        |
| Consistenza di memoria - Lock e Unlock.....                                    | 142        |
| Strict consistency.....                                                        | 142        |
| Sequential consistency.....                                                    | 142        |
| Memory Consistency Model V1.....                                               | 142        |
| Memory Consistency Model V2.....                                               | 143        |
| Memory Consistency Model V3.....                                               | 143        |
| Test & Set.....                                                                | 144        |
| Load Reserved (lr).....                                                        | 144        |
| Store Conditional (sc).....                                                    | 144        |
| Funzionamento ld e sc.....                                                     | 144        |
| Performance di questo sistema.....                                             | 146        |
| Effetto della dimensione del blocco di cache sulle miss.....                   | 146        |
| <b>Architetture specifiche per applicazione.....</b>                           | <b>150</b> |
| Domain-Specific Device.....                                                    | 151        |
| Regole di progettazione dei microcontrollori.....                              | 151        |
| Deep Neural Network.....                                                       | 152        |
| Fase di Training.....                                                          | 152        |
| Funzionamento di una CNN.....                                                  | 155        |
| Come Funziona una RNN.....                                                     | 157        |
| Cache NUCA.....                                                                | 161        |
| <b>Domande Esame/Presentazione.....</b>                                        | <b>162</b> |
| <b>GPU - Graphic Processing Unit.....</b>                                      | <b>164</b> |
| Architettura NVIDIA Fermi.....                                                 | 164        |
| SM - Streaming Multiprocessor.....                                             | 165        |
| CUDA - Compute Unified Device Architecture.....                                | 165        |

|                                              |            |
|----------------------------------------------|------------|
| CUDA core.....                               | 165        |
| Memoria condivisa.....                       | 166        |
| Warp Scheduler.....                          | 166        |
| Sistema eterogeneo - CPU + GPU.....          | 167        |
| Comunicazione CPU-GPU.....                   | 167        |
| Differenza di performance tra CPU e GPU..... | 168        |
| Utilizzo di un sistema eterogeneo.....       | 168        |
| <b>Linguaggio CUDA C.....</b>                | <b>169</b> |
| CUDA API.....                                | 170        |
| Sintassi di CUDA C.....                      | 171        |
| Utilizzo della memoria del device.....       | 171        |
| Trasferimento tra GPU e CPU.....             | 172        |
| Kernel.....                                  | 174        |
| CUDA Blocks.....                             | 174        |
| CUDA Threads.....                            | 175        |
| CUDA Grid Index.....                         | 176        |
| Cuda Warps.....                              | 177        |
| Warp Scheduler.....                          | 178        |
| Divergenza degli Warp.....                   | 180        |
| Migliorare lo stalling dei warp.....         | 181        |
| CUDA Grid.....                               | 183        |
| Gestione della memoria della GPU.....        | 185        |
| Texture memory.....                          | 185        |
| Shared memory.....                           | 186        |
| Registri.....                                | 186        |
| Local memory.....                            | 186        |
| Shared memory.....                           | 186        |
| Cache della GPU.....                         | 186        |
| Global memory.....                           | 187        |
| Constant memory.....                         | 187        |
| Qualifier.....                               | 187        |
| Race condition.....                          | 187        |
| <b>GPU in Python.....</b>                    | <b>188</b> |
| <b>Progetto.....</b>                         | <b>192</b> |
| Parte CPU.....                               | 192        |
| Parte GPU.....                               | 192        |
| <b>Domande d'esame.....</b>                  | <b>193</b> |
| Pipeline.....                                | 194        |
| Cache.....                                   | 197        |

# Il trend nelle micro-architetture

[1986 - 2003]

Illustriamo un breve excursus sull'evoluzione delle architetture negli ultimi quarant'anni

Tra il 1986 e il 2003 succedono molte cose: viene inventato il microprocessore, nasce UNIX e il linguaggio C.

## UNIX

UNIX è stata una svolta nei sistemi operativi perchè era gratis e aggiornato di continuo.

## Linguaggio C

UNIX è stato totalmente sviluppato in assembler e portarlo in un'altra macchina avrebbe comportato di riscrivere un altro assembler (*ogni processore ha il suo assembler perché quest'ultimo rispecchia l'instruction set della CPU*).

Allora era conveniente costruire un compilatore che dato in input un linguaggio restituiva un programma in assembler.

## Architettura RISC - Reduced Instruction Set Computer

Approccio progettuale per i processori che mira a semplificare l'instruction set, **concentrandosi su istruzioni di lunghezza fissa e solo per le operazioni di base** (*quindi riducendo anche il numero di istruzioni, quindi meno hardware e più spazio per cache più complesse o altro*).

Questo approccio cerca di migliorare le prestazioni del processore, riducendo il tempo necessario per eseguire istruzioni complesse.

I processori RISC tendono ad essere **più efficienti**, con un'architettura che favorisce una **maggiore parallelizzazione** delle operazioni e una **pipeline più breve**. Questo porta a una migliore gestione delle risorse e a una maggiore velocità di esecuzione delle istruzioni.

## Architettura CISC - Complex Instruction Set Computer

Approccio progettuale per i processori che si contrappone all'architettura RISC. Contrariamente a RISC, CISC mira a fornire un set di istruzioni più ricco e complesso.

In un'architettura CISC, le istruzioni possono eseguire più operazioni alla volta, rispetto a RISC. Ciò può includere operazioni complesse come il caricamento e lo

scaricamento di dati dalla memoria, operazioni matematiche complesse e istruzioni di controllo del flusso più sofisticate.

L'obiettivo principale di CISC è quello di rendere più facile la programmazione, consentendo ai programmatori di scrivere codice più conciso e di ridurre il numero di istruzioni necessarie per eseguire determinate operazioni. Tuttavia, a volte questo può comportare una complessità aggiuntiva nell'implementazione hardware del processore.

### **Instruction Level Parallelism**

Tecnica utilizzata nell'architettura dei processori per migliorare le prestazioni eseguendo più istruzioni contemporaneamente o sovrapponendo l'esecuzione di istruzioni indipendenti. L'obiettivo è **umentare il throughput di singole istruzioni** e ridurre il tempo di esecuzione dei programmi.

### **Personal Computer**

I grossi e costosi mainframe iniziarono a sparire e furono sostituiti dai comuni microprocessori general purpose, compaiono anche i primi tablet e telefonini.

### **Processori ARM**

ARM divenne dominante, su questi sistemi si poteva spegnere il microprocessore quando non serviva, e veniva risvegliato dall'I/O, il quale non faceva niente finché l'utente non interagiva, quindi il risparmio energetico era tutto lì.

### **Warehouse**

Nascono le warehouse con internet, le quali sono viste dagli utenti come grandi macchine a cui richiedere servizi, costruendo un sistema con tante macchine si possono spegnere le macchine quando non c'è traffico e riaccenderle man mano che servono. Inoltre con l'avvento del telefonino, che era una cosa nuova, il microprocessore veniva utilizzato anche per voce, suoni, immagini e video.

### **[2003 - 2011]**

Se si vuole costruire da zero un nuovo sistema i costi sono elevatissimi ma si guadagna in performance.

Se i componenti vengono acquistati già fatti dal mercato, è il mercato che paga la ricerca, quindi conviene copiare da sistemi già esistenti.

### **Multiprocessori**

Dai microprocessori si passò ai multiprocessori; anche se in realtà esistevano già però erano tanti microprocessori collegati tramite un bus.

## Thread e le Macchine dataflow

Il parallelismo era cambiato, era basato sui thread.

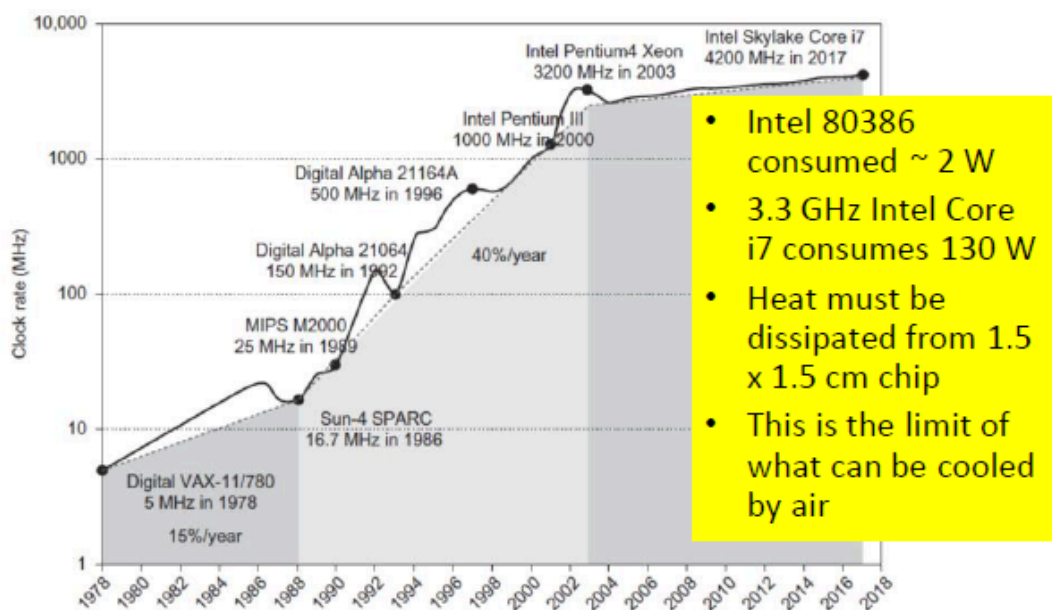
Inizialmente nati nelle macchine data flow, ossia macchine in cui il sistema globale è costituito da tante “palline”: Il dato entra da qualche parte, attraversa le palline e viene dato il risultato di uscita.

Ogni pallina è un processo, il problema è che se ho tanti processi, la macchina non regge a causa dei continui cambi di contesto, allora è stato creato il concetto di thread.

I microprocessori dovettero imparare a gestire i thread in hardware: la novità introdotta dai thread è che tutti i thread nati dallo stesso padre hanno lo spazio di memoria virtuale condiviso.

I thread furono implementati subito in UNIX e successivamente anche in Windows.

## Potenza dissipata nel tempo



In figura sopra è possibile vedere il consumo di energia elettrica dei processori, dal 1978 al 2018.

## Trend delle tecnologie

All'interno dei calcolatori inizialmente, la legge di Moore stabiliva che la densità dei transistor raddoppiava ogni anno, ma nel corso del tempo questa tendenza si è rallentata e si è stabilizzata su un raddoppio di ogni due anni. Questo fenomeno è stato osservato nel settore della tecnologia dei semiconduttori e ha avuto conseguenze significative sulle prestazioni e sulla complessità dei circuiti integrati.

Riguardo alle memorie RAM, mentre le prestazioni sono migliorate nel corso degli anni, è diventato sempre più difficile ottenere incrementi significativi nella densità e nelle prestazioni. Di conseguenza, i ricercatori stanno esplorando nuove tecnologie, come le memorie flash, che hanno rivoluzionato l'archiviazione nei dispositivi portatili e hanno contribuito all'evoluzione dei dispositivi elettronici moderni.

### Banda e throughput

I due punti vitali per le prestazioni sono la **banda** (o *throughput*) e la **latenza** (o *tempo di risposta*).

Il throughput indica il numero di lavori svolti nell'unità di tempo.

Il response time riguarda il settore real time in cui c'è un'interazione diretta con l'utente e più ridotto è il tempo, più l'utente percepisce qualità (soft real time, ovvero sistemi in cui l'obiettivo è rispondere entro determinati limiti di tempo, ma occasionali violazioni di questi limiti possono essere tollerate).

### Disco

Il disco ha avuto un'evoluzione positiva: Il throughput nel disco è incrementato di 100 volte, mentre la latenza di 10, se seguisse la linea retta nel grafico, andrebbero di pari passo.

### Memoria RAM

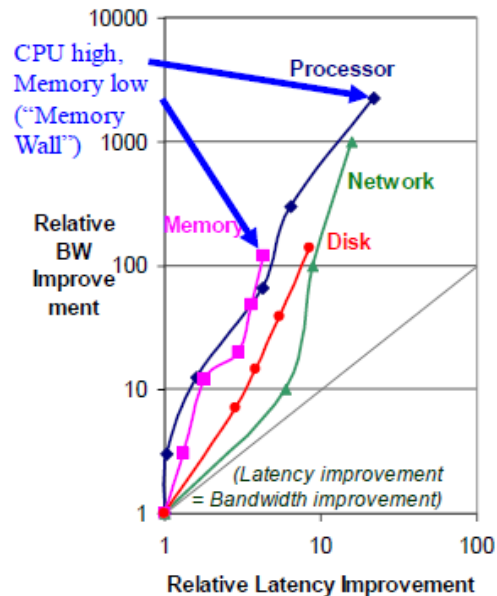
Nella memoria l'evoluzione è anche peggiore del disco, nel disco è stato utilizzato il trucco del throughput, nella memoria la latenza è diminuita di 6, il throughput è aumentato di 100.

### Rete

Grazie alla fibra, il throughput è migliorato di 1000 volte, la latenza è migliore negli altri casi.

## Processore

Nel processore è andata ancora meglio, sia nel throughput che nella latenza, il problema è la grande distanza tra la memoria ed il processore dato che devono lavorare insieme, il processore ha bisogno della memoria.



## Come colmare il divario tra CPU e RAM?

Ad esempio nel processore Intel i7, la velocità del microprocessore è di 409,6 GB/sec, mentre la memoria ha una velocità di 25 GB/sec, come si risolve questo problema?

### Multiporting

Permettere a più dispositivi di accedere a diverse parti della memoria in parallelo per migliorare le prestazioni del sistema. Quindi se un processore accede ad una zona di memoria, allora un altro processore può accedere ad un'altra zona.

### Cache multilivello

Utilizzata per ridurre il tempo di accesso alla memoria principale conservando una copia delle informazioni frequentemente utilizzate nella cache più vicina al processore.

Riduce il tempo di accesso alla memoria principale e migliora le prestazioni complessive del sistema.

Il tempo di accesso in cache è il tempo di hit per la probabilità di hit + il tempo di miss per la probabilità di miss.



## Categorie di calcolatori

Nella figura sotto ci sono le caratteristiche di ogni dispositivo visto e i relativi prezzi delle componenti dominanti sono quelle evidenziate.

| Feature                       | Personal mobile device (PMD)                    | Desktop                                         | Server                                        | Clusters/warehouse-scale computer                     | Internet of things/ embedded                    |
|-------------------------------|-------------------------------------------------|-------------------------------------------------|-----------------------------------------------|-------------------------------------------------------|-------------------------------------------------|
| Price of system               | \$100–\$1000                                    | \$300–\$2500                                    | \$5000–\$10,000,000                           | \$100,000–\$200,000,000                               | \$10–\$100,000                                  |
| Price of microprocessor       | \$10–\$100                                      | \$50–\$500                                      | \$200–\$2000                                  | \$50–\$250                                            | \$0.01–\$100                                    |
| Critical system design issues | Cost, energy, media performance, responsiveness | Price-performance, energy, graphics performance | Throughput, availability, scalability, energy | Price-performance, throughput, energy proportionality | Price, energy, application-specific performance |

### Sistemi embedded

Come ad esempio lavatrici, macchine, stampanti ecc, è un settore in cui l'Europa è leader nella vendita di prodotti.

### Internet of Things

Consiste nel gestire una periferica senza l'intervento dell'umano che abilita una connessione di rete, il software che si vuole costruire deve essere indipendente dal device, come ad esempio i contatori dell'Enel, vogliamo gestirli senza conoscere il tipo che è montato.

Amplia le possibilità di questi settori, dato che standardizza e permette di creare nuove cose, in questo settore si mette il processore che serve e nulla di più, al massimo il 10% in più, si hanno dei microcontrollori, ovvero hanno un microprocessore, con la memoria, dei contatori e dispositivi per inviare onde quadre per comunicare con l'esterno.

### Personal Mobile Device

Si sono diffusi principalmente grazie alle memorie flash.

Questi dispositivi utilizzano batterie e quindi il consumo deve essere minimizzato. Un altro limite inedito è dato dal design: L'hardware si adatta al design e non viceversa. Le applicazioni in questo caso sono principalmente web-based e orientate a video e immagini.

## Desktop

Il mercato è il rapporto prezzo-prestazione, le applicazioni sono tipicamente web, interattive e ci aspettiamo buone prestazioni. Molte soluzioni nelle aziende vengono risolte tramite un cloud.

## Server

Nascono con internet, il primo era di posta elettronica, e hanno le seguenti caratteristiche:

- **Availability:** La macchina deve essere attiva 24 ore su 24 e 7 giorni su 7.
- **Scalability:** La capacità di aumentare memoria, storage e bandwidth in modo semplice ed economico.
- **Efficient throughput:** Performance generale, in termini di transazioni per minuto, il 90% delle operazioni devono essere eseguite entro 3 secondi, del restante 10% non ci interessa.
- **Cluster:** Hanno una lista degli utenti condivisa tra tutti, nati con l'idea di dare una grande potenza di calcolo, su queste macchine l'80% del costo è il consumo, si consuma per raffreddare (vengono raffreddate a liquido), quindi si cerca di metterle in zone fredde per risparmiare.

# Benchmark

Noi ci confrontiamo con il VAX 11/780, il benchmark misurato è lo SPEC integer, il benchmark è un insieme di programmi, è un punto di riferimento standard utilizzato per valutare o confrontare le prestazioni di un sistema, un dispositivo o un'altra entità rispetto agli altri.

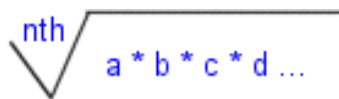
I benchmark sono spesso utilizzati per valutare le prestazioni dei computer, dei processori, dei dispositivi di archiviazione, dei software e di altri sistemi o componenti hardware.

## Indici di performance

### Media quadratica

$$\sqrt{\frac{x_1^2 + x_2^2 + \dots + x_n^2}{n}}$$

### Media geometrica



Per calcolare una media geometrica, si moltiplicano insieme tutti i risultati dei test e si prende la radice n-esima del prodotto, dove n è uguale al numero di test.

### Tempo di esecuzione

Il tempo di esecuzione dal punto di vista dell'utilizzatore è detto anche "*latenza*". E' il tempo che trascorre dall'evento alla risposta del sistema.

### Throughput

Il numero di lavori (*sta a me decidere cosa è una unità di lavoro*) che riesco a fare nell'unità di tempo, ad esempio il numero di pagine web che posso dare ai clienti in un periodo di tempo.

Da questa metrica si possono valutare anche i costi hardware di una pagina web.

### MIPS - Million Instructions Per Second

Si possono usare i MIPS come base di confronto tra due architetture? Non è ideale, perché è un'unità di misura della velocità di elaborazione di un processore, ma **non** tiene conto della complessità delle istruzioni o delle differenze nel set di istruzioni tra i processori.

Due sistemi con velocità di clock simili e diverse architetture di istruzioni possono avere prestazioni molto diverse anche se misurati in MIPS.

Inoltre, il MIPS non considera fattori come la pipeline del processore, la cache, l'architettura del bus e altri aspetti dell'implementazione hardware che influenzano le prestazioni complessive di un sistema. Pertanto, confrontare il throughput utilizzando solo il MIPS potrebbe essere fuorviante e non fornirebbe una valutazione accurata delle prestazioni relative dei due sistemi.

## Speed-Up

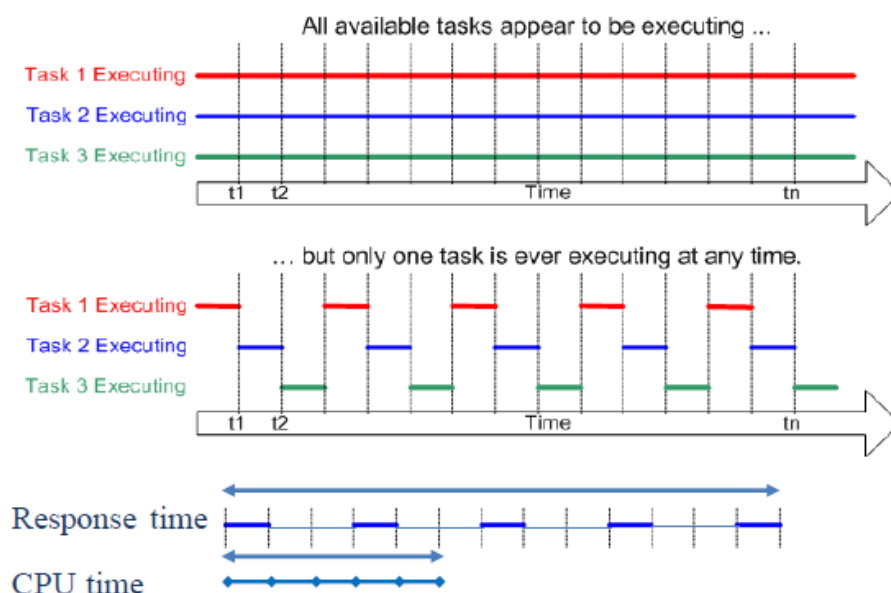
Serve per vedere se un calcolatore è più veloce rispetto ad un calcolatore che conosco.

## Come utilizzare il benchmark

Si utilizzano i benchmark, in essi ci sono i kernel che sono un insieme di applicazioni utilizzate in un OS, di cui calcoliamo i tempi di risposta, si calcola il rapporto e indichiamo la media quadratica di questi valori.

Le suite di benchmark sono importanti, sono gestite da consorzi in cui ci sono tutti i fornitori, definiscono le condizioni di esecuzione e effettuano le misure. Il set di programmi del kernel vengono cambiati ogni anno perché alcuni processori si adattano ai programmi e quindi le prestazioni risultano migliori. (da verificare)

Se ho tanti task in esecuzione, la risposta non è sempre uguale perché c'è anche il sistema operativo e bisogna definire anche qual è il carico.



## Quale benchmark usare?

Si deve trovare il più adatto al nostro scopo e al tipo di architettura che vogliamo realizzare.

Nel mondo ci sono tanti sistemi che misurano i benchmark e ci sono consorzi che offrono vari benchmark per vari settori applicativi:

- **SPEC:** E' stato il primo benchmark ad uscire ed è specializzato nell' analisi di programmi con dati di tipo intero o floating point.
- **TPC:** Analizza programmi che utilizzano database.
- **TPC-A:** Riguarda operazioni **OLTP** (*on-line transaction processing application*), ovvero letture e scritture eseguite attraverso la rete.
- **TPC-H:** Riguarda grandi collezioni di dati.

## Benchmark ULP

### ULPMark

Il benchmark esegue un carico di lavoro attivo per un periodo di tempo, quindi va in modalità sleep. La misurazione dell'energia durante il ciclo di lavoro riflette un test reale di bassa potenza incorporata oltre un semplice numero per la sleep.

### ULPMark-PeripheralProfile

Esamina il costo energetico di quattro periferiche: real time clock, modulazione di larghezza di impulso, conversione da analogico a digitale e comunicazione SPI.

### ULPMark-CoreMark

Misura l'energia di CoreMark in un ambiente consistente. Il punteggio ULPMark-CM è proporzionale al numero di iterazioni CoreMark che un dispositivo può eseguire per millijoule.

## Benchmark per telefoni e tablet

### BrowsingBench:

Misura le prestazioni del browser con Browsing Bench, una raccolta di pagine web caricate utilizzando un server Nginx locale tramite una connessione LAN cablata.

### AndEBench-Pro

Disponibile gratuitamente nel Play Store di Android, AndeBench è un metodo accettato dal settore per valutare le prestazioni della piattaforma Android.

## Come calcolare il punteggio del benchmark

Il punteggio riportato per ciascun dispositivo è numero singolo calcolato prendendo la media geometrica dei punteggi AutoBench individuali e dividendola per **307.455**.

Questo fattore di normalizzazione (307.455) deriva dal punteggio più basso in questa categoria il 5 dicembre 2000 (*quindi ogni punteggio è misurato in base a questa prestazione*).

I punteggi per ciascuno dei singoli benchmark all'interno di questa suite consentono ai progettisti di aggregare i benchmark per soddisfare requisiti applicativi specifici.

## Elenco di alcuni benchmark

### **SPEC MPI2007**

La suite di benchmark di SPEC per la valutazione delle prestazioni MPI-parallele, in virgola mobile e ad alta intensità di calcolo su un'ampia gamma di cluster e hardware SMP.

### **SPEC OMP2012**

progettato per misurare le prestazioni utilizzando applicazioni basate sullo standard OpenMP 3.1 per l'elaborazione parallela di memoria condivisa.

### **SOA**

SPEC per implementazioni tipiche di middleware, database e deployment hardware di applicazioni basate sulla Service Oriented Architecture.

### **Virtualization**

Il primo benchmark di SPEC che affronta la valutazione delle prestazioni dei server nei datacenter utilizzati nei server virtualizzati.

## Elaborazione di transazioni

### **Benchmark sull'elaborazione delle transazioni (TP)**

I benchmark di elaborazione delle transazioni (TP) misurano la capacità di un sistema di **gestire transazioni che consistono in accessi e aggiornamenti al database**.

I sistemi di prenotazione aerea e i sistemi ATM bancari sono tipici e semplici esempi di TP; i sistemi TP più sofisticati coinvolgono database e processi decisionali complessi.

## TPC-C

Simula un ambiente informatico completo in cui una popolazione di **utenti** esegue transazioni su un **database**.

Il benchmark è incentrato sulle principali attività (*transazioni*) di un ambiente di immissione degli ordini.

Queste transazioni includono l'inserimento e la consegna degli ordini, la registrazione dei pagamenti, il controllo dello stato degli ordini e il monitoraggio del livello delle scorte nei magazzini.

Il benchmark ritrae l'attività di un fornitore all'ingrosso, rappresenta qualsiasi settore che deve gestire, vendere o distribuire un prodotto o servizio.

## TPC-H

consiste in una suite di query ad hoc orientate al business e modifiche concorrenti dei dati.

Le query e i dati che popolano il database sono stati scelti per avere un'ampia rilevanza a livello di settore.

Questo benchmark illustra i sistemi di supporto decisionale che esaminano grandi volumi di dati, eseguono query con un elevato grado di complessità e forniscono risposte a domande aziendali critiche.

La metrica delle prestazioni riportata da TPC-H è denominata **metrica delle prestazioni delle query composite per ora TPC-H** (QphH@Size).

## TPC-E

Prevede un mix di dodici transazioni simultanee di diverso tipo e complessità, eseguite online o attivate da criteri di prezzo o temporali.

Il database è composto da trentatré tabelle con un'ampia gamma di colonne, cardinalità e proprietà di ridimensionamento.

## TPC-E

Viene misurato in **transazioni al secondo** (tpsE); non si limita all'attività di un particolare segmento di business, ma rappresenta piuttosto qualsiasi settore che deve riferire ed eseguire transazioni di natura finanziaria.

## SPEC (Standard Performance Evaluation Corporation)

E' un consorzio in cui ci sono tutti i produttori di microprocessori, i quali definiscono kernel, i quali sono programmi che vengono eseguiti nei loro processori, usando questi programmi ed eseguendoli sul sistema VAX si ottengono certe prestazioni da utilizzare come riferimento.

*(VAX, "Virtual Address eXtension", è una famiglia di computer e microprocessori sviluppati da Digital Equipment Corporation - DEC a partire dagli anni 70. I sistemi VAX sono noti per la loro architettura complessa e flessibile).*

Successivamente questi programmi si eseguono sul microprocessore da produrre e si prende il tempo di esecuzione confrontandolo con il tempo ottenuto dall'esecuzione nel VAX, *(si fa il rapporto tra i due tempi, si fa la radice, dopodichè si prende i quadrato).*



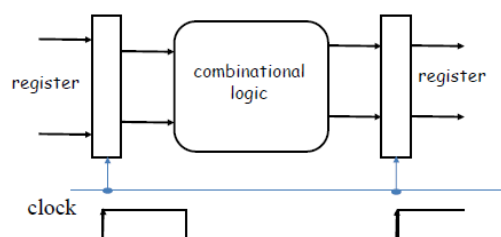
# Progettazione di un calcolatore

Ora vedremo quali sono le principali componenti da progettare quando si progetta un calcolatore.

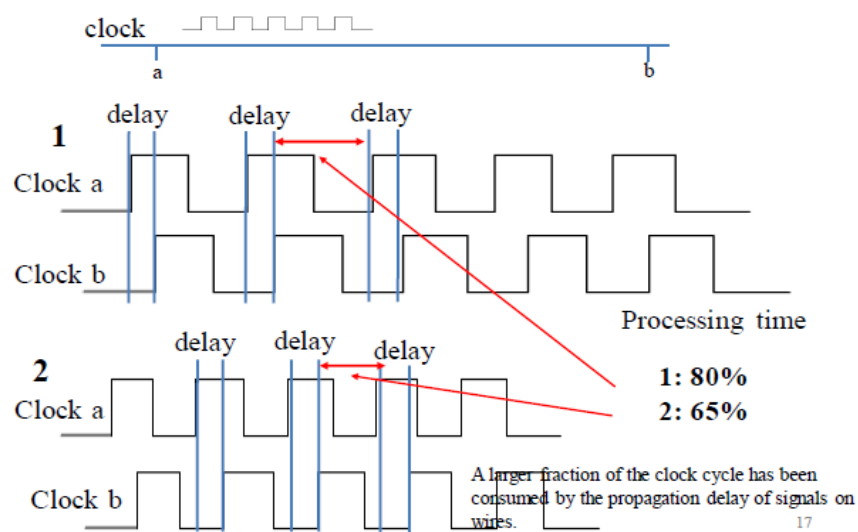
## Clock e le reti sincronizzate

Un microprocessore può essere immaginato come una rete sincronizzata molto complessa la quale ha come funzionamento base il seguente:

1. Arriva il fronte di salita del clock.
2. Il registro di entrata campiona (*richiede una quantità di tempo quasi costante*).
3. Poco dopo il registro di entrata dà in uscita il valore.
4. Si passa la logica combinatoria (*di solito molto veloce*).
5. Il registro di uscita campiona (*richiede una quantità di tempo quasi costante*).
6. Poco dopo il registro di uscita dà in uscita il valore.



Nota che il tempo necessario ad un registro per memorizzare l'informazione non è nullo ed è quasi costante, quindi aumentando la frequenza di lavoro aumenta anche la penalty data da questo ritardo ineliminabile.



## Instruction set

L'Instruction set individua l'insieme di istruzioni più utilizzate dal processore e dipende dai programmi e dalle mode del momento (*il calcolatore deve avere le migliori caratteristiche rispetto all'uso che se ne farà al momento*); è molto importante poiché condiziona tutte le fasi successive.

Come detto prima: una buona progettazione è anche lungimirante, ossia deve implementare (*o comunque avere componenti che le appoggiano*) le caratteristiche che saranno richieste nel futuro. Da questo punto di vista il settore più difficile è quello delle console da gaming.

## Tipologia di applicazione

L'Instruction set deve essere adattato a quello che vogliamo fare, in modo tale che le operazioni più utilizzate siano ottimizzate e il più veloci possibili.

Abbiamo generalmente due tipi di applicazioni:

### **Settore Embedded**

Le applicazioni sono fortemente orientate ad un uso specifico: ad esempio microcontrollori di robot, switch di rete, etc..

### **General Purpose**

Le applicazioni non sono orientate ad un uso specifico; In questo secondo caso le applicazioni da considerare sono quelle in uso nel presente e quelle che si pensa vengano usate nel futuro.

L'idea di base è di costruire una macchina che al suo interno ha acceleratori adatti a fare in modo che la macchina abbia tempi di risposta adatti alle richieste degli utenti.

## Linguaggi di programmazione

Periodicamente cambiano i linguaggi di programmazione più in voga e il nostro instruction set deve permettere a questi linguaggi di essere eseguiti in modo veloce ed efficiente.

Ad esempio in Java non ci interessano le specifiche hardware della macchina in cui avviene l'esecuzione perché la Java Virtual Machine permette di astrarre sia l'hardware del calcolatore che il sistema operativo.

### **Retrocompatibilità**

Un software scritto anni fa (*il suo eseguibile, scritto in linguaggio binario e quindi assembly*) dovrebbe poter essere mandato in esecuzione anche sulle macchine attuali.

## Binary code e Object code

In dettaglio, il binary code e l'object code sono entrambi tipi di codice che derivano dalla compilazione di un programma scritto in un linguaggio di programmazione ad alto livello, come C, C++, Java, ecc. Tuttavia, ci sono delle differenze significative tra i due:

Quindi ci sono due tipi di codice: l'**object code** e il **binary code**.

### **Binary Code (Codice Binario)**

Il codice binario è il risultato diretto della compilazione di un programma sorgente.

È scritto in linguaggio macchina quindi una sequenza di 0 e 1.

Non contiene informazioni sulla struttura del programma o sulle variabili utilizzate nel codice sorgente.

### **Object Code (Codice Oggetto)**

L'object code, nel contesto delle architetture dei calcolatori, è il risultato della fase di compilazione di un programma sorgente. È il codice binario che rappresenta il programma in una forma che può essere eseguita direttamente dalla CPU del computer. L'object code è una sequenza di istruzioni macchina specifiche per l'architettura del processore di destinazione, scritte in linguaggio macchina comprensibile per il processore. Questo codice è in genere non leggibile per gli esseri umani, essendo costituito da una serie di numeri binari che corrispondono alle istruzioni e ai dati del programma. Una volta che l'object code è stato generato, può essere eseguito direttamente dalla CPU del computer senza bisogno di ulteriori traduzioni.

## Sistema Operativo

Il sistema operativo che sarà usato col microprocessore che stiamo progettando non può essere ignorato, dobbiamo sapere quanta memoria richiede il SO, bisogna saper dimensionare lo spazio d'indirizzo sia virtuale, utilizzato nella programmazione, che quello fisico.

- In un processore general purpose ci aspettiamo che vengano usati sistemi operativi come Windows o Ubuntu.
- In un processore da server invece ci aspettiamo sistemi operativi basati su Linux o addirittura Hypervisor bare metal.
- In microcontrollori o sistemi embedded ci aspettiamo sistemi operativi molto leggeri basati su Linux come ad esempio Lubuntu.

## Memory Management

Quindi dobbiamo tenere conto anche il memory management, c'è anche la segmentazione utile per la protezione.

## Multiutente e virtualizzazione

C'è da tener conto della virtualizzazione, ovvero consentire ad utenti diversi di accedere alla stessa macchina utilizzando OS diversi.

## Standard

Standard come: floating point, I/O, OS, rete, linguaggi di programmazione, tutto questo influisce nella scelta dell'architettura e nel modo in cui la si progetta perché non adottarli potrebbe causare una uscita dal mercato di riferimento.

Uno standard obsoleto o che lo sarà a breve potrebbe non essere implementato, ma un qualcosa che pensiamo lo diventerà a breve è saggio implementarlo in qualche modo

## Le mode del momento

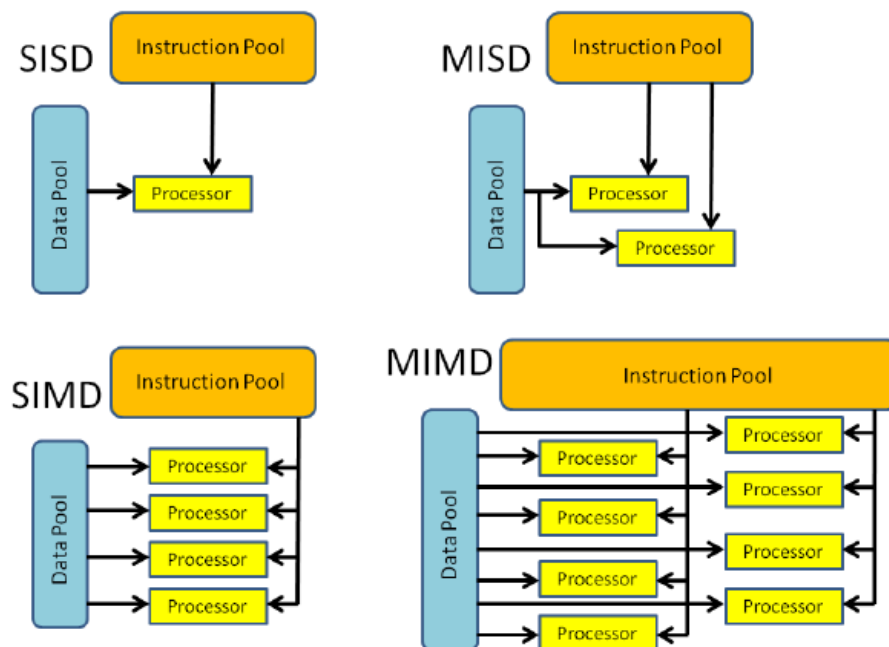
Ad esempio in questo momento il settore delle automobili sta crescendo molto (*guida autonoma, controllori, etc*).

Altri settori in cui il mercato è in crescita sono i device personali portatili, dispositivi a casa, sistemi per le smart city e sistemi per la salute. Le opportunità del momento:

- Il **multicore** ha sfondato nel settore della telefonia però nei sistemi embedded non ha sfondato.
- **L'IoT** nel quale l'applicazione viene sviluppata indipendentemente dal dispositivo.
- **Intelligenza artificiale**, abbate il costo di competenza che bisogna avere per realizzare dei prodotti, non è una moda, allargherà l'insieme di persone che possono progettare sistemi; buona cosa introdurre acceleratori per l'intelligenza artificiale.

## Tassonomia di Flynn

Consiste in una classificazione di vari tipi di macchine:



|                     |          | Data Streams            |                               |
|---------------------|----------|-------------------------|-------------------------------|
|                     |          | Single                  | Multiple                      |
| Instruction Streams | Single   | SISD: Intel Pentium 4   | SIMD: SSE instructions of x86 |
|                     | Multiple | MISD: No examples today | MIMD: Intel Core i7           |

### SISD

Si ha un singolo processore e un singolo flusso di istruzioni.

### SIMD

Si ha un singolo flusso di istruzioni ma su più flussi di dati: Esempi di questo tipo sono le macchine vettoriali ed estensioni multimediali.

### MISD

Multipli flussi di Istruzioni che vengono applicati su un unico flusso di dati, in questo caso non hanno un'implementazione commerciale.

In realtà questo concetto è utilizzato nelle pipeline, non esiste a livello hardware però tutte le pipeline sono organizzate così.

## MIMD

Architettura di computer in cui più processori eseguono istruzioni diverse su dati diversi contemporaneamente. La comunicazione dei dati e delle istruzioni in MIMD può essere influenzata da fattori come l'accesso in memoria e i tempi di comunicazione sulla rete.

## Gestione degli Errori

### SLA - Service Level Agreement

Fornisce delle regole, in cui sono definite delle penali se non vengono mantenute certe tempistiche nella gestione degli errori.

### Error detection

La distanza temporale tra **la comparsa di un errore e il malfunzionamento del sistema**.

Questo significa che può passare del tempo tra l'occorrenza di un errore e il momento in cui il sistema manifesta effettivamente un comportamento errato, che può essere osservato dagli utenti o dagli operatori del sistema.

### Incapsulamento

#### Incapsulamento

Questa distanza temporale può essere ridotta utilizzando tecniche come l'**incapsulamento** dei dati con tecniche come CRC o altro:

**Quando un dato entra ed esce dalla memoria, viene effettuato un controllo per garantire che il dato sia corretto, riducendo così la possibilità che un errore si propaghi nel sistema e causi malfunzionamenti.**

Tuttavia, l'implementazione di queste tecniche comporta costi aggiuntivi in termini di complessità del software e utilizzo delle risorse di sistema.

### System reconfiguration

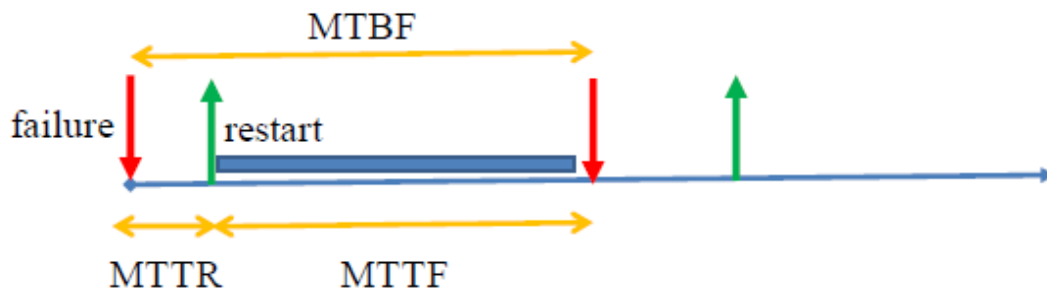
Trovare un'altra forma del nostro sistema per garantire almeno le funzionalità più importanti (*è la stessa cosa di avere in macchina una quinta ruota che non viene utilizzata, viene utilizzata solo quando serve*).

Ad esempio la modalità provvisoria di windows.

### System recovery

Ovvero consentire al sistema di ripartire da uno stato consistente ed andare avanti.

## Metriche per l'analisi dei guasti



Nel grafico sopra:

- La freccia rossa indica il guasto.
- La freccia verde indica che il sistema è ripartito

Il numero di coppie di frecce rosse/verdi è il numero di guasti in un dato periodo di tempo.

### Metriche numeriche

Mean time to failure (MTTF)

Mean time to repair (MTTR)

Mean time between failures (MTBF) = MTTF + MTTR

Availability = MTTF / MTBF

### Esempio

Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 ATA controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 ATA cable, 1,000,000-hour MTTF

Supponi che il tempo di vita del modulo sia una variabile distribuita in modo esponenziale. Supponi che i fallimenti siano tutti indipendenti.

Calcola il MTTF dell'intero sistema.

I fallimenti sono tutti indipendenti, quindi la probabilità di un guasto è la somma delle probabilità. Tutto moltiplicato per 10 perché ho 10 dischi.

$$\begin{aligned}\text{Failure rate}_{\text{system}} &= 10 \times \frac{1}{1,000,000} + \frac{1}{500,000} + \frac{1}{200,000} + \frac{1}{200,000} + \frac{1}{1,000,000} \\ &= \frac{10 + 2 + 5 + 5 + 1}{1,000,000 \text{ hours}} = \frac{23}{1,000,000} = \frac{23,000}{1,000,000,000 \text{ hours}}\end{aligned}$$

Quindi il MTTF del sistema si calcola in questo modo:

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1,000,000,000 \text{ hours}}{23,000} = 43,500 \text{ hours}$$

1 power supply, 200,000-hour MTTF

1 fan, 200,000-hour MTTF

$$\text{MTTF}_{\text{system}} < \frac{(\text{MTTF}_{\text{power\_supply}} + \text{MTTF}_{\text{fan}})}{2}$$

Quindi deduco che il prossimo guasto si verificherà tra 43000 ore, ossia tra circa 5 anni.

Cosa succede se metto una ridondanza nel sistema di alimentazione?

$$\text{MTTF}_{\text{power supply pair}} = \frac{\text{MTTF}_{\text{power supply}}/2}{\frac{\text{MTTR}_{\text{power supply}}}{\text{MTTF}_{\text{power supply}}}} = \frac{\text{MTTF}_{\text{power supply}}^2/2}{\text{MTTR}_{\text{power supply}}} = \frac{\text{MTTF}_{\text{power supply}}^2}{2 \times \text{MTTR}_{\text{power supply}}}$$

If we assume it takes on average 24 hours to replace the power supply, and that

$$\text{MTTF}_{\text{power\_supply}} = 200,000\text{-hour}$$

The reliability of the fault tolerant pair of power supplies is

$$\text{MTTF}_{\text{power supply pair}} = \frac{\text{MTTF}_{\text{power supply}}^2}{2 \times \text{MTTR}_{\text{power supply}}} = \frac{200,000^2}{2 \times 24} \cong 830,000,000$$

4150 times more reliable than a single power supply

7

Ridondanza

Ridondanza nel tempo e nelle risorse:

### Ridondanza nel tempo

Nel primo caso si guarda se il guasto è temporaneo, ad esempio il disco, se c'è un errore prova a scrivere 5 volte prima di generare l'errore, visto che il guasto potrebbe essere temporaneo.



## **Ridondanza nelle risorse**

Si introducono elementi architetture aggiuntivi.

Nel sistema dell'esempio di prima avevamo scoperto che l'alimentatore dava problemi di velocità, quindi sono stati introdotti due alimentatori e le prestazioni migliorano.

## Valutazione delle performance di un calcolatore

Si utilizza una metrica chiamata CPI.

$$CPU\ time = \text{Number of clock cycles for a program} * \text{Clock cycle time}$$

$$CPU\ time = \text{Instruction count} * \text{Cycles per instruction} * \text{Clock cycle time}$$

Con *Clock cycle time* la durata di un ciclo di clock.

$$CPI = \frac{\text{Number of clock cycles for a program}}{\text{Clock rate}} = \frac{\text{Number of clock cycles for a program}}{\text{instruction count}}$$

Ma il CPI dipende anche dal tipo di istruzione che si sta eseguendo.

$$CPU\ clock\ cycles = \sum_{i=1}^n IC_i * CPI_i$$

Con  $IC_i$  il numero di volte che viene eseguita l'istruzione  $i$

Con  $CPI_i$  il numero di cicli di clock necessari per eseguire l'istruzione  $i$

$$\text{E quindi: } CPU\ time = \left( \sum_{i=1}^n IC_i * CPI_i \right) * \text{Clock cycle time}$$

## Come migliorare le performance

Ci sono varie azioni che possiamo fare, basate sui nostri problemi:

### Trarre vantaggi dal parallelismo

- Utilizzare più processori, dischi, banchi di memoria, pipeline, molteplici unità funzionali.
- Progettare una soluzione parallela efficiente.

### Garantire il bilanciamento del carico di ciascun componente

In caso di molteplici core, schedulare i thread in modo da non sovraccaricare uno.

### Ridurre al minimo l'overhead

Ossia ridurre il tempo impiegato nelle comunicazioni (*quantità di dati scambiati, velocità della tecnologia di comunicazione, collegamento privato/condiviso, traffico, ...*) e il tempo impiegato per gestire il parallelismo (*processi, distribuzione del lavoro, raccolta dei risultati, ...*)

### Come sfruttare il principio di località

Tramite un furbo riutilizzo di dati e istruzioni

- La **località temporale** indica che è probabile che gli elementi a cui si è avuto accesso di recente saranno accessibili nel prossimo futuro.
- La **località spaziale** dice che gli elementi i cui indirizzi sono vicini tra loro tendono a essere referenziati insieme nel tempo.

### Concentrarsi sul caso comune - Legge di Amdahl

Ossia sfruttare la legge di Amdahl, quindi dobbiamo decidere: Qual è il caso frequente e quello che rallenta di più (*bottleneck*); e quanto è possibile migliorare le prestazioni rendendo il caso più veloce.

$$\text{Speedup overall} = \frac{\text{Execution time for entire task without using the enhancement}}{\text{Execution time for entire task using the enhancement when possible}}$$

- **Fraction\_e**, The fraction of the computation time in the original solution that can be converted to take advantage of the enhancement
- **Speedup\_e**, The improvement gained by the enhanced execution mode for Fraction=e,

(Execution time for entire task using the enhancement when possible) =  
 (Execution time for **fraction of task** without using the enhancement) +  
 (Execution time for **fraction of task** using the enhancement) =

$$(1 - \text{Fraction\_e}) * T_{\text{old}} + \text{Fraction\_e} * \frac{T_{\text{old}}}{\text{Speedup\_e}}$$

$$\text{Speedup overall} = \frac{1}{(1 - \text{Fraction\_e}) + \frac{\text{Fraction\_e}}{\text{Speedup\_e}}}$$

en

| Solution | Enhancement     | Computation | Waiting for I/O | Overall time |
|----------|-----------------|-------------|-----------------|--------------|
| Original | ...             | 7 s         | 3 s             | 10 s         |
| Disks    | I/O Speedup = 3 | 7 s         | 1 s             | 8 s          |
| CPU 1    | CPU speedup = 2 | 3,5 s       | 3 s             | 6,5 s        |
| CPU 2    | CPU speedup = 3 | 2,33 s      | 3 s             | 5,33 s       |

Disks Fraction\_e=3/10, Speedup\_e=3, Speedup overall=10/8=1,25

CPU 1 Fraction\_e=7/10, Speedup\_e=2, Speedup overall=20/13=10/6,5=1,54

CPU 2 Fraction\_e=7/10, Speedup\_e=3, Speedup overall=30/16=10/6,5=1,88

Disk 100 Fraction\_e=3/10, Speedup\_e=100, Speedup overall=10/8=1,42

### **Sfruttare le richieste di servizio asincrone**

Progettazione per aumentare la percentuale di servizio asincrono richieste. Quindi coprire il ritardo indotto dalle operazioni sincrone con altre attività. In parole povere: fare in modo che lo “spin time”, ossia il tempo in cui la CPU non fa nulla sia ridotto al minimo.

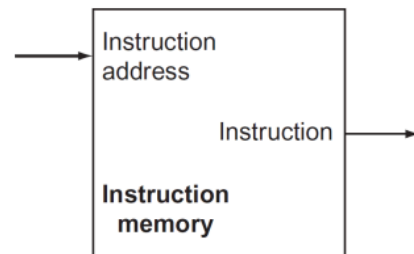
# Processore

## Componenti

### Instruction Memory

L'Instruction memory richiede solo di fornire l'accesso in lettura perché il percorso dei dati non scrive istruzioni.

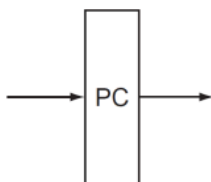
L'output in qualsiasi momento riflette il contenuto della posizione specificata dall'indirizzo di input e non è necessario alcun segnale di controllo di lettura.



a. Instruction memory

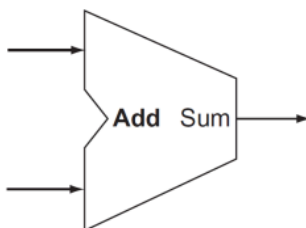
### Program Counter

Il program counter è un registro scritto alla fine di ogni ciclo di clock e quindi non necessita di un segnale di controllo di scrittura.



b. Program counter

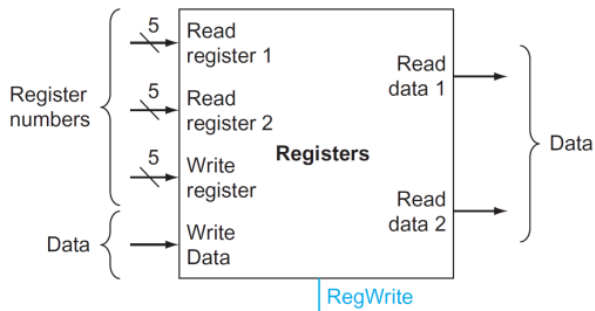
### ALU



c. Adder

### Register File

Il file di registro multiporta contiene tutti i registri e dispone di due porte di lettura e una porta di scrittura.



## Lettura

Il file di registro emette sulle uscite il contenuto dei registri corrispondenti agli ingressi del registro di lettura.

## Scrittura

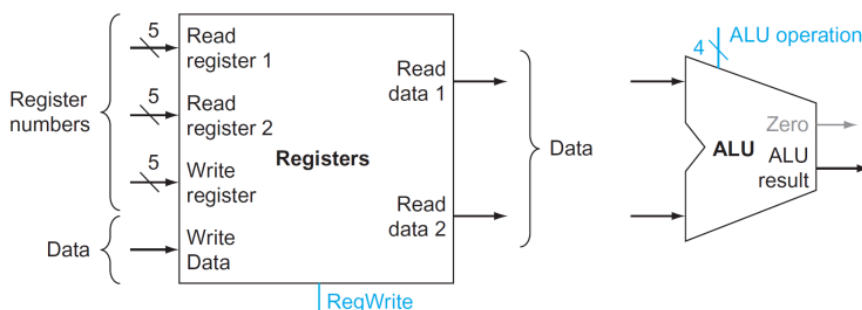
Una scrittura nel registro deve essere esplicitamente indicata affermando il segnale di controllo di scrittura `Write Data`.

Le scritture vengono attivate dal fronte, pertanto tutti gli ingressi di scrittura devono essere validi sul fronte del clock; il nostro progetto può leggere e scrivere lo stesso registro all'interno di un ciclo di clock: la lettura otterrà il valore scritto in un ciclo di clock precedente, mentre il valore scritto sarà disponibile per la lettura in un ciclo di clock successivo.

Gli ingressi che portano il numero di registro nel file di registro sono tutti larghi 5 bit, mentre le linee che portano i valori dei dati hanno una larghezza di 64 bit.

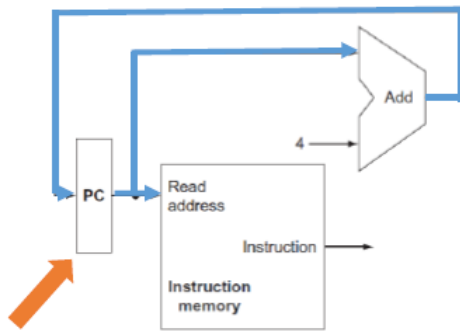
L'operazione che deve essere eseguita dalla ALU è: controllata con il segnale operativo ALU, che sarà largo 4 bit.

Utilizzeremo l'output di rilevamento dello zero dell'ALU per implementare i rami condizionali.



## Fetch dell'istruzione ed incremento del PC

Il program counter va incrementato ad ogni nuova istruzione.



## Metodologia del segnale del fronte del clock

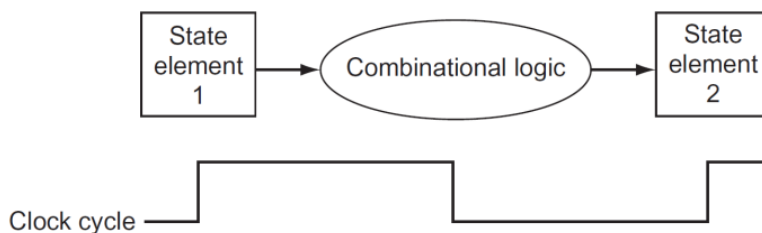
Definisce quando i segnali possono essere letti e quando possono essere scritti; importante per specificare i tempi di lettura e scrittura.

### Clock positive/negative edge-triggered

Significa che qualsiasi valore viene memorizzato in un elemento di una logica sequenziale viene aggiornato solo sul fronte del clock, che è una transizione rapida da basso ad alto o viceversa.

Gli ingressi sono valori che sono stati scritti in un ciclo di clock precedente, mentre le uscite sono valori che possono essere utilizzati in un ciclo di clock successivo.

### Esempio



Due elementi di stato che circondano un blocco di logica combinatoria, che opera in un unico ciclo di clock:

Tutti i segnali devono propagarsi dall'elemento di stato 1, attraverso la logica combinatoria e per dichiarare l'elemento 2 nel tempo di un ciclo di clock.

### Frequenza di clock

Definisce il tempo necessario affinché i segnali raggiungano l'elemento di stato 2, definisce la durata del ciclo di clock.

# Pipeline

Il pipelining è una tecnica che permette di “eseguire” più istruzioni contemporaneamente.

Una caratteristica della pipeline è la trasparenza: non richiede uno sforzo al programmatore per utilizzarla.

Un'istruzione, durante la sua esecuzione, percorre una linea di montaggio nel quale, ad ogni stage, vengono eseguite delle micro-operazioni (*ovvero delle operazioni compiute dal hardware*) atte ad eseguire:

- 1) Il prelievo dell'istruzione (*instruction fetch*).
- 2) La decodifica (*instruction decode*).
- 3) L'esecuzione (*instruction execution*).
- 4) Un eventuale accesso alla memoria (*memory access*).
- 5) Un eventuale scrittura su registro (*write-back*).

In ogni stage della pipeline, il processore utilizza delle unità funzionali (*circuiti*) diverse; ciò rende ogni stage indipendente dallo stage precedente e dallo stage successivo.

L'indipendenza tra gli stage è una proprietà che riguarda solo le unità funzionali del processore. In realtà gli stage non sono indipendenti tra loro: I risultati prodotti dallo stage  $i$  sono necessari per le operazioni condotte dallo stage  $i + 1$ .

Quindi presa in considerazione una singola istruzione, ad ogni ciclo di clock può essere attivato solo uno stage della linea di montaggio.

Ad ogni ciclo di clock, solo un sottoinsieme delle unità funzionali del processore è attivo poiché mentre eseguo una fase i circuiti delle altre fasi sono inutilizzati. Non stiamo sfruttando a pieno le risorse del processore: abbiamo un CPI di 5 (*5 cicli di clock per completare un'istruzione*).

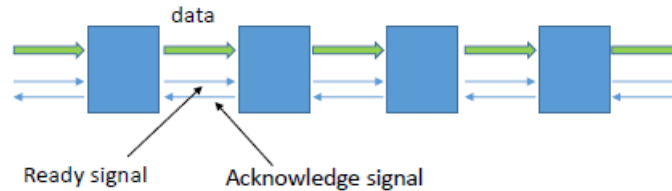
Sovrapponiamo l'esecuzione di più istruzioni visto che ad ogni ciclo di clock un'istruzione occupa un solo stage della linea di montaggio!

Il pipelining permette di ottenere, in teoria, un CPI di 1.



## Pipeline con metodo asincrono

Come facciamo passare le informazioni da uno stage all'altro? Seguiremo il paradigma produttore-consumatore per far passare l'informazione alla successiva.



- `ready signal` Informa lo stage successivo che lo stage corrente è pronto a passare i dati.
- `acknowledge signal` Informa lo stage precedente che lo stage corrente ha accettato i dati.

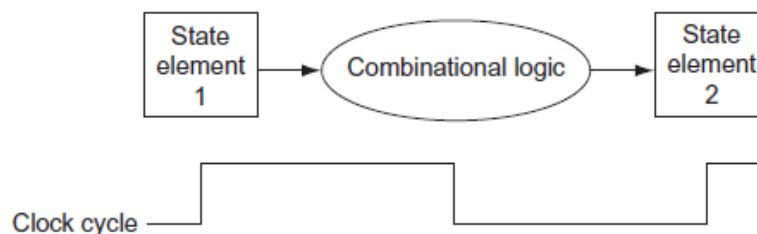
Questo metodo fornisce la massima flessibilità nei tempi operativi di stadio e dovrebbe far funzionare il pipeline al massimo della sua velocità; la quale è limitata dallo stage più lento.

Ovviamente si deve inserire della logica aggiuntiva per gestire i segnali di `ready` e di `acknowledge`.

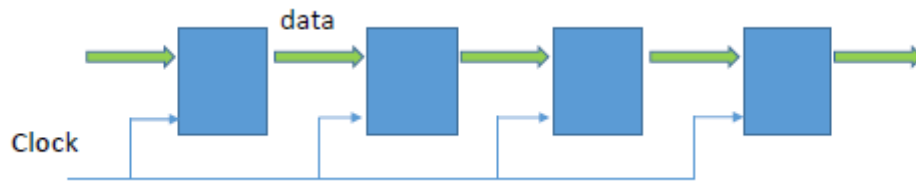
## Pipeline con metodo sincrono

La maggior parte delle pipeline di istruzioni e di aritmetica utilizzano il metodo sincrono per ridurre la complessità logica e i problemi di implementazione.

La pipeline lavora grazie ad un segnale di clock (*1 filo che arriva a tutti i componenti*) che determina quando gli elementi con stato (*registri*) scriveranno i nuovi valori nella propria memoria interna.



Qualsiasi ingresso a un elemento di stato deve raggiungere un valore stabile prima che il fronte attivo del clock provochi l'aggiornamento dello stato.



Il clock fa in modo che tutti gli stage mostrino in entrata agli stage successivi i prossimi valori. Il segnale di clock è periodico con un periodo deciso tenendo conto dello stage più lento.

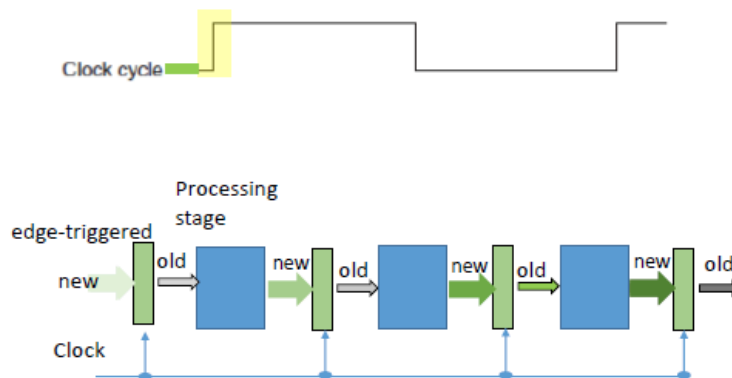
Una metodologia a fronte di salita ci consente di fare queste cose nello stesso ciclo di clock:

- Leggere i contenuti in entrata e scrivere nel registro.
- Mostrare il valore in uscita attraverso qualche logica combinatoria.

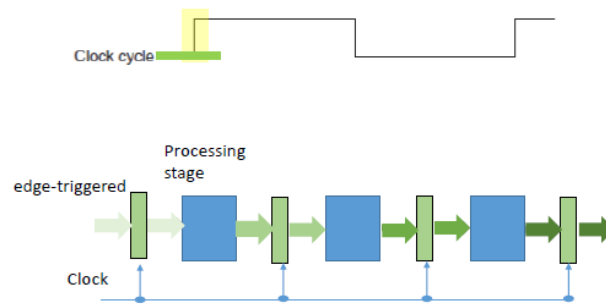
### Stabilizzare i dati

All'inizio, prima del fronte di salita del clock i dati sono in ingresso ai registri, stabili e pronti per essere memorizzati.

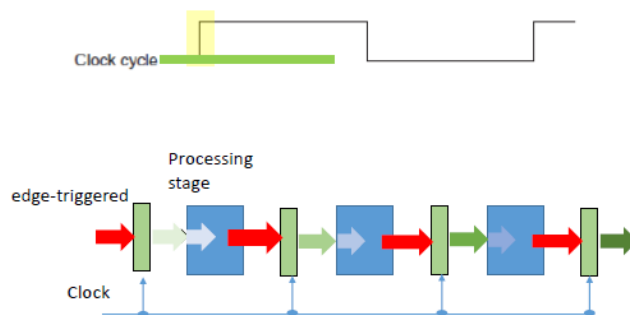
Per stabilizzare si intende l'attesa che si carichino le capacità parassite e che siano alimentati tutti i circuiti d'ingresso.



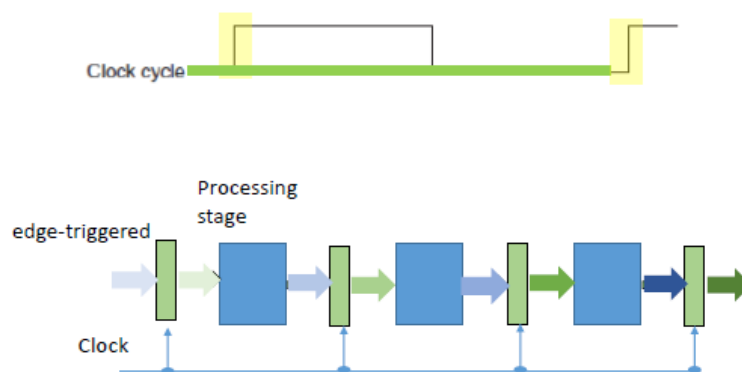
Al fronte di salita del clock i dati vengono memorizzati dai registri inter-stage.



Dopo un tempo di propagazione dovuto alla logica dei registri, i dati vengono presentati stabili in uscita.



Al fronte di clock i dati nuovi vengono memorizzati dai registri inter-stage.

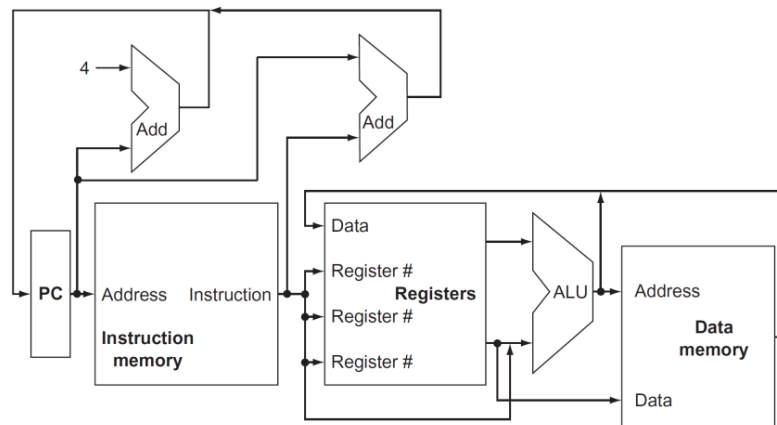


## Hardware del processore e della pipeline

Il circuito base che abbiamo immaginato è composto da:

- Una memoria per le istruzioni.
- Una memoria per i dati.
- Un file di registri.

- Una ALU.



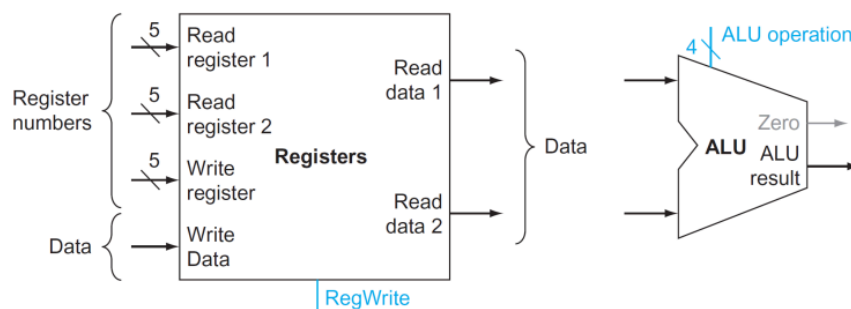
## Program counter

Ci serve anche un circuito per modificare il program counter: necessario per individuare le nuove istruzioni e farle uscire dalla instruction memory.

Un'istruzione può essere proposta dal program counter solo se è terminata quella precedente.

## Multi-port register file

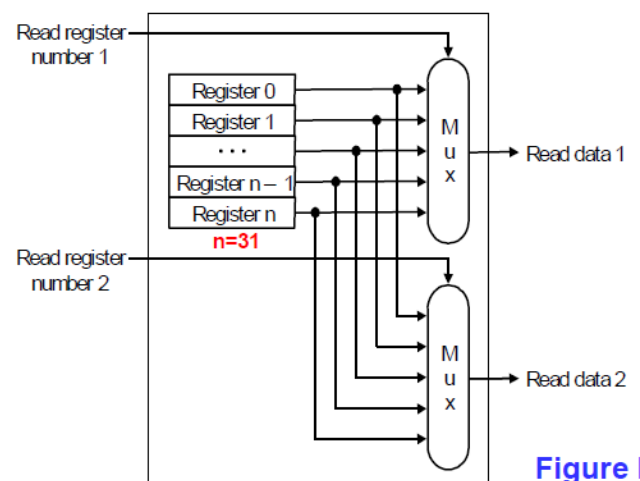
Contiene tutti i registri e dispone di due porte di lettura e una porta di scrittura.



## Letture nel register file

Restituisce sempre sulle uscite il contenuto dei registri corrispondenti agli ingressi del registro di lettura.

Nella figura sotto è rappresentato lo schema della parte di lettura: Ogni registro ha l'uscita indipendente, con i due multiplexer viene selezionato il registro da cui si vuole leggere.

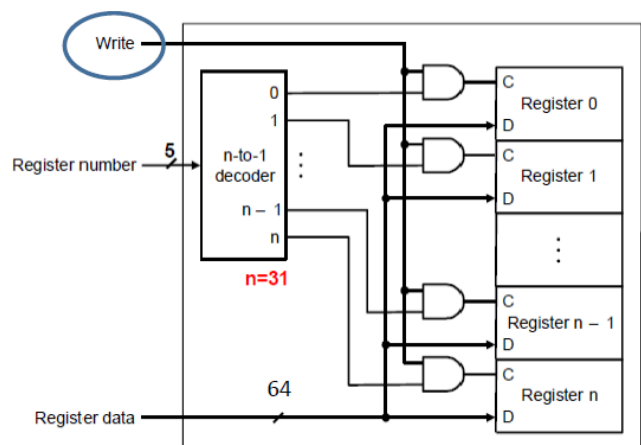


**Figure B.**

## Scritture nel register file

Una scrittura in un registro deve essere esplicitamente indicata mettendo ad 1 il segnale di controllo della scrittura *write*, il quale, come si vede nell'immagine sotto, è inviato a tutti i registri.

Le scritture vengono effettivamente attivate dal fronte di salita del clock, quindi tutti gli input devono essere stabili sul fronte di salita del clock.



Si può leggere e scrivere nello stesso registro all'interno di un ciclo di clock:

- La lettura al ciclo di clock corrente darà il valore scritto nel ciclo di clock precedente.
- Il valore scritto al ciclo di clock corrente sarà disponibile per la lettura nel ciclo di clock successivo.

## La scalabilità - il problema del register file

Il problema di questi due circuiti è la quantità di hardware necessaria, che aumenta esponenzialmente con l'aumento del numero di registri.

Quindi aumentando il numero dei registri nel il register file si riscontra:

- **Maggiore complessità:** Quindi maggior costo di progettazione.
- **Maggiore overhead:** i segnali devono percorrere distanze più lunghe e possono verificarsi molti ritardi di propagazione.
- **Minore frequenza massima di clock:** Poiché il percorso critico è più lungo.
- **Maggiore probabilità di guasto:** Poiché ho più componenti hardware.

## ALU - Arithmetic Logic Unit

Insieme di circuiti che permettono di eseguire un determinato set di operazioni.

L'operazione desiderata si seleziona tramite un segnale di controllo detto *ALUOp* (il quale andrà in entrata ad un multiplexer interno alla ALU).

| ALU control lines | Function     |
|-------------------|--------------|
| 0000              | AND          |
| 0001              | OR           |
| 0010              | add          |
| 0110              | subtract     |
| 0111              | pass input b |
| 1100              | NOR          |

Queste linee di controllo derivano dallo `OPCODE` dell'istruzione, poiché ogni istruzione può avere bisogno in qualche modo della ALU:

### ALUOp

Segnale che indica, in modo generico, il tipo di operazione da far eseguire alla ALU (*somma, confronto o indirizzi*).

| Instruction | ALUOp | Instruction operation      | Opcode field | Desired ALU action | ALU control input |
|-------------|-------|----------------------------|--------------|--------------------|-------------------|
| LDUR        | 00    | load register              | XXXXXXXXXX   | add                | 0010              |
| STUR        | 00    | store register             | XXXXXXXXXX   | add                | 0010              |
| CBZ         | 01    | compare and branch on zero | XXXXXXXXXX   | pass input b       | 0111              |
| R-type      | 10    | ADD                        | 10001011000  | add                | 0010              |
| R-type      | 10    | SUB                        | 11001011000  | subtract           | 0110              |
| R-type      | 10    | AND                        | 10001010000  | AND                | 0000              |
| R-type      | 10    | ORR                        | 10101010000  | OR                 | 0001              |

Sviluppando il circuito possiamo immaginare che una possibile soluzione operativa possa essere la seguente.

| ALUOp  |        | Opcode field |       |       |       |       |       |       |       |       |       |       | Operation |
|--------|--------|--------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----------|
| ALUOp1 | ALUOp0 | I[31]        | I[30] | I[29] | I[28] | I[27] | I[26] | I[25] | I[24] | I[23] | I[22] | I[21] |           |
| 0      | 0      | X            | X     | X     | X     | X     | X     | X     | X     | X     | X     | X     | 0010      |
| X      | 1      | X            | X     | X     | X     | X     | X     | X     | X     | X     | X     | X     | 0111      |
| 1      | X      | 1            | 0     | 0     | 0     | 1     | 0     | 1     | 1     | 0     | 0     | 0     | 0010      |
| 1      | X      | 1            | 1     | 0     | 0     | 1     | 0     | 1     | 1     | 0     | 0     | 0     | 0110      |
| 1      | X      | 1            | 0     | 0     | 0     | 1     | 0     | 1     | 0     | 0     | 0     | 0     | 0000      |
| 1      | X      | 1            | 0     | 1     | 0     | 1     | 0     | 1     | 0     | 0     | 0     | 0     | 0001      |

## Formato delle istruzioni

L'obiettivo è quello di ottimizzare (*usare il numero minimo di fili*) le istruzioni, il quale deve essere tale da mantenere le operazioni principali.

### Formato ADD, SUB, AND e OR

Hanno tre registri operandi: `Rn`, `Rm` e `Rd`:

- `Rn` e `Rm` sono sorgenti.
- `Rd` è la destinazione.

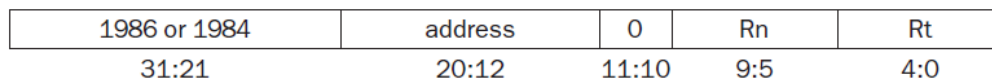
| opcode | Rm    | shamt | Rn  | Rd  |
|--------|-------|-------|-----|-----|
| 31:21  | 20:16 | 15:10 | 9:5 | 4:0 |

## Formato LOAD e STORE

Un'operazione di `load` e `store` contiene la codifica che dobbiamo dare alla ALU (*terzo valore*).

Il registro  $R_n$  è il registro di base che viene aggiunto al campo di indirizzo di 9 bit (*offset*) per formare l'indirizzo di memoria.

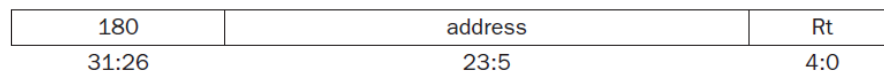
- Load:  $R_t$  è il registro di destinazione per il valore caricato.
- Store:  $R_t$  è il registro sorgente, il cui valore deve essere memorizzato in memoria.



## Istruzione di salto CBZ Addr, $R_t$

Formato dell'istruzione per leggere il registro  $R_t$  e saltare all'indirizzo specificato se è zero.

Il campo di indirizzo di 19 bit viene esteso per segno, shiftato e aggiunto al PC per calcolare l'indirizzo di destinazione del salto.

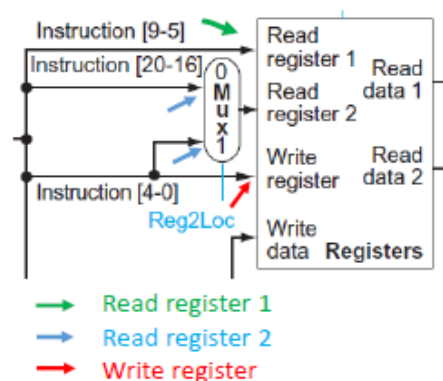


## Decodificare lo OPCODE

Il campo `OPCODE` in questo caso ha una larghezza compresa tra 6 e 11 bit e si trova nei bit 31:26 a 31:21.

Bisogna evitare di inviare gli stessi bit in ingresso a più registri, per questo useremo il multiplexer, che invierà i bit dell'istruzione solo ai circuiti che devono riceverli.

Il **primo registro operando**  $R_n$  è nelle posizioni dei bit 9:5 per le istruzioni di tipo aritmetico che per le istruzioni di `load` e `store`.



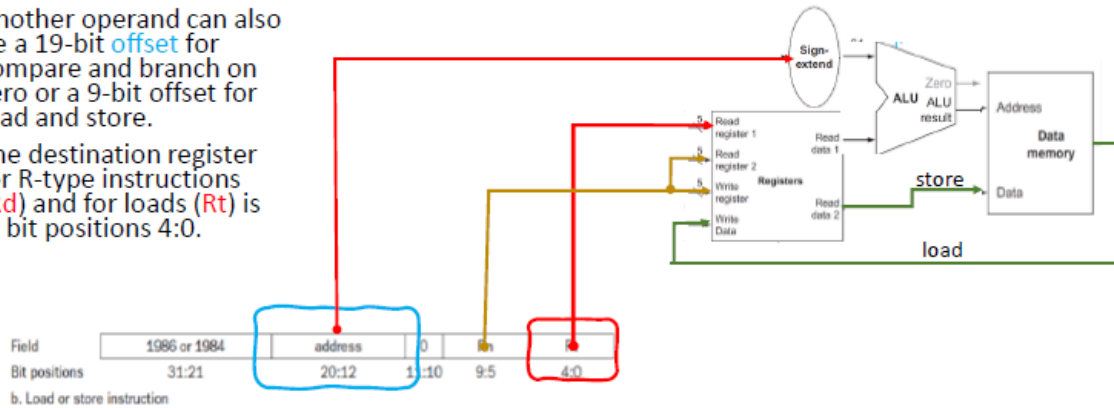
L'altro **registro operando** è nelle posizioni dei bit 20:16 ( $R_m$ ) per le istruzioni di tipo aritmetico; e nelle posizioni dei bit 4:0 ( $R_t$ ) per il registro da scrivere in un caricamento.

Un altro operando può anche essere un **offset** di 19 bit per confrontare e saltare se zero o un offset di 9 bit per caricare e memorizzare.

Il registro di destinazione per le istruzioni di tipo R (**Rd**) e per i caricamenti (**Rt**) si trova nelle posizioni dei bit 4:0.

Another operand can also be a 19-bit **offset** for compare and branch on zero or a 9-bit offset for load and store.

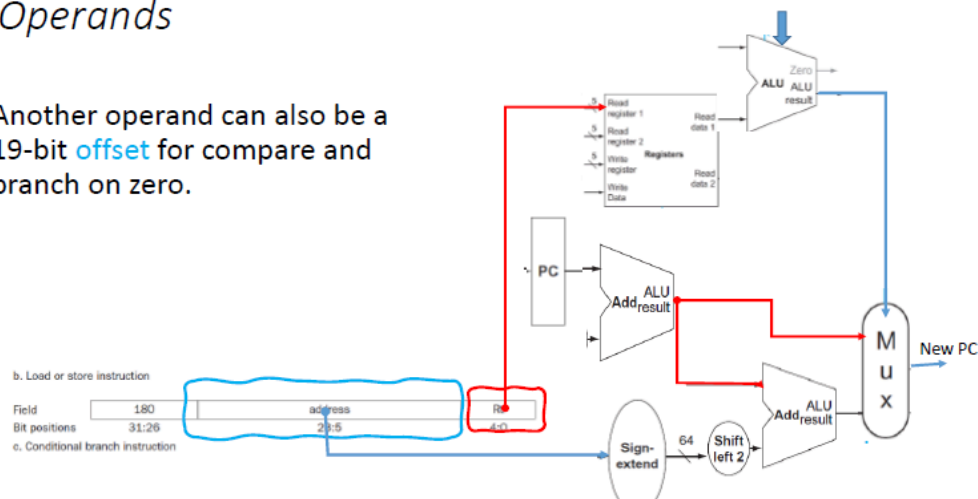
The destination register for R-type instructions (**Rd**) and for loads (**Rt**) is in bit positions 4:0.



Un altro operando può anche essere un offset di 19 bit per confrontare e saltare se zero.

## Operands

Another operand can also be a 19-bit **offset** for compare and branch on zero.



Divisione in stage delle istruzioni - ADD X1, X2, X3

Possiamo pensare a 4 passaggi necessari per eseguire l'istruzione ADD.

1. L'istruzione viene fetchata e il program counter viene incrementato.
2. I valori dei registri X2 e X3, vengono letti dal register file.
  - Inoltre, l'unità di controllo principale calcola l'impostazione delle linee di controllo durante questa fase.
3. Esegue la ADD sui dati letti dal file dei registri.
  - La ADD viene selezionata utilizzando lo ALUOp, ottenuto con una porzione dello OPCODE.

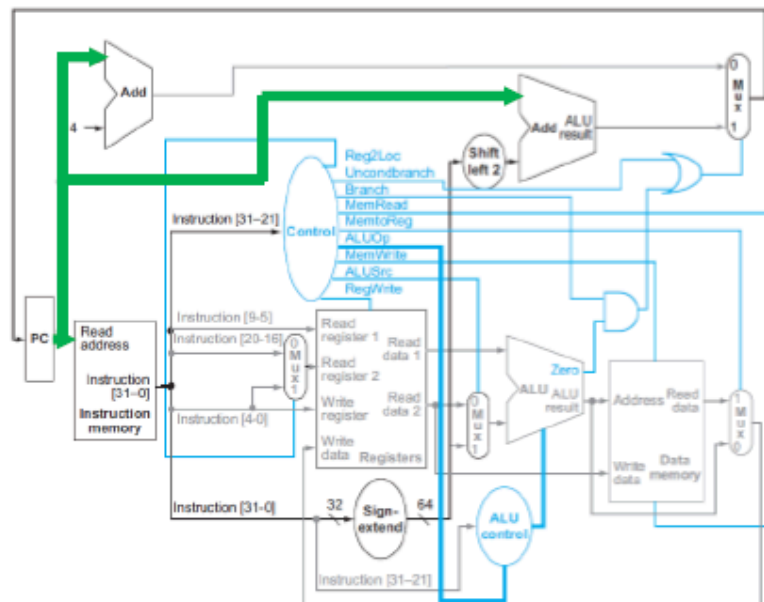


4. Il risultato viene scritto nel registro x1 nel register file.

Dato che tutto avviene in un solo ciclo di clock, allora prima del prossimo fronte di salita, è necessario completare tutte le 4 fasi. Questo assicura che l'output dell'istruzione sia stabile e pronto per essere utilizzato nel ciclo successivo.

### 1° fase - Appena dopo il fronte di salita del clock

1. Il PC mette in output dai sommatori l'indirizzo della istruzione corrente.
  - Il valore del PC è stato generato al ciclo precedente.
2. Il dato si stabilizza e va in ingresso a:
  - **Instruction memory:** Fa uscire lo OPCODE.
  - **Ai due sommatori:** Per calcolare l'indirizzo della prossima istruzione (*il prossimo valore del PC*).

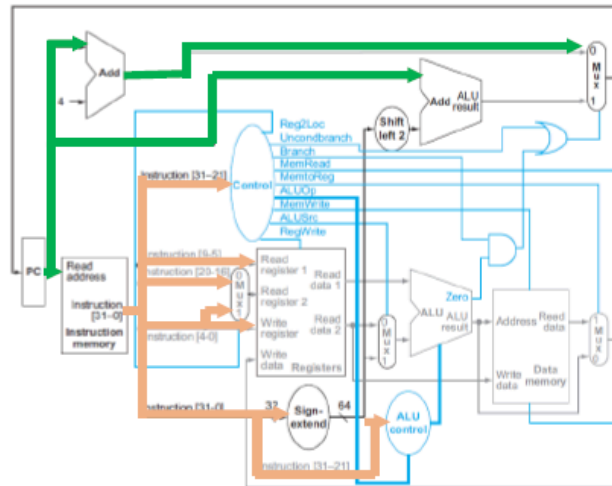


### 2° fase - Primo semiperiodo del clock (vale 1)

Il segnale relativo all'istruzione si stabilizza in vari componenti dell'architettura: quindi tutti i componenti di cui ha bisogno l'istruzione hanno a disposizione questa istruzione (*linee arancioni*).

In particolare l'istruzione ricevuta è inviata al:

- Register file.
- Unità di controllo.

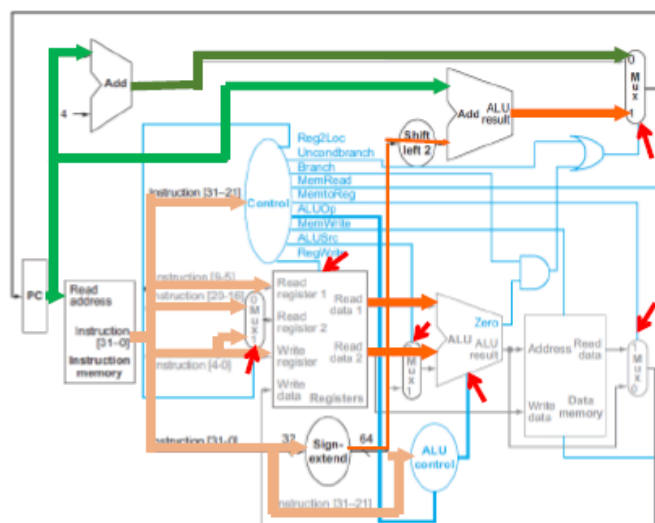


### 3° fase - Secondo semiperiodo del clock (*vale o*)

Si ha la lettura dei registri e la decodifica dell'istruzione.

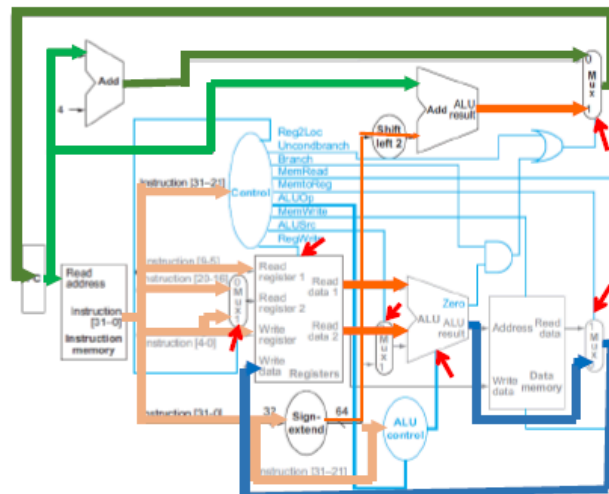
La ALU è pronta a fare il suo lavoro, quindi viene abilitata per fare la **ADD**.

Per effettuare la somma opera sui dati letti dal file dei registri, selezionati utilizzando porzioni dello `OPCODE`.



#### 4° fase - Ci stiamo avvicinando al prossimo fronte di salita del clock

Il risultato della ALU è pronto e viene portato al multiplexer (*linee blu*).



## Registri intermedi

Questa soluzione di pipelining è troppo lenta, poiché per fare tutto in un solo ciclo di clock, la frequenza deve essere bassissima.

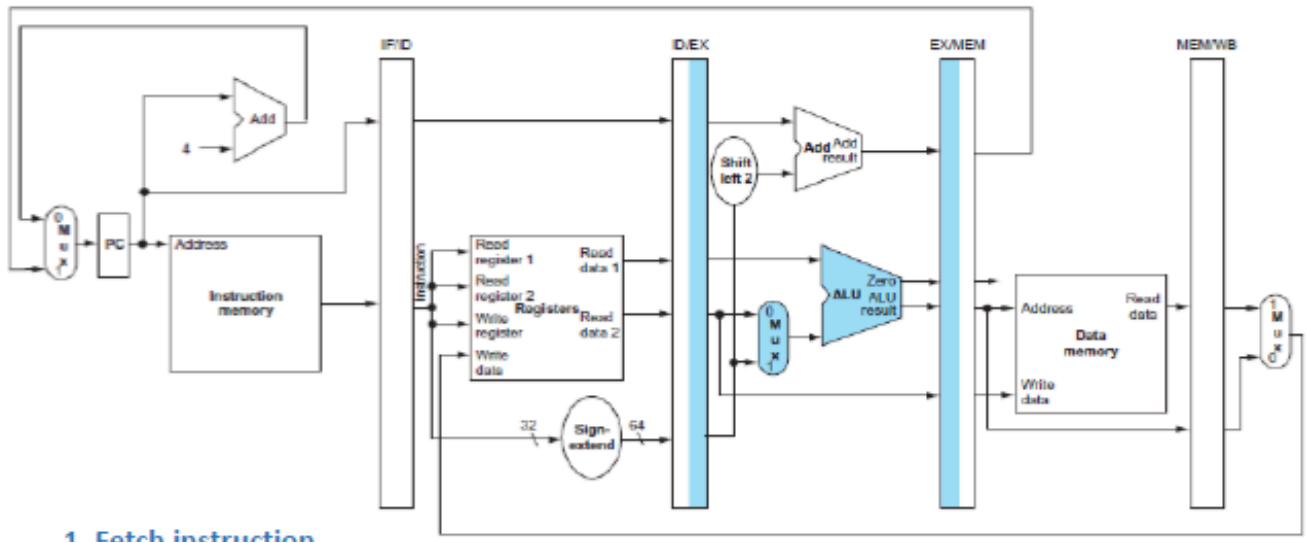
Per definire il clock che deve essere fornito al registro PC, bisogna prendere il più alto tra i valori, dato che bisogna garantire che ogni operazione venga completata.

Andiamo quindi a separare gli step con dei registri, in modo tale da procedere all'elaborazione successiva dell'istruzione senza impegnare la precedente.

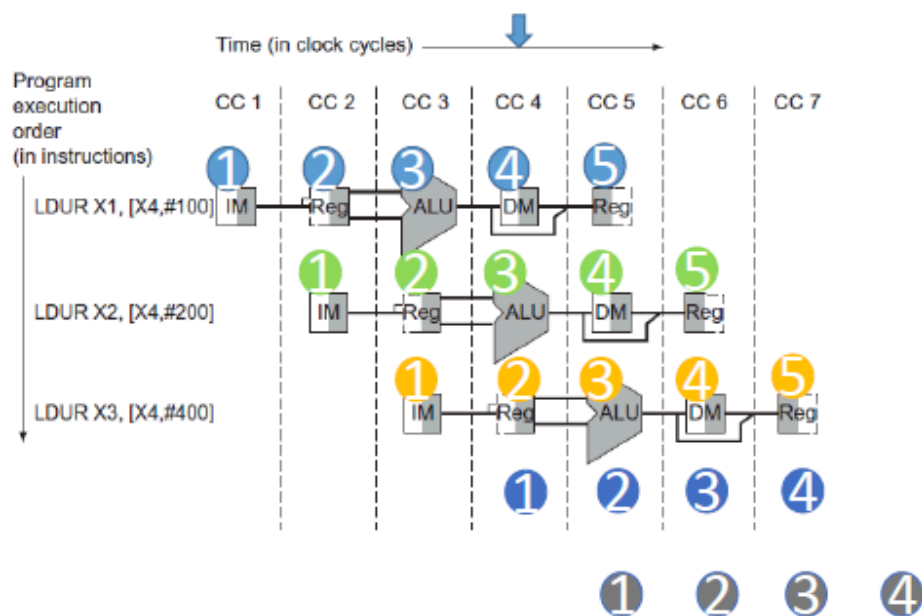
Tutti i registri oltre il program counter hanno lo stesso clock.

Durante il ciclo di clock, il registro mantiene i dati necessari per l'istruzione successiva nella prima zona del registro, mentre nella seconda zona memorizza il risultato dell'elaborazione dell'istruzione corrente. Quando il fronte del clock arriva, il risultato memorizzato viene reso disponibile come output del registro per l'istruzione successiva.

Questo approccio consente di massimizzare l'utilizzo delle risorse e l'efficienza del processore, eseguendo più istruzioni contemporaneamente.



Il circuito combinatorio totale è stato spezzettato, traendo il vantaggio che ogni pezzo è più veloce dell'intero e quindi la frequenza di clock può essere più alta.



Il problema di questa soluzione è che è lenta, per fare qualcosa ci metto 5 cicli di clock; devo creare una strategia per utilizzare anche le unità non utilizzate.

## Pipeline hazards

La pipeline introduce dei problemi:

- Overhead introdotto dalla logica che gestisce la pipeline.
- Stalli dovuti ad hazards.

## Hazard strutturale

Gli hazard strutturali sono causati dai conflitti che possono accadere quando due istruzioni necessitano, durante lo stesso ciclo di clock, di accedere alla stessa unità funzionale per proseguire l'esecuzione.

Supponiamo di avere una unica memoria sia per le istruzioni che per i dati.

Analizziamo, per ogni ciclo di clock, quali unità funzionali vengono accedute da ogni istruzione:

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ | $C_7$ | $C_8$ |
|----------------|-------|-------|-------|-------|-------|-------|-------|-------|
| LD R2, \$0[R1] | Mem   | Reg   | ALU   | Mem   | Reg   |       |       |       |
| ADD R5, R4, R3 |       | Mem   | Reg   | ALU   | Mem   | Reg   |       |       |
| SUB R8, R7, R6 |       |       | Mem   | Reg   | ALU   | Mem   | Reg   |       |
| ST R3, \$4[R1] |       |       |       | Mem   | Reg   | ALU   | Mem   | Reg   |

Al ciclo di clock  $C_4$ :

- LD vuole accedere alla memoria per recuperare il contenuto della cella di memoria puntato da  $\$0[R1]$ .
- ST vuole accedere alla memoria per recuperare l'istruzione e quindi eseguire la fetch.

Entrambi le istruzioni vogliono accedere alla memoria nello stesso ciclo di clock.

Per poter proseguire è necessario stallare (*fermare la pipeline per qualche ciclo di clock*) la pipeline fino a quando l'istruzione LD R2, \$0[R1] giunge a termine.

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$            | $C_6$ | $C_7$ | $C_8$ | $C_9$ |
|----------------|-------|-------|-------|-------|------------------|-------|-------|-------|-------|
| LD R2, \$0[R1] | Mem   | Reg   | ALU   | Mem   | Reg              |       |       |       |       |
| ADD R5, R4, R3 |       | Mem   | Reg   | ALU   | Mem <sup>3</sup> | Reg   |       |       |       |
| SUB R8, R7, R6 |       |       | Mem   | Reg   | ALU              | Mem   | Reg   |       |       |
| ST R3, \$4[R1] |       |       |       | Stall | Mem              | Reg   | ALU   | Mem   | Reg   |

Questo caso specifico si risolve dividendo la memoria tra istruzioni e dati. In questo modo la fase di fetch e quella di memorizzazione agirebbero in due memorie diverse.

## Control hazards

BZ R1, taken

SUB R2, R3, R4

B cont

taken:

ADD R2, R3, R4

cont:

...

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ |
|----------------|-------|-------|-------|-------|-------|-------|
| BZ R1, taken   | IF    | ID    | EX    | MEM   | WB    |       |
| SUB R2, R3, R4 |       | IF    | ID    | EX    | MEM   | WB    |

Se la condizione è falsa: Tutto procede normalmente.

Se la condizione è vera: L'esecuzione parallela non può continuare.

Il risultato della BZ si saprà al termine della fase di decodifica della SUB.

Nel mentre la pipeline ha già eseguito la fetch seguendo il program counter, e quindi ha fetchato la SUB.

Ma nel caso si dovesse saltare, l'istruzione che la pipeline dovrebbe aver decodificato dovrebbe essere la ADD.

Per garantire la corretta esecuzione del programma, la pipeline deve:

1. Stallare.
2. Svuotarsi.
3. Ricominciare dalla ADD.

## Branch prediction

Si cerca di prevedere il risultato dell'istruzione di salto in modo da non far stallare la pipeline.

Un approccio popolare consiste nel mantenere la cronologia per ciascun ramo condizionale e quindi utilizzare il comportamento del passato recente per prevedere il futuro.

Quando l'ipotesi è sbagliata:

- Il controllore della pipeline deve garantire che le istruzioni che seguono il ramo condizionale previsto erroneamente non abbiano nessun effetto
- Occorre riavviare la pipeline dall'indirizzo di ramo corretto.

## Data Hazards

Gli hazard sui dati sono causati da istruzioni che presentano dipendenze sugli operandi.

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ |
|----------------|-------|-------|-------|-------|-------|-------|
| ADD R1, R2, R3 | IM    | Reg   | ALU   | DM    | Reg   |       |
| SUB R5, R4, R1 |       | IM    | Reg   | ALU   | DM    | Reg   |

L'istruzione `ADD` è disponibile sul registro `R1` al ciclo di clock  $C_5$ .

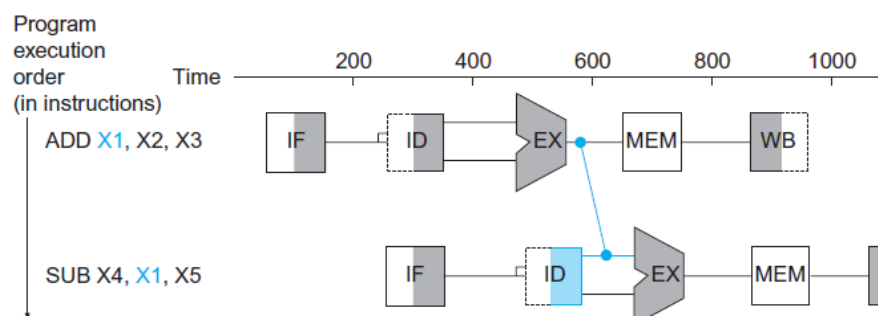
L'istruzione `SUB` preleva il contenuto del registro `R1` al ciclo di clock  $C_3$ .

Ma il risultato prodotto dalla `ADD` non è ancora pronto nel momento in cui la `SUB` lo necessita. Quindi possiamo riprendere al ciclo  $C_4$  e stallare per 2 cicli di clock.

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ | $C_7$ | $C_8$ |
|----------------|-------|-------|-------|-------|-------|-------|-------|-------|
| ADD R1, R2, R3 | IM    | Reg   | ALU   | DM    | Reg   |       |       |       |
| SUB R5, R4, R1 |       | Stall | Stall | IM    | Reg   | ALU   | DM    | Reg   |

## Forwarding

Aggiunta di hardware extra per recuperare anticipatamente l'elemento mancante dalle risorse interne. Non appena la ALU esegue la somma per la `ADD`, possiamo fornirla come input per la `SUB`.



Si aggiunge un collegamento diretto tra:

- La fine della fase di esecuzione della `ADD`.
- L'inizio della fase di esecuzione della `SUB`.

La `SUB` avrà i dati necessari appena prima di iniziare la sua fase di esecuzione.

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ | $C_7$ | $C_8$ |
|----------------|-------|-------|-------|-------|-------|-------|-------|-------|
| ADD R1, R2, R3 | IM    | Reg   | ALU   | DM    | Reg   |       |       |       |
| SUB R5, R2, R1 |       | IM    | Reg   | ALU   | DM    | Reg   |       |       |
| AND R5, R3, R1 |       |       | IM    | Reg   | ALU   | DM    | Reg   |       |
| OR R7, R6, R1  |       |       |       | IM    | Reg   | ALU   | DM    | Reg   |

## Load-use data hazard

Si verifica quando un'istruzione di caricamento (`load`) viene eseguita e il risultato di questo caricamento viene utilizzato immediatamente da un'istruzione successiva nella pipeline.

Una lettura dalla cache o dalla RAM richiede sempre più di un ciclo di clock.

Per affrontare questo problema, potrebbe essere necessario stallare per consentire al caricamento di completarsi.

### Spostare un'istruzione indietro

C'è anche un'altra opportunità che consente di evitare stalli, ovvero spostare un'istruzione all'indietro.

Ovvero eseguiamo l'istruzione di caricamento molto prima di quando ci serve in modo da avvantaggiarsi e non stallare nel momento del bisogno.

Un'istruzione però non può essere spostata dove vogliamo, soprattutto se questa istruzione modifica dei flag, poiché causerebbe un cambiamento nell'esecuzione del programma.

Detto ciò possiamo spostare solo le istruzioni che non modificano i flag.

Nell'esempio sotto le due istruzioni evidenziate (*una ADD e una STORE*) vengono posticipate di uno step.

L'istruzione di caricamento che inizialmente era quinta viene eseguita come terza.

|                   |            |                   |
|-------------------|------------|-------------------|
| LDUR X1, [X0,#0]  | // Load b  | LDUR X1, [X0,#0]  |
| LDUR X2, [X0,#8]  | // Load e  | LDUR X2, [X0,#8]  |
| ADD X3, X1,X2     | // b + e   | LDUR X4, [X0,#16] |
| STUR X3, [X0,#24] | // Store a | ADD X3, X1,X2     |
| LDUR X4, [X0,#16] | // Load f  | STUR X3, [X0,#24] |
| ADD X5, X1,X4     | // b + f   | ADD X5, X1,X4     |
| STUR X5, [X0,#32] | // Store c | STUR X5, [X0,#32] |

## Gestione delle eccezioni nella pipeline

L'azione fondamentale che il processore deve eseguire quando si verifica un'eccezione è salvare l'indirizzo dell'istruzione interrotta e poi trasferire il controllo al sistema operativo a un indirizzo specificato dalla tabella delle interruzioni. Il sistema operativo può quindi intraprendere l'azione appropriata e dopo aver eseguito qualsiasi azione richiesta a causa dell'eccezione, il sistema operativo può riprendere l'esecuzione del programma.



La pipeline tratta le eccezioni come control hazard: Dobbiamo svuotare la pipeline dalle istruzioni che seguono l'istruzione corrente e recuperare le istruzioni dal nuovo indirizzo, quello dell'handler.

Quale program counter deve essere salvato? Quello dell'ultima istruzione completata prima dell'eccezione, il quale ha prodotto l'indirizzo dell'istruzione successiva, perchè è da lì che bisogna ripartire.

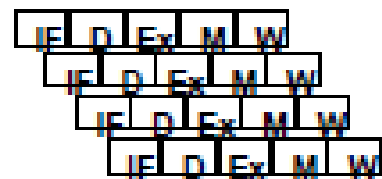
## Tipologie di Pipeline

### Super-pipeline

Andiamo a dividere in step anche le fasi più piccole.

Aumentando il numero di step (*pipeline depth*), ogni step diventa più corto.

Dato che ogni step è una rete sequenziale sincronizzata si accorcia anche il percorso critico.

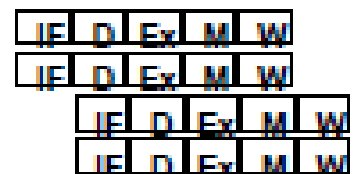


Quindi posso aumentare la frequenza di clock del processore in maniera logaritmica (*più aggiungo step e meno miglioro le prestazioni*). La frequenza di clock è limitata superiormente dai limiti fisici dati dalla circuiteria (*registri e unità funzionali*) la quale ha un tempo di propagazione piccolo ma non trascurabile.

Inoltre ci sono componenti (*come le unità di floating point*) che per fare il loro lavoro richiedono più di un ciclo di clock.

### Super-scalare

Il processore è progettato per eseguire più istruzioni indipendenti per ciclo di clock, aumentando così il throughput.

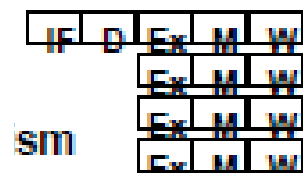


Il compilatore riconosce e riordina le istruzioni a coppie di istruzioni indipendenti in modo da massimizzare la resa di queste tecnologie.

### VLIW (Very Large Instruction Word)

Progettata per eseguire un'operazione molte volte in parallelo.

Ogni istruzione è una parola molto lunga che contiene multiple operazioni indipendenti da eseguire simultaneamente. Queste operazioni sono destinate a diverse unità funzionali del processore e sono eseguite in parallelo.



Le unità funzionali eseguono le operazioni contenute nella stessa istruzione lunga in parallelo. Questo riduce il numero di cicli di clock necessari per completare le operazioni, migliorando il throughput.

Il parallelismo nel caso VLIW è meno efficiente di un parallelismo reale, in cui tutte le istruzioni vengono eseguite effettivamente in parallelo.

Particolarmente affetta da hazard strutturali.

## Dipendenza dal compilatore

L'efficienza di una pipeline VLIW dipende fortemente dal compilatore, che deve analizzare il codice e raggruppare le operazioni (*assembla le istruzioni in blocchi di più istruzioni*) in modo ottimale nelle parole lunghe. Il compilatore è responsabile di gestire le dipendenze tra le operazioni e di organizzare le istruzioni per massimizzare il parallelismo.

I controlli sulle dipendenze vengono lasciati al compilatore che le determina in modo statico (*secondo logiche sue*) e include nelle istruzioni dei dati di controllo per il processore.

## Operazioni vettoriali

Creata per eseguire operazioni su vettori in modo molto efficiente. Aniché utilizzare la tecnica di pipeline solo con le istruzioni, la utilizzano anche sui dati stessi: si applica la medesima istruzione ad un grosso lotto di dati in parallelo, senza decodificare la stessa istruzione di memoria più volte.

Nella maggior parte dei processori a emissione statica, questo processo è almeno parzialmente gestito dal compilatore;



## Processori a doppia emissione

Il processore è in grado di prelevare, decodificare ed eseguire due istruzioni contemporaneamente in ogni ciclo di clock.

Si utilizzano due pipeline parallele, che permettono l'esecuzione simultanea delle istruzioni. Ogni pipeline può essere specializzata per certi tipi di istruzioni (*ad esempio, una per le operazioni aritmetiche e l'altra per le operazioni di memoria*).

| Instruction type          | Pipe stages |    |    |     |     |     |     |    |
|---------------------------|-------------|----|----|-----|-----|-----|-----|----|
| ALU or branch instruction | IF          | ID | EX | MEM | WB  |     |     |    |
| Load or store instruction | IF          | ID | EX | MEM | WB  |     |     |    |
| ALU or branch instruction |             | IF | ID | EX  | MEM | WB  |     |    |
| Load or store instruction |             | IF | ID | EX  | MEM | WB  |     |    |
| ALU or branch instruction |             |    | IF | ID  | EX  | MEM | WB  |    |
| Load or store instruction |             |    | IF | ID  | EX  | MEM | WB  |    |
| ALU or branch instruction |             |    |    | IF  | ID  | EX  | MEM | WB |
| Load or store instruction |             |    |    | IF  | ID  | EX  | MEM | WB |

### Lecture e Scritture di Registri

Supponiamo che in un ciclo di clock dobbiamo:

- Leggere due registri per un'operazione ALU (Unità Logica e Aritmetica).
- Leggere altri due registri per un'operazione di memorizzazione.
- Scrivere un risultato di un'operazione ALU.
- Scrivere un risultato di un'operazione di caricamento.

In totale, sono necessarie 4 operazioni di lettura e 2 operazioni di scrittura in un singolo ciclo di clock.

Serve un addizionatore separato per calcolare l'indirizzo effettivo per le operazioni di trasferimento dati (*caricamenti e memorizzazioni*).

### Prestazioni

Il processore a doppia emissione può teoricamente migliorare le prestazioni fino a un fattore di due, eseguendo due istruzioni per ciclo di clock.

Tuttavia, questo aumento di prestazioni è limitato dagli hazard:

### Hazard Strutturali

Senza le risorse aggiuntive necessarie (*come più porte di lettura/scrittura o un addizionatore separato*), il pipeline a doppia emissione può incontrare hazard

strutturali, cioè situazioni in cui le risorse hardware richieste non sono sufficienti per eseguire tutte le operazioni contemporaneamente.

### **Hazard di Dati**

Nei pipeline a cinque stadi i caricamenti hanno una latenza di utilizzo di un ciclo di clock. Ciò significa che un'istruzione che dipende dal risultato di un caricamento non può utilizzarlo immediatamente nel ciclo successivo.

Con la doppia emissione, due istruzioni successive non possono utilizzare il risultato di un caricamento nel ciclo di clock successivo, causando ulteriori rallentamenti.

### **Hazard di Controllo**

Anche le istruzioni ALU che non avevano latenza di utilizzo nel pipeline a cinque stadi ora hanno una latenza di un'istruzione, perché i risultati non possono essere utilizzati immediatamente nel ciclo successivo di caricamento o memorizzazione.

## **Compilazione e pianificazione hardware**

Per sfruttare efficacemente il parallelismo in questo tipo di processore sono necessarie:

- Il compilatore deve riordinare le istruzioni in modo da ridurre gli hazard di dati e massimizzare l'uso delle risorse disponibili.
- Il compilatore deve inserire istruzioni di riempimento (nop) o altre tecniche per gestire i ritardi dovuti agli hazard di dati e controllo.
- L'hardware deve avere tecniche di ridenominazione dei registri per ridurre gli hazard sui dati.
- L'hardware deve avere tecniche di branch prediction per minimizzare gli hazard di controllo.

## **Static two-issue datapath**

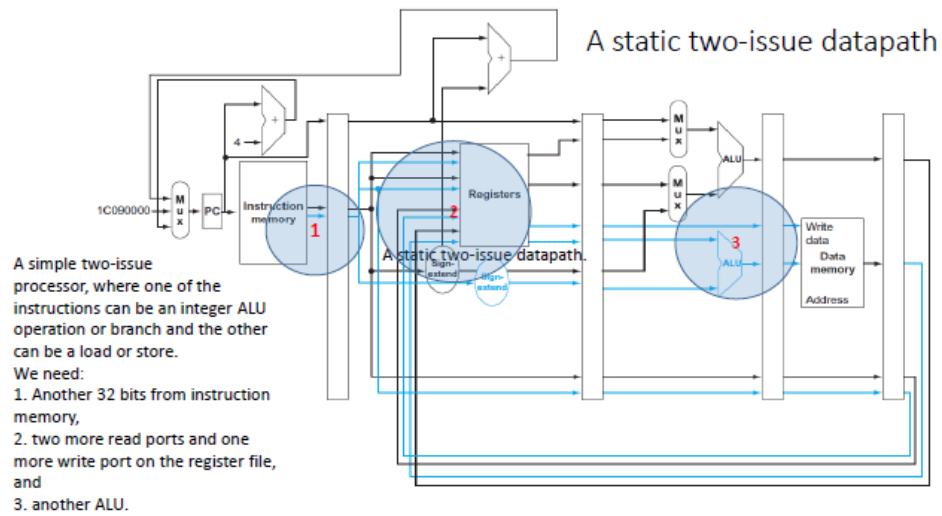
Per raddoppiare le istruzioni eseguite per unità di tempo si può raddoppiare tutto l'hardware necessario per eseguire un'istruzione.

### **Dipendenza dal compilatore**

Il termine "*static*" è dovuto al fatto che il processore utilizza un metodo di dispatch statico: il compilatore è responsabile di programmare le istruzioni in modo che possano essere eseguite in parallelo senza conflitti.

Il compilatore deve anche gestire le dipendenze tra le istruzioni per evitare stalli. Poiché le due istruzioni devono essere indipendenti per poter essere eseguite in

parallelo, il compilatore deve organizzare il codice in modo che le dipendenze siano minimizzate.

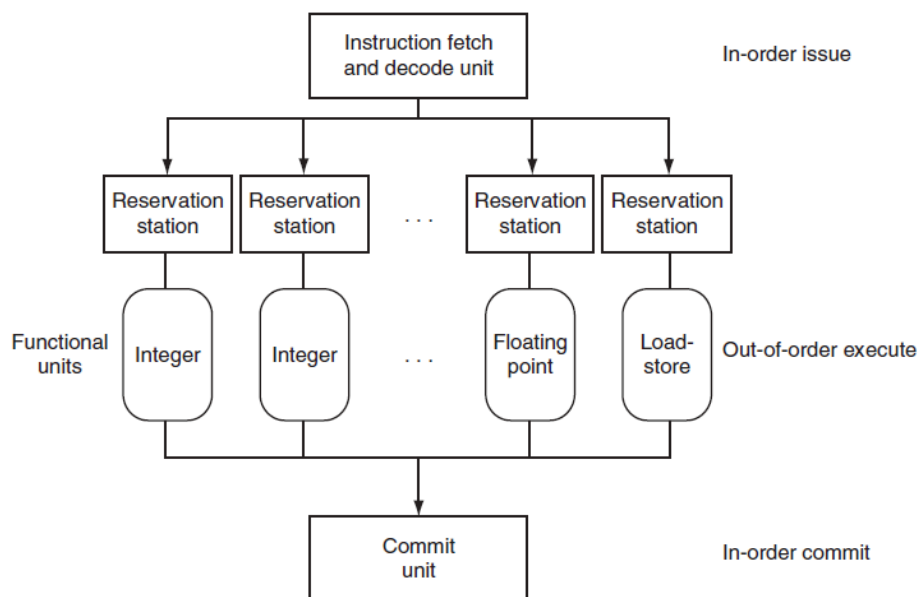


## Esecuzione out-of-order

Si tratta di una tecnica che permette di eseguire le istruzioni di un programma senza rispettare l'ordine imposto dal programmatore (*ossia il modo in cui è scritto il programma*) ma ovviamente mantenendo lo stesso risultato finale.

Il processore analizza il codice assembly e individua istruzioni indipendenti (*non sono vincolate tra loro*) e le esegue in parallelo.

Ciò permette di migliorare le prestazioni dato che permette di riempire unità funzionali che altrimenti rimarrebbero inutilizzate.



1. La prima unità preleva, decodifica e invia ciascuna istruzione a una unità funzionale per l'esecuzione.

Ogni unità funzionale ha un buffer, chiamato "*reservation station*", che contiene gli operandi e l'operazione.

2. Non appena il buffer contiene tutti gli operandi necessari, l'unità funzionale è pronta per l'esecuzione.
3. Il risultato viene calcolato dall'unità funzionale.
4. Il risultato viene inviato a eventuali reservation station in attesa del risultato appena prodotto (*usando il forwarding*) e all'unità di commit.
5. L'unità di commit memorizza il risultato fino a quando sarà possibile compiere la scrittura nel file dei registri o nella memoria in caso di *store*.

Il buffer nell'unità di commit, spesso chiamato "*reorder buffer*", viene anche utilizzato per fornire operandi, in modo molto simile a quanto fa la logica di forwarding in una pipeline statica.

6. Una volta che un risultato viene committato nel file dei registri, può essere prelevato direttamente da lì, proprio come in una pipeline normale.

La commit unit garantisce che i risultati delle istruzioni vengano consegnati in ordine e in modo coerente rispetto all'ordine dato dal programmatore. Dopo una commit è possibile trovare il risultato nel register file come accadrebbe in una normale pipeline.

## RAW - Read After Write Hazard

Caso di data hazard che può accadere in caso di esecuzione fuori ordine.

```
ADD R1, R2, R3
```

```
LD R1, [R4]
```

LD cerca di leggere il risultato prima che la ADD l'abbia prodotto. Questo tipo di hazard è il più comune e corrisponde ad una dipendenza dei dati.

L'ordine del programma deve essere preservato per garantire che LD R1, [R4] riceva l'operando corretto dalla ADD.

## WAW - Write After Write Hazard

Caso di data hazard che può accadere in caso di esecuzione fuori ordine in pipeline che:

- Effettuano scritture in più di una fase della pipeline
- Consentono a un'istruzione di procedere quando l'istruzione precedente è bloccata.

```
i: LD R1, [R5]
```

```
j: LD R1, [R4]
```

Quando un'istruzione j cerca di scrivere un operando prima che sia scritto dall'istruzione i (*i precede j*).

Le scritture vengono effettuate nell'ordine sbagliato, lasciando il valore scritto da i anziché il valore scritto da j nella destinazione.

L'ordine del programma deve essere preservato.

## WAR - Write After Read Hazard

Caso di data hazard che può accadere in caso di esecuzione fuori ordine.

```
i: ADD R1, R2, R3
```

```
j: ADD R2, R4, R5
```

Quando un'istruzione  $j$  cerca di scrivere su un registro prima che lo stesso registro venga letto dall'istruzione  $i$  ( $i$  precede  $j$ ).

Si risolve con il renaming dei registri.

## Renaming dei registri

Tecnica che va a risolvere gli hazards WAR e WAW dell'esecuzione out-of-order.

Non possiamo aumentare il numero di registri architetturali, perché non c'è spazio nella dimensione del formato dell'istruzione (*dove dobbiamo indicare l'indirizzo del registro e quindi aumentare il numero di bit che lo riguardano*).

## Registri architetturali

I registri architetturali sono quelli usati dal programmatore (EAX, EBX, ...).

## Registri fisici

Componenti fisicamente identici a quelli architetturali ma senza un nome particolare; addirittura in alcune macchine i registri architetturali non esistono nemmeno fisicamente, si hanno solo quelli fisici.

Il renaming dei registri è eseguito nella stessa fase in cui si fa l'esecuzione out of order.

## Idea di base

Si scrive nel registro più a destra.

1: **ADD R1**, R2, R3

2: **ADD R4**, R5, **R1**

La 2 non può andare avanti finché la 1 non è completata; però la 2 sovrascrive quello che ha scritto la 1 senza neanche leggere il contenuto.

Quindi alla 2 non importa veramente su quale registro deve scrivere, allora si cambia nella 2 il registro R1 a R1\*.

Si scrive nel registro più a destra.

1: **ADD R1**, R2, R3

2: **ADD R4**, R5, **R1\***

Effettuando il renaming, entrambe le istruzioni possono essere effettuate in parallelo poiché diventano indipendenti.



Questa idea venne in mente perché di solito i registri sono usati come parcheggi per i dati e nel 99% dei casi non mi interessa quale registro in particolare conterrà il dato.

### Primo esempio - WAW

In questo esempio sotto entrambe le istruzioni scrivono in R3; se R3 venisse rinominato in R3\*, le due istruzioni potrebbero essere eseguite in parallelo visto che nella seconda istruzione R3 viene mappato in un altro registro fisico.

Si scrive nel registro più a destra.

ist1: op R1,R2,**R3**  
- instructions using R3  
ist2: op R4,R5,**R3**  
- instructions using R3

ist1: op R1,R2,**R3**  
- instructions using R3  
ist2: op R4,R5,**R3\***  
- instructions using R3\*

### Secondo esempio - RAW

Si scrive nel registro più a destra.

1 - op R1,R2,**R3**  
2 - op R3,R2,R4  
3 - op R4,R3,R5

1 - op R1\*,R2\*,**R3\***  
2 - op R3\*,R2\*,R4\*  
3 - op R4\*,R3\*,R5\*

4 - op R2,R4,**R3**  
5 - op R3,R2,R6

4 - op R2\*,R4\*,**R3\*\***  
5 - op R3\*\*,R2\*,R6\*

All'inizio ogni registro architetturale viene mappato in un registro fisico.

La seconda e la terza istruzione leggono il risultato della prima (*la quale ha scritto in R3\**). Sono dipendenti e il registro fisico R3\* deve coincidere nelle tre istruzioni.

Nelle ultime due istruzioni si utilizzano i doppi asterischi visto che il registro non è lo stesso di prima (*con R3\*\* indico che sto usando lo stesso registro architetturale R3 ma un altro registro fisico ancora, diverso da R3\**).

Perché l'istruzione quattro sta sovrascrivendo ciò che ha fatto la prima, quindi può benissimo scrivere su un altro registro fisico, in modo da non avere dipendenze con le prime tre istruzioni.

Quindi lo stesso registro architetturale può essere mappato in più registri fisici (64 registri architetturali, 1K registri fisici).

### Terzo esempio

Ad R1 viene assegnato il registro fisico R2\*, mentre ad R2 viene assegnato R4\* ed R3 viene assegnato R1\*.

Nella tabella a destra (*ptr*) indica l'indice del registro architetturale assegnato al registro fisico.

Si scrive nel registro più a destra.

| Arch. reg. | Original Instr.  | Renamed Instr.      | Physical/arch. ptr. |   |
|------------|------------------|---------------------|---------------------|---|
| R1= 4      | 1) op R1, R2, R3 | 1) op R2*, R4*, R1* | R1*=                | 3 |
| R2= 3      | 2) op R3, R2, R4 |                     | R2*= 4              | 1 |
| R3= 7      | 3) op R1, R5, R3 |                     | R3*=                |   |
| R4= 1      | 4) op R3, R1, R5 |                     | R4*= 3              | 2 |
| R5= 9      |                  |                     | R5*=                |   |
| R6= 5      |                  |                     | R6*=                |   |
|            |                  |                     | R7*=                |   |
|            |                  |                     | R8*=                |   |
|            |                  |                     | R9*=                |   |
|            |                  |                     | R10*=               |   |

Notiamo subito che l'ordine (1) → (2) non può essere cambiato dato che la (2) legge il risultato della (1).

Notiamo anche che l'ordine (2) → (3) può essere cambiato dato che la (3) sovrascrive il risultato della (1) e quindi di fatto basta che la (3) scriva da un'altra parte.

Si scrive nel registro più a destra.

| Arch. reg. | Original Instr.  | Renamed Instr.      | Physical/arch. ptr. |   |
|------------|------------------|---------------------|---------------------|---|
| R1= 4      | 1) op R1, R2, R3 | 1) op R2*, R4*, R1* | R1*=                | 3 |
| R2= 3      | 2) op R3, R2, R4 |                     | R2*= 4              | 1 |
| R3= 7      | 3) op R1, R5, R3 | 3) op R2*, R7*, R5* | R3*=                |   |
| R4= 1      | 4) op R3, R1, R5 |                     | R4*= 3              | 2 |
| R5= 9      |                  |                     | R5*=                | 3 |
| R6= 5      |                  |                     | R6*=                |   |
|            |                  |                     | R7*= 9              | 5 |
|            |                  |                     | R8*=                |   |
|            |                  |                     | R9*=                |   |
|            |                  |                     | R10*=               |   |

Il registro architetturale R3 viene allocato nel registro fisico R5\*.

Non essendoci più vincoli di registri, la (1) e la (3) possono essere eseguite in parallelo (*solo se il sistema permette multiple letture in contemporanea da R2\**).

Come si può notare dalla tabella a destra sia  $R5^*$  che  $R1^*$  sono mappati sul registro architetturale  $R3$ .

Si scrive nel registro più a destra.

| Arch. reg. | Original Instr.  | Renamed Instr.      | Physical/arch. ptr. |
|------------|------------------|---------------------|---------------------|
| R1= 4      | 1) op R1, R2, R3 | 1) op R2*, R4*, R1* | R1*= 3              |
| R2= 3      | 2) op R3, R2, R4 | 3) op R2*, R7*, R5* | R2*= 4              |
| R3= 7      | 3) op R1, R5, R3 | 4) op R4*, R2*, R6* | R3*=                |
| R4= 1      | 4) op R2, R1, R5 |                     | R4*= 3              |
| R5= 9      |                  |                     | R5*= 3              |
| R6= 5      |                  |                     | R6*= 5              |
|            |                  |                     | R7*= 9              |
|            |                  |                     | R8*=                |
|            |                  |                     | R9*=                |
|            |                  |                     | R10*=               |

La (4) deve leggere i registri architetturali  $R2$  ed  $R1$ , quindi riuso il registro fisico  $R4^*$  ed  $R2^*$ .

La (4) e la (1) utilizzano  $R2^*$  in lettura e possono essere eseguite in parallelo.

Si scrive nel registro più a destra.

| Arch. reg. | Original Instr.  | Renamed Instr.      | Physical/arch. ptr. |
|------------|------------------|---------------------|---------------------|
| R1= 4      | 1) op R1, R2, R3 | 1) op R2*, R4*, R1* | R1*= 2              |
| R2= 3      | 2) op R3, R2, R4 | 3) op R2*, R7*, R5* | R2*= 4              |
| R3= 2      | 3) op R1, R5, R3 | 4) op R4*, R2*, R6* | R3*= 4              |
| R4= 1      | 4) op R2, R1, R5 | 2) op R1*, R4*, R3* | R4*= 3              |
| R5= 9      |                  |                     | R5*= 3              |
| R6= 5      |                  |                     | R6*= 5              |
|            |                  |                     | R7*= 9              |
|            |                  |                     | R8*=                |
|            |                  |                     | R9*=                |
|            |                  |                     | R10*=               |

- Eseguiamo in modo out of order la (1), (3) e (4).
  - La (1) **termina**, produce il valore **2** e lo mette in  $R1^*$
  - La (3) **termina**, produce il valore **3** e lo mette in  $R5^*$
  - La (4) **termina**, produce il valore **5** e lo mette in  $R6^*$
- La (2) può essere eseguita dato che  $R1^*$  ora non è più bloccato in scrittura.

Si scrive nel registro più a destra.

| Arch. reg. | Original Instr.   | Renamed Instr.       | Physical/arch. ptr. |
|------------|-------------------|----------------------|---------------------|
| R1= 4      | 1) op R1, R2, R3  | 1) op R2*, R4*, R1*  | R1*=2 3             |
| R2= 3      | 2) op R3, R2, R4  | 3) op R2*, R7*, R5*  | R2*= 4 1            |
| R3= 2      | 2b) op R3, R3, R6 | 4) op R4*, R2*, R6*  | R3*= 4              |
| R4= 1      | ... Using R3      | 2) op R1*, R4*, R3*  | R4*= 3 2            |
| R5= 9      | 3) op R1, R5, R3  | 2b) op R1*, R1*, R8* | R5*=10 3            |
| R6= 5      | 4) op R2, R1, R5  | ... Using R1*        | R6*= 5              |
|            |                   |                      | R7*= 9 5            |
|            |                   |                      | R8*= 6              |
|            |                   |                      | R9*=                |
|            |                   |                      | R10*=               |

Quando posso “deallocare” un registro fisico?

Per “deallocare” si intende il rompere il legame tra il registro fisico e quello architetturale.

Quando si esegue la commit di un'istruzione che scrive su un architetturale che era stato allocato in un'istruzione precedente.

ADD R1, R2, R3 // Alloca il registro R3\*

ADD R3, R5, R1 // legge su R3\*

ADD R1, R2, R3 // Sovrascrive ciò che ha fatto la 1°

ADD R1, R2, R6 // Scrivo su un altro registro architetturale.

ADD R1\*, R2\*, R3\* // Alloca il registro R3\*

ADD R3\*, R5\*, R1\* // legge su R3\*

ADD R1\*, R2\*, R7\* // Sovrascrive ciò che ha fatto la 1°.

// Si rompe il legame tra R3 e R3\*

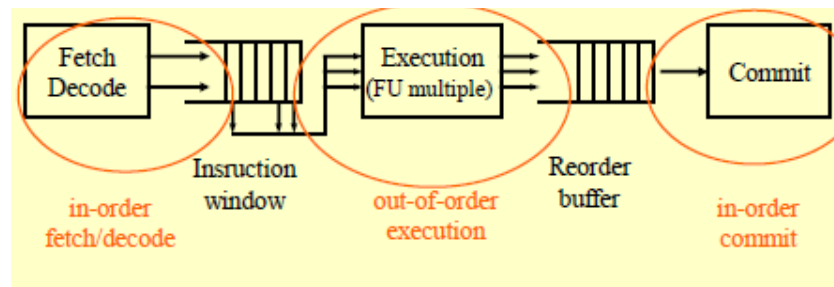
ADD R1\*, R2\*, R3\* // Sovrascrive ciò che ha fatto la 3°.

// Si rompe il legame tra R3 e R7\*

In parole povere: quando sovrascrivo, e che quindi perdo il valore inserito precedentemente. Quando accade questa cosa il registro architetturale in questione smette di essere parte di un flusso di esecuzione ben preciso e quindi lo posso associare ad un altro registro fisico.

Nell'esempio di prima quando si committa la (2), quella che scriveva qualcosa di nuovo in  $R3^*$ .

### Come avviene il renaming in hardware



La pipeline viene divisa in 3 zone, divise da 2 buffer:

- Instruction window: Si occupa di fare il renaming.
  - L'implementazione dipende dal processore e non segue una logica standard.
- Reorder buffer.

### Branch prediction - Esecuzione speculativa

In questo caso ho due possibili scenari:

1. instr. 1
2. instr. 2
3. instr. 4

Oppure:

1. instr. 3
2. instr. 4

```
Es.: original code
if ( x > 3 ) {
    instr. 1
    instr. 2
}
else
    instr. 3
instr. 4
```

A seconda del valore della variabile  $x$ .

Invece di aspettare il risultato della verifica del predicato ( $x > 3$ ), si eseguono entrambi gli scenari. Appena si conosce l'esito del predicato prendo il ramo corretto e scarto l'altro.

### Vantaggi dell'esecuzione speculativa

- Ridurre il ritardo dovuto alle dipendenze di dati e control hazards.
- Eseguire più istruzioni in parallelo quando possibile.

## Svantaggi dell'esecuzione speculativa

- Speculazione errata: aumento del numero di istruzioni eseguite senza risultati utili, se il ramo speculativo scartato alla fine si rivela essere quello corretto.

### if-else-if annidati

| <i>original code</i>              | <i>modified code</i>                           |
|-----------------------------------|------------------------------------------------|
| <code>if ( x &gt; 3 ){</code>     | <code>Rpred1 = (x&gt;3)</code>                 |
| <code>  instr. 1</code>           | <code>instr.1 (Rpred1 - true)</code>           |
| <code>  instr. 2</code>           | <code>instr.2 (Rpred1 - true)</code>           |
| <code>    if ( y &gt; 5 ){</code> | <code>Rpred2 = (y&gt;5) (Rpred1 - true)</code> |
| <code>      instr. 5</code>       | <code>instr.5 (Rpred2 - true)</code>           |
| <code>      instr. 6</code>       | <code>instr.6 (Rpred2 - true)</code>           |
| <code>    }</code>                |                                                |
| <code>  }</code>                  |                                                |
| <code>else</code>                 |                                                |
| <code>  instr. 3</code>           | <code>instr.3 (Rpred1 - false)</code>          |
| <code>instr. 4</code>             | <code>instr. 4 (true)</code>                   |

Quando si incontrano `if-else-if` annidati il processore non sa immediatamente quale ramo eseguire, poiché il risultato della seconda condizione può dipendere dai risultati delle istruzioni all'interno del primo ramo (*ad esempio, il valore di `y` può dipendere da `instr.1` o `instr.2`*).

Invece di aspettare passivamente che la condizione venga risolta (*la quale può richiedere anche un accesso in memoria in caso di miss*), il processore esegue tutti i rami in parallelo e nel mentre monitora costantemente lo stato delle istruzioni eseguite in modo speculativo.

Appena si conosce l'esito del 1° `if`, il processore scarta l'esecuzione dell'altro ramo e procede con l'esecuzione del ramo "*prescelto*".

Un esempio in cui viene usato sono i cicli `for`:

```
int a = 0;

for(int i = 0 ; i < 1000 ; i++){

    a++;

}
```

Invece che eseguire il controllo della condizione su `i` ad ogni iterazione, si va a diritto senza controllarla.

Su 1000 iterazioni (*dando per scontato che si continui sempre a ciclare*) si sbaglierà predizione solo 1 volta. Ma in compenso ho velocizzato di molto il ciclo `for`.

### Su cosa si basa questa tecnica

Questa tecnica si basa molto sui dati ottenuti tramite profiling; ergo si basa su informazioni statistiche sul programma.

## Multithreading

Un processo è composto da uno o più flussi di esecuzione detti “*thread*” che condividono lo stesso spazio di indirizzamento.

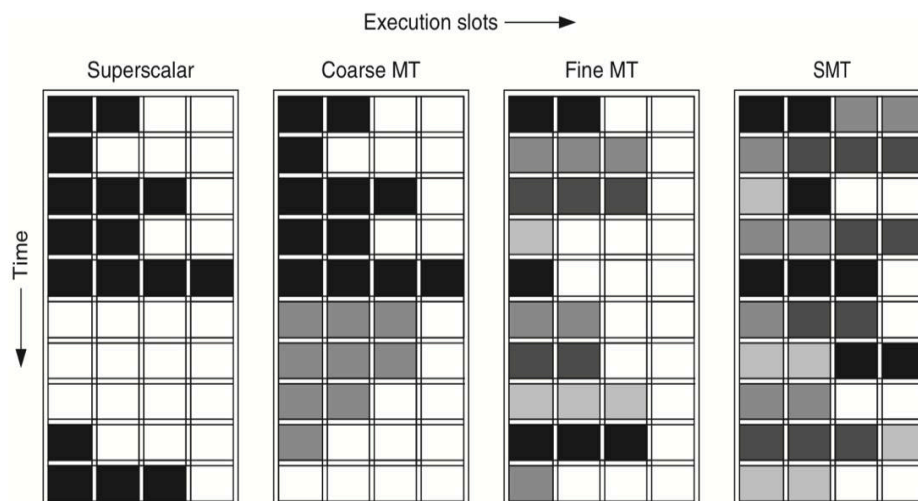
Due thread dello stesso padre si differenziano per:

- **Stack:** Le variabili private.
- **Register file:** Il valore dei registri.
- **Program Counter:** Due thread dello stesso padre eseguono lo stesso codice ma da due punti diversi.

### Cambio di contesto tra thread

L’hardware di un core fisico deve permettere di passare rapidamente a un thread diverso. Il cambio di thread è molto più efficiente rispetto a un cambio di processo, che di solito richiede centinaia o migliaia di cicli di processore.

### Utilizzo dei thread

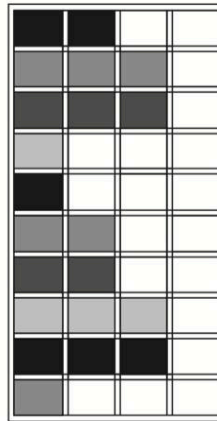


Il grafico sopra mostra come potrebbero essere eseguiti più thread nella realtà con diversi stili di esecuzione: ognuna delle 4 matrici mostrate ha 10 righe e 4 colonne: ciò significa che il sistema in esempio ha 4 unità funzionali.

Ogni colore (*ogni sfumatura di grigio*) si riferisce ad un thread, se un quadrato è colorato significa che quella unità funzionale in quell'istante di tempo è utilizzata da un thread.

Si nota bene che poche volte si riesce ad utilizzare queste 4 unità funzionali contemporaneamente.

## Fine MT - Fine-grained MultiThreading



Si passa frequentemente da un thread all'altro in modo round-robin, causando quindi l'interleaving dell'esecuzione di istruzioni appartenenti a thread diversi.

### Vantaggi

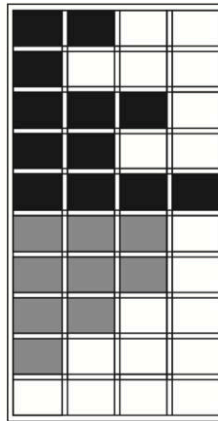
Può nascondere le perdite di throughput che derivano da stalli brevi e lunghi, poiché le istruzioni degli altri thread possono essere eseguite quando un thread si blocca, anche se il blocco dura solo alcuni cicli.

### Svantaggi

La complessità (*il cambio di thread deve essere velocissimo*) e rallenta l'esecuzione del singolo thread: un thread pronto per l'esecuzione verrà ritardato dalle istruzioni degli altri thread.



## Coarse MT - Coarse-grained MultiThreading



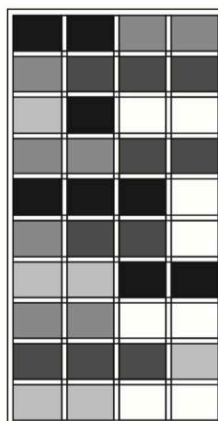
Il coarse-grained multithreading effettua un cambio di contesto tra thread solo in caso di lunghe attese:

- Stalli della pipeline dovuti a miss in L2 o L3.
- Lock bloccanti su una risorsa.

Questa tecnica:

- **Meno complessità hardware:** Evita la necessità che il cambio di thread sia poco costoso, poiché i cambi di thread sono meno frequenti e quindi impattano meno sul throughput.
- Rallenta meno l'esecuzione del singolo thread.

## SMT - Simultaneous MultiThreading

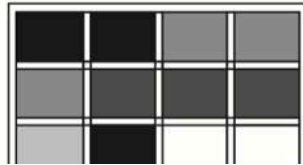


L'implementazione più comune del multithreading.

Questa tecnica è applicabile su un processore con dynamic scheduling, poiché un processore schedulato in modo dinamico ha già molti dei meccanismi hardware necessari per sostenere questa tecnica.

La ridenominazione dei registri e il dynamic scheduling consentono l'**esecuzione in contemporanea di più istruzioni provenienti da thread indipendenti** senza tener conto delle dipendenze tra di loro.

Questo permette di **nascondere gli stalli della pipeline**, aumentando così l'utilizzo delle unità funzionali.



In figura sopra, nella 1° riga, vediamo come il thread **nero** è vicino al **grigio**. Questo perchè le istruzioni di questi due thread vengono eseguite in parallelo.

# Processore Intel

Vediamo ora gli approcci adottati da Intel per realizzare i suoi processori, in particolare vedremo i7.

## CISC - Complex Instruction Set Computing

Intel è una macchina con istruzioni CISC, quindi:

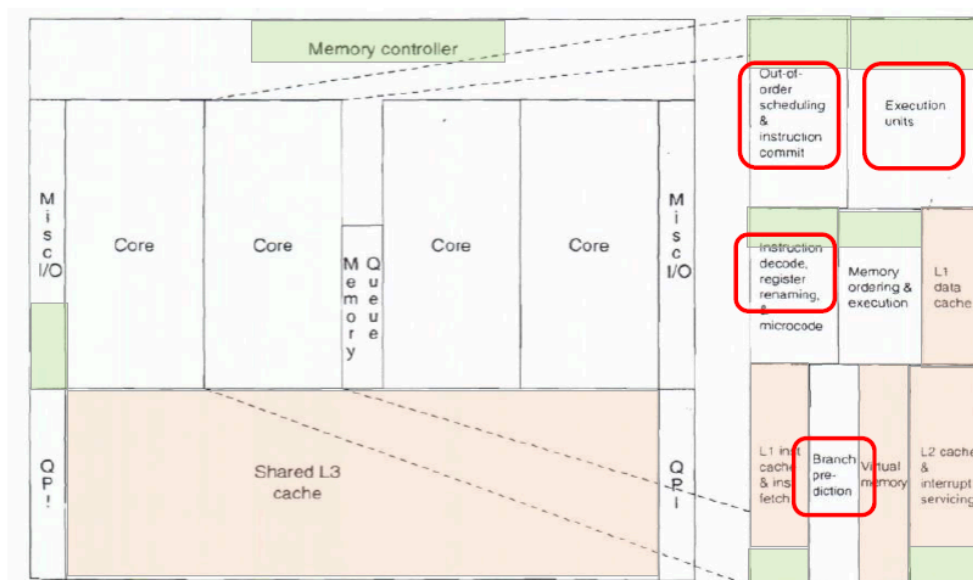
- Lunghezza delle istruzioni variabile.
- Semantica delle istruzioni complessa.
- Tempo di esecuzione variabile.
- Numerosi accessi alla memoria richiesti (*operandi in memoria*)

Il vantaggio delle istruzioni CISC è che possono eseguire operazioni complesse direttamente in un'unica istruzione.

A livello interno, i moderni processori Intel utilizzano tecniche di progettazione RISC per eseguire le istruzioni in modo più efficiente. Questo viene fatto tramite un'unità di decodifica che traduce le istruzioni x86 CISC in micro-operazioni (*micro-ops*) più semplici, che sono quindi eseguite da un'architettura interna simile a RISC.

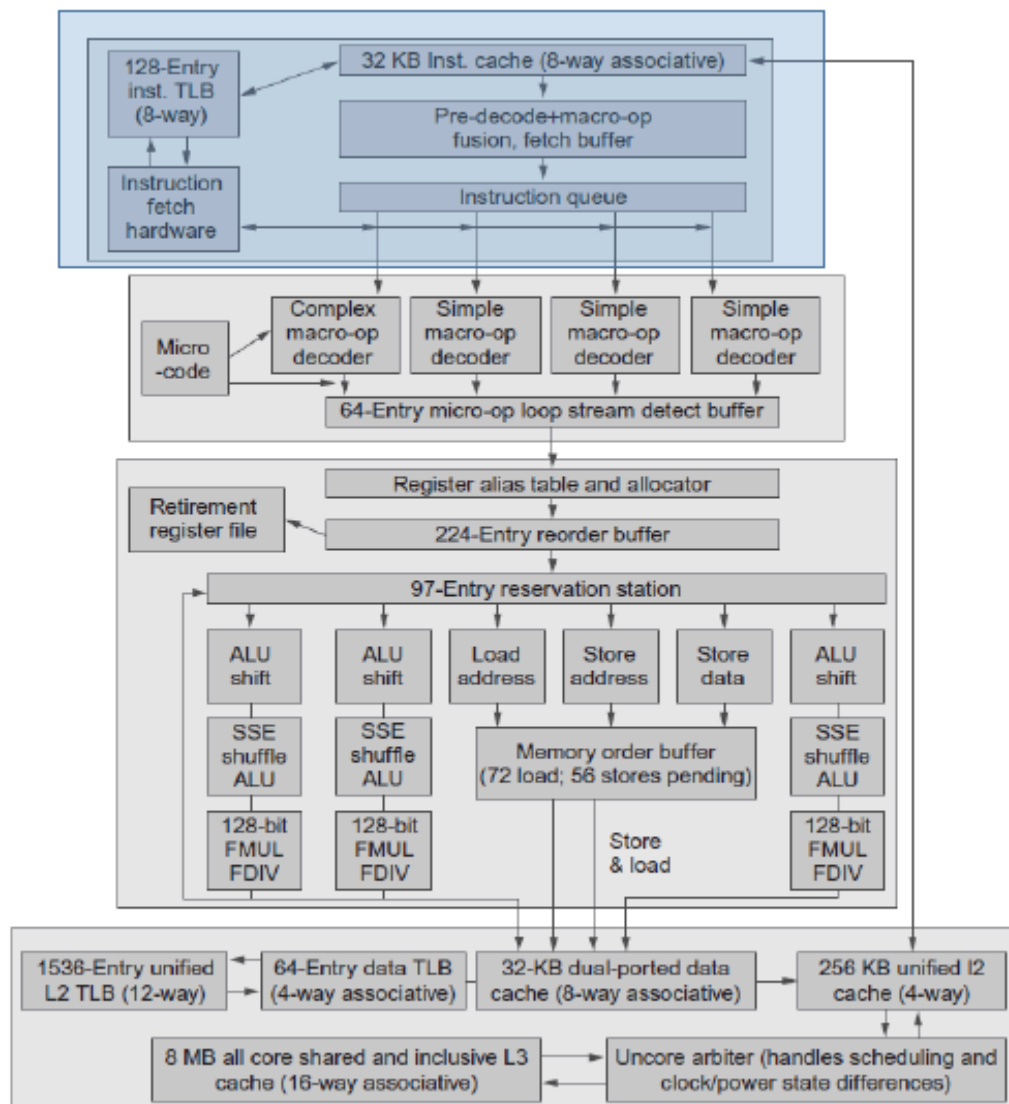
Quindi il set di istruzioni dei processori Intel a basso livello è RISC, mentre al programmatore appaiono come CISC.

## Intel i7



Il passaggio ad una architettura RISC come permette di mantenere la compatibilità dell'istruzione set corrente con le architetture precedenti? L'architettura precedente a livello di linguaggio macchina è CISC, mentre il microcodice effettivamente eseguito è RISC.

## Fase di fetch



Presenta anche un return address stack per velocizzare l'operazione di return nelle funzioni.

La fase di fetch si basa su un sofisticato branch predictor multilivello per ottenere un bilanciamento tra velocità e precisione delle previsioni.

Sbagliare le previsioni dei salti causa una penalità di circa 17 cicli di clock; invece utilizzando l'indirizzo previsto, l'unità di fetch delle istruzioni, fetcha 16 byte dalla cache delle istruzioni.

## Pre-decodifica delle istruzioni

I 16 byte prelevati durante la fase di fetch sono collocati nel buffer di predecodifica delle istruzioni.

La fusione delle macro-operazioni prende combinazioni di istruzioni e le fonde in un'unica operazione con un risultato equivalente.

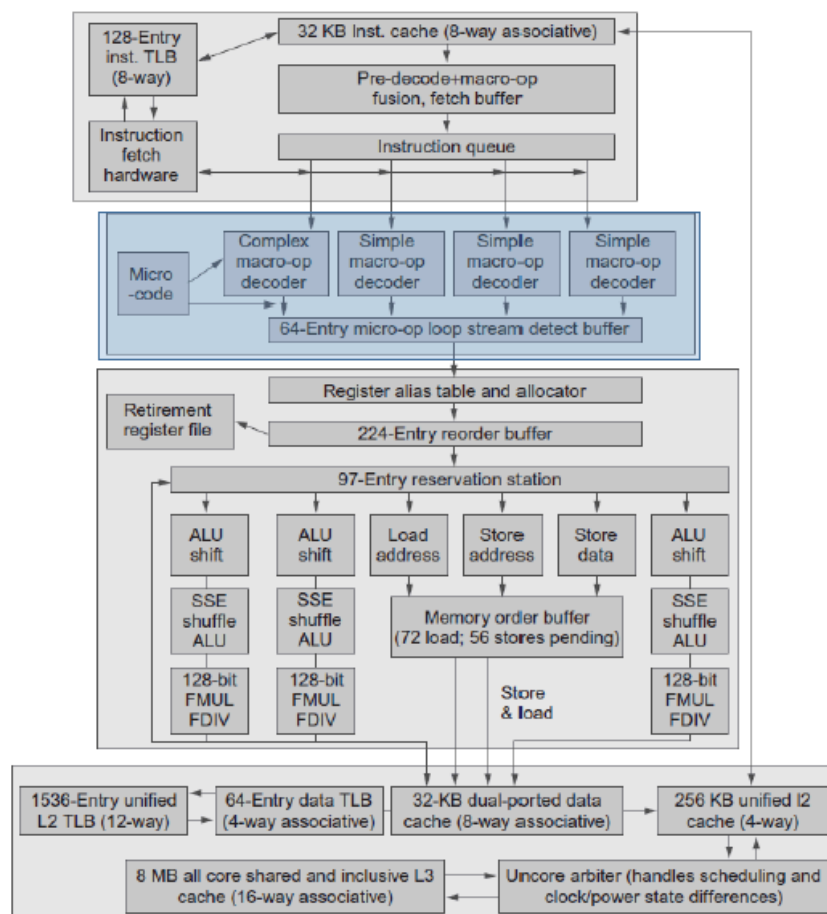
Solo alcuni casi speciali possono essere fusi, poiché dobbiamo sapere che l'unico uso del primo risultato avviene con la seconda istruzione (*ossia, confronto e salto*).

La macrofusione influisce molto sulle prestazioni dei programmi interi, con un aumento medio delle prestazioni compreso tra l'8% e il 10%, con alcuni programmi che mostrano risultati negativi. Sui programmi in virgola mobile non influisce molto.

Lo stadio di predecodifica divide anche i 16 byte in singole istruzioni x86.

Le singole istruzioni x86 (*include alcune istruzioni fuse*) sono collocate quindi nella coda delle istruzioni.

## Decodifica delle micro-operazioni



Dalla coda delle istruzioni le singole istruzioni x86 vengono tradotte in micro-operazioni. Le micro-operazioni sono istruzioni semplici simili a quelle di RISC che possono essere eseguite direttamente dalla pipeline.

Tre dei decoder traducono le istruzioni x86 direttamente in una micro operazione (*Simple macro-op decoder*). Invece per le istruzioni x86 con una semantica più complessa, esiste un apposito motore di microcodice (*Complex macro-op decoder*) che viene utilizzato per produrre una sequenza di micro-operazioni.

Le micro-operazioni de-codificate vengono posizionate in base all'ordine delle istruzioni x86 nel buffer di micro operazioni con 64 entry.

## Rilevamento del loop stream

Se c'è una piccola sequenza di istruzioni (*meno di 64 istruzioni*) che comprende un ciclo, il rilevatore di flusso del ciclo: troverà il ciclo e emetterà direttamente le micro-operazioni dal buffer.

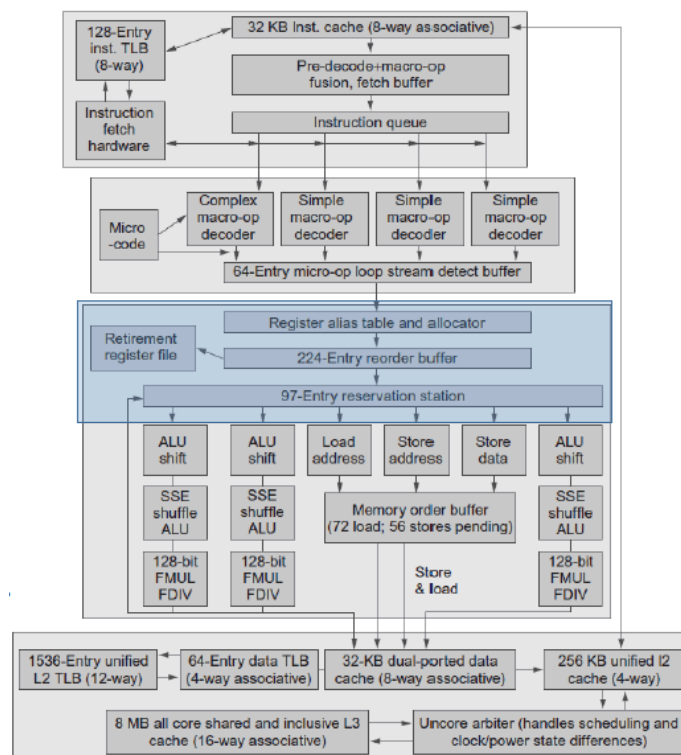
Ciò elimina la necessità di attivare molteplici fasi di fetch e decodifica delle istruzioni.

## Microfusione

Questa tecnica consente di combinare più micro-ops derivate da una singola istruzione in una singola micro-operazione micro, riducendo così il numero totale di micro-ops che devono essere gestite dalla pipeline del processore.

La fusione delle micro-operazioni produce guadagni minori per i programmi con numeri interi e maggiori per quelli in virgola mobile.

## Emissione dell'istruzione base

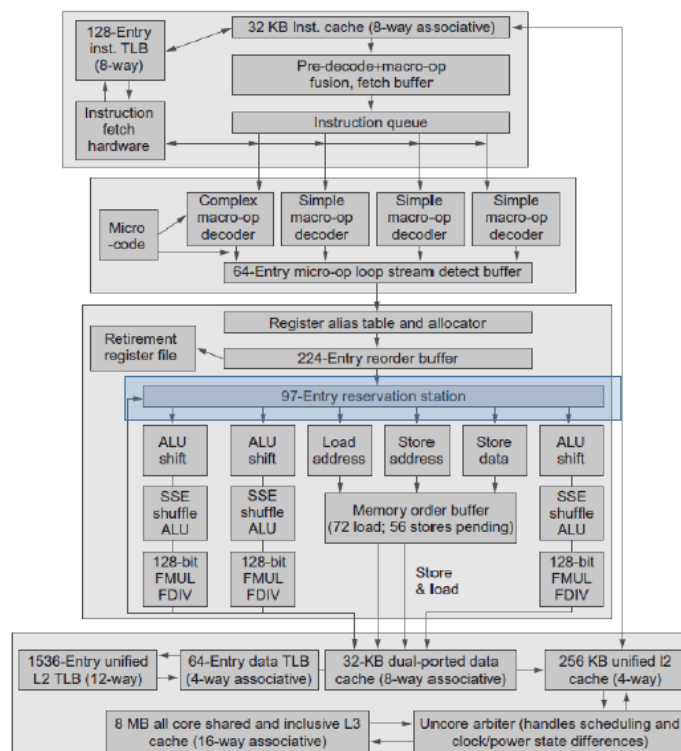


In questa fase avviene:

- Ricerca della posizione del registro nelle tabelle dei registri.
- Rinomina dei registri.
- Assegnazione di un'entrata nel buffer di riordinamento.
- Fetch dei risultati dai registri o dal buffer di riordinamento prima di inviare le micro-operazioni alle reservation station.

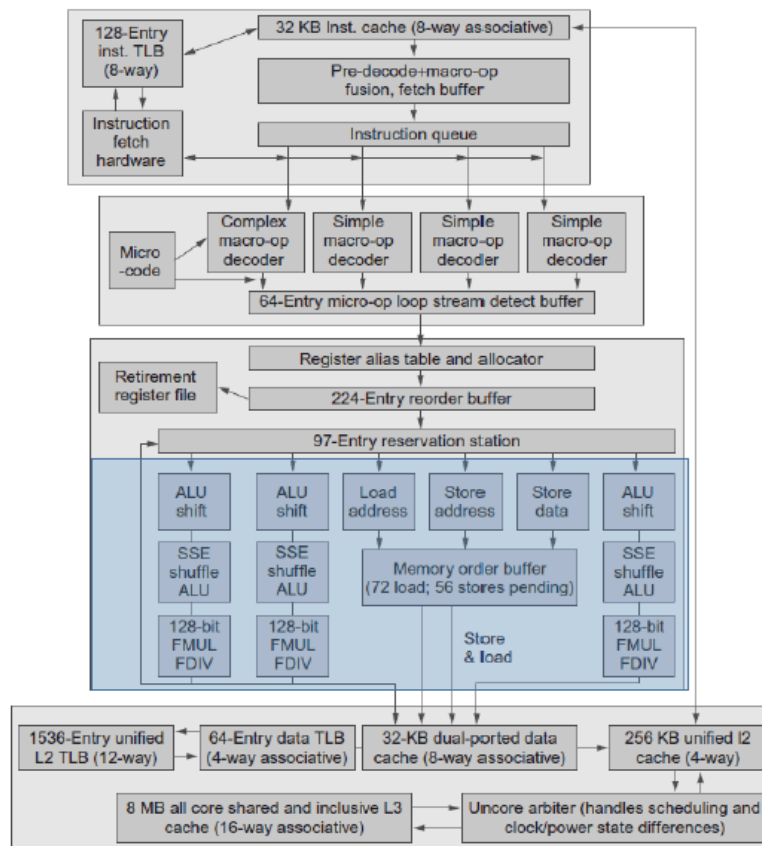
Fino a quattro micro-operazioni possono essere elaborate ad ogni ciclo di clock; vengono assegnate le successive entrate disponibili nel buffer di riordinamento.

## Reservation station centralizzata



La reservation station centralizzata è condivisa da sei unità funzionali. Fino a sei micro-operazioni possono essere inviate alle unità funzionali ad ogni ciclo di clock.

## Esecuzione delle micro-operazioni

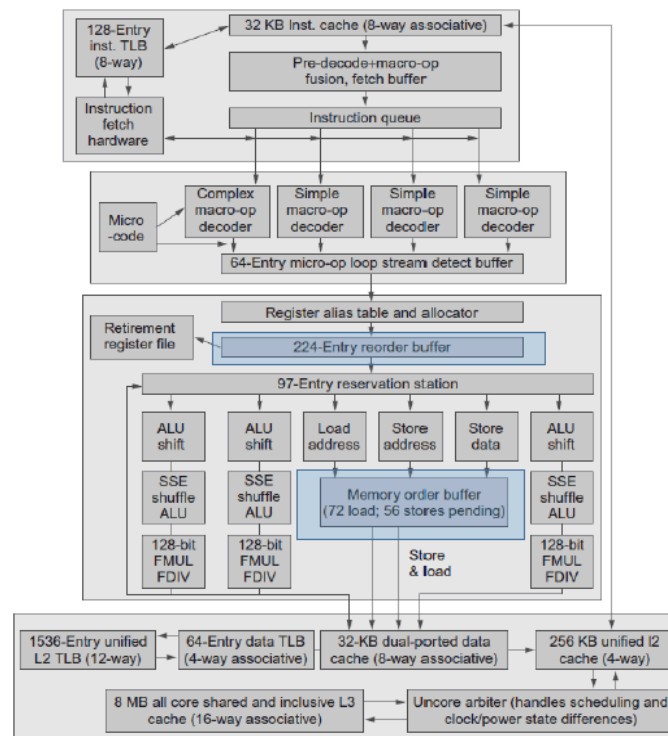


Le micro-operazioni vengono eseguite dalle unità funzionali individuali e i risultati vengono inviati sia alle reservation station in attesa che all'unità di ritiro dei registri.

Una volta noto che l'istruzione non è più speculativa, verrà aggiornato lo stato del registro. L'entrata corrispondente all'istruzione nel buffer di riordinamento viene segnata come completa.



## Commit



Quando una o più istruzioni all'inizio del buffer di riordinamento sono state contrassegnate come complete:

- Le scritture in sospeso nell'unità di ritiro dei registri vengono effettivamente eseguite.
- Le istruzioni vengono rimosse dal buffer di riordinamento.

## Fase di rename

Successivamente c'è la fase di rename, le istruzioni vengono mantenute ordinate, le quali vanno verso la zona in cui ci sono le unità funzionali, queste lavorano a completare l'informazione e memorizzarla, dopo questa fase vanno verso la memoria e c'è la gestione della memoria, in questa struttura c'è il TLB che ha due livelli in cache.

Il TLB è la cache delle ultime traduzioni.

La parte in azzurro serve per l'Instruction fetch ed effettuare la prediction.

Nella fase di rename vengono messe le micro-istruzioni nel reorder-buffer, quando sono pronte vengono inviate al reservation station, le stazioni a disposizione sono delle ALU semplici e delle ALU specializzate per le load e store, se ne mettono due anziché una perché l'assembler è CISC, nelle esecuzioni CISC .....

Fase di commit, è sdoppiata, fa due tipi di operazioni, fa il write back, il quale può essere o sui registri o in memoria.

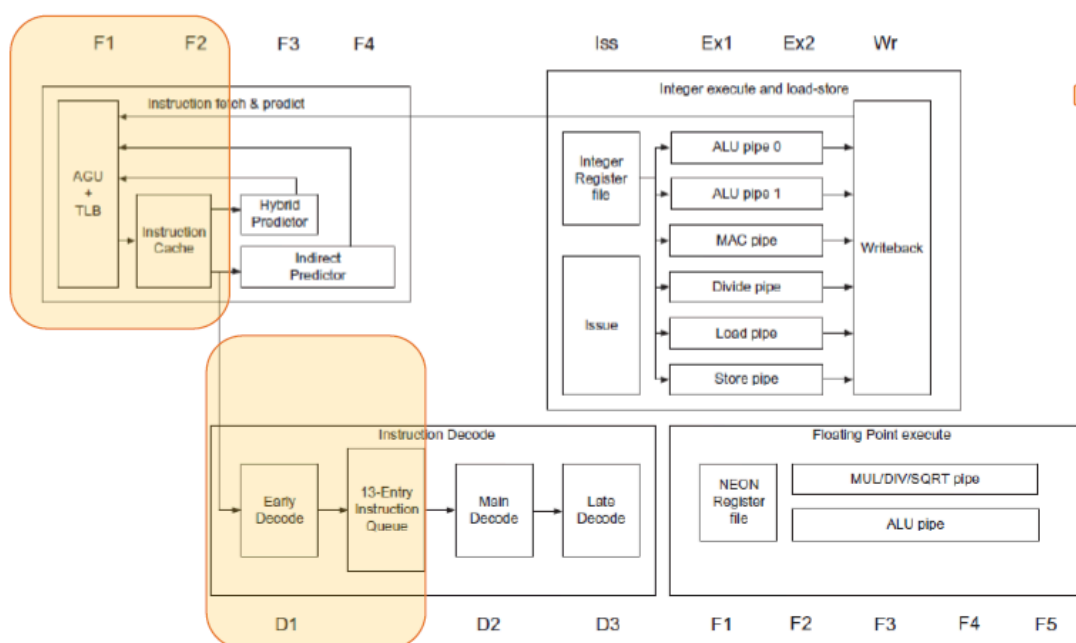
# Processori ARM

Confrontando l'ARM con l'i7 920, l'ARM è molto più efficiente in termini di consumo di energia elettrica poiché ha una frequenza di clock più bassa.

Nella prima parte c'è il sistema di prediction, con una parte dove avviene il prefetch dell'istruzione, quindi c'è la fase di prefetch del TLB e il prediction, i primi due cicli servono per il prefetch, mentre gli altri due cicli servono per fare prediction.

Vediamo in dettaglio l'architettura dell'ARM Cortex-A53:

## Fase 1



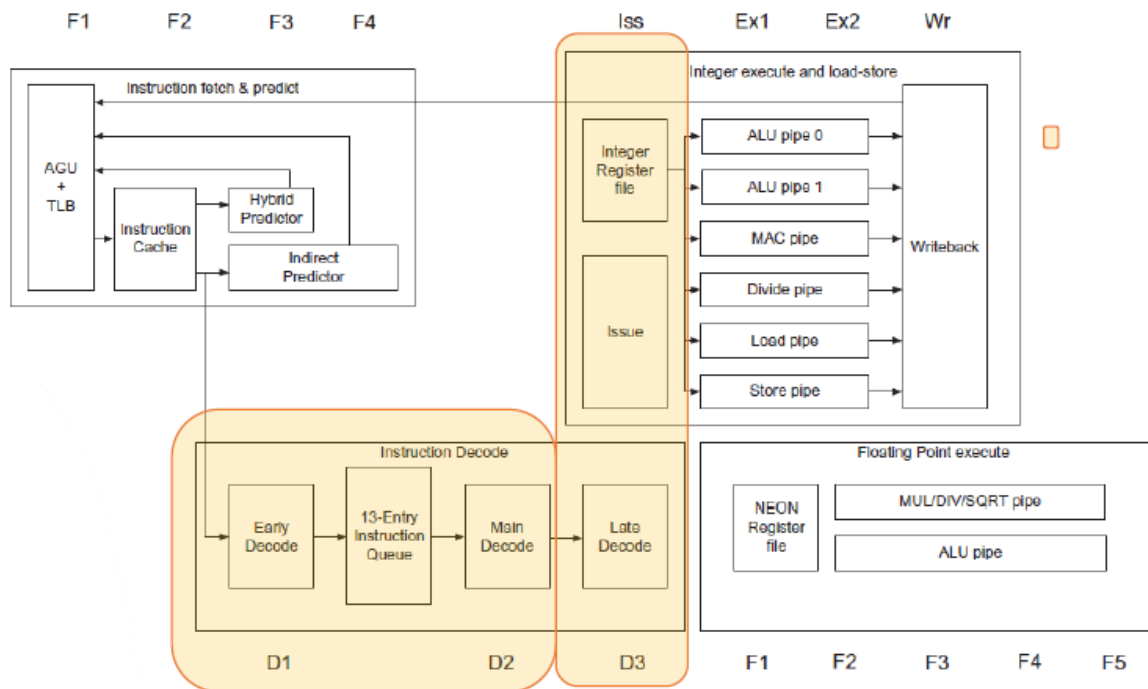
Le prime fasi dell'esecuzione avvengono nelle componenti in giallo, l'unità di generazione degli indirizzi (AGU) utilizza un **predittore ibrido**, un **predittore indiretto** e uno **stack di ritorno** per effettuare branch prediction e mantenere una coda di istruzioni completa.

Nelle prime tre fasi, le istruzioni vengono caricate in una coda di istruzioni con 13 voci. I quattro cicli di fetch delle istruzioni coinvolgono un'unità di generazione degli indirizzi che genera il prossimo valore del PC incrementando il valore precedente del PC o utilizzando uno dei tre predittori in caso di istruzioni di salto.

Durante il primo ciclo di fetch, viene verificata la cache di destinazione; se avviene hit, le due istruzioni successive si trovano nella cache di destinazione.

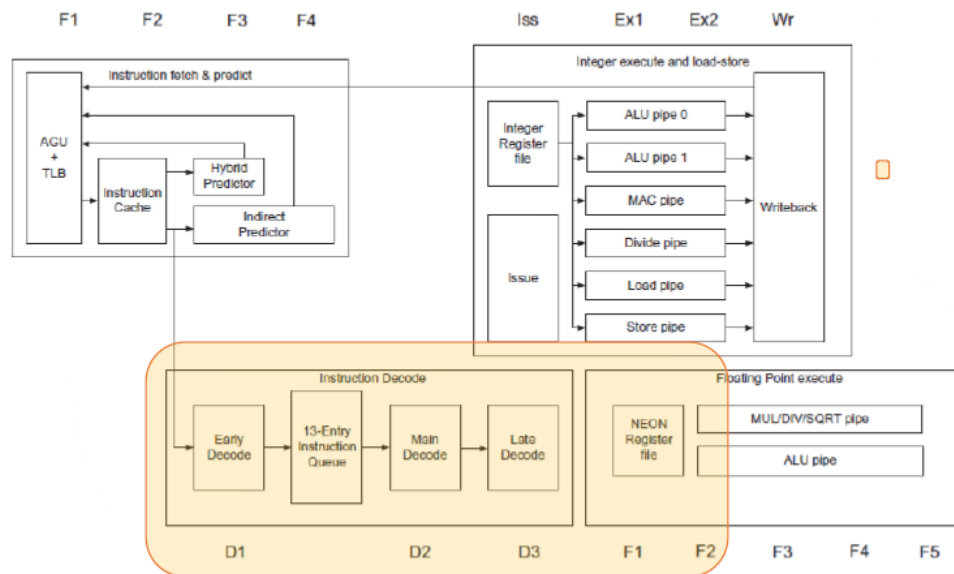
Quando c'è un hit e una previsione corretta, il branch viene eseguito senza alcun ciclo di ritardo.

## Fase 2



Successivamente D1 e D2, rappresentati in figura, eseguono la decodifica di base, mentre D3 decodifica alcune istruzioni più complesse e si sovrappone alla prima fase della pipeline di esecuzione (ISS). Dopo ISS, le fasi Ex1, Ex2 e WB completano la pipeline per gli interi.

## Fase 3



Come è possibile osservare nella zona gialla, la pipeline di esecuzione in virgola mobile è profonda 5 cicli, oltre ai 5 cicli necessari per il fetch e la decodifica, per un totale di 10 stadi. l'ARM ha avuto un effetto strano con il floating point, all'inizio era in virgola fissa, quindi necessitò di uno sforzo notevole.

Le fasi di decodifica della pipeline determinano se ci sono dipendenze tra una coppia di istruzioni, che obbligherebbe l'esecuzione sequenziale, e in quale pipeline delle fasi di esecuzione inviare le istruzioni.

La sezione di esecuzione delle istruzioni occupa principalmente tre stadi di pipeline e fornisce una pipeline per le istruzioni di caricamento (load), una per le istruzioni di memorizzazione (store), due per le operazioni aritmetiche intere e pipeline separate per le operazioni di moltiplicazione e divisione intere.

Entrambe le istruzioni della coppia possono essere emesse alle pipeline di caricamento o memorizzazione.

Le fasi di esecuzione hanno un inoltro completo tra le pipeline. Le operazioni in virgola mobile e SIMD aggiungono due stadi di pipeline alla sezione di esecuzione delle istruzioni e presentano una pipeline per le operazioni di moltiplicazione/divisione/radice quadrata e una pipeline per altre operazioni aritmetiche.

## Core IP

Cortex-A53 può essere distribuito come core IP (Intellectual Property), i core IP sono la forma predominante di distribuzione tecnologica nei mercati embedded, dei dispositivi mobili personali e correlati; miliardi di processori ARM e MIPS sono stati creati da questi core IP.

Un core IP (che può essere a sua volta multicore) è progettato per essere incorporato con altre logiche (quindi è il “core” di un chip), compresi processori specifici per applicazioni (come un codificatore o decodificatore video), interfacce I/O e interfacce di memoria, e quindi fabbricati per produrre un processore ottimizzato per una particolare applicazione.

Sebbene il core del processore sia quasi identico logicamente, i chip risultanti possono avere molte differenze.

Un parametro importante è la dimensione della cache L2, che può variare fino a un fattore di 16

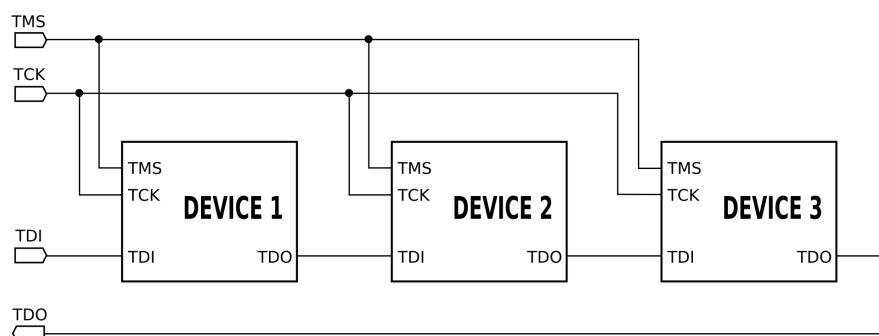
Struttura ARM7, non è molto diversa dal circuito che abbiamo creato noi, abbiamo le due situazioni in cui il contenuto del PC indirizza la memoria istruzioni, subito c'è il sommatore, successivamente entra nella zona in cui ci sono i banchi di registri.

### Power saving instructions

Novità introdotta da ARM, quando il processore non sa cosa fare, chiama la funzione **WFI** (*wait for interrupt*), ovvero finché non accade nulla va in risparmio (ovvero *gli viene tolto il clock e tutto rimane così, questo perché è una logica sequenziale asincrona*), l'altra istruzione è la **WFE** (*wait for event*), questo avviene nei sistemi multiprocessore, un processore va in pausa finché non viene risvegliato da un altro processore.

### JTAG

Protocollo utilizzato per il testing, la diagnosi e la programmazione di circuiti integrati, come microcontrollori, FPGA (*Field-Programmable Gate Array*) e altri dispositivi integrati.



Il JTAG consente di accedere ai pin interni di un dispositivo, consentendo il debug e la programmazione del dispositivo anche dopo che è stato montato su una scheda elettronica.

Con il protocollo JTAG, è possibile leggere e scrivere dati nei registri interni del dispositivo, eseguire il debug dei circuiti e controllare varie funzionalità, come la programmazione della memoria e il controllo delle porte di I/O.

Tra le funzionalità principali del JTAG ci sono la catena di shift register (*shift chain*) che consente di comunicare con i singoli pin del dispositivo, l'accesso alla memoria interna del dispositivo per la programmazione e il test, e la possibilità di eseguire il debug del codice firmware o hardware attraverso l'interfaccia JTAG.

### **Come funziona il JTAG?**

I registri vengono messi in sequenza e sono tutti collegati da un unico filo e possono lavorare in modalità shift. Se su un grosso registro shift dò un colpo di clock, in uscita ho il primo valore del registro più esterno.

# Cache

La cache è un componente hardware (*il cui operato è invisibile agli occhi dei programmi*) utilizzato per diminuire la latenza d'accesso alla memoria principale.

In particolare, nei moderni processori si utilizza una gerarchia multilivello di memorie avente lo scopo di minimizzare il tempo speso dal processore in attesa del completamento di un'operazione (*lettura/scrittura*) sulla memoria.

## SRAM - Static RAM

Tecnologia per realizzare le memorie molto veloce, costosa e che consuma molto; per questo è usata solo nelle cache e non sulla RAM. La tecnologia SRAM permette una latenza di accesso nell'ordine dei 5 nanosecondi.

## DRAM - Dynamic RAM

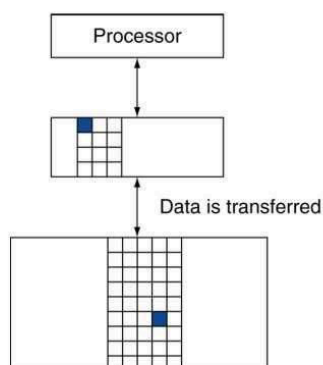
La tecnologia utilizzata nelle RAM presenti in commercio; consuma e costa molto meno; ma è molto più lenta ed è più difficile da gestire elettronicamente (*appaiono come dei condensatori*) dal punto di vista dei cali di tensione.

La tecnologia SRAM permette una latenza di accesso nell'ordine dei 70 nanosecondi. Ma circa ogni 8 nanosecondi occorre refreshare la memoria (*per ricaricare i condensatori*).

## Principio di località dei dati

Il blocco (*chiamato anche "cacheline" o "line"*) è l'unità di informazione trasferibile dalla memoria RAM alla cache e viceversa.

I dati che servono al programma sono spesso molto limitati perché tende ad usare sempre la stessa zona di memoria per fare le sue operazioni. Se quella zona di memoria fosse caricata in una memoria molto più veloce allora tutto si velocizzerebbe.



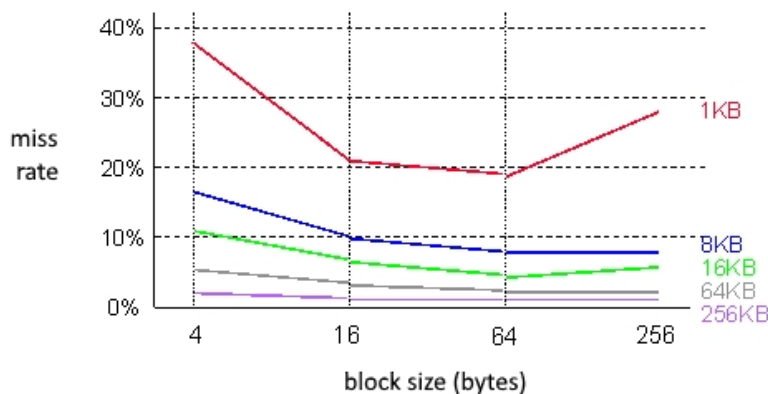


## Località temporale

Se la CPU esegue un accesso in memoria, è altamente probabile che a breve ne sia fatto un altro. Per sfruttarla al meglio si preferisce avere tanti blocchi piccoli.

## Località spaziale

Se la CPU esegue un accesso in memoria, è altamente probabile che il prossimo accesso sia fatto nelle immediate vicinanze. Per sfruttarla al meglio si preferisce avere pochi blocchi ma grandi.



## Indici di prestazione delle cache

### Cache-hit

Quando il blocco è presente in cache, la copia del blocco viene acceduto direttamente in cache senza utilizzare la RAM.

### Cache-miss

Quando il blocco non è presente in cache, la copia del blocco viene trasferita dalla RAM nella cache.

Il trasferimento richiede tempo (*il blocco ha dimensione maggiore del bus che collega RAM e cache*) poiché per trasferire un blocco è necessario fare molteplici trasferimenti.

### Miss rate

La probabilità di fare una miss:  $Miss_{ratio} = \frac{\text{Numero Miss}}{\text{Numero di Accessi}}$

Per abbassare il miss rate si guarda quali sono le locazioni di memoria (*che possono contenere istruzioni o operandi*) più utilizzate e facciamo in modo che siano tutte raggruppate in meno blocchi possibili; ossia si sfrutta il principio di località dei dati.

## Tempo di accesso medio

Il nostro obiettivo è quello di minimizzare il tempo medio di accesso in cache, il quale si calcola come: la somma dei tempi di esecuzione delle operazioni diviso il numero di operazioni.

La formula del tempo medio di accesso alla memoria ci fornisce un quadro per presentare le ottimizzazioni rivolte a migliorare le prestazioni della cache.

$$T_{\text{medio accesso cache}} = \frac{\sum_{i=0}^N T_{op_i}}{\text{Numero Operazioni}}$$

Oppure un'altra formula è:

$$T_{\text{medio accesso cache}} = hit_{\text{time}} + Miss_{\text{Penalty}} * Miss_{\text{Rate}}$$

## Miss penalty

Indica quanti cicli di clock sono stati necessari per poter accedere a quell'elemento in RAM in caso di miss rispetto al caso ideale (*cache-hit*).

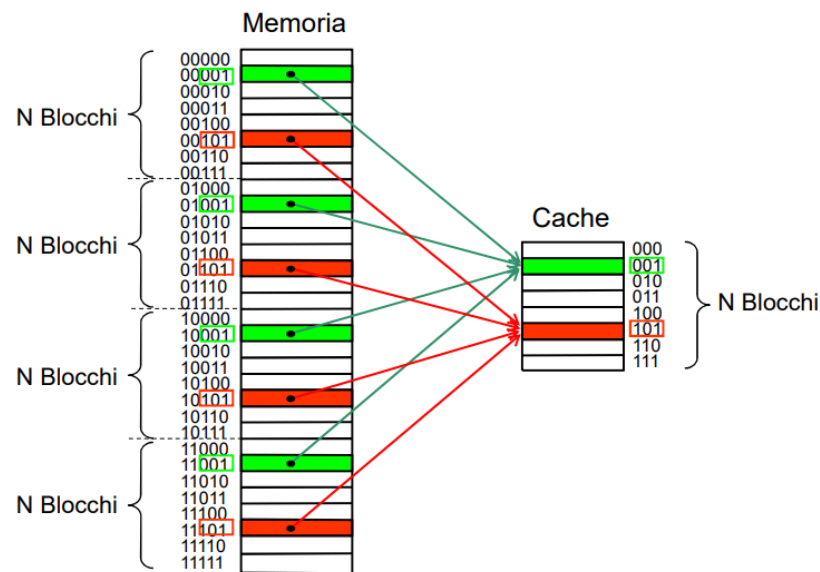
Dopo il trasferimento, si ritenta di nuovo l'accesso in cache, per questo c'è quella sottrazione.

$$Miss_{\text{Penalty}} = T_{\text{Accesso con Miss}} - T_{\text{Accesso con Hit}}$$

$T_{\text{Accesso con Miss}}$  → Numero di cicli di clock per prelevare i dati dalla RAM e inserirli in cache.

$T_{\text{Accesso con Hit}}$  → Numero di cicli di clock per prelevare i dati dalla cache.

## Cache ad accesso diretto



Ad ogni indirizzo di memoria corrisponde una ed una sola posizione nella cache. Quindi un blocco memorizzato in cache, sarà sempre nello stesso identico posto in cache.

La posizione in cache è spesso derivata in modo deterministico dall'indirizzo stesso: di solito si prendono alcuni dei bit dell'indirizzo o si applica una funzione modulo/hash.

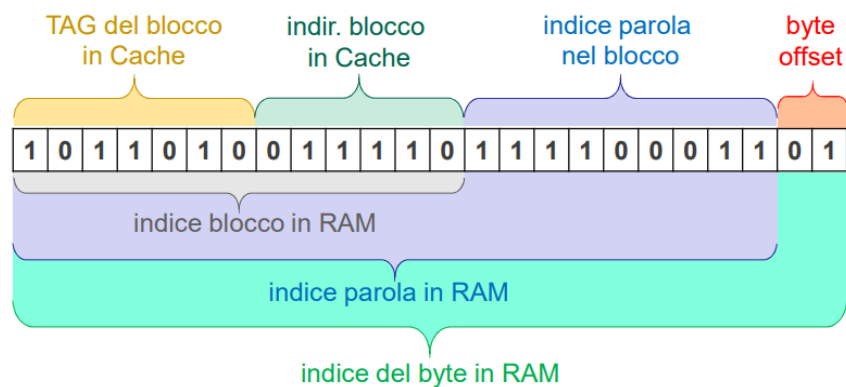
### Conflitto

Quando più blocchi di memoria vengono inseriti nella stessa posizione in cache. Questa problematica può portare ad un grave calo di performance: le miss di conflitto.

### La parola

La dimensione della parola è data dalla dimensione del bus che collega la cache al processore. La dimensione del bus è la dimensione tipica di un operando e quindi di un registro (*quindi la dimensione delle istruzioni influenza la dimensione della parola*).

La dimensione del bus esterno in alcuni casi è minore rispetto alle parole; perchè sul bus esterno abbiamo maggiore probabilità di guasto, quindi meno bit abbiamo, meno probabilità di errore/guasto c'è.



Supponiamo di avere:

- RAM di 16 MegaByte ( $2^{24}$ ).
- Parole da 32 bit.
- Blocchi di 512 parole.
- Memoria cache di 128 KiloByte ( $2^{15}$ ).

La parola verrà suddivisa nel seguente modo:

### Index

I 6 bit più significativi dopo il tag rappresentano la posizione del blocco in cache.

### Tag

I 7 bit più significativi rappresentano il TAG del blocco. Serve a capire se il blocco è quello a cui si vuole effettivamente accedere o se è un altro blocco con stesso indice (conflitto).

### Offset

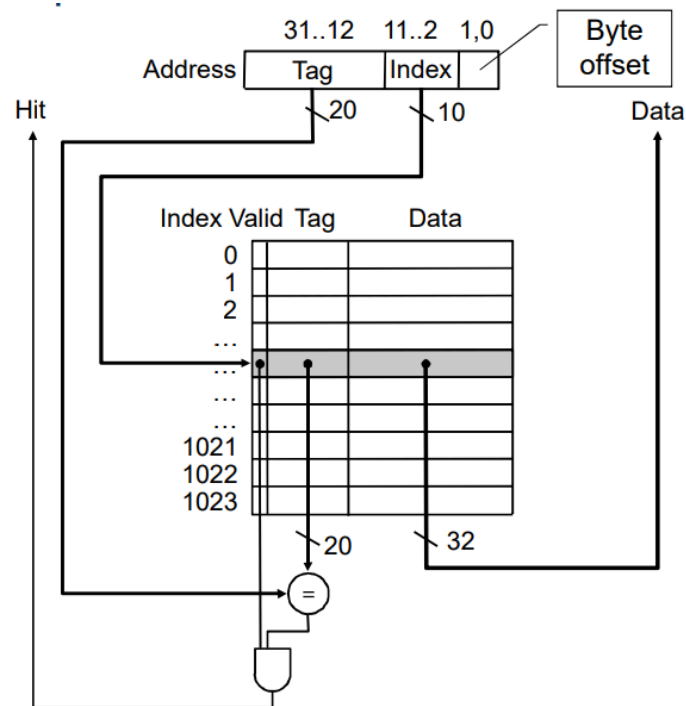
I rimanenti 2 bit meno significativi indicano l'indirizzo del byte all'interno del blocco.

### Validity Bit (v)

Indica se il blocco in RAM è uguale al blocco in cache. Le informazioni in cache non sempre sono buone, anzi possono essere non valide o non aggiornate.

V indica se l'attuale contenuto del blocco di cache ha il dato aggiornato oppure no.

## Accesso ad un blocco - Cache ad accesso diretto



1. I fili di indirizzo della cache prendono come valore il campo `index` della parola e quindi in output la cache restituisce il blocco corrispondente.
2. Il comparatore confronta i primi 20 bit dell'indirizzo di memoria con il campo `tag` del blocco in cache: Se sono uguali allora il blocco voluto potrebbe essere contenuto in cache; altrimenti **miss**.
3. Se  $V = 1$  il blocco in cache è quello voluto ed è anche aggiornato: **hit**.

|        | Valid | Tag | Data |
|--------|-------|-----|------|
| 0 (00) | 1     | 10  | xyz  |
| 1 (01) | 1     | 11  | aaa  |
| 2 (10) | 1     | 10  | bbb  |
| 3 (11) | 1     | 00  | ccc  |

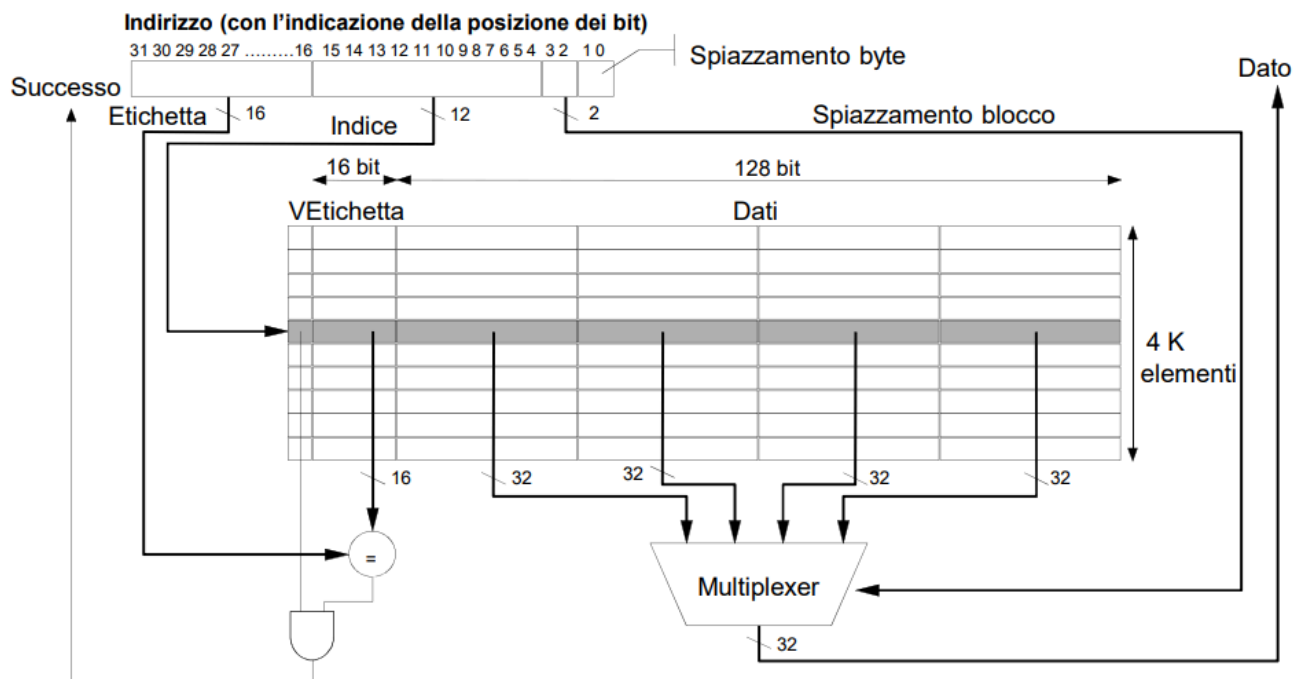
## Aggiornare un blocco - Cache ad accesso diretto

Per aggiornare un blocco M in cache basta forzare due miss (provocare 2 miss dovute ad un conflitto).

1. Si accede ad un blocco K; con lo stesso index di M ma diverso tag.
2. **Causerà una miss** dato che il tag è diverso.
3. La miss si risolve:
  - La copia del blocco M in cache viene salvata in RAM (solo se è stato modificato).
  - Viene copiato il blocco K in cache.
4. Vogliamo rimettere M in cache, quindi si accede al blocco M.
5. **Causerà una miss** dato che il tag di K è diverso da quello di M.
6. La miss si risolve:
  - La copia del blocco K in cache non viene salvata in RAM (non è stato modificato).
  - Viene copiato il blocco M in cache.

## Cache ad accesso diretto con molteplici blocchi

In questo caso ogni riga della cache contiene 16 blocchi di 4 byte ciascuno; nella parola si specifica anche il singolo blocco voluto.



## Cache e pipeline

La gestione di una cache miss è uguale alla gestione di un load-use hazard:

1. La pipeline stalla.
  - *Si rischia il cambio di contesto del thread.*
2. Viene risolta la miss (*prelievo del blocco di memoria dal livello superiore della gerarchia di cache*).
3. Quando il blocco è diventato disponibile, la pipeline:
  - Ripete il ciclo di fetch se la miss è stata causata da un'istruzione.
  - Completa l'accesso in memoria se la miss è stata causata da un operando in memoria.

### Overlap delle miss

Per mitigare questo problema si utilizza questa tecnica: **Nel mentre che si risolve la miss la cache non si blocca ma accetta una nuova richiesta nella speranza che vengano prodotte attività all'interno del microprocessore.**

## Tipologie di cache-miss e come migliorarle

$$T_{\text{medio accesso cache}} = Hit_{\text{time}} + Miss_{\text{Penalty}} * Miss_{\text{Rate}}$$

Il nostro obiettivo è quello di minimizzare la formula sopra. Vediamo le tipologie di miss e per ognuna come ridurre il loro miss rate.

### Compulsory miss (*cold start o migrazione di processo*)

Accadono quando l'immagine di un programma non è presente in cache e quindi quasi ogni operazione di quel programma produrrà una miss; può accadere quando:

- Un programma viene eseguito per la prima volta.
- Un programma non viene eseguito da tanto tempo;
- Alla chiamata di una funzione.
- Durante la gestione di un interrupt.
- A seguito di un page fault.

Si può aumentare la dimensione dei blocchi?

Aumentando la dimensione del blocco si riducono queste miss: impiegheremo meno accessi alla memoria per caricare l'immagine del programma in esecuzione.

Tale diminuzione avviene perché il principio della località ha due componenti: la località temporale e la località spaziale. Blocchi più grandi sfruttano meglio la località spaziale.

Tuttavia, all'aumentare della dimensione dei blocchi:

- Aumenta il miss penalty.
  - Non c'è alcun beneficio nel ridurre il miss rate se ciò comporta un aumento del tempo medio di accesso alla memoria.
- Potrebbero aumentare le miss di conflitto e le miss di capacità:
  - **Conflitto**: avendo meno blocchi è più probabile che avvenga un conflitto.
  - **Capacità**: avendo meno blocchi ma più grandi riempio la cache in meno tempo.

Come misurare/stimare il numero di compulsory miss

Si conta il numero di blocchi unici che vede il programma.

Perché l'immagine della memoria riferita dagli indirizzi in questione non è ancora presente in cache.



## Capacity miss

Accadono quando la cache non riesce a contenere tutti i blocchi del programma:

- Strutture dati molto grandi.
- Lunghezza estrema del codice.

Si può aumentare la dimensione della cache?

Il modo per ridurre le miss di capacità (e anche quelle di conflitto) è aumentare la capacità della cache.

Gli svantaggi:

- Aumento dello hit time.
- Aumento dei costi.
- Aumento del consumo di energia.

Come misurare/stimare il numero di miss di capacità

Possono essere stimate simulando l'esecuzione di un programma in un sistema avente un cache fully associative.

Oltre alle compulsory miss, le cache fully associative, per loro natura (*non possono avere miss di conflitto secondo la definizione che abbiamo dato*), possono produrre solo miss di capacità visto che un blocco di memoria può essere mappato in un blocco qualsiasi della cache.

## Conflict miss

Dovuto ad accessi a locazioni della RAM mappate nella stessa locazione della cache (*stesso indice*).

“Ognuno caccia via l'altra” poiché un accesso ad una produce una miss che distrugge la copia dell'altra.

La soluzione a questo tipo di cache la vedremo in seguito con le cache set-associative.

Come misurare/stimare il numero di miss di conflitto

Possono essere stimate eseguendo il programma su un sistema avente una cache n-ways set associative.

Il numero di miss di conflitto è dato dalla differenza tra le miss totali osservate sul sistema con cache set-associative e le miss totali osservate quando si è fatto girare il programma su un sistema avente una cache fully associative.

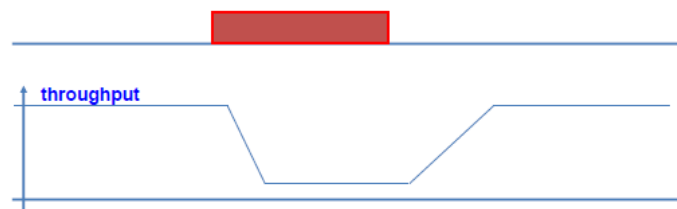
## Distribuzione e conseguenze delle Miss

Supponiamo che ogni 100 accessi in memoria si hanno 2 miss.

Contrariamente a quanto si possa pensare, avvengono di solito tutte in un breve intervallo di tempo. Di conseguenza durante questo intervallo il processore è lento.



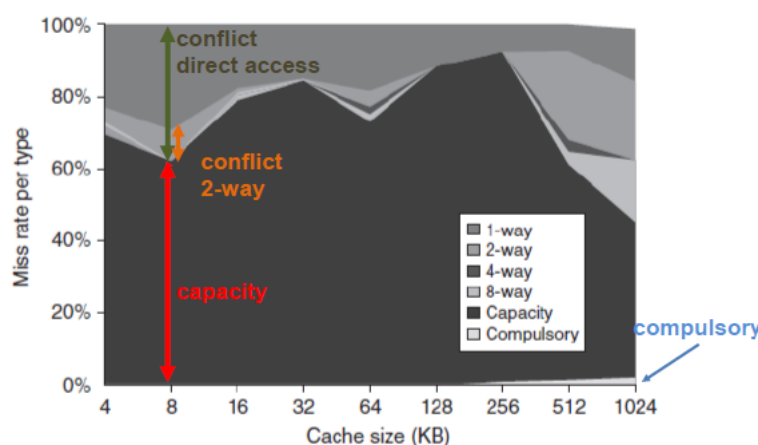
Nella figura sotto vediamo che durante l'intervallo di tempo in cui abbiamo molte miss le prestazioni della CPU crollano, questa irregolarità può essere migliorata.



La conseguenza delle miss è un calo del throughput (*più del 90%*).

La probabilità di miss in caso di read è importante ma è molto difficile cambiarla; molto meglio anzi lavorare sulle write, ossia scegliere tra una strategia tra write-through e write-back.

La seconda ha il difetto che deve per forza modificare tutto il blocco anche se è stato modificato (*update*) solo una volta.

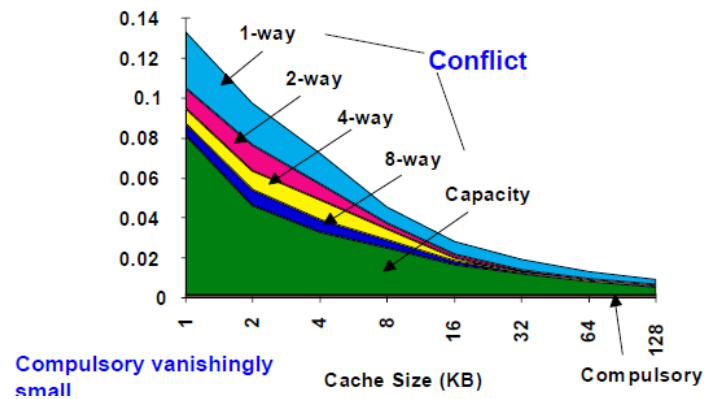


Dal grafico sopra si nota come varia la ripartizione delle miss variando la dimensione della cache.

- Le miss rosse (*quelle di capacità*) non sono costanti.

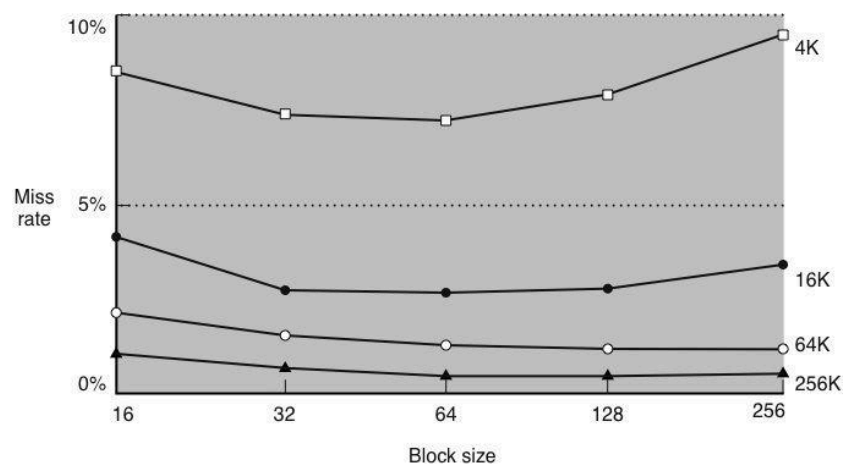
- Le miss rosse e verdi sono quelle ad “accesso diretto”.

Raddoppiando la dimensione della cache diminuiscono le miss di conflitto ma rimangono quelle di capacità.



Nel grafico sopra si nota come:

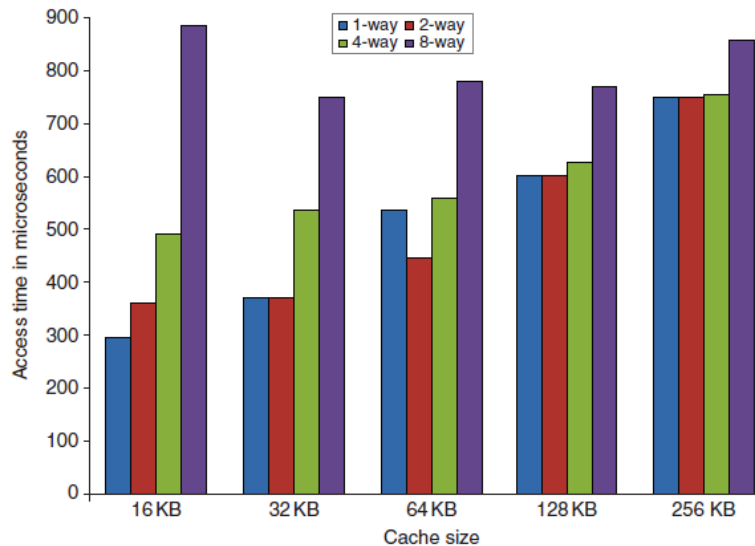
- All'aumentare della capacità della cache le miss di conflitto diminuiscono ovviamente e anche la probabilità di miss complessiva.
- Diminuendo la memoria aumentano in modo non lineare le miss di conflitto.



Nel grafico sopra si nota come la probabilità di miss nella cache di 1° livello cambia all'aumentare della dimensione del blocco di cache.

Aumentare la dimensione del blocco favorisce la località spaziale; ma se la aumentiamo fino ad un punto tale che abbiamo pochi blocchi, allora la località temporale è fortemente penalizzata in questa situazione e il miss rate aumenta di nuovo.

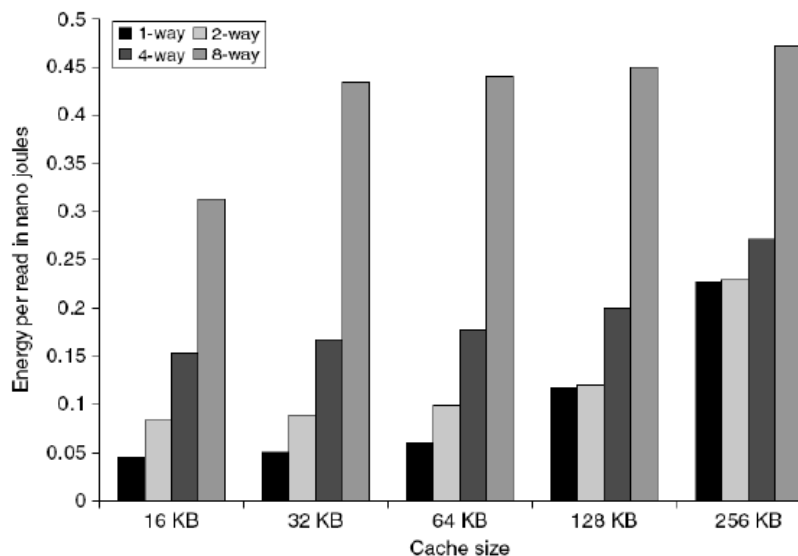
Inoltre l'aumento della dimensione del blocco aumenta anche miss penalty (*blocchi più grandi richiedono più tempo per essere trasferiti sul bus*).



Nel grafico sopra vediamo (a parità di tecnologia ma con differenti dimensioni e numeri di vie) quanto cambia il tempo di accesso.

**Come mai a blocchi di 256KB le differenze si appiattiscono?**

All'aumentare della dimensione i tempi di accesso aumentano e quindi dato che la parte di comparazione rimane uguale a prima, incide sempre meno e diventa sempre meno rilevante rispetto al tempo di accesso effettivo.



Nel grafico sopra vediamo il consumo di energia elettrica: Più vie ho e più consumo energia elettrica; infatti con una cache ad 8 vie ho consumi troppo alti.

## Algoritmi di rimpiazzamento

Per liberare blocchi di cache ci sono varie tecniche che consentono di buttare i blocchi che non utilizzeremo.

I blocchi scartati non si buttano subito via, vengono messi in un'altra cache (*una di livello inferiore, più lenta ma più capiente*), perché non si sa mai che possano servire.

### LRU - Least Recently Used

LRU richiede un ordinamento sui blocchi per funzionare, il problema è che va effettuato durante ogni accesso in hit.

Con molti blocchi risulta inefficiente, inoltre molto complicata.

### pseudo-LRU (FIFO)

Imita LRU ma in modo molto più semplice e veloce.

### Random

Il blocco da scartare si sceglie in modo randomico. Usata spesso nelle cache fully associative.

## Cache Fully Associative

Ogni blocco di memoria può essere memorizzato in qualsiasi posizione in cache.

Non esiste una relazione fissa tra l'indirizzo di memoria e la posizione in cache. Per questo motivo occorre utilizzare moltissimi comparatori (*uno per ogni riga*) perché a priori non possiamo sapere in quale indice di cache si trova il blocco.



La cache Fully Associative ha un'alta efficienza ma è costosa in termini di circuiti poiché si ha un comparatore per ogni entry della cache, ciò aumenta molto il costo dell'hardware.

Perciò le dimensioni di questa cache sono in genere molto piccole ed è usata solamente per casi specifici (*ad esempio il TLB*).

L'algoritmo di rimpiazzamento ha un ruolo cruciale per evitare miss inutili.

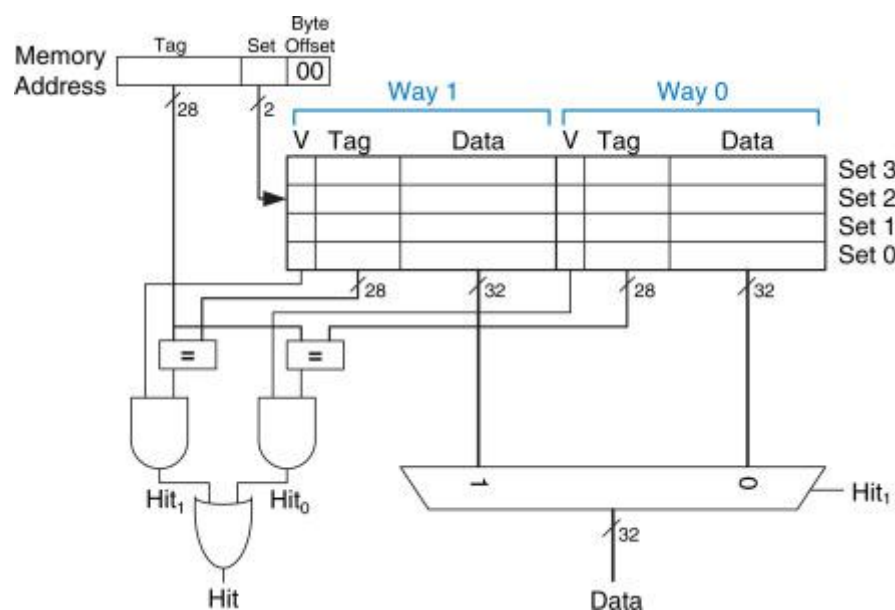
## Cache Set Associative

Detta anche multi-via-direct-mapped è una combinazione dei due approcci precedenti ed è usata per ridurre le miss di conflitto (*blocchi con lo stesso index*).

Un blocco può andare in un unico set; stesso meccanismo delle cache ad accesso diretto, quindi l'indice determina il set.

Ogni set ha molteplici vie. Un blocco può occupare una via qualsiasi; stesso meccanismo delle fully associative.

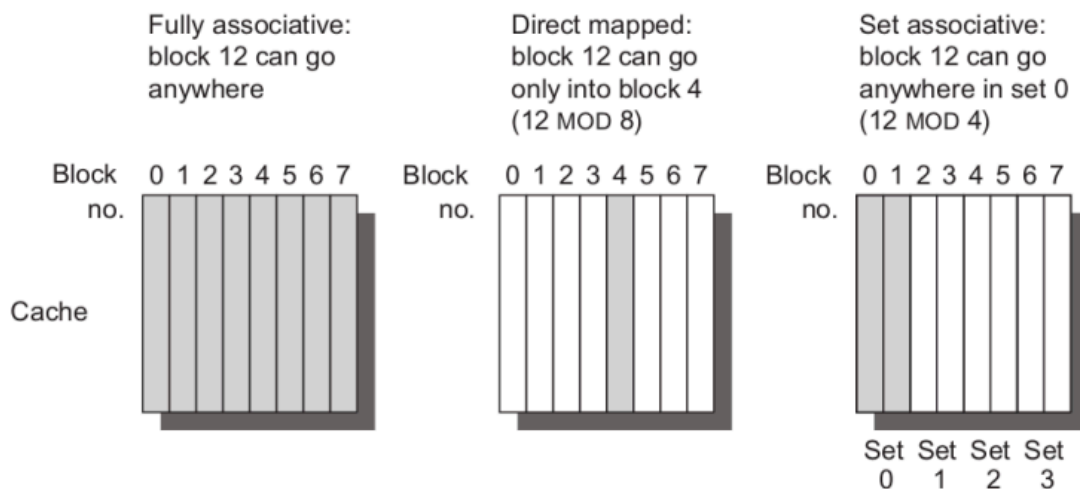
Se il set scelto ha una via libera, il blocco viene memorizzato lì; altrimenti si usa una tecnica di rimpiazzamento come pseudo-LRU.



### Funzionamento di una cache set associative

1. Il set è individuato come in una cache ad accesso diretto.
2. Ogni via del set selezionato sputa fuori il contenuto.
  - Per minimizzare il tempo di accesso: Il campo tag di ogni via del set viene confrontato con il campo tag dell'indirizzo.
  - Per minimizzare il tempo di accesso: il campo V di ogni via del set viene confrontato con il risultato del comparatore.
3. I segnali provenienti dalle AND vanno a pilotare una OR;
  - Restituisce 1 (*cache-hit*) se c'è almeno una via che ha prodotto un blocco con lo stesso tag e con V = 1.
4. Inoltre il segnale in uscita dalla AND che ha prodotto 1 pilota il multiplexer in modo da far uscire i dati voluti.

Una cache di questo tipo è ovviamente più complicata e consuma di più (*mitigata con la way prediction*); quindi non ci si può allargare molto perchè sono molto onerose in termini di energia consumata e tempi di sviluppo



## Way Prediction

Per verificare una hit si deve accendere tutte le vie, questo consuma corrente.

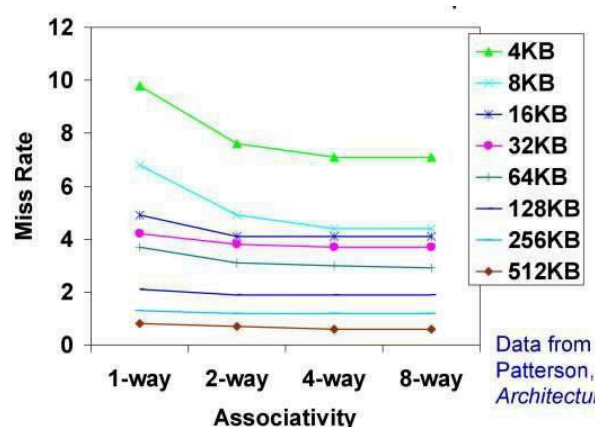
Per risparmiare energia si può accendere (*far scorrere corrente*) una via sola, quella più probabile.

Questa cosa serve anche per settare i multiplexer in anticipo e quindi abbassa l'overhead.

La predizione può essere fatta con vari algoritmi statistici, nella quale alcuni raggiungono anche più del 90% di accuratezza in una cache a 2 vie.

## Statistiche di miss e numero di vie

Nel grafico sotto abbiamo dei casi reali riguardo alla probabilità di miss in percentuale al variare della dimensione della cache e del numero di vie.



Nella curva **verde** vediamo che aumentando il numero di vie si abbassa la probabilità di miss (*diminuiscono le miss di conflitto*).

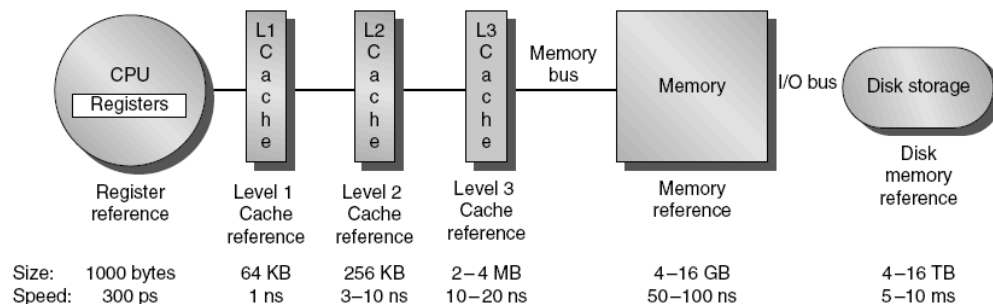
Nella curva **marrone** notiamo che aumentando anche la dimensione della cache il numero di vie diventa sempre meno importante.

Quindi il numero di vie aiuta molto in caso di cache piccole; in caso di cache grandi non ne vale molto la pena.

Da notare anche che all'aumentare del numero di vie la cache è più complessa, più lenta e più costosa poiché per pareggiare le prestazioni richiede hardware più veloce e più costoso.

## Cache Multilivello

Il multilivello risolve il problema dell'impossibilità di una cache grande e molto veloce (*poiché consuma tanto*).



In ogni livello si ha un tipo di cache con velocità e capacità diversa: più il livello è basso (L1) e più le cache sono veloci e piccole.

Le cache di primo livello si concentrano sul minimizzare il hit time e infatti sono contenute nel chip del processore.

Le cache di secondo e terzo livello si concentrano sul minimizzare il miss rate.

$$t_{\text{access}} = t_{\text{hitL1}} + p_{\text{missL1}} \times t_{\text{penaltyL1}}$$

$$t_{\text{penaltyL1}} = t_{\text{hitL2}} + p_{\text{missL2}} \times t_{\text{penaltyL2}}$$

$$t_{\text{access}} = t_{\text{hitL1}} + p_{\text{missL1}} \times (t_{\text{hitL2}} + p_{\text{missL2}} \times t_{\text{penaltyL2}})$$

Le cache multilivello sono anche dette “**cache inclusive**” poiché si basano sull’idea che se una copia di un blocco è presente nella cache di primo livello, allora abbiamo una ulteriore copia anche nella cache di secondo livello e nella cache di terzo livello.

### Victim cache

Sono piccole cache in cui si memorizzano blocchi scartati dall’algoritmo di rimpiazzamento, ed è molto utile per le cache ad accesso diretto.



La cache dipende fortemente dalla sequenza di accessi in memoria, questo dipende dall'algoritmo, come è stato scritto e come è stato tradotto.

## Strategie per hit in scrittura

Supponiamo che nel mentre il controllore della cache determina se l'indirizzo intercettato ha generato una hit o una miss, l'operazione richiesta dal processore viene eseguita sul blocco di memoria acceduto.

- Se l'operazione fosse una read, alla peggio, i dati letti dal processore vengono ignorati con uno spreco di energia ma senza alcuna conseguenza distruttiva.
- Se l'operazione fosse una write, nel caso in cui l'indirizzo abbia generato una miss, la scrittura avrebbe sovrascritto un blocco non interessato dall'operazione.

Perciò una write può essere eseguita solo dopo che il controllore cache abbia decretato hit o miss.

## Write-Through

Con questa tecnica la scrittura attraversa la cache e va direttamente in RAM: quindi sia la cache che la memoria sono sempre aggiornate.

La politica write-through è poco performante: ad ogni scrittura si deve accedere comunque alla memoria principale.

D'altro canto la politica write-through è poco costosa da implementare e permette una gestione della coerenza tra cache molto più semplice in presenza di multiprocessori.

## Write-Back

Le informazioni sono scritte solo nel blocco di memoria cache e le modifiche saranno propagate anche in RAM solo quando il blocco di memoria cache sarà rimpiazzato.

Questo meccanismo necessita di ricordarsi quali blocchi di memoria devono essere aggiornati nel momento in cui devono essere rimpiazzate le cacheline associate: al momento del rimpiazzamento, il controllore della cache legge il bit `dirty` e aggiorna il blocco di memoria se il bit `dirty == 1`.

## Strategie per miss in scrittura

### Write allocate

Il blocco di memoria puntato dall'indirizzo viene allocato in cache e si compiono le operazioni previste dalla politica in caso di hit.

### No-Write allocate

Il blocco di memoria puntato dall'indirizzo viene aggiornato nel livello della gerarchia su cui risiede **ma non allocato in cache**

## Strategie per il caricamento dei blocchi in cache

Le cache (*riguardo questo aspetto*) possono essere di due tipologie:

### Blocking

In caso di miss, la cache non accetta più richieste dalla CPU finché non risolve la miss (*molto più semplice ma meno veloce*).

La miss è risolta quando il blocco è caricato in cache.

### Non-blocking

La cache è molto richiesta da tutti i processi, soprattutto sul primo e sul secondo livello.

**La cache mentre risolve la miss accetta altre operazioni provenienti dalla CPU; in questo modo faccio un overlap delle richieste soddisfacendole in modo parallelo.**

Ciò aumenta molto la complessità della cache ma il risultato è che **viene ridotto molto il miss-penalty**.

### No-Caching

E' possibile proibire ad alcuni programmi di mettere alcuni blocchi in cache.

**Molto vantaggiosa se a certi blocchi accedo solo una volta e poi basta.**

## Prestazioni di una cache

A noi interessa il numero di cicli che utilizza la CPU durante l'esecuzione di un'istruzione.

$$CPU_{Time} = IC * CPI_{Stall} * CC$$

Con

$$CPI_{Stall} = CPI_{Ideale} + MemoryStall_{Cycles}$$

A noi interessa quanto overlap questa struttura (*la cache*) riesce a coprire.

- $MemoryStall_{Cycles}$  → Numero di cicli di clock persi dalla CPU mentre sta attendendo il completamento di una istruzione in memoria RAM.
- $IC$  → Numero totale di istruzioni eseguite durante l'esecuzione del programma.
- $CPI_{Stall}$  → Numero medio di cicli di clock necessari per eseguire un'istruzione.
  - In altre parole, rappresenta l'efficienza con cui il processore esegue le istruzioni.
  - Un CPI basso indica un processore più efficiente, in quanto impiega meno cicli per istruzione.
- $CPI_{Ideale}$  → Valore ideale di cicli di clock per eseguire un'istruzione.
- $CC$  → Numero totale di cicli di clock impiegati per eseguire l'intero programma.

Ad esempio In una cache con write-through:

$$MSC = Accesses * Miss_{Rate} * Miss_{Penalty}$$

$MSC$  → Numero medio di cicli di clock spesi per le cache miss in un programma.

- $Accesses$  → Numero totale di accessi alla cache nel programma.
- $Miss_{Rate}$  → La frequenza con cui si verifica una cache miss, cioè la percentuale di accessi alla cache che risultano in una cache miss.
- $Miss_{Penalty}$  → Numero medio di cicli di clock persi per ogni cache miss.
  - Questo può includere il tempo necessario per caricare i dati dalla memoria principale nella cache e il tempo di stallo del processore durante l'attesa.

Given

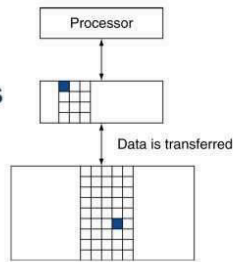
- I-cache miss rate = 2%
- D-cache miss rate = 4%
- Miss penalty = 100 cycles
- Base CPI (ideal cache) = 2
- Load & stores are 36% of instructions

Miss cycles per instruction

- I-cache:  $0.02 \times 100 = 2$
- D-cache:  $0.36 \times 0.04 \times 100 = 1.44$

Actual CPI =  $2 + 2 + 1.44 = 5.44$

- Ideal CPU is  $5.44/2 = 2.72$  times faster



Sintesi sul miglioramento delle performance della cache

### Ridurre lo hit time

- Cache piccola e direct mapped.
- Blocchi piccoli.

### Ridurre il miss rate

- Cache grande e fully-associative con un buon algoritmo di rimpiazzamento.
- Victim cache per i blocchi rimpiazzati da poco.
- Blocchi grandi (*almeno 16 bytes*).

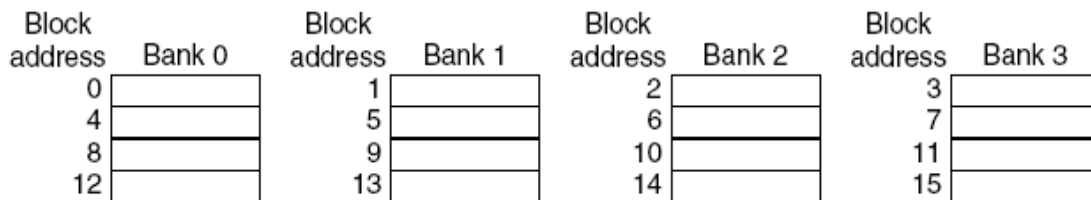
### Ridurre il miss penalty

- Blocchi piccoli.
- Usare molteplici livelli di cache con L1 incorporata alla CPU.
- Victim cache per i blocchi rimpiazzati da poco e eseguire letture anche lì.
- Cache non bloccante.
- Write-back.
- Memory banking.

## Cache multibanked

La cache multibanked è una tecnica di progettazione della cache che prevede la suddivisione della cache stessa in più banchi o sotto-cache.

Ogni banco è una cache a sé (*con molteplici vie o altro*).



E' possibile eseguire operazioni di lettura/scrittura parallele su più banchi contemporaneamente, aumentando così l'efficienza complessiva del sistema.

Le richieste di dati possono essere gestite in parallelo, migliorando la larghezza di banda complessiva della cache e riducendo il tempo di accesso complessivo (*hit time*).

Può ridurre i conflitti di accesso che si verificano quando più richieste di dati cercano di accedere alla stessa linea di cache contemporaneamente, poiché le richieste possono essere smistate tra i banchi, riducendo il rischio di conflitti e migliorando le prestazioni complessive.

## Ottimizzazione usando la cache

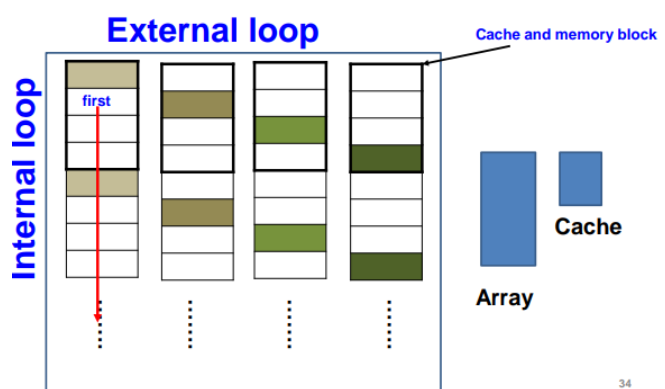
Moltissima ottimizzazione si può ottenere in questo modo. Il problema è che il compilatore non conosce l'organizzazione della cache e a volte può peggiorare la situazione.

Supponiamo di avere due loop, uno interno e uno esterno; tutto per visitare tutte le locazioni dell'array.

Il loop interno accede al primo elemento del primo blocco e poi al primo elemento del secondo blocco e così via.

Il loop esterno avanza di uno e il loop interno accede al secondo elemento del primo blocco e poi al secondo elemento del secondo blocco e così via.

Questa cosa può danneggiare il lavoro della cache perché in cache ci tengo roba che non userò più.

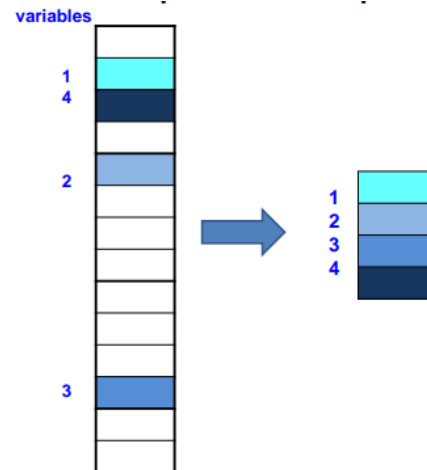


34

In questo caso si dice che l'accesso non è compatibile alla località spaziale. Un programma ha un'ottima località spaziale se una volta portato dentro un blocco uso solo locazioni in quel blocco.

Un modo migliore è accedere più spesso possibile a variabili vicine. Quindi si accede prima ad ogni elemento del primo blocco, successivamente ad ogni elemento del secondo blocco e così via.

Mettere le variabili a cui so che accederai a breve tempo l'una dall'altra il più possibile vicino tra loro e ordinate in base all'ordine in cui avverrà l'accesso.

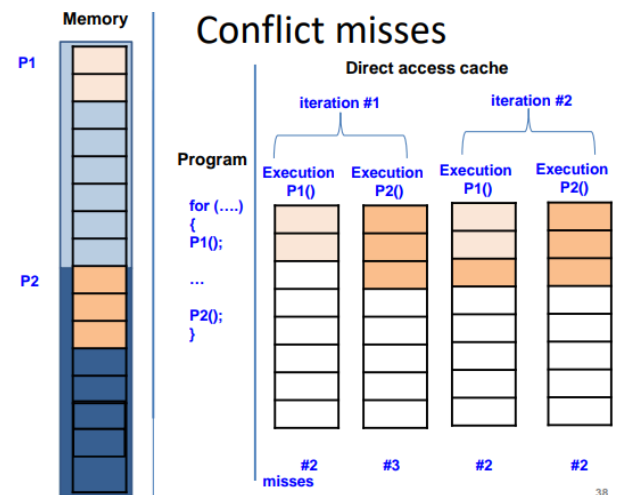


## Migliorare le miss di conflitto

Le zone azzurra e blu sono pagine (il blocco non sempre coincide con la pagina).

Il link editor colloca la funzione  $P1()$  nei primi 2 blocchi e la funzione  $P2()$  nei primi tre blocchi di un'altra pagina.

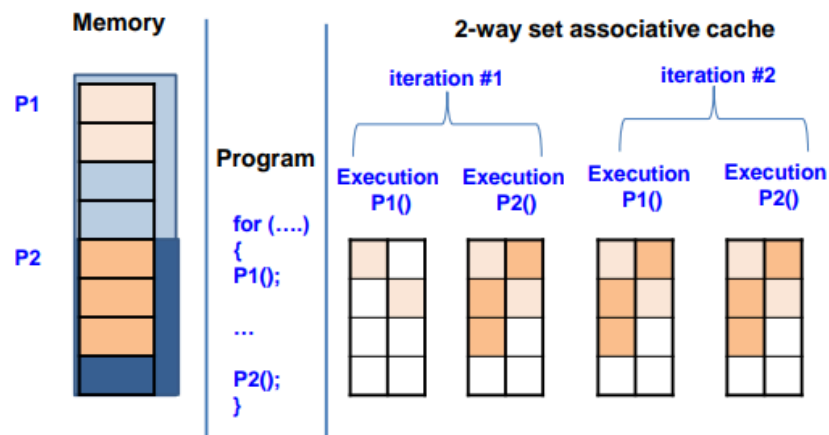
Se abbiamo un programma con un loop nel quale utilizza prima  $P1()$  e dopo  $P2()$  in sequenza:



1. Si portano in cache i due blocchi di  $P1()$ , provoco 2 miss compulsory.
2. Si portano in cache i tre blocchi di  $P2()$ , provoco 3 miss compulsory.
  - a. Si sovrappongono ai due blocchi di  $P1()$ .
3. Viene chiamata  $P1()$ , provoco 2 miss di conflitto.
4. Viene chiamata  $P2()$ , provoco altre 2 miss di conflitto.

Quindi in due iterazioni del ciclo si sono generate in tutto 9 miss, di cui 4 di conflitto e 5 compulsory.

## Soluzione Hardware



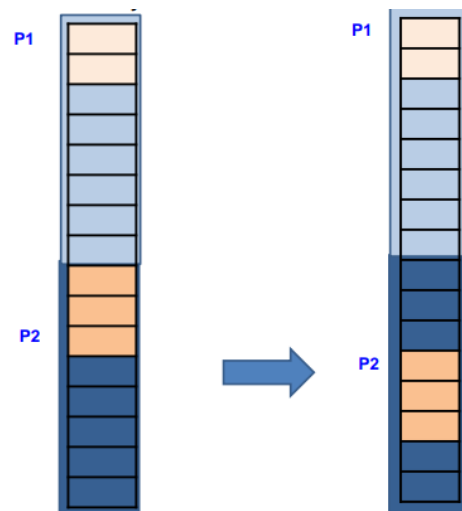
La soluzione hardware è quella di una cache 2-way set-associative.

1. Si portano in cache i due blocchi di  $P_1()$ , provo 2 miss compulsory.
2. Si portano in cache i tre blocchi di  $P_2()$ , provo 3 miss compulsory.
3. Viene chiamata  $P_1()$ , ho 2 cache hit.
4. Viene chiamata  $P_2()$ , ho 3 cache hit.

Quindi in due iterazioni del ciclo si sono generate in tutto 5 miss, di cui 0 di conflitto e 5 compulsory.

## Soluzione Software

Senza utilizzare una cache più complessa, si adotta una soluzione software in cui si sposta l'allocazione dei blocchi nelle pagine, in modo che i blocchi siano in locazioni diverse nelle due pagine (prima entrambi i blocchi iniziavano dalla prima locazione in entrambe le pagine ed essendo che l'indice della pagina è lo stesso in cui viene caricato il blocco in cache, si genera il conflitto).



# Memoria virtuale

Ogni processo vorrebbe tutta la memoria fisica possibile, inoltre vorrebbe che gli indirizzi partissero dallo 0. La virtualizzazione è un trucco per mostrare ai processi le risorse fisiche in modo diverso.

Un requisito è quello di fare in modo che i processi vedano infinita memoria disponibile per loro.

## Protezione

Un processo non può accedere a pagine di altri processi o comunque a lui vietate.

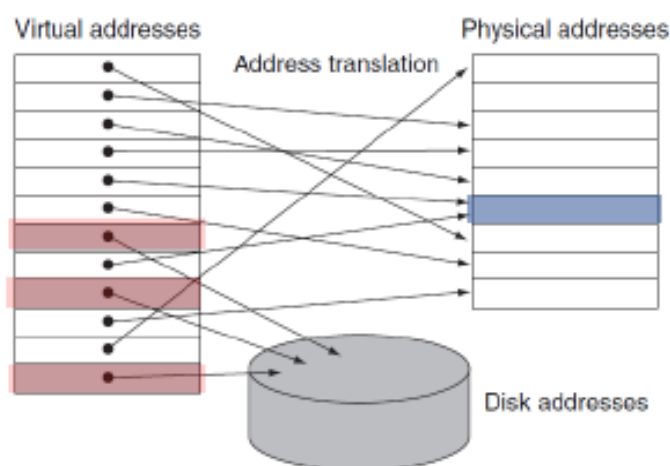
Per questo dentro l'architettura ci devono essere almeno due stati (*user* e *supervisor*), tramite meccanismi come *trap* o *system call* è possibile cambiare stato da *user* a *supervisor*, ovviamente devono esserci istruzioni che consentono il cambio di stato inverso.

## Indirizzi virtuali

Mappiamo le pagine virtuali dei processi nelle pagine fisiche.

Alcune di queste pagine virtuali sono effettivamente nella memoria fisica, però altre potrebbero essere sul disco (*quando la memoria virtuale è più grande della memoria fisica*).

Anche qui occorre avere dei meccanismi per scegliere quale pagina rimuovere dalla memoria per lasciare entrare le pagine che hanno bisogno di essere caricate.

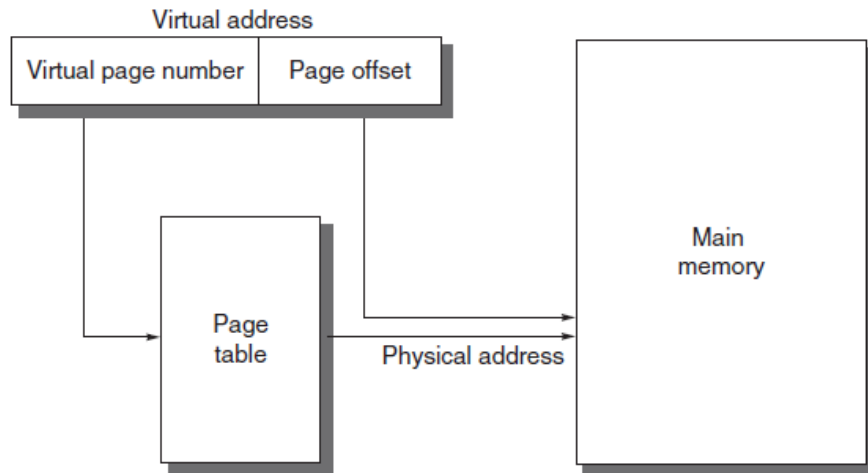


Dal punto di vista pratico divido l'indirizzo virtuale in due parti:

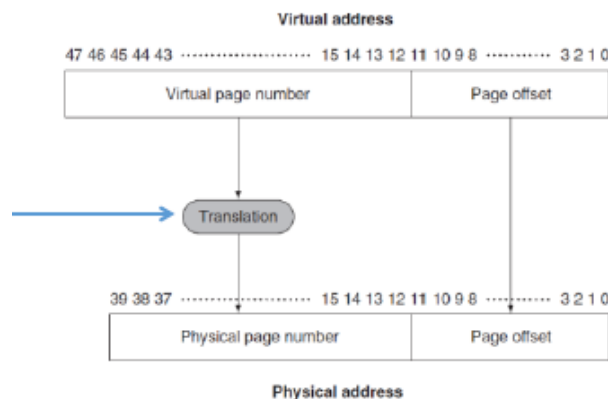
- **Numero della pagina virtuale:** La parte di indirizzo che verrà tradotta da virtuale a fisico e viceversa.



- **Offset nella pagina:** L'indirizzo all'interno della pagina, contenente l'elemento che ci interessa.



La traduzione dell'indirizzo coinvolgerà solo il numero della pagina virtuale, lasciando l'offset inalterato.



## La pagina virtuale

### Page fault

Quando una pagina virtuale non è nella memoria fisica, ma nel disco si genera un **page fault**.

Sono molto costosi e spesso richiedono milioni di cicli di clock per risolversi, perché in caso di page fault si va a recuperare la pagina dal disco.

### Working-set

Il **working set** è l'insieme di pagine che dovremmo avere caricate in memoria virtuale per garantire il minimo numero di page fault durante l'esecuzione di un programma.

Ovviamente non è noto a priori; ma usare il working-set minimizza il numero di hit nel TLB e il numero di page fault.

### La dimensione della pagina

Il nostro obiettivo è quello di **diminuire il numero di page fault**, poiché possono bloccare un processo per milioni di cicli di clock.

Un altro nostro obiettivo è quello di **ridurre la frammentazione** (*spreco di memoria*) interna ed esterna.

In molti sistemi è possibile cambiare la dimensione della pagina; questo aspetto è importante e **dipende dal tipo di processi che dovremo gestire**.

### Molti processi con piccolo spazio di memoria

**Conviene avere pagine più piccole.**

In questo modo, è possibile allocare solo la quantità di memoria necessaria per ciascun processo e quindi **sprecare meno memoria** (*si riduce la frammentazione interna*).

Tuttavia, ciò comporta:

- **Maggiore overhead**
  - Il sistema deve tenere traccia di più pagine, quindi serve un TLB (*hardware*) e una tabella delle traduzioni (*in RAM*) più grandi.
- **Aumento dei page fault**
  - Avendo pagine più piccole è anche più probabile che un processo causi un page fault.
- **Aumento della frammentazione esterna**

### Molti processi con grande spazio di memoria

**Conviene utilizzare pagine più grandi.**

In questo modo, i dati correlati saranno più probabilmente vicini tra loro e **la probabilità di page fault diminuirà**.

Tuttavia, ciò comporta:

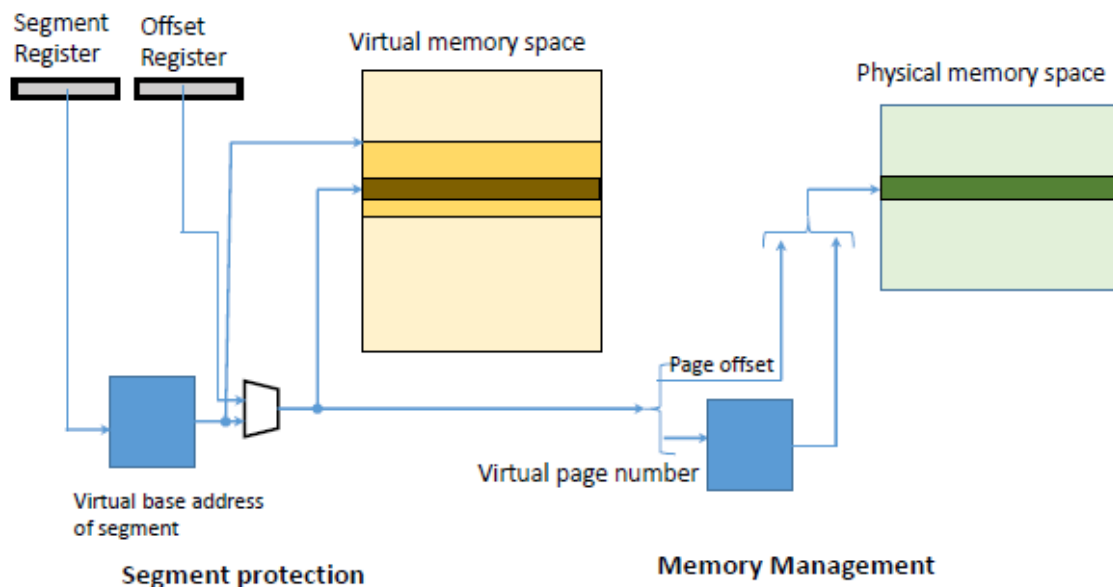
- **Aumento della frammentazione interna**
  - Poiché potrebbe essere allocata più memoria di quella effettivamente utilizzata da ciascun processo.
  - Avendo pagine molto grandi, ogni processo occuperà un grande spazio di memoria RAM.

## Segmentazione

Ogni oggetto del programma viene classificato (*codice, costanti, etc*) e mappato in una sezione (*segmento*) della memoria virtuale a seconda della sua categoria.

Il meccanismo dei segmenti garantisce un'organizzazione della protezione più fine.

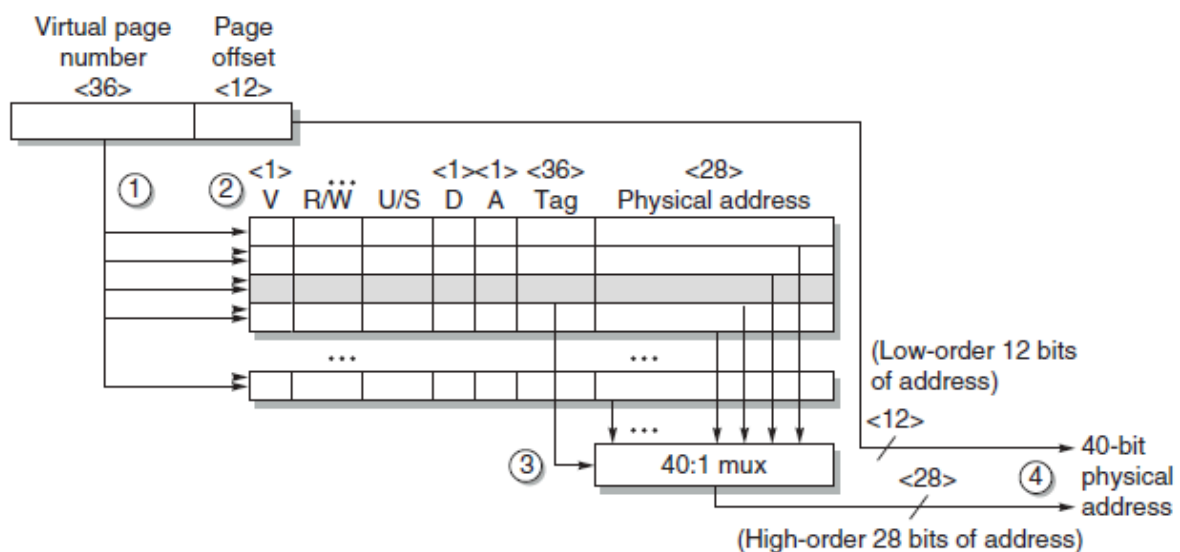
Nella realtà sistemi così non ce ne sono più in circolazione.



## TLB

Per evitare di tradurre più volte lo stesso indirizzo si introduce una cache per le traduzioni, il TLB.

Spesso realizzata con tecnologia fully associative, in quanto piccola in dimensione.



Assumiamo che avvenga una hit.

1. Il `virtual page number` va ad accendere la linea del TLB corrispondente.
2. La linea sputa fuori il `tag` che va a pilotare il multiplexer.
3. Ogni linea sputa fuori il campo `physical address`.
4. Il multiplexer fa uscire il `physical address` giusto.
5. Si concatenano `physical address` e `offset`.

C'è inoltre:

- `V`: Un bit di validità come nella cache.
- `R/W`: Informazioni che indicano cosa può fare chi accede a quella pagina.
- `A`: Informazioni per capire se una pagina è stata acceduta di recente.

### Dimensione del TLB

La dimensione del TLB è data dal blocco di cache.

Una riga del TLB è nominata registro perchè contiene tutte le informazioni relative alla rilocalizzazione di una pagina fisica.

## Cache e TLB

Cache L1 e TLB possono essere disposti in diversi ordini e di conseguenza possono generare diversi pattern d'accesso alla memoria.

### Physically Indexed Physically Tagged

L'indirizzo virtuale generato dalla CPU prima viene intercettato dal TLB (*quindi tradotto da virtuale a fisico*) e poi visto dalla cache.

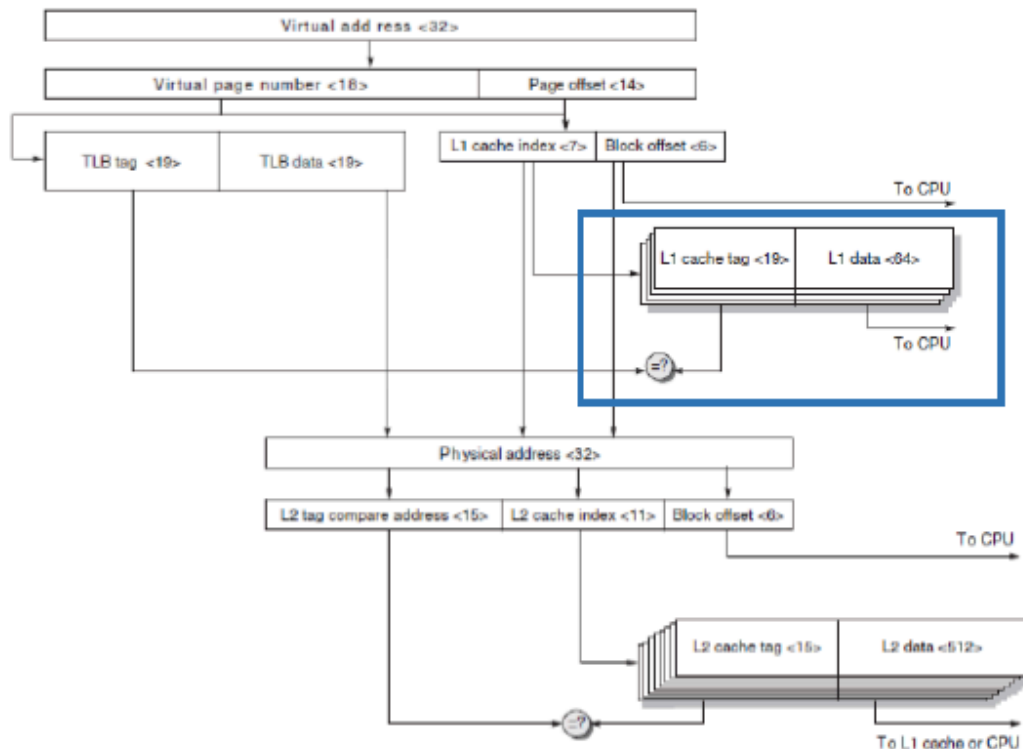
Quindi la L1 vede solo indirizzi fisici.

Molto facile da fare perché la struttura della cache non viene cambiata.

## Virtually Indexed Virtually Tagged

L'indirizzo virtuale generato dalla CPU prima viene intercettato dalla cache (*quindi il campo tag viene controllato usando l'indirizzo virtuale*).

Quindi la L1 vede solo indirizzi virtuali.



In caso di miss in L1, l'accesso al livello sottostante richiede una traduzione dell'indirizzo compiuta dal TLB. Questo approccio presenta due problemi:

### Context switch

Ad ogni cambio di contesto (e quindi avviene anche un cambio della tabella di traduzione) si deve fare il flush della L1 perchè indirizzi virtuali prodotti dal processo entrante mappano in indirizzi fisici diversi dal processo uscente.

Non ci si può fare nulla per risolvere questo problema.

### Aliasing

Supponiamo che due processi condividano un blocco di memoria.

Ognuno ne avrà una copia in cache, ma con indirizzi virtuali diversi e di conseguenza occupando due posti diversi in cache L1.

Se uno dei due processi scrive sul blocco condiviso, in realtà scrive nella sua copia in cache. L'altro processo non vede la modifica, poiché il suo blocco ha indirizzo virtuale e posizione in cache differente.

Per risolvere l'aliasing serve della logica aggiuntiva che aumenterebbe la complessità della cache L1 (e di conseguenza aumenterebbe l'hit time, il punto di forza della L1).

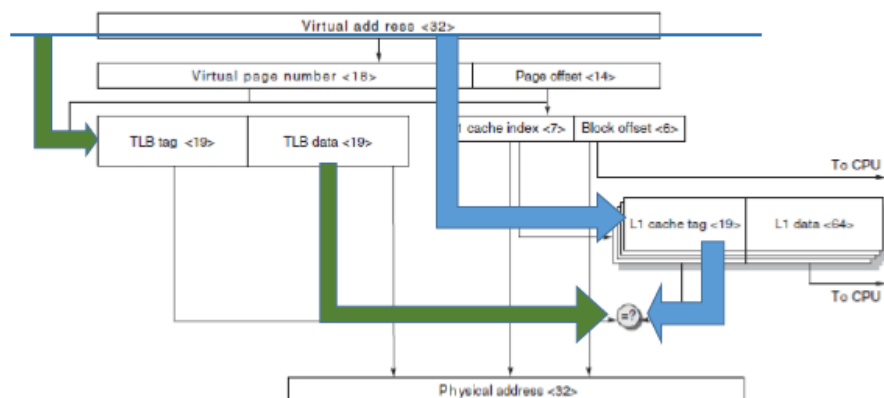
Esempio di Virtually Indexed cache

Tecnica usata nei processori ARM.

Abbiamo un requisito:  $Dim_{Pagina\ Virtuale} \geq Grandezza_{Via}$

Per  $Grandezza_{Via} = \frac{Cache_{Size}}{N_{vie}}$  è la grandezza di una via della cache L1.

Questo vincolo consente di utilizzare i bit dell'offset come indice nella cache (per selezionare la via).



L'idea è di accedere alla L1 solo utilizzando l'offset di pagina virtuale (che non viene tradotto), mentre in parallelo eseguo la traduzione nel TLB.

1. L'offset dell'indirizzo virtuale viene usato come campo `index` nella cache L1.
2. La cache sputa fuori la parte alta dell'indirizzo (`tag`).
3. L'output del TLB viene confrontato con il campo `tag`.
  - Se coincidono, significa che l'indirizzo è corretto e può essere usato direttamente.

## TLB Multilivello

### TLB L1 e L2

Alcuni processori (*come ad esempio Intel i7*) includono due livelli di TLB.

Questi due livelli di TLB svolgono ruoli diversi nella gestione delle traduzioni degli indirizzi virtuali in indirizzi fisici durante l'esecuzione delle istruzioni da parte del processore.

#### L1 TLB

Questo TLB è il più vicino alla CPU e di solito ha dimensioni più piccole e latenze più basse. Come la cache L1 memorizza le traduzioni degli indirizzi virtuali più recentemente utilizzati.

#### L2 TLB

Memorizza un numero più ampio di traduzioni degli indirizzi virtuali con ha latenze leggermente più elevate.

Agisce come un backup per il L1 TLB e fornisce traduzioni aggiuntive degli indirizzi virtuali, aiutando a ridurre il numero di miss nel TLB complessivo del processore.

La località tra dati e istruzioni è diversa, perciò sono diverse anche le loro cache: ad esempio cambiano il numero di vie mantenendo la dimensione costante.

### TLB Dati e Istruzioni

I processori ARM (*come ad esempio Cortex A8*) invece hanno due TLB separati e paralleli.

#### TLB per le istruzioni

Responsabile della traduzione degli indirizzi di memoria virtuali in indirizzi di memoria fisici per le istruzioni del programma.

#### TLB per i dati

Responsabile della traduzione degli indirizzi di memoria virtuali in indirizzi di memoria fisici per i dati (*letture o scritture*).

# MIMD - Multiple Instruction Multiple Data

Architettura di calcolo parallelo in cui più unità di elaborazione (*processori o computer completi*) eseguono simultaneamente diverse istruzioni su diversi set di dati.

Comunemente utilizzate nei sistemi multiprocessore e nei cluster di computer, dove possono gestire compiti complessi e distribuiti, come simulazioni scientifiche, rendering grafico avanzato e analisi di grandi quantità di dati.

## Vantaggi delle MIMD

- **Indipendenza e parallelismo delle istruzioni:** Ogni processore esegue il proprio flusso di istruzioni contemporaneamente agli altri processori.
- **Indipendenza dei dati:** Ogni processore opera su un proprio set di dati, il che facilita l'elaborazione di compiti eterogenei.
- **Scalabilità:** È possibile aggiungere ulteriori processori per aumentare la capacità di calcolo del sistema.

## Svantaggi delle MIMD

- Progettazione del software più complessa.
- Progettazione dell'hardware molto più complessa.

## Tipologie di parallelismo di istruzioni

### Instruction-Level Parallelism (ILP)

Sfrutta il parallelismo all'interno di un singolo flusso di istruzioni, utilizzando tecniche come la pipeline.

### Vector-Level Architecture

Realizzata applicando una singola istruzione ad un insieme di dati in parallelo; ad esempio usata nelle GPU.

### Thread-Level Parallelism

Un insieme di thread che collaborano per eseguire un compito comune sugli stessi dati.

Ad esempio un programma che suddivide un array in N sezioni e ogni sezione verrà elaborata da un thread.



## Overhead introdotto dai thread

I thread possono anche essere utilizzati per sfruttare il parallelismo ma introducono overhead. L'overhead introdotto dalla gestione dei thread è “*giustificato*” se la dimensione del task assegnato a ciascun thread è sufficientemente grande per sfruttare efficientemente il parallelismo.

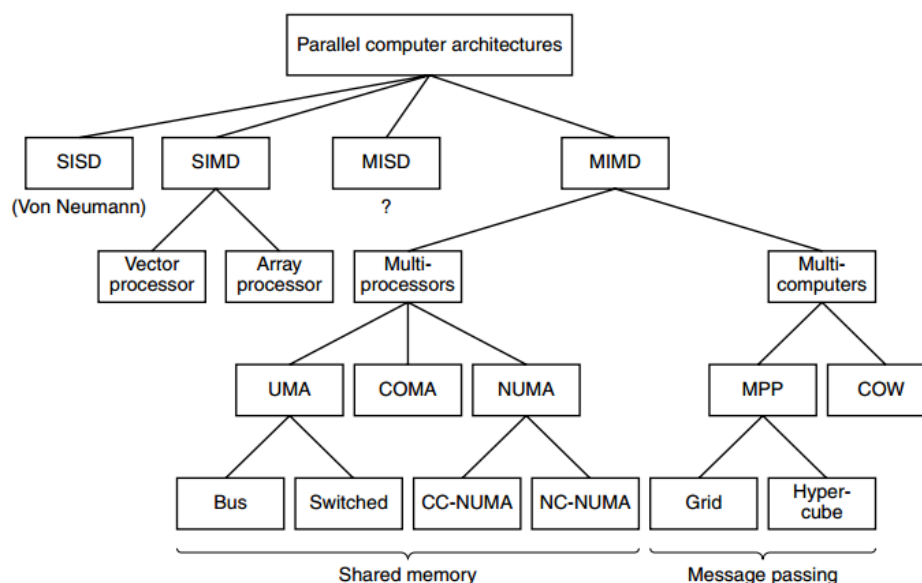
## Request-level parallelism (RLP)

Questa tipologia di parallelismo è comune nelle applicazioni il cui carico di lavoro è dettato dalle richieste.

I thread non lavorano più per un task comune ma per un task individuale, quindi i dati sono separati. Questa tipologia di parallelismo diminuisce il response time delle singole richieste.

Ad esempio un DBMS può sfruttare questa tipologia di parallelismo istanziando un nuovo thread per ogni query in arrivo.

## Multiprocessori e Multicomputer



## Multiprocessori - Accoppiamento stretto

Consentono di avere molteplici unità di elaborazione (*processori*).

Questa cosa permette di avere un sistema più scalabile e performante grazie all'elevato potenziale di parallelizzazione rispetto ad un'architettura con un singolo processore.

## Per cosa è adatta?

Per applicazioni in cui è richiesta una comunicazione frequente e veloce tra i processori, come nel calcolo parallelo o nel calcolo scientifico.

## Limite superiore delle prestazioni

Aumentare il numero di processori non dà sempre vantaggi; infatti c'è un limite superiore a questi vantaggi oltre il quale le prestazioni non crescono più, ma addirittura potrebbero peggiorare.

## Memoria condivisa e comunicazione tra processori

I processori condividono lo stesso spazio di memoria fisica e i collegamenti sono fatti tramite un bus di sistema veloce.

La memoria condivisa permette:

- Comunicazione più veloce (*non serve eseguire lunghe procedure di condivisione*).
- Sincronizzazione tra processori più efficiente.

Il costo è un aumento di complessità dell'hardware.

## UMA - Uniform Memory Access

Ogni processore ha accesso alla memoria principale con lo stesso tempo di latenza, indipendentemente dall'indirizzo a cui si sta accedendo.

## NUMA - Non Uniform Memory Access

Il tempo di accesso alla memoria può variare in base all'indirizzo di memoria richiesto.

In parole povere: l'accesso alla memoria può essere più veloce se i dati sono memorizzati localmente vicino al processore, mentre può richiedere più tempo se i dati devono essere recuperati da una memoria lontana.

## Utilizzazione e migrazione dei processi

In fase di esecuzione va effettuato un bilanciamento dinamico (*load balancing*), ossia una migrazione di processi, per evitare che un processore vada in sovraccarico rispetto agli altri

Nella condizione ideale tutti i processori in una macchina dovrebbero fare qualcosa, non devono essere in stato di idle.

In caso di processori in stato di idle allora si effettua la migrazione di processi in modo da occupare i processori in stato di idle.

## Multicomputer - Accoppiamento lasco

I processori hanno i propri spazi di memoria fisica e comunicano attraverso una rete di interconnessione come ad esempio una LAN.

L'accesso alla memoria è più lento rispetto ai sistemi ad accoppiamento stretto a causa del bisogno di trasferire messaggi attraverso la rete.

### Per cosa è adatta?

Utilizzato in sistemi che devono compiere calcoli altamente distribuiti, in cui i processori sono fisicamente separati e devono comunicare tra loro attraverso una rete.

### Vantaggi

- Molto più scalabile rispetto ai multiprocessori.

### Svantaggi

- Più complesse rispetto ai multiprocessori.
- Lo scambio di messaggi è molto complicato da realizzare.
- Load balancing più complicato da fare rispetto al multiprocessore.

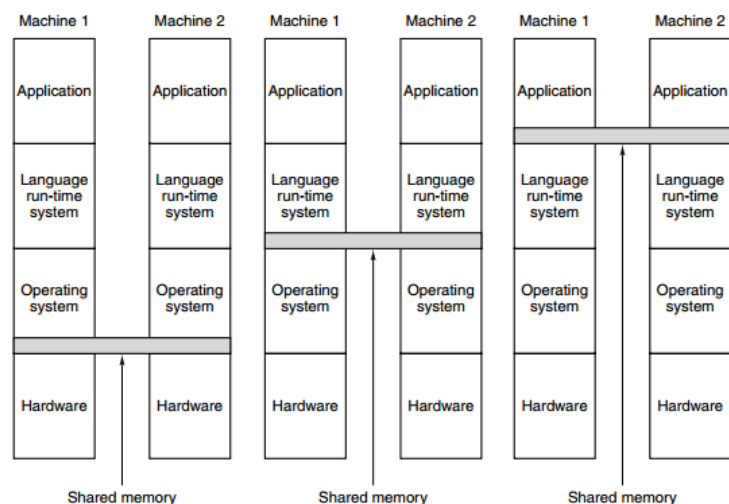
### Posizione della memoria condivisa

Nella figura sotto vediamo tre esempi di piazzamento della memoria condivisa.

Nel blocco a sinistra i processori hanno la memoria condivisa dai sistemi operativi.

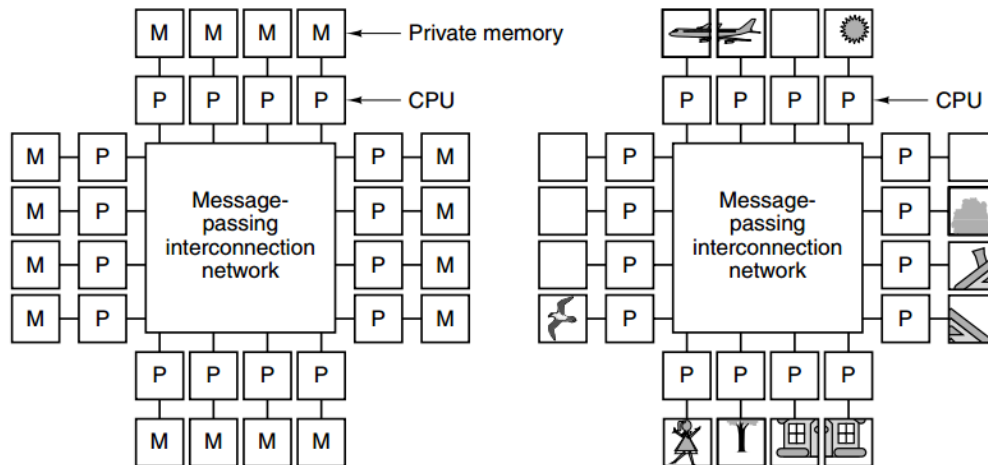
Nel blocco centrale c'è una soluzione ibrida in cui i multicomputer assorbono parte del SO, come ad esempio lo scheduler per gestire i processi su ogni processore.

Nel terzo blocco abbiamo il modello multicomputer reale, le macchine hanno un proprio SO, sopra c'è solo l'applicazione, è la soluzione adottata nei cloud.



### Rete di interconnessione

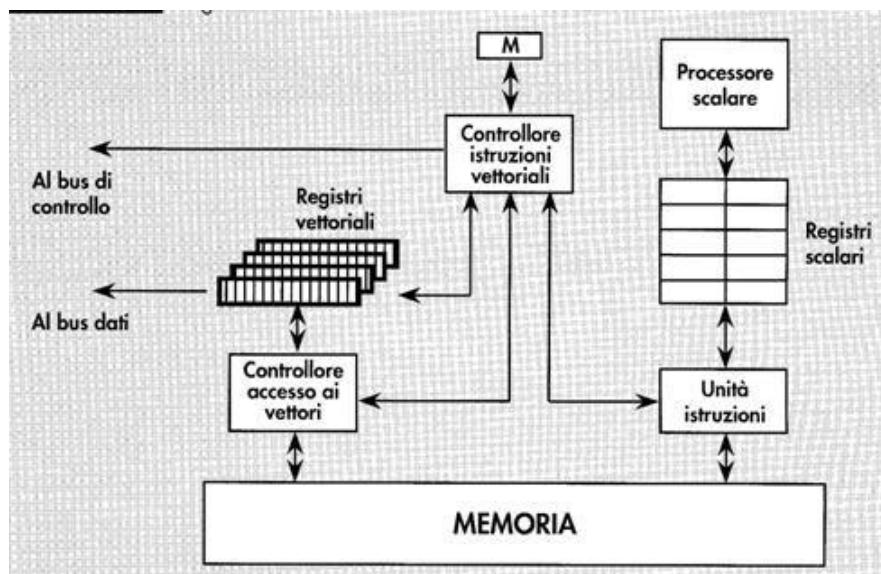
La rete d'interconnessione consente di aumentare la scalabilità ma a quel punto le prestazioni dipendono anche dalla condizione della rete: se aumenta il traffico aumenta anche il tempo di accesso.



## Macchine Vettoriali

Ma in aggiunta si hanno dei registri vettoriali e delle istruzioni speciali che eseguono operazioni vettoriali come somma, sottrazione, moltiplicazione di vettori in un unico ciclo di clock (*non sempre*).

In questo caso non sempre si può prendere un programma scritto per CPU e trasferirlo direttamente in questo tipo di macchine, a volte bisogna “tradurlo”.



## Istruzioni vettoriali

In teoria potrebbero essere eseguite in un solo ciclo di clock, però in realtà le cose non stanno sempre così.

| Instruction | Operands | Function                                                       |
|-------------|----------|----------------------------------------------------------------|
| ADDVV.D     | V1,V2,V3 | Add elements of V2 and V3, then put each result in V1.         |
| ADDVS.D     | V1,V2,F0 | Add F0 to each element of V2, then put each result in V1.      |
| SUBVV.D     | V1,V2,V3 | Subtract elements of V3 from V2, then put each result in V1.   |
| SUBVS.D     | V1,V2,F0 | Subtract F0 from elements of V2, then put each result in V1.   |
| SUBSV.D     | V1,F0,V2 | Subtract elements of V2 from F0, then put each result in V1.   |
| MULVV.D     | V1,V2,V3 | Multiply elements of V2 and V3, then put each result in V1.    |
| MULVS.D     | V1,V2,F0 | Multiply each element of V2 by F0, then put each result in V1. |
| DIVVV.D     | V1,V2,V3 | Divide elements of V2 by V3, then put each result in V1.       |
| DIVVS.D     | V1,V2,F0 | Divide elements of V2 by F0, then put each result in V1.       |
| DIVSV.D     | V1,F0,V2 | Divide F0 by elements of V2, then put each result in V1.       |
| LV          | V1,R1    | Load vector register V1 from memory starting at address R1.    |
| SV          | R1,V1    | Store vector register V1 into memory starting at address R1.   |

Si vuole eseguire questa operazione:  $Y = (a * X) + Y$  dove  $Y$  e  $X$  sono due vettori di numeri interi senza segno.

Nell'immagine sotto vediamo la soluzione scalare; la quale richiede un loop per scorrere tutti gli elementi degli array  $Y$  e  $X$ .

```

        L.D F0,a          ;load scalar a
        DADDIU R4,Rx,#512 ;last address to load
Loop:   L.D F2,0(Rx)      ;load X[i]
        MUL.D F2,F2,F0    ;a × X[i]
        L.D F4,0(Ry)      ;load Y[i]
        ADD.D F4,F4,F2    ;a × X[i] + Y[i]
        S.D F4,9(Ry)      ;store into Y[i]
        DADDIU Rx,Rx,#8   ;increment index to X
        DADDIU Ry,Ry,#8   ;increment index to Y
        DSUBU R20,R4,Rx   ;compute bound
        BNEZ R20,Loop     ;check if done

```

Nell'immagine sotto vediamo la soluzione vettoriale.

```

        L.D F0,a          ;load scalar a
        LV V1,Rx          ;load vector X
        MULVS.D V2,V1,F0   ;vector-scalar multiply
        LV V3,Ry          ;load vector Y
        ADDVV.D V4,V2,V3   ;add
        SV V4,Ry          ;store the result

```

### Implementare la logica vettoriale - Soluzione sequenziale

Un processore vettoriale dispone di hardware che permette di eseguire il ciclo in  $N$  cicli di clock ma con una sola istruzione.

Ad ogni ciclo di clock fa la somma degli elementi  $i$ -esimi dei vettori  $A$  e  $B$  tramite una ALU.

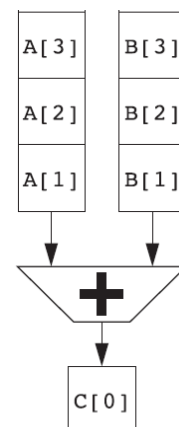
Utilizziamo 3 registri in tutto.

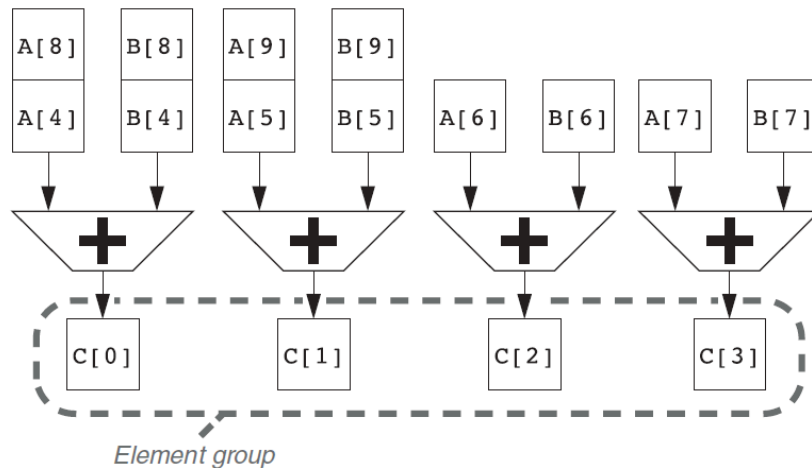
### Implementare la logica vettoriale - Soluzione Ibrida

La soluzione a 4 cicli di clock consente di parallelizzare i circuiti che servono per utilizzare i registri.

In questo esempio ci servono 12 registri (8 per gli input e 4 per l'output), e quindi conseguentemente servirebbero 12 bus.

Se i registri necessari fossero in numero maggiore, il numero di bus sarebbe troppo grande e non realizzabile poiché più soggetto a guasti e richiederebbe più area nel chip del processore.

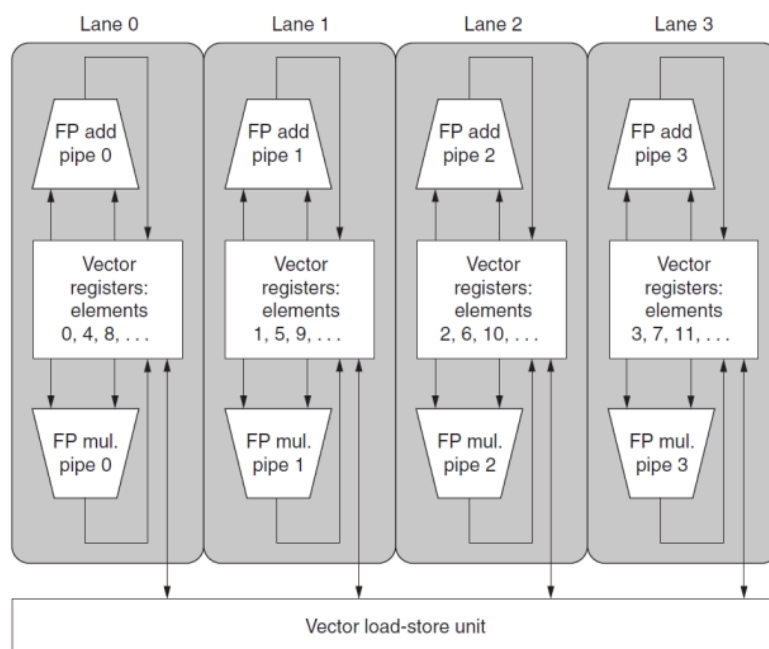




Questa è l'architettura più utilizzata ed è molto simile alla GPU. Da notare che è un'architettura SIMD.

### Come funziona l'hardware di questa soluzione

La memorizzazione nel registro vettoriale  $C$  è divisa in corsie: ciascuna corsia contiene uno dei quattro elementi del registro vettoriale:  $C[0]$ ,  $C[1]$ ,  $C[2]$  e  $C[3]$ .



La figura sopra mostra le quattro corsie: Ogni corsia  $i$  contiene 3 unità funzionali vettoriali:

- FP add  $i$ .
- FP mul  $i$ .
- Unità di load/store, ossia il registro  $C[i]$ .

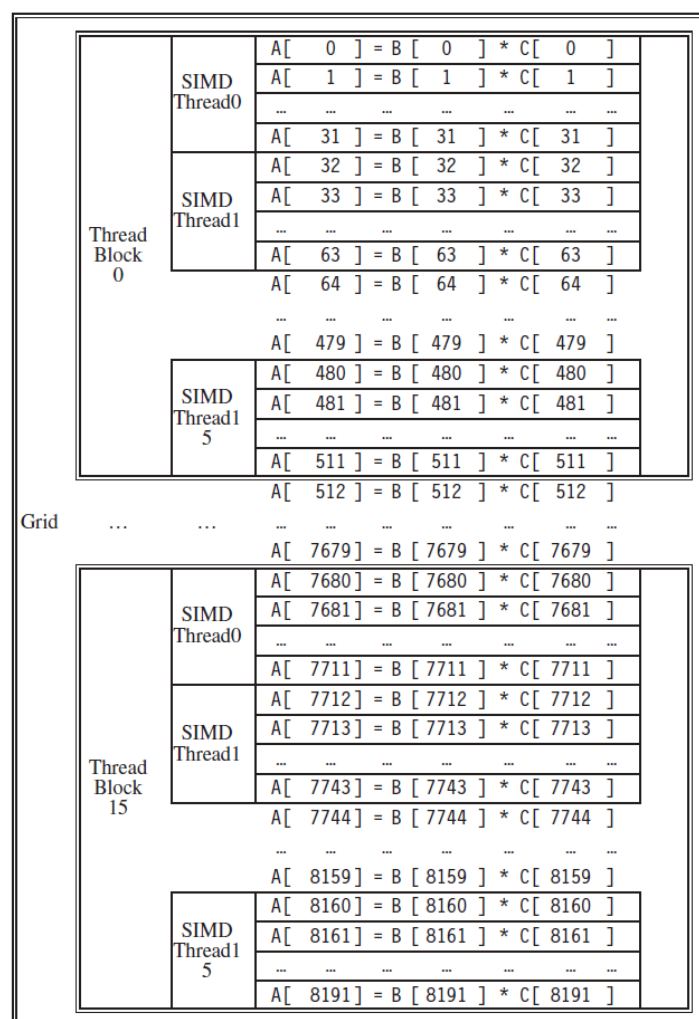
Questi quattro circuiti agiscono allo stesso modo per completare una singola istruzione vettoriale.

Nota come ciascuna sezione del file dei registri vettoriali deve fornire solo porte sufficienti per le pipeline locali nella sua corsia.

## Mappatura a griglia nelle GPU

I principali problemi delle macchine vettoriali sono la latenza delle load/store i quali sono pesanti e pertanto si implementa a banchi.

La memoria della GPU è partizionata, quindi il numero di elementi che possono accedere ad un'area di memoria è fortemente limitato.





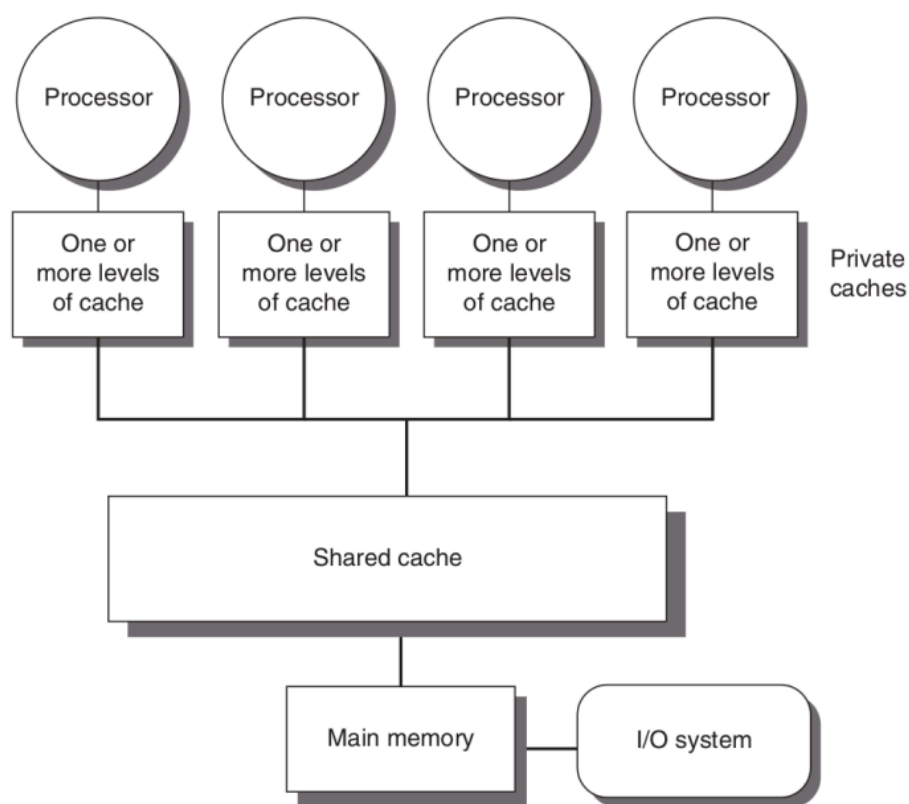
# Multiprocessori

Sistemi composti da un insieme di processori strettamente accoppiati ed il cui controllo è affidato ad un singolo sistema operativo.

I processori condividono tra loro la memoria (*quindi possiamo parlare di un sistema a memoria condivisa*) attraverso l'utilizzo dello stesso spazio d'indirizzamento.

Notare che condividere lo spazio d'indirizzamento non significa condividere necessariamente la stessa memoria fisica.

## SMP - Symmetric MultiProcessors



Tutti i processori sono fisicamente uguali e hanno la stessa vista.

I processori condividono una memoria centralizzata a cui tutti i processori hanno accesso, da qui il termine "*simmetrico*".

Negli odierni chip multicore (*ovvero nelle moderne CPU*), la memoria è condivisa in modo centralizzato tra i core il che rende tutti i multicore esistenti SMP.

## UMA - Uniform Memory Access

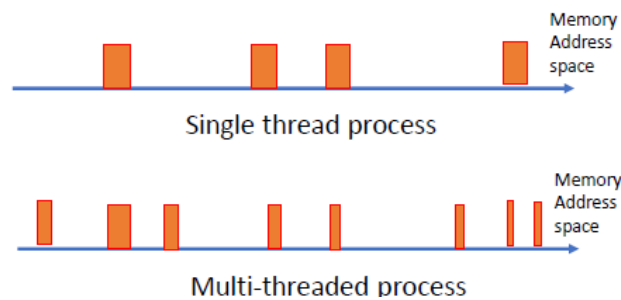
Le architetture SMP sono talvolta chiamate anche multiprocessori con accesso uniforme alla memoria (*Uniform Memory Access, UMA*), derivante dal fatto che tutti i processori hanno un tempo d'accesso alla memoria uniforme.

In questo caso l'accesso alla cache condivisa è UMA, ovvero tutti i processori hanno un ritardo uniforme negli accessi in memoria.

### Accesso alla memoria condivisa e prestazioni

Le miss dell'ultimo livello di cache causano un'operazione nella memoria condivisa, e sono queste che causano un grosso abbassamento del throughput.

Un problema dei thread è che ogni thread visita aree distinte di memoria (*ad esempio per prelevare i dati*), ciò può provocare miss di conflitto ed è per questo che abbiamo cache n-way.



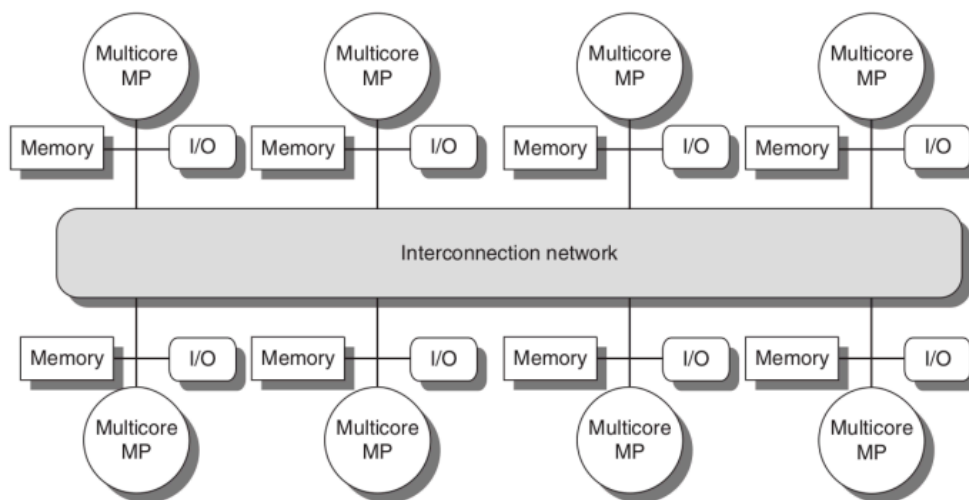
I thread agiscono su dati diversi quindi le aree visitate dall'intero programma in quell'intervallo di tempo sono piuttosto sparse, come ben sappiamo questa cosa può causare un aumento delle miss di conflitto.

## DSM - Distributed Shared Memory

L'approccio alternativo a SMP consiste in multiprocessori con memoria fisicamente distribuita.

Il termine “*memoria condivisa*” si riferisce al fatto che lo spazio degli indirizzi di memoria è condiviso tra i processori.

Ciò significa che ogni processore può accedere a qualsiasi locazione di memoria all'interno dello spazio degli indirizzi condiviso, a prescindere dalla memoria fisica in cui effettivamente risiede il dato.



### Vantaggi di un sistema DSM

- **Maggiore scalabilità**
  - Per supportare un numero maggiore di processori, la memoria deve essere distribuita anziché centralizzata.
  - Se la memoria fosse centralizzata, sistemi di questo tipo andrebbero in crisi perchè la mole di richieste d'accesso alla memoria centralizzata è tale da saturare la banda disponibile (*con un conseguente aumento della latenza d'accesso*).
- **La memoria condivisa appare come unica ai programmatori.**

### Svantaggi di un sistema DSM

- **Comunicazione più complessa:**
  - I processori possono comunicare tra loro scrivendo e leggendo dalla memoria condivisa. La comunicazione tra processori è più complessa e richiede maggiori sforzi dei programmatori per sfruttare al massimo la larghezza di banda della memoria.
- **Latenza dovuta ai trasferimenti variabile:**

- Il tempo di accesso che impiega un nodo per accedere ad un indirizzo è diverso rispetto al tempo necessario per accedere ad un altro indirizzo.
- **Protocolli software per l'accesso alla memoria fisica di altri processori:**
  - La memoria fisica di un processore non può essere direttamente accessibile da un altro processore senza l'ausilio di protocolli software che gestiscono la comunicazione tra i processori.
  - In queste architetture, vengono utilizzati protocolli di messaggistica per consentire la comunicazione e lo scambio di dati tra i processori.
- **Bottleneck dovuto alla sincronizzazione:**
  - Serve usare delle primitive (*lock & unlock*) gestire la concorrenza in quanto possono verificarsi problemi di conflitto e/o sincronizzazione quando più thread cercano di accedere e modificare contemporaneamente la memoria condivisa.

## NUMA - NonUniform Memory Access

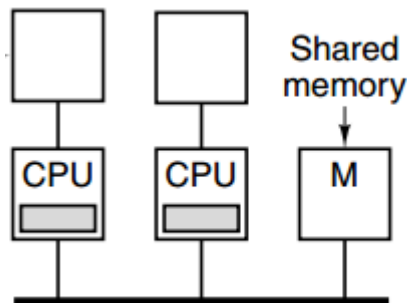
Il tempo di accesso dipende dalla posizione fisica dei dati in memoria.

I processori condividono lo stesso spazio d'indirizzamento ma i banchi di memoria fisica possono essere molteplici, tutti i processori possono accedere al dato ma il tempo d'accesso tra un processore ed un altro cambia in funzione alla distanza tra il processore e la memoria fisica su cui risiede il dato.

E' buona norma fare in modo che i thread che necessitano di comunicare vengano messi in posizioni tali da ridurre il tempo di accesso alle variabili condivise.

## Cache + Memoria condivisa

In questa soluzione abbiamo una cache per ciascun processore e anche una memoria condivisa tra tutti i processori.



I conflitti vengono diminuiti, però la complessità aumenta notevolmente e.

La soluzione con memoria cache:

- Riduce il tempo di accesso medio, in quanto a volte si andrà in cache.
- Riduce il numero di operazioni di accesso alla memoria principale per accedere ai dati.
- Si hanno svantaggi in caso di cambi di contesto: occorre svuotare la cache.
- La complessità aumenta.

Vantaggi di avere una cache in questo contesto

### Vantaggi della cache in caso di dati privati

I dati privati sono quelli utilizzati da un singolo processore.

Nel caso di dati privati:

- Riduce la latenza (*la velocità di lettura/scrittura delle memorie cache è molto maggiore rispetto a quella di una RAM*).
- Riduce il traffico di dati sul bus (*tutte le volte che il dato viene acceduto dal processore, ad esclusione della prima volta, la lettura si propaga fino alla cache senza andare sul bus*).

### Vantaggi della cache in caso di dati condivisi

I dati condivisi sono utilizzati da più processori, fornendo essenzialmente comunicazione tra i processori tramite letture e scritture dei dati condivisi.

Nel caso di dati condivisi, il dato può essere replicato su più cache private.

- Riduzione della latenza.
- Riduzione della banda.

- Riduzione dei conflitti di accesso in lettura sui dati in caso di più letture contemporaneamente da più processori diversi.

Svantaggi di avere una cache in questo contesto

### **Il traffico delle scritture devono avvenire quando il bus è libero**

Per questo c'è una soluzione hardware che previene che due scritture siano fatte insieme.

### **Risolvere le operazioni di scrittura su copie condivise**

Su copie singole si opera in write-back (*si bufferizzano le scritture in cache e poi si aggiorna la memoria*).

Invece su copie condivise lavoriamo in write-through (*si scrive direttamente in memoria condivisa oltre che in cache*).

### **Gestire le interruzioni I/O**

Gli interrupt possono causare la perdita di molte istruzioni in corso di esecuzione. Per minimizzare questo problema, utilizziamo due livelli di interrupt: principale e secondario.

L'interrupt secondario comunica al principale quando ha terminato un'operazione.

Nel nostro caso, applichiamo un protocollo di coerenza per garantire che le operazioni di I/O non interferiscano con l'esecuzione corrente.

La soluzione più diffusa prevede di risvegliare un processore con priorità più bassa per gestire l'interrupt, mantenendo l'efficienza del sistema.

### **Gestione della coerenza dei dati nelle cache**

Il problema della coerenza esiste perché abbiamo sia uno stato globale, definito dalla memoria principale, che uno stato locale, definito dalle cache, che sono private per ogni core del multiprocessore.

Poiché la visione della memoria detenuta da due diversi processori è data dalle loro cache, i processori che eseguono le interrupt potrebbero finire per vedere valori diversi per la stessa locazione di memoria.

In parole povere, avendo molte cache, potremmo avere molte copie (*anche diverse in valore*) dello stesso dato.

## Esempio di incoerenza

| Time | Event                       | Cache contents for processor A | Cache contents for processor B | Memory contents for location X |
|------|-----------------------------|--------------------------------|--------------------------------|--------------------------------|
| 0    |                             |                                |                                | 1                              |
| 1    | Processor A reads X         | 1                              |                                | 1                              |
| 2    | Processor B reads X         | 1                              | 1                              | 1                              |
| 3    | Processor A stores 0 into X | 0                              | 1                              | 0                              |

Assumiamo che all'inizio nessuna delle cache contenga la variabile X.

Assumiamo che X abbia il valore 1.

Assumiamo che la cache adotti una politica write-through, quindi una scrittura in cache viene subito fatta anche sulla memoria condivisa.

1. I processori A e B leggono X, fanno miss e salvano una copia nelle loro rispettive cache.
2. Il valore di X è stato scritto dal processore A; ora X vale 0.
3. La cache di A e la memoria condivisa contengono entrambe il nuovo valore di X. La cache di B contiene ancora il valore vecchio.
4. B legge il valore di X dopo che A lo ha modificato.
5. B legge ancora il valore 1 perchè lo stato di X è dato dal valore contenuto dalla cache di B, quindi viene dato un cache-hit non coerente.

Serve un meccanismo che permette di notificare quando una copia in cache deve essere aggiornata.

Quindi su questi multiprocessori la memoria è quasi sempre in stato inconsistente e viene aggiornata principalmente quando il bus è libero o in caso di scrittura.

### Approccio snooping based - SMP

Ogni cache che ha una copia dei dati di un blocco di memoria fisica può tenere traccia dello stato di condivisione del blocco.

In un SMP, le cache sono tipicamente tutte accessibili tramite un mezzo condiviso (esempio: un bus collega le cache di ogni core alla cache condivisa o alla memoria), e tutti i controller della cache monitorano (o fanno snooping) il mezzo per determinare se hanno o meno una copia di un blocco richiesto sul bus.

### Write Invalidate Protocol

Per usare questo protocollo bisogna garantire che un processore abbia accesso esclusivo a un elemento di dati prima di scriverlo.

Se i controllori vedono che c'è stata una scrittura invalidano la copia, in caso di accesso ci sarà una miss.

I multiprocessori recenti hanno scelto di implementare un protocollo write invalidate.

### **Perché l'accesso esclusivo?**

L'accesso esclusivo garantisce che non vengano effettuate due scritture contemporaneamente e quindi che esistano altre copie leggibili o scrivibili di un elemento quando avviene la scrittura.

### **Come avviene l'invalidazione?**

L'invalidazione avviene mediante una comunicazione broadcast utilizzando un bus condiviso tra tutte le cache.

#### Write Update Protocol

Se i controllori vedono che c'è stata una scrittura aggiornano la copia non appena si conclude la scrittura, in caso di accesso non ci sarà una miss.

Un protocollo write update deve trasmettere in broadcast tutte le scritture, richiede una quantità di banda considerevolmente maggiore.

#### Approccio directory based - DSM e SMP

La directory è una tabella che risiede in una memoria condivisa (*directory-based centralizzato*) o distribuita tra i nodi (*directory-based distribuito*).

Ogni entry della tabella corrisponde a un blocco di memoria e contiene informazioni su quali nodi hanno una copia di quel blocco e lo stato del blocco (*ad esempio, se è modificato o meno*).

### **Operazioni di Lettura**

Quando un nodo vuole leggere un blocco di memoria:

1. Se il nodo non ha una copia locale, invia una richiesta alla directory.
2. La directory risponde indicando dove si trova la copia più aggiornata del blocco o se altri nodi hanno copie condivise.

### **Operazioni di Scrittura**

Quando un nodo vuole scrivere un blocco di memoria:

1. Il nodo invia una richiesta alla directory.
2. La directory invalida le copie di quel blocco in possesso di altri nodi e concede al nodo richiedente il permesso di scrittura.



## Protocollo directory-based distribuito

Nel caso di DSM:

- Non c'è un bus condiviso che consente ai processori di fare snooping, ossia di vedere i blocchi di memoria condivisi.
- Non ha senso avere una singola directory, poiché ciò creerebbe un singolo punto di accesso (*quindi un collo di bottiglia*) e renderebbe difficile scalare a molti chip multicore.

Questa soluzione è molto complessa perchè il protocollo deve andare verso la memoria condivisa quando deve effettuare un'operazione di lettura o una scrittura.

## MESI (*Modified Exclusive Shared Invalid*)

Protocollo di tipo **write invalidate** che **mantiene la coerenza delle cache dei processori tenendo traccia dello stato di ogni cache line.**

### Stati di una cacheline

Una entry (*cacheline*) della cache può avere uno di questi 4 stati:

#### **Modified (M)**

Il contenuto è stato modificato (*dal processore associato alla cache in questione*) e non è consistente con la memoria principale.

Altre cache che possiedono una copia del dato devono invalidare le loro copie per garantire la coerenza.

#### **Exclusive (E)**

Nessun'altra cache mantiene quel blocco, la memoria è aggiornata e coerente.

#### **Shared (S)**

Questo stato indica che il dato contenuto nella cacheline è presente in più cache ed è consistente con la memoria principale.

La cacheline può essere letta dai processori, ma le modifiche richiedono la transizione allo stato Modified.

#### **Invalid (I)**

Questo stato indica che una cacheline non è valida o non esiste nella cache; un accesso a tale cacheline provoca una miss.

## Funzionamento di MESI

Dato un blocco di memoria A e dei processori  $P_i$ .

1. P1 richiede A e provoca una miss sulla sua cache privata.

2. A non è presente nella cache condivisa, quindi viene recuperato dalla RAM condivisa e caricato in cache condivisa.

3. A diventa *exclusive* e la copia viene presa direttamente dalla cache condivisa.

4. P2 richiede A e provoca una miss sulla sua cache privata.

5. P2 richiede la condivisione di A.

6. La copia privata di P1 va in stato *shared* e viene data anche a P2.

7. P2 scrive sulla sua copia privata di A; e la copia di P2 va in stato *modified*.

8. Viene inviato un segnale broadcast e la copia di P1 diventa *invalid*.

9. P3 richiede la condivisione di A.

10. P2 monitora il bus e dato che ha eseguito una scrittura, la copia di A nella sua cache privata viene inviata alla memoria RAM condivisa.

11. La copia privata di P2 torna in stato *shared* e viene data anche a P3.

12. P2 scrive sulla sua copia privata di A; e la copia di P2 va in stato *modified*.

13. Viene inviato un segnale broadcast e la copia di P3 diventa *invalid*.

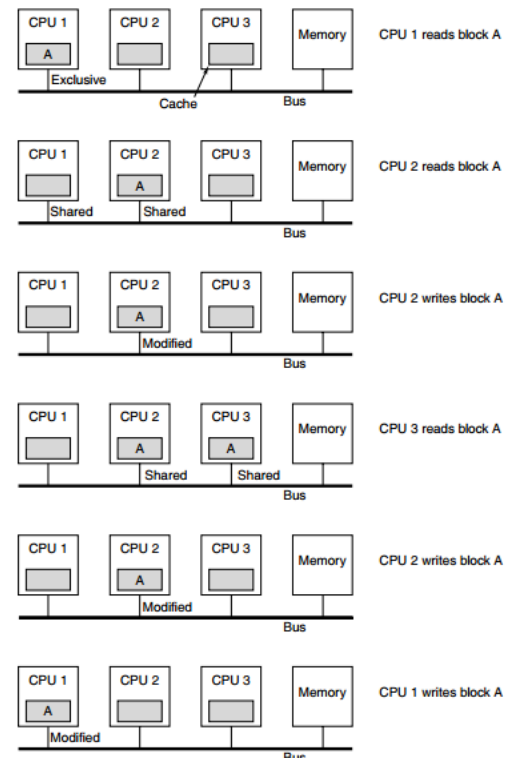
14. P1 richiede la condivisione di A.

15. P2 monitora il bus e dato che ha eseguito una scrittura, la copia di A nella sua cache privata viene inviata alla memoria RAM condivisa.

16. La copia privata di P2 torna in stato *shared* e viene data anche a P1.

17. P1 scrive sulla sua copia privata di A; e la copia di P1 va in stato *modified*.

18. Viene inviato un segnale broadcast e la copia di P2 diventa *invalid*.



## Monitorare il bus

La chiave per implementare un protocollo di invalidazione in un multicore è l'uso del bus, o un altro mezzo di trasmissione broadcast, per eseguire le invalidazioni.

Tutti i processori monitorano continuamente il bus, osservando gli indirizzi trasmessi.

Verificano se l'indirizzo sul bus corrisponde a una delle cacheline presenti nella propria cache. Se individua una corrispondenza tra l'indirizzo sul bus e un tag nella propria cache, esegue un'operazione di cambio di stato.

## Scrittura di un blocco *shared*

Se due processori cercano di scrivere due blocchi (*contrassegnati come shared*) contemporaneamente, i loro tentativi di trasmettere un'operazione di invalidazione verranno serializzati: **Il primo processore a ottenere l'accesso al bus provocherà l'invalidazione di tutte le altre copie private del blocco che sta scrivendo.**

Se i processori stavano cercando di scrivere lo stesso blocco, la serializzazione imposta dal bus renderà le loro scritture seriali.

Una conseguenza di questo schema è che una scrittura su un dato condiviso non può essere effettivamente completata fino a quando non ottiene l'accesso al bus.

## Funzionamento di MESI visto in un altro modo

Il funzionamento del protocollo si può vedere anche grazie a queste tabelle: si nota come le azioni di un processore si riflettono sulla entry e sugli altri processori.

| State         | This Processor   |             | Other Processor  |                  |
|---------------|------------------|-------------|------------------|------------------|
|               | Load             | Store       | Load Miss        | Store Miss       |
| Invalid (I)   | Miss<br>→ S or E | Miss<br>→ M | ---              | ---              |
| Shared (S)    | Hit              | Miss<br>→ M | ---              | → I              |
| Exclusive (E) | Hit              | Hit<br>→ M  | Send Data<br>→ S | Send Data<br>→ I |
| Modified (M)  | Hit              | Hit         | Send Data<br>→ S | Send Data<br>→ I |

Si nota come una load su una entry invalida (I) provoca una miss e la transizione di quella entry a Shared o Exclusive (*poiché il processore ottiene l'accesso al blocco di memoria condivisa*).

Notare anche come una store in una copia shared provoca l'invalidazione della copia dell'altro processore.

## In caso di cache-miss

Oltre a invalidare le copie di un blocco di cache in fase di scrittura, è anche necessario individuare un dato quando si verifica un cache miss.

## Cache write-through

In una cache write-through, è facile trovare il valore più recente di un dato, poiché tutti i dati scritti vengono sempre inviati alla memoria, dalla quale è sempre possibile recuperare il valore più recente (*I buffer di scrittura possono portare a qualche complessità aggiuntiva e devono essere adeguatamente trattati*).

## Cache write-back

Per una cache write-back, il problema di trovare la copia più recente è più complesso, poiché il valore più recente di un dato può trovarsi in una cache privata anziché nella cache condivisa o nella memoria.

## L3 Condivisa

Fortunatamente, le cache write-back possono utilizzare lo stesso schema di “snooping” anche per le read miss:

1. Ogni processore sonda ogni indirizzo posizionato sul bus condiviso.
2. Se un processore scopre di avere una copia “dirty” del blocco di cache richiesto, fornisce quel blocco di cache in risposta alla richiesta di lettura e provoca l’annullamento dell’accesso alla memoria RAM (o L3, la quale è condivisa).

## Gestione della coerenza in cache multi-livello

In un sistema con cache multi-livello è fondamentale gestire correttamente le operazioni di coerenza della cache.

## Consultazione dei tag

Quando una transazione avviene sul bus i campi `tag` delle cache private (L1 e L2) devono essere consultati per verificare se la transazione in corso sul bus riguarda un blocco di memoria presente in una delle cache.

Ma tutto questo è possibile farlo mentre il processore lavora? Se usiamo gli stessi campi `tag` sia per le operazioni del processore che per le operazioni di coerenza del bus, potremmo avere conflitti o inefficienze.

## Duplicazione dei Tag

La duplicazione dei tag è una tecnica utilizzata per migliorare l'efficienza delle operazioni di coerenza e accesso alla cache.

### Copia per il processore

Una copia dei campi tag viene usata esclusivamente quando il processore vuole accedere a un blocco di memoria.

Questo permette al processore di verificare se il blocco è presente in cache (*hit*) o se deve essere caricato dal livello inferiore (*miss*).

### Copia per la coerenza

Una seconda copia dei campi tag viene usata per le operazioni di coerenza del bus.

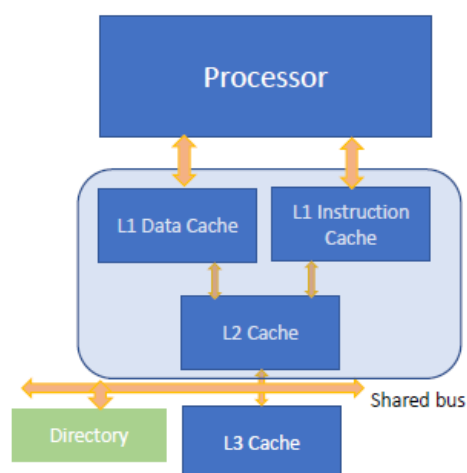
Quando avviene una transazione sul bus, questa copia dei tag viene consultata per verificare se il blocco coinvolto nella transazione è presente nella cache e se è necessario aggiornare o invalidare quel blocco per mantenere la coerenza.

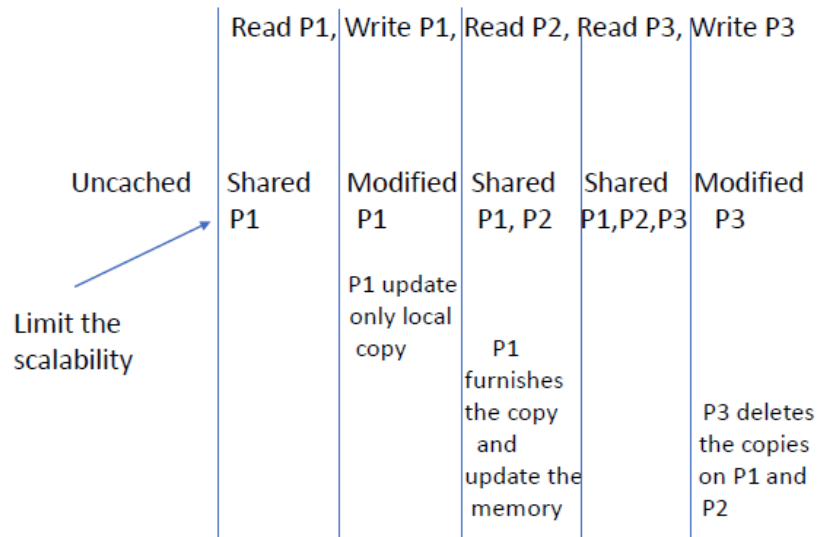
### Utilizzo della directory

Parte parallela alla memoria L3, essa indica lo stato delle copie dei blocchi in cache.

Una copia può essere:

- **Uncached:** Nessun nodo ne ha una copia nelle cache private.
- **Shared:** Ci sono dei nodi ne hanno una copia nelle cache private.
- **Modified:** Ci sono dei nodi che ne hanno una copia nelle cache private; e uno di questi ha scritto sulla sua copia in cache.

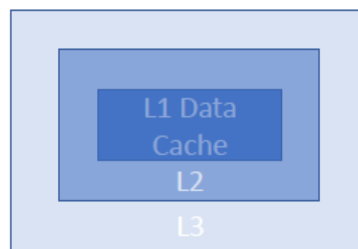




Gestione della presenza dei dati nelle cache e la proprietà di inclusione

Proprietà di inclusione: **“Se una copia non è presente in L3, allora non è presente né in L2 né in L1”.**

Questo può portare a inefficienze e spreco di memoria poiché le stesse copie di dati occupano spazio in più livelli di cache.



## L1 Cache

È la cache più piccola e veloce, spesso fisicamente situata nel chip del processore. Tutti i dati presenti in L1 dovrebbero essere presenti anche nei livelli superiori (L2 e L3).

In L1 si trovano i dati usati più spesso dal singolo core.

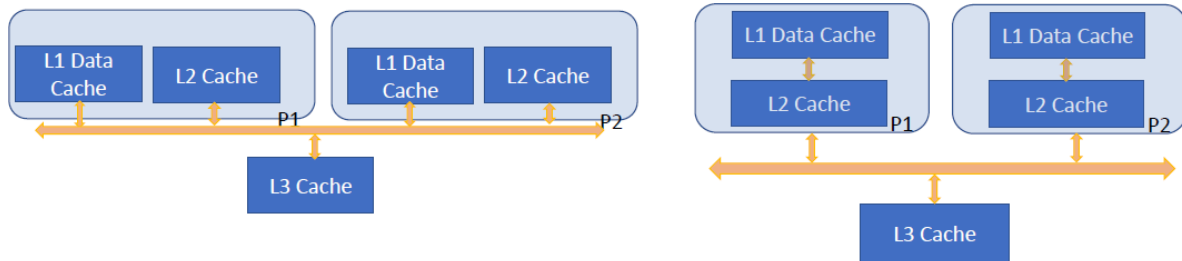
## L2 Cache

È di dimensioni intermedie e velocità intermedia. I dati in L2 dovrebbero essere presenti anche in L3.

In L2 si trovano i blocchi che prima erano in L1 ma sono stati scartati dall'algoritmo di rimpiazzamento.

## L3 Cache

È la cache più grande e lenta, molto spesso condivisa tra tutti i core del processore. Contiene una copia dei dati presenti nelle cache L1 e L2.



## Il rimpiazzamento e il problema della duplicazione

Il rimpiazzamento è il processo attraverso il quale un blocco viene rimosso da una cache per fare spazio ad un nuovo blocco.

Questo può avvenire indipendentemente nei vari livelli di cache, quindi può succedere che un dato venga rimosso da L1 ma rimanga in L2 e L3, o viceversa (*duplicazione*).

### Rimpiazzamento in L1

Se un dato viene rimosso da L1, esso può ancora essere presente in L2 o L3.

Questo avviene perché L1 potrebbe rimpiazzare i dati basandosi sulla storia locale degli accessi, mentre L2 e L3 non necessariamente seguono lo stesso schema di rimpiazzamento.

### Rimpiazzamento in L3

Quando un dato viene rimosso da L3, il controllore cache controlla se il dato è presente in L2 e nel caso si rimuove. Se non è presente in L2, non è necessario controllare in L1.

## Consistenza di memoria - Lock e Unlock

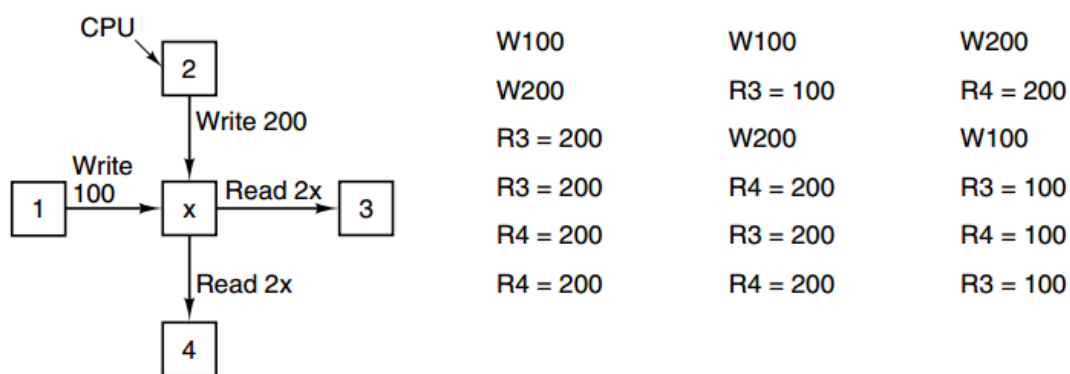
Ci sono vari paradigmi tra cui possiamo scegliere:

### Strict consistency

Ogni lettura ad una locazione  $x$ , restituisce sempre il valore della scrittura più recente in  $x$ .

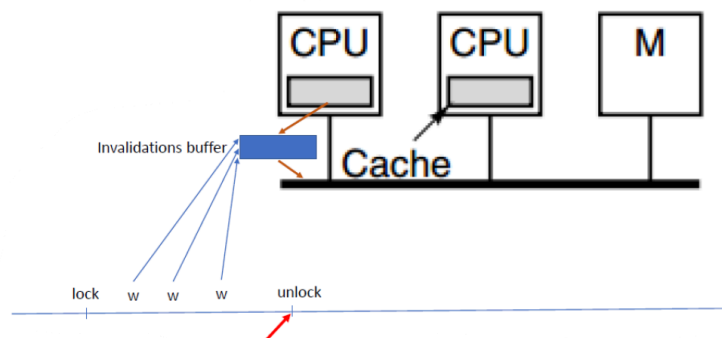
### Sequential consistency

Si mantiene l'ordine delle operazioni di lettura e scrittura.



### Memory Consistency Model V1

Un processore fa una `lock` e fa una scrittura e chiede (*invia in broadcast*) agli altri processori di effettuare le invalidazioni delle altre copie prima di effettuare una `unlock`, ovvero quando il processore che effettua una scrittura, effettua una `lock`, si ferma e controlla che tutte le altre copie sono state invalidate.

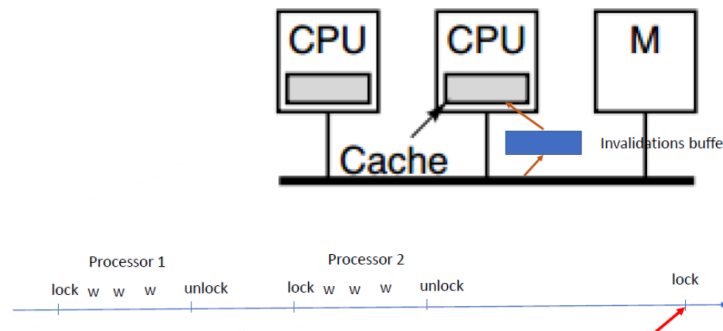




## Memory Consistency Model V2

Le operazioni di invalidazione vengono immagazzinate in ingresso all'invalidation buffer.

Quando un processore effettua la `lock`; la `lock` non è ultimata finché non vengono eseguite tutte le invalidazioni che ha ricevuto e che sono contenute nel buffer.

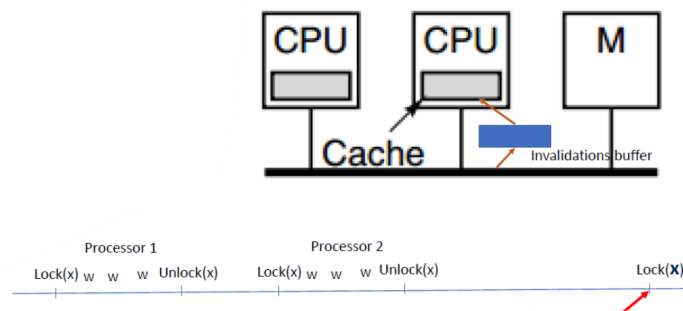


## Memory Consistency Model V3

In questo modello vengono indicate le variabili condivise.

Questa soluzione era eccezionale, però sul codice già scritto non funzionava, perché occorreva dichiarare tutte le variabili condivise.

Il processore effettua la `unlock` quando ha risolto tutte le invalidazioni.



## Test & Set

Si testa un valore (*tipicamente per 0*) e lo si setta solo se il valore supera il test. Questa coppia di istruzioni è usata per implementare i semafori; e quindi le primitive `lock` e `unlock`.

Due istruzioni speciali utilizzate nei processori per implementare meccanismi di sincronizzazione e gestione delle risorse condivise in modo efficiente.

Queste istruzioni vengono spesso impiegate per realizzare primitive di sincronizzazione, come i lock, senza ricorrere a istruzioni atomiche più costose.

## Load Reserved (`lr`)

Legge il valore di una locazione di memoria e "*riserva*" (*prenota*) tale locazione per un'operazione successiva di store conditional.

Questa riserva implica che il processore terrà traccia della locazione di memoria letta per garantire che nessun altro processore modifichi quel valore prima che l'operazione di store conditional venga eseguita.

## Store Conditional (`sc`)

Istruzione che tenta di scrivere un nuovo valore nella locazione di memoria precedentemente "*riservata*" dalla load reserved.

La scrittura ha successo solo se nessun altro processore ha modificato la locazione di memoria da quando è stata eseguita la load reserved.

## Funzionamento `ld` e `sc`

1. Il processore P1 esegue una `lr` su una variabile di `lock`.
  - Copia una copia di `lock` in cache con stato `shared`.
2. Il processore P2 esegue una `lr` su una variabile di `lock`.
  - Può farlo perché la `lr` evita che qualcuno modifichi quel valore, non che lo legga.
  - Copia una copia di `lock` in cache con stato `shared`.
3. P1 determina se `lock` è disponibile (se `lock = 0`).
4. Se disponibile, il P1 tenta di acquisire il lock impostando il valore a 1 tramite una `sc`.
5. Se nessun altro processore ha modificato `lock` da quando è stata eseguita la `lr`: La `sc` avrà successo e il processore avrà acquisito il lock.
  - Se la `sc` fallisce, significa che un altro processore ha modificato `lock`, e il processo deve ripetere il ciclo.

6. La copia di `lock` in P2 in cache viene invalidata.
7. La copia di `lock` in P1 in cache diventa `exclusive`.

```
do {  
    lr r1, [lock] // Carica il valore di lock in r1 e riserva la locazione  
    if (r1 == 0) { // Se il lock è disponibile (valore 0).  
        sc r2, [lock] // Store Conditional: prova a impostare lock a 1.  
        // Se un'altra modifica è stata fatta, fallisce.  
    }  
} while (r2 == 0); // Se sc fallisce, ripeti il ciclo.
```

Se due processori entrano nella stessa locazione per una `lr`, le `lr` hanno successo ed entrambe le loro copie vanno nello stato `shared`.

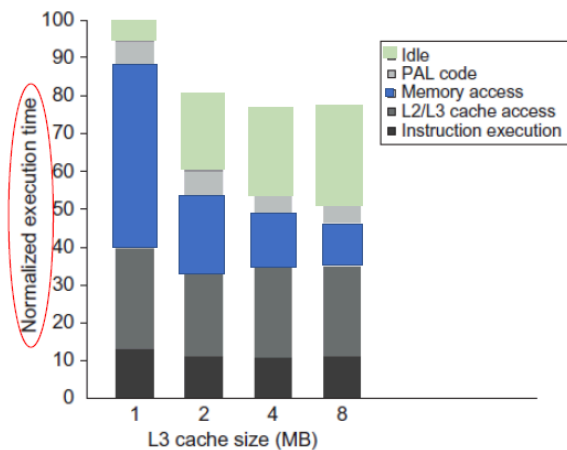
Se due processori entrano nella stessa locazione per una `sc`:

1. Uno dei due vince la lock e il vincitore invia il segnale di invalidazione sul bus.
2. L'altro (*il perdente*) prova comunque a fare l'invalidazione, però fallisce perché vede il segnale di invalidazione del vincitore.

## Performance di questo sistema

Vediamo come è speso il tempo di esecuzione, in basso nel grafico vediamo che il tempo è speso sulle miss L2 ed L3, quindi abbiamo:

- Il tempo di esecuzione vero e proprio.
- L'overhead dell'accesso in memoria.
- Il PAL code.
- Il tempo di attesa nelle sincronizzazioni.

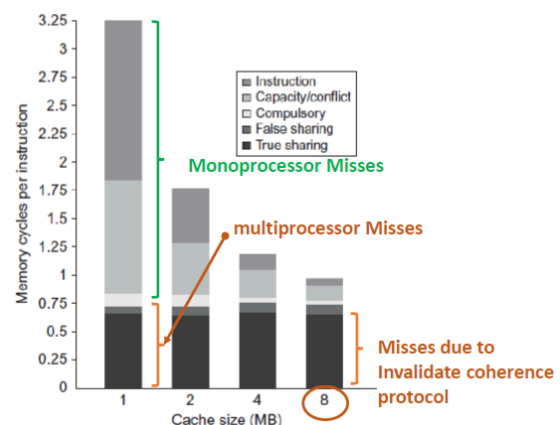


Aumentando le dimensioni della cache alcune metriche migliorano e altre no.

In questo esempio abbiamo il numero di cicli per eseguire un'istruzione; vediamo come i vari fattori influiscono a questa metrica:

- Le miss di conflitto.
- Le true sharing e le false sharing.
- Alcune miss sono dovute anche alla migrazione dei processi.

Le miss in verde sono per il monoprocesso, mentre in arancione sono le miss per il multiprocessore.



Aumentando le dimensioni della cache

diminuiscono le miss per il monoprocesso, però aumentano quelle per il multiprocessore a causa del protocollo di coerenza.

Il CPI aumenta all'aumentare delle dimensioni perché aumentano il true sharing ed il false sharing.

Le compulsory sono quelle che aumentano di più, in realtà non è vero che aumentano, se vengono distribuiti gli accessi aumentano le compulsory in percentuale.

## Effetto della dimensione del blocco di cache sulle miss

La dimensione del blocco di cache si riferisce alla quantità di dati che viene trasferita tra la memoria principale e la cache in un'unica operazione. Cambiare la dimensione del blocco di cache può avere effetti significativi sulla coerenza della cache, in particolare sulle miss di coerenza.

## Scenario con Blocchi Piccoli

Immaginiamo di avere blocchi di cache di piccole dimensioni e due blocchi di cache contigui che contengono dati condivisi tra più processori.

Quando un processore modifica un dato in uno dei blocchi, il protocollo di coerenza deve invalidare o aggiornare le copie del blocco nelle altre cache. Se entrambi i blocchi sono invalidati separatamente, si verificano due miss di coerenza.

## Scenario con Blocchi Più Grandi

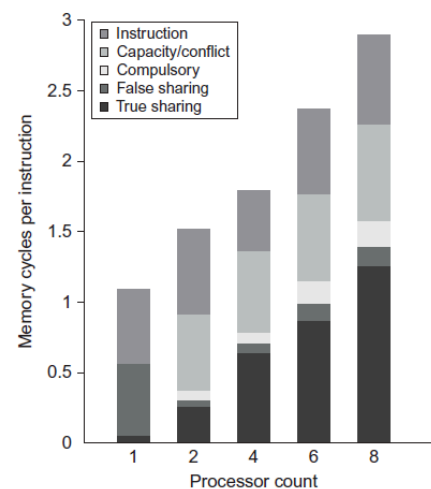
Ora, consideriamo l'effetto di aumentare la dimensione del blocco di cache.

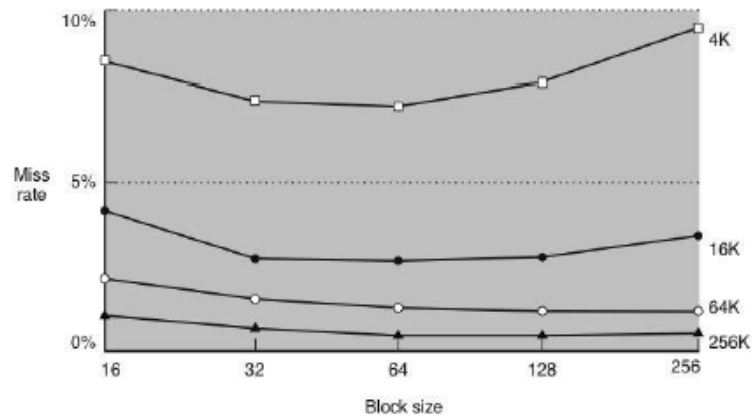
Se i due blocchi contigui sono uniti in un unico blocco più grande, quando un processore modifica un dato, il protocollo di coerenza invaliderà o aggiornerà solo questo blocco più grande. Questo significa che, invece di avere due miss di coerenza (una per ciascun blocco), ci sarà solo una miss di coerenza per il blocco unito.

## Vantaggio della Dimensione del Blocco Più Grande

Il vantaggio principale di avere blocchi di cache più grandi è quindi la riduzione del numero di miss di coerenza.

- **Diminuzione del Costo di Coerenza:**  
Avere meno miss di coerenza riduce il carico sul sistema di coerenza della cache, migliorando l'efficienza.
- **Efficacia della Cache:** Con blocchi più grandi, una singola operazione di invalidazione o aggiornamento può coprire più dati, riducendo il numero complessivo di operazioni di coerenza necessarie.



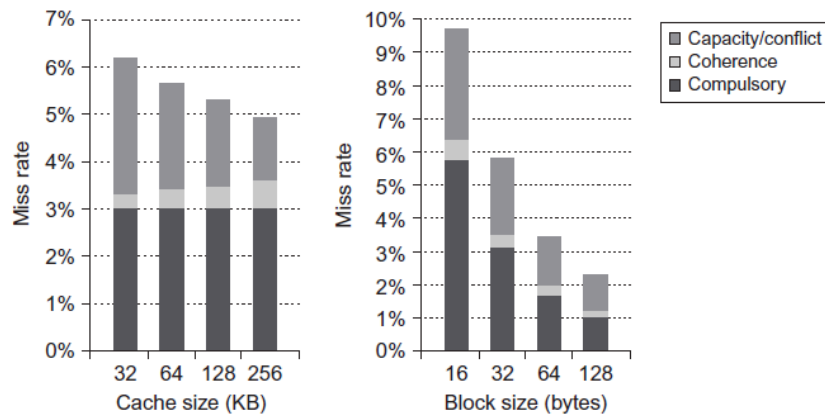


### Monoprocessor

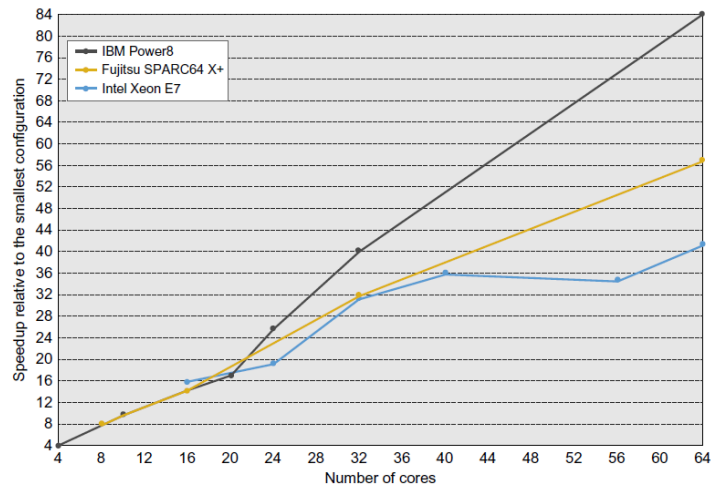
Che cosa succede quando queste operazioni avvengono considerando la cache L1 dati (*L1 istruzioni non ci interessa perché non produce coerenza*).

Nel grafico a sinistra, variando la dimensione della cache diminuiscono le capacity/conflict, le altre rimangono costanti.

Nel grafico a destra al variare delle dimensioni dei blocchi, diminuiscono tutte, quindi sembrerebbe che la soluzione sia ingrandire i blocchi, però siccome il trasferimento avviene a blocchi non è conveniente.



In questo grafico vediamo la scalabilità di tre architetture DSM.



Le prestazioni di un multiprocessore al variare delle caratteristiche delle cache dipendono fortemente dalla velocità del sistema utilizzato per condividere le informazioni (*bus o un sistema più evoluto come in alcune architetture di Intel*), dipende fortemente dal tipo di applicazione, molte generano molte false sharing e altre meno.

# Architetture specifiche per applicazione

## Costi energetici delle istruzioni

Vediamo la totale inefficienza delle macchine che abbiamo visto finora.

|                   |      |          |     |            |
|-------------------|------|----------|-----|------------|
| RISC instruction  |      | Overhead | ALU | 125 pJ     |
| Load/Store        | D-\$ | Overhead | ALU | 150 pJ     |
| SP floating point |      |          | +   | 15–20 pJ   |
| 32-bit addition   |      |          | +   | 7 pJ       |
| 8-bit addition    |      |          | +   | 0.2–0.5 pJ |

## Macchine basate su Intelligenza artificiale

Macchine che operano su dati statistici; si rinuncia alla precisione per ottenere qualcosa in più.

Ad esempio la GPU può essere uno strumento necessario nella fase di learning (*la fase in cui si progettano le regole*) grazie al suo alto grado di parallelismo.

## Macchine basate su FPGA

Hardware programmabile. In un FPGA non ci sono cache né altro, ma comunque il parallelismo è facile da programmare; inoltre può essere programmato più volte.

Lo svantaggio è che le prestazioni sono un decimo/un ventesimo rispetto alle prestazioni iniziali.

## Macchine basate su SoC

A differenza di un FPGA, un SoC integra porte logiche progettate specificamente per applicazioni particolari durante la fabbricazione, rendendolo più efficiente in termini di velocità e consumo energetico.

Tuttavia, la progettazione di un SoC è più laboriosa e costosa rispetto alla configurazione di un FPGA, che può essere riprogrammato post-fabbricazione per diverse applicazioni.



## Domain-Specific Device

Detti anche “*microcontrollori*”, sono dispositivi progettati per trattare casi specifici.

Spesso non hanno un microprocessore, ma un chip che incorpora tutte le risorse hardware necessarie per quel contesto.

Le uniche parti che non possono essere integrate sono quelle che devono lavorare ad alta frequenza o ad alta tensione.

### Regole di progettazione dei microcontrollori

#### **Minimizzare la distanza tra elaborazione e dati**

La prima regola di progetto è quella di minimizzare la distanza tra le unità di elaborazione e le memorie. Quindi conviene **usare memorie dedicate in modo da minimizzare questa distanza**.

**Non seguire questa regola porta ad uno spreco di area del chip ed energia elettrica (*entrambi elementi cruciali in un sistema embedded*).**

Per mitigare questo problema si può aggiungere una cache.

Molti microcontrollori hanno una memoria interna del chip, che solitamente basta, se non basta, può essere comandata per lavorare con la cache.

I dati più importanti vengono messi dentro la memoria, quelli meno utilizzati vengono messi all'esterno, però il costo è 2,5 volte l'equivalente di una memoria.

Ad esempio l'aggiunta di una cache set-associative causa un consumo pari 2,5 volte rispetto alla normale memoria utilizzata on-demand.

#### **Ottimizzare le operazioni più costose con l'hardware**

L'hardware possibilmente deve fare le operazioni che sono più costose in consumo di tempo in una macchina general purpose.

L'hardware è molto più veloce del software e può abbattere il consumo di tempo ed energia.

L'idea è quella di creare un chip eliminando la complessità dell'esecuzione out of order, del multithreading, del prefetching ecc e utilizzare tante unità funzionali che ci servono e la memoria possibilmente dentro il chip.

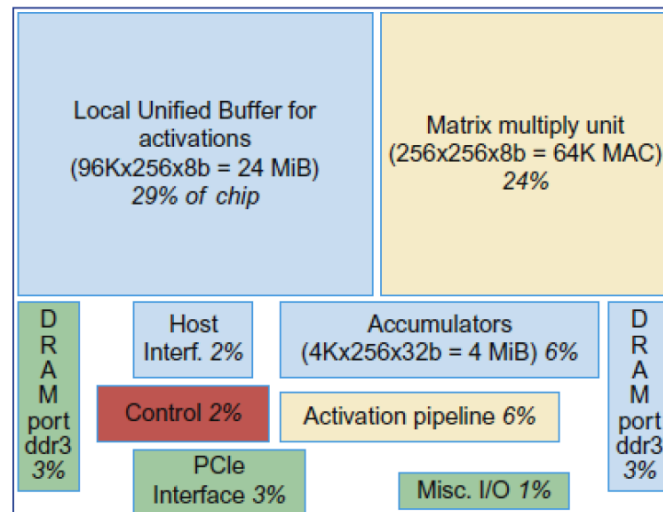
Realizzare il parallelismo in hardware è più facile, quindi dato un algoritmo occorre trovare le parti parallelizzabili e realizzare quindi un acceleratore hardware per lo scopo.

Il linguaggio utilizzato deve essere adatto, l'hardware progettato deve essere adeguato al sistema.

## TPU

Nella figura sotto vediamo un TPU, ovvero un acceleratore per AI.

C'è la possibilità di controllare il mondo esterno, c'è un co-processore, ovvero un processore general purpose che comanda la comunicazione col mondo esterno.



## Deep Neural Network

C'è la fase di training (*che noi indichiamo come progettazione delle regole*), le regole si basano su una struttura di elaborazione di queste regole stesse. Prendiamo in considerazione la rete globale, si prendono dei punti per ottenere dei dati intermedi e si va avanti fino ad ottenere i dati finali. Si va avanti così finché non si raggiunge un errore massimo, il quale rappresenta il nostro obiettivo.

Una Deep Neural Network (*DNN*) è una rete neurale artificiale con molteplici strati tra l'input e l'output. Ogni strato è composto da neuroni artificiali che elaborano i dati, ed è chiamata "profonda" perché ha più strati nascosti.

### Fase di Training

La fase di training di una DNN è il processo in cui la rete impara a svolgere un compito specifico (ad esempio, classificare immagini, riconoscere la voce, ecc.). Durante questa fase, la rete modifica i propri parametri (i pesi delle connessioni tra i neuroni) per minimizzare l'errore tra le predizioni della rete e i valori reali.

Ecco una descrizione più chiara e dettagliata del processo di training di una DNN:

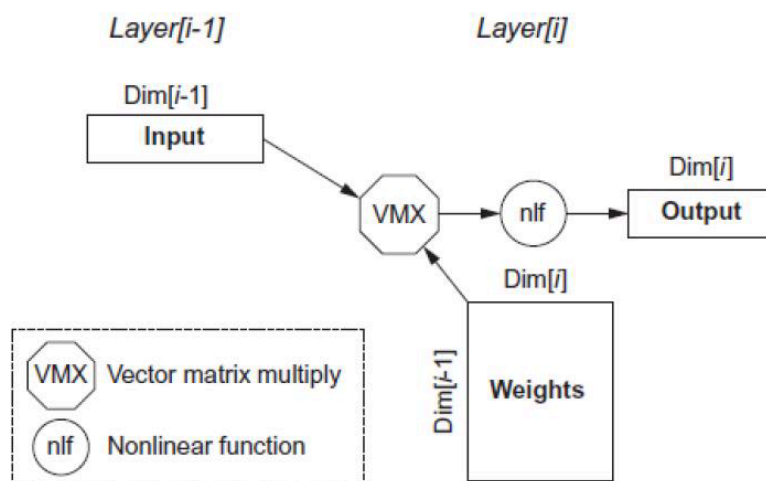
#### 1. Inizializzazione

- La rete neurale viene inizializzata con pesi casuali o specifici.
  - Definiamo la struttura della rete, ovvero il numero di strati e di neuroni per strato.
- 2. Forward Pass**
- Durante il forward pass, un input (ad esempio, un'immagine) viene passato attraverso la rete.
  - Ogni neurone elabora i dati ricevuti dall'input o dai neuroni del livello precedente e passa il risultato al livello successivo fino a raggiungere l'output finale.
- 3. Calcolo dell'Errore**
- L'output della rete viene confrontato con l'output desiderato (il valore reale o etichetta).
  - Si calcola l'errore (ad esempio, utilizzando una funzione di perdita come l'errore quadratico medio o l'entropia incrociata).
- 4. Backward Pass (Backpropagation)**
- Durante il backward pass, l'errore calcolato viene propagato all'indietro attraverso la rete.
  - I gradienti dell'errore rispetto ai pesi della rete vengono calcolati utilizzando la derivata della funzione di perdita.
  - I pesi vengono aggiornati usando un algoritmo di ottimizzazione come la discesa del gradiente, per ridurre l'errore.
- 5. Iterazione:**
- Questo processo di forward pass, calcolo dell'errore e backward pass viene ripetuto per molti cicli (epoch).
  - Ogni iterazione mira a ridurre l'errore della rete, migliorando la sua capacità di fare previsioni accurate.
- 6. Convergenza:**
- Il processo di training continua fino a quando l'errore di validazione non raggiunge un valore minimo accettabile (errore massimo tollerato) o un certo numero di epoche è completato.
  - A questo punto, la rete ha appreso a generalizzare dai dati di training, idealmente anche ai nuovi dati non visti durante il training.

In tabella vediamo la dimensione dei dati utilizzati per allenare dei sistemi e vediamo l'architettura in cui è avvenuto l'allenamento e la durata di esso.

| Type of data | Problem area               | Size of benchmark's training set | DNN architecture  | Hardware             | Training time |
|--------------|----------------------------|----------------------------------|-------------------|----------------------|---------------|
| text [1]     | Word prediction (word2vec) | 100 billion words (Wikipedia)    | 2-layer skip gram | 1 NVIDIA Titan X GPU | 6.2 hours     |
| audio [2]    | Speech recognition         | 2000 hours (Fisher Corpus)       | 11-layer RNN      | 1 NVIDIA K1200 GPU   | 3.5 days      |
| images [3]   | Image classification       | 1 million images (ImageNet)      | 22-layer CNN      | 1 NVIDIA K20 GPU     | 3 weeks       |
| video [4]    | activity recognition       | 1 million videos (Sports-1M)     | 8-layer CNN       | 10 NVIDIA GPUs       | 1 month       |

Prendiamo una rete DNN, la quale prende dei dati in input, li moltiplica per una matrice utilizzando dei pesi e produce un output.



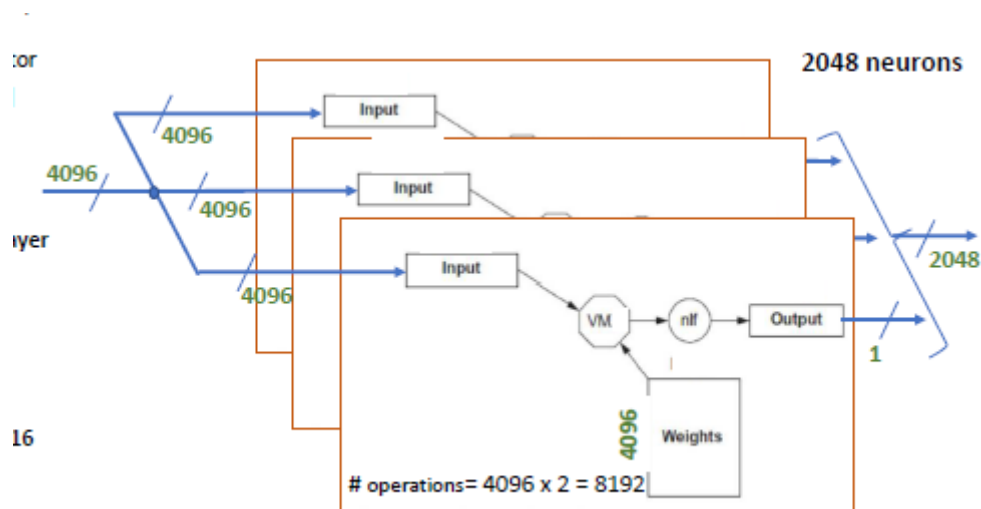
$$y_n = F(W \times y_{n-1})$$

In questo tipo di rete esistono due tipi di parallelismi:

1. Il primo parallelismo si trova nella moltiplicazioni tra il vettore in input e la matrice;
2. L'altro parallelismo sta nel fatto che questa figura è un neurone della rete, all'interno di una sola rete ci sono molti neuroni.

Il costo è nel numero di dati trattati e nel riutilizzo dei dati, c'è il rischio che la matrice sia molto più grande delle capacità di elaborazione, quindi deve essere scomposta.

In questo esempio abbiamo 4096 dati in ingresso, vengono dati in ingresso a 2048 neuroni, ciascuno genera un dato in output, quindi l'output finale è di 2048. Il numero dei pesi è sugli 8 milioni, mentre il numero di operazioni totali sono 16 milioni.



## Convolutional Neural Network

Utilizzata in computer vision, è basata sull'estrazione di caratteristiche su porzioni d'immagine, abbiamo rispetto a prima un ulteriore elemento di parallelismo, le immagini vengono estratte in modo da ottenere informazioni dal resto.

Gli ingressi si splittano e si distribuiscono.

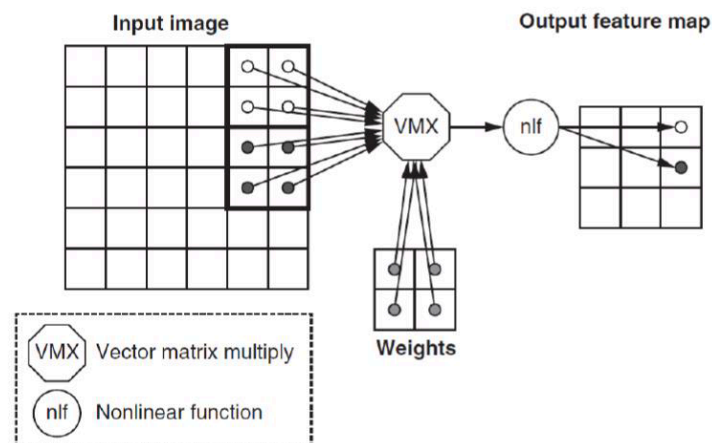
## Funzionamento di una CNN

1. **Input:** Una CNN prende in input un'immagine, rappresentata come una matrice di valori di pixel.
2. **Convoluzione:** Gli strati di convoluzione applicano vari filtri sull'immagine di input per creare diverse feature map, evidenziando caratteristiche specifiche come bordi, angoli, e texture.
3. **ReLU:** La funzione di attivazione ReLU viene applicata alle feature map per introdurre non-linearità, permettendo alla rete di apprendere rappresentazioni più complesse.
4. **Pooling:** Gli strati di pooling riducono la dimensione delle feature map, aggregando informazioni da regioni vicine e

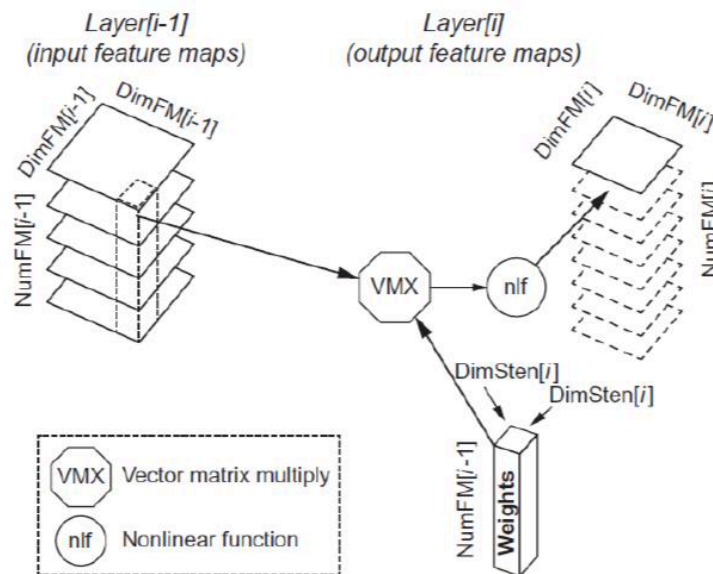
rendendo la rappresentazione più compatta e resistente a traslazioni e distorsioni nell'immagine di input.

5. **Ripetizione:** I passaggi di convoluzione, ReLU, e pooling vengono ripetuti più volte. Ogni ripetizione forma un livello più profondo della rete, che apprende caratteristiche sempre più astratte e complesse dell'immagine.

6. **Flattening:** Dopo diversi strati di convoluzione e pooling, le feature map finali sono "appiattite" in un vettore unidimensionale. Questo vettore contiene tutte le caratteristiche estratte dall'immagine e viene passato agli strati completamente connessi.
7. **Strati Completamente Connessi:** Questi strati elaborano il vettore appiattito. Ogni neurone in questi strati è connesso a tutti i neuroni del livello precedente, permettendo alla rete di combinare le caratteristiche estratte in previsioni finali.
8. **Output:** L'ultimo strato completamente connesso produce l'output finale. Per una rete di classificazione delle immagini, l'output potrebbe essere un vettore di probabilità che indica la probabilità che l'immagine appartenga a ciascuna delle classi possibili.



Un layer prende in considerazione le finestre e fa questo lavoro per ogni finestra, utilizzando i pesi e produce delle informazioni (un rettangolo d'informazioni più piccolo) che vengono utilizzate dal livello successivo.



Supponiamo che abbiamo 28 come dimensione dei dati d'ingresso, detti **feature**, e vogliamo ottenere un output di 14, il numero di mappe è 64, il numero di feature in output è 18. Facendo dei conti avremmo bisogno di 25 mila neuroni, 73 mila pesi ed il numero di operazioni sono circa 29 milioni.

## RNN (Recurrent Neural Network)

Reti per riconoscere il linguaggio, introduce una novità, quello che abbiamo visto finora è una rete combinatoria, effettua solo operazioni, questa è una rete combinatoria, durante l'elaborazione costruisce delle informazioni aggiuntive che utilizza nelle elaborazioni successive.

### Come Funziona una RNN

#### 1. Struttura delle RNN:

- **Neuroni Ricorrenti:** Ogni neurone in una RNN riceve input sia dall'attuale elemento della sequenza sia dal proprio stato precedente.
- **Stato Nascosto (Hidden State):** Il neurone mantiene un "stato nascosto" che viene aggiornato ad ogni passo della sequenza. Questo stato nascosto funge da memoria, conservando informazioni sulla parte precedente della sequenza.

#### 2. Elaborazione di una Sequenza:

- **Input:** Ad ogni passo temporale  $t$ , la RNN prende un elemento della sequenza  $x_t$  come input.
- **Calcolo dello Stato Nascosto:** Lo stato nascosto  $h_t$  viene aggiornato utilizzando l'input corrente  $x_t$  e lo stato nascosto del passo precedente.

ht-1.

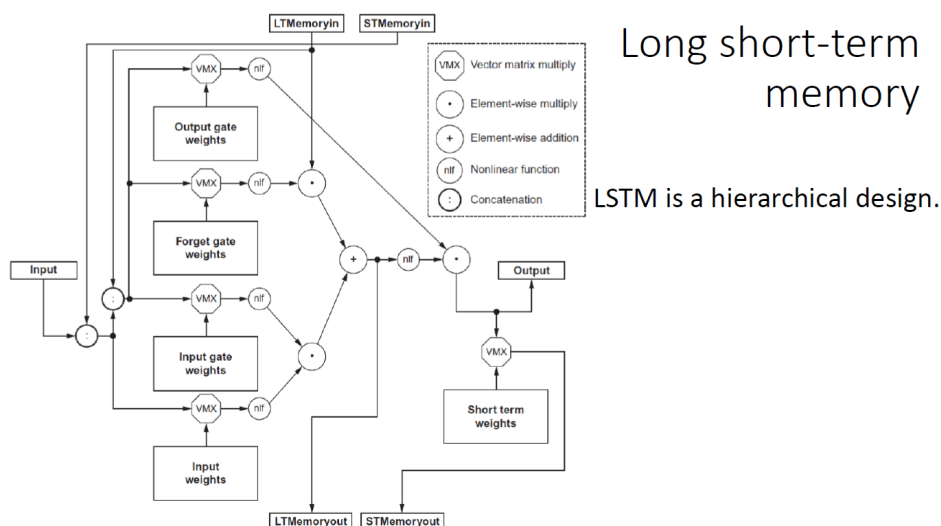
$$h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

- Dove  $W_h$  e  $W_x$  sono pesi di connessione,  $b$  è il bias, e  $f$  è una funzione di attivazione (ad esempio, tanh o ReLU).
- **Output:** L'output della RNN può essere generato in ogni passo temporale basandosi sullo stato nascosto corrente  $h_t$ .

### 3. Propagazione nel Tempo (Backpropagation Through Time, BPTT):

- Durante il training, l'errore viene propagato all'indietro nel tempo attraverso l'intera sequenza per aggiornare i pesi della rete, un processo noto come BPTT. Questo consente alla rete di apprendere dipendenze a lungo termine nella sequenza.

Ci sono varie strutture come riferimento, una è questa in figura.



L'output è costituito anche dai pesi della moltiplicazione finale, ovvero gli short term weights.

Il numero delle operazioni è molto elevato, i dati d'ingresso non sono molti essendo campioni di voce.

Quale potrebbe essere il processore che soddisfa questo sistema?

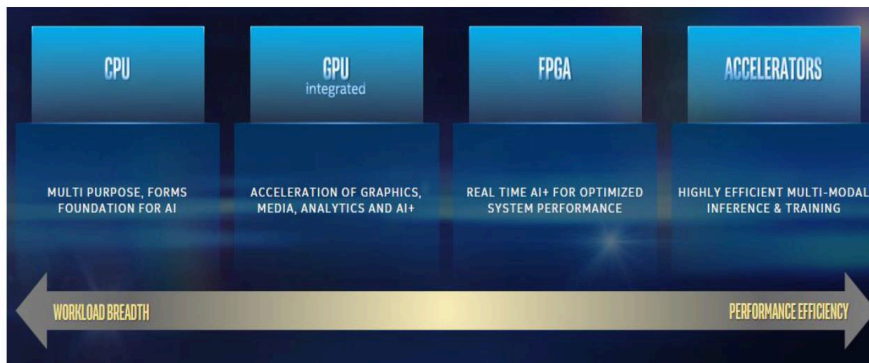
In figura vediamo l'efficienza delle prestazioni dell'esecuzione sui vari componenti.



## Multi models

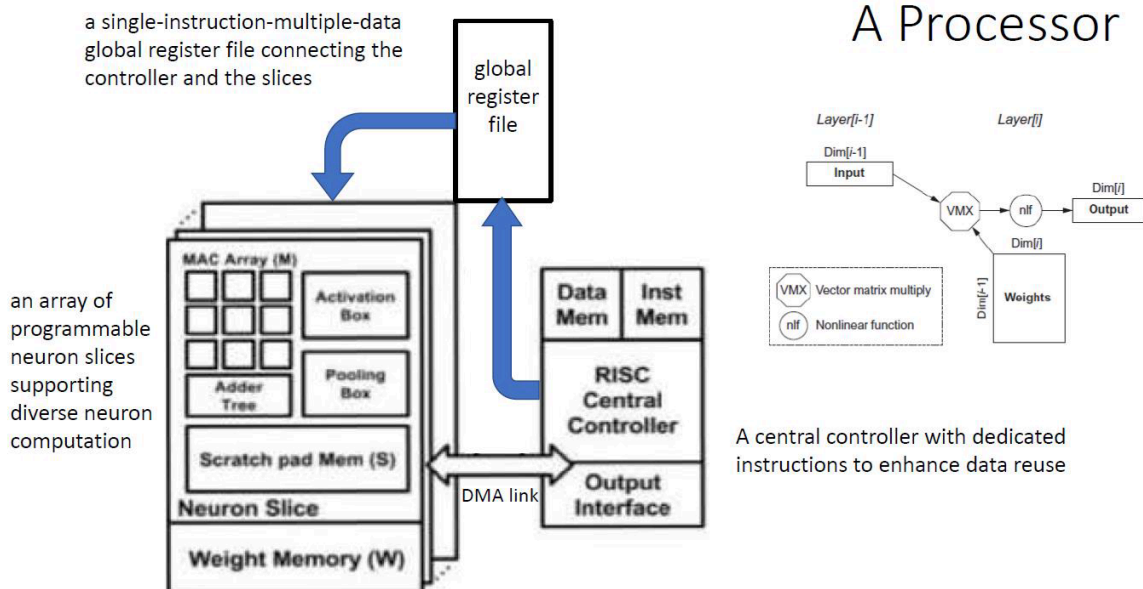
High performance across broad set of existing and emerging DL models for applications:

Natural Language Processing, Speech & Text, Recommendation Engines, and Video & Image search & filtering



Una possibile soluzione per un acceleratore esterno al processore è la seguente in figura.

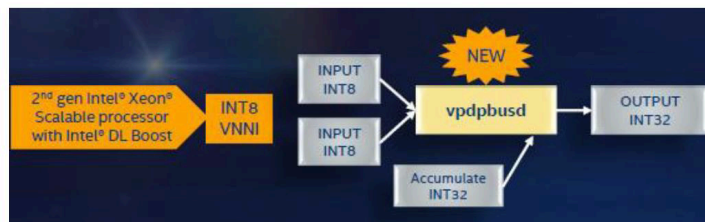
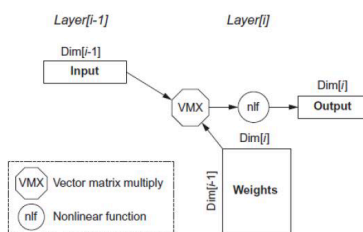
I dati entrano tramite un collegamento DMA, quindi il trasferimento deve avere velocità significative. Il componente a destra è un computer normale, il quale scrivendo i comandi realizza la rete sulla destra.



Le istruzioni principali sono le seguenti:

1. **Read\_Memory**: legge i dati dalla memoria della CPU, e li mette nella memoria del pezzo di neurone.
2. **Read\_Weights**: legge i pesi dalla memoria della CPU, nella memoria FIFO dei pesi.
3. **MatrixMultiply/Convolve**: l'unità di moltiplicazione della matrice effettua moltiplicazioni matrice-matrice, moltiplicazioni vettore-matrice, moltiplicazione elemento per elemento della matrice, moltiplicazione elemento per elemento di un vettore, convoluzione.
4. **Activate**: effettua la funzione non lineare del neurone artificiale.
5. **Write\_Host\_Memory**: scrive i dati dall'unified buffer nella memoria della CPU.

Intel ha preso lo Xeon, parte da un software che produce i risultati che mette in output.



I dati sono frutto di un'elaborazione statistica, quindi sono già affetti da errore, se vengono ridotti i dati, vengono ridotti anche i tempi di esecuzione notevolmente.

Un soluzione consiste nel partire da 32 bit ed in fase di inferenza si passa a 16 bit, mentre nell'altra soluzione si passa ad 8 bit.

## Cache NUCA

Il microprocessore simmetrico è un processore in cui dal punto di vista di ogni processo, tutti i processori sono uguali. Il bilanciamento del carico è banale, se un processo si ferma, un processore esegue un altro processo.

La cache NUCA cerca di risolvere il problema del tempo di accesso per le cache grandi, le cache grandi sono più lontane dal processore. La soluzione consiste nel suddividere la cache in blocchi, i blocchi sono collegati in rete, il messaggio viene inviato in broadcast dal controllore, il blocco che ha questo elemento risponde (caso di hit), se nessun blocco ha questo elemento è un caso di miss. Ovviamente queste sono le cache di livello L3.

Le NUCA sono di due tipi SNUCA (statiche) e DNUCA (dinamiche), nel primo caso un blocco rimane sempre nella stessa posizione. Mentre nelle dinamiche i blocchi più utilizzati ad ogni hit scendono in una riga più bassa, quindi i blocchi più utilizzati dai programmi vanno vicino alla CPU, mentre quelli meno utilizzati rimangono “in alto” più lontani. Nel caso di miss un blocco viene inserito “in alto” nella griglia.

I processori possono essere messi o tutti in una zona, ad esempio nel lato in basso della griglia, oppure in due zone, metà nel lato alto e metà nel lato basso.

Nel primo caso le hit si verificano quasi totalmente in basso, mentre nella seconda soluzione le hit avvengono in tutti i livelli, i livelli sopra quello più basso hanno molte più hit rispetto al primo caso, da notare che la linea del livello 1 e dell'ultimo livello sono quelle veloci, le altre linee sono più lontane, quindi più lente.

Si crea il fenomeno del ping-pong, ovvero nel caso 2, con i processori sia nella zona alta e bassa, entrambi i livelli attraggono i blocchi che fanno hit, quindi quei blocchi tendono a rimanere al centro, ovvero nelle linee centrali.

Per ottimizzare gli hit, ovvero fare in modo che gli hit avvengano nella linea più vicina al processore, è possibile adottare una soluzione software che consiste nel cambiare l'ordine di esecuzione delle istruzioni del programma.

Il mapping della cache avviene prima della compilazione, in modo tale da posizionare le hit nelle zone migliori.

Intervenendo solo sul software è stato ottenuto un miglioramento del 20%.

## Domande Esame/Presentazione

Presentazione: mettere una roadmap, se la presentazione è lunga mettere un lucido di stacco.

Ha chiesto cos'è un CUDA core

il confronto con la CPU non importa

che legame c'è tra warp e blocco?

grafici differenza dal profiler con 32 thread per blocco e con 64 thread per blocco

dopo questi grafici viene messo il grafico con nelle x i valori di 64 thread per blocco (1 50 100 150 200..... 1000 sono il numero di blocchi)

mettere alla fine, con quali valori conviene maggiormente la CPU o la GPU

anche nella GPU mettere speedup e tempo di esecuzione.

giustificare la dimensione della griglia utilizzata (noi abbiamo una matrice, però nella versione ottimizzata non la usiamo nella GPU, quindi si può usare 1x1 come dimensione e variare il numero di blocchi).

se siamo fortunati possiamo cambiare il numero di blocchi, se non riusciamo, possiamo cambiare il numero di thread per blocco.

Guardare anche lo speed up al variare del carico.

Inviare codice e presentazione entro giovedì 30 sera.

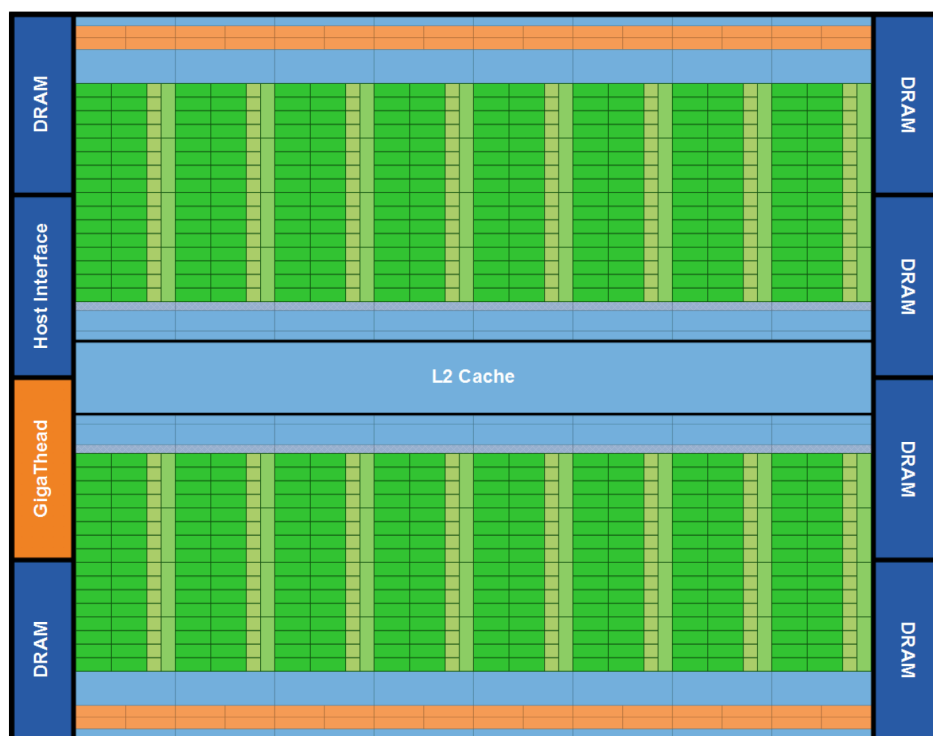
# GPU - Graphic Processing Unit

Le schede video sono dei sistemi eterogenei che spesso sono dei veri sottosistemi separati molto simili alla scheda madre, in quanto includono sia una CPU, ovviamente una GPU e anche una memoria interna che può funzionare da memoria RAM.

Le GPU sono progettate per essere dei motori per il rendering grafico; possono essere integrate come quelle della Intel o anche delle periferiche esterne come quelle della Nvidia.

## Architettura NVIDIA Fermi

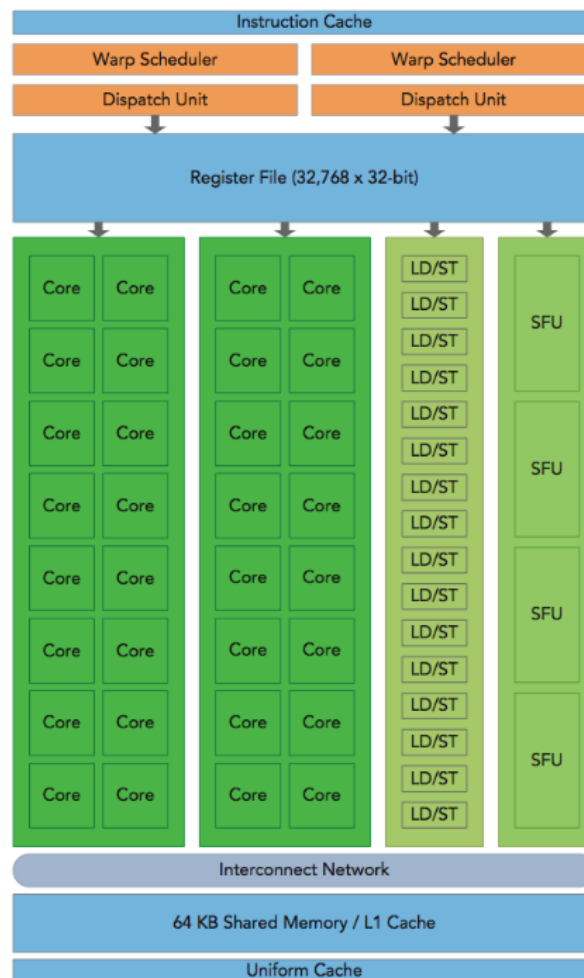
In questo esempio vediamo uno schema semplificato del modello architetturale della scheda Nvidia Fermi.



Fermi's 16 SM are positioned around a common L2 cache. Each SM is a vertical rectangular strip that contain an orange portion (scheduler and dispatch), a green portion (execution units), and light blue portions (register file and L1 cache).

## SM - Streaming Multiprocessor

È l'unità di calcolo di base che consente l'elaborazione parallela nella GPU.



## CUDA - Compute Unified Device Architecture

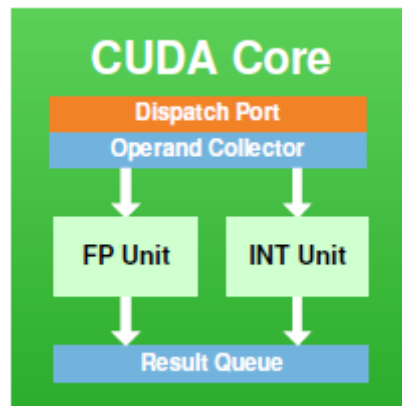
Una piattaforma di elaborazione parallela e un modello di programmazione sviluppati da NVIDIA che permette agli sviluppatori di utilizzare la potenza di calcolo della GPU per scopi generici, non limitati alla grafica.

### CUDA core

I CUDA Cores sono i nuclei fondamentali all'interno delle GPU NVIDIA che eseguono le operazioni aritmetiche e logiche.

Ogni CUDA Core è un'unità di calcolo semplice e relativamente poco potente, ma la loro risiede nel fatto che lo streaming multiprocessor di una GPU può contenere migliaia di questi core che lavorano in parallelo.

Oltre ai core CUDA, un SM include anche altre unità funzionali come unità di calcolo per operazioni specializzate e unità di calcolo per operazioni in virgola mobile e interi.



Un kernel è una funzione scritta in C/C++ con estensioni specifiche per CUDA che viene eseguita in parallelo da più thread sulla GPU.

I CUDA Cores vengono utilizzati in vari contesti che richiedono un'elevata capacità di calcolo parallelo.

### Vantaggi dei CUDA Cores

- **Parallelismo Massivo:** La capacità di eseguire migliaia di thread in parallelo permette di accelerare notevolmente i tempi di elaborazione per compiti che possono essere parallelizzati.
- **Efficienza Energetica:** Le GPU, grazie alla loro architettura parallela, possono eseguire molte operazioni con un consumo energetico relativamente basso rispetto alle CPU per lo stesso tipo di carico di lavoro.
- **Flessibilità:** La piattaforma CUDA permette di utilizzare la GPU per una vasta gamma di applicazioni, non limitandosi solo alla grafica.

### Memoria condivisa

Gli SM dispongono di una memoria condivisa, che è una memoria veloce accessibile da tutti i core all'interno dell'SM. Questa memoria è utile per la condivisione di dati tra i core e per minimizzare l'accesso alla memoria globale, che è più lenta.

Ogni SM ha anche un set di registri che vengono utilizzati per memorizzare i dati temporanei durante l'esecuzione delle istruzioni.

### Warp Scheduler

Gli SM utilizzano un concetto chiamato "*warp*" per gestire l'esecuzione delle istruzioni.



Un warp è un gruppo di 32 thread che eseguono la stessa istruzione contemporaneamente. Questo approccio consente di massimizzare l'efficienza di esecuzione.

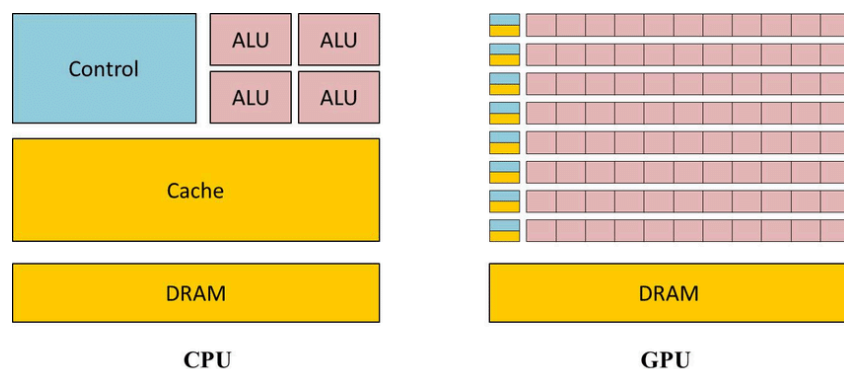
Il warp scheduler all'interno dell'SM decide quale warp deve essere eseguito in un dato ciclo, permettendo un'alta efficienza e un utilizzo ottimale delle risorse dell'SM.

## Sistema eterogeneo - CPU + GPU

La CPU comunica direttamente con la GPU tramite uno scambio di messaggi tra la RAM del calcolatore alla RAM presente nella scheda video.

In questo contesto la GPU è detta anche “*device*”, mentre la CPU in questo contesto prende il nome di “*host*”.

Lo scambio di informazioni prevede che i dati vengano copiati e incollati tra le due memorie, ovviamente questo procedimento richiede tempo e spesso rappresenta un collo di bottiglia.

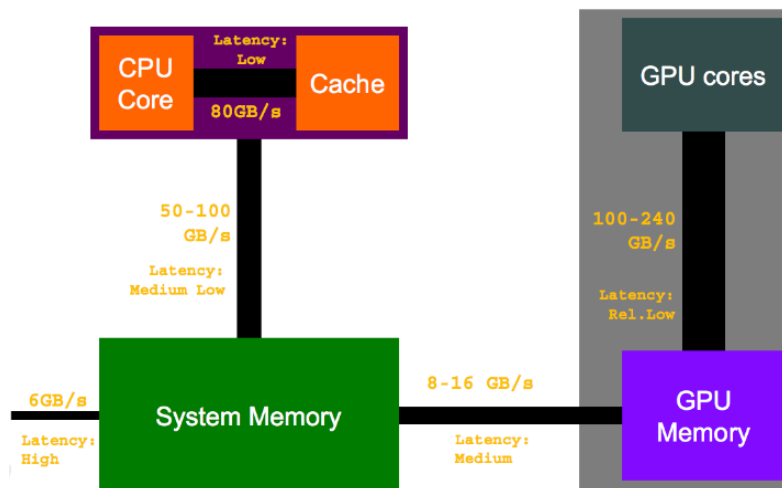


Per fare questo scambio di informazioni si utilizza una memoria nella GPU chiamata “*memoria globale*” (almeno 2 GB); in questa memoria vi risiedono inizialmente tutti i dati in comune tra CPU e GPU.

### Comunicazione CPU-GPU

Nella figura sotto vediamo la capacità dei collegamenti tra ogni componente:

- La CPU comunica con la propria RAM con una latenza medio-bassa.
- Invece tra la memoria host e la memoria della GPU (anche a causa del *PCI-Express*) la latenza può essere medio-alta perché di solito la quantità di dati da trasferire può essere molto elevata.



## Differenza di performance tra CPU e GPU

CPU e GPU possono essere entrambe molto utili per il calcolo di un algoritmo, ma non hanno le stesse caratteristiche hardware; non è detto che passando alla GPU le prestazioni degli algoritmi siano migliori rispetto alla CPU, anzi, possono anche peggiorare!

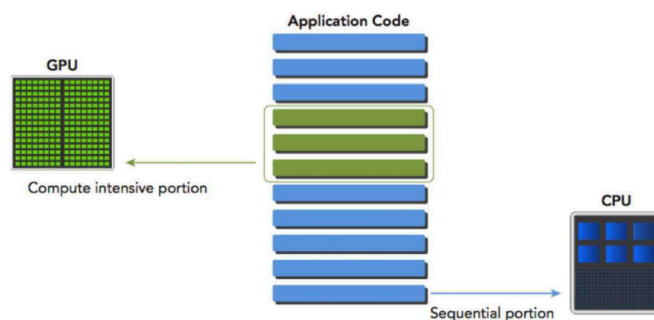
## Differenza di frequenza di clock

Notare che di solito la CPU ha una frequenza di clock anche superiore a 5 GHz; mentre la frequenza di clock di un CUDA core che si aggira intorno a 1,5 Ghz.

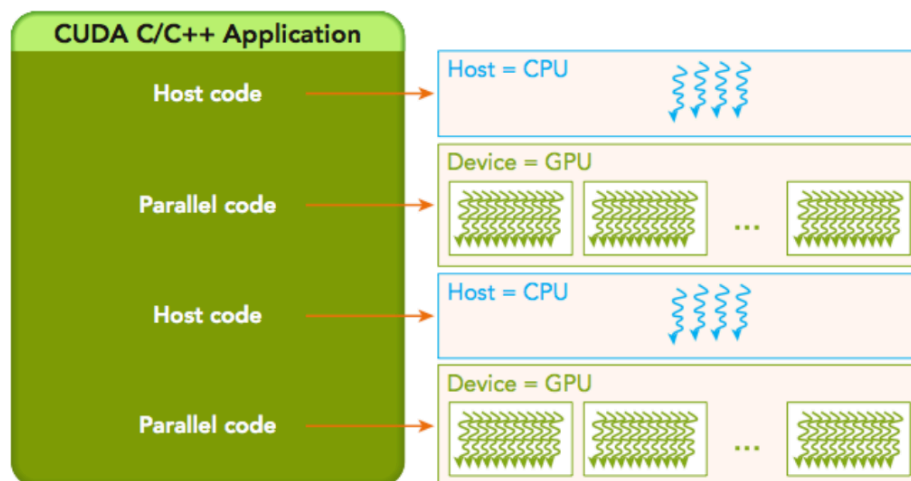
## Utilizzo di un sistema eterogeneo

Nella figura sotto il codice è stato diviso in due parti (*blu e verde*):

- Il codice azzurro è più idoneo ad essere eseguito nella CPU.
- Il codice verde è più idoneo ad essere eseguito nella GPU.



Per sfruttare entrambe al massimo è buona norma far eseguire ad ognuno dei due dispositivi il codice più idoneo alle loro capacità.



Mentre la GPU è impegnata a eseguire il codice la CPU può fare altro. Però prima o poi dovrà sincronizzarsi con la GPU per prelevare i risultati.

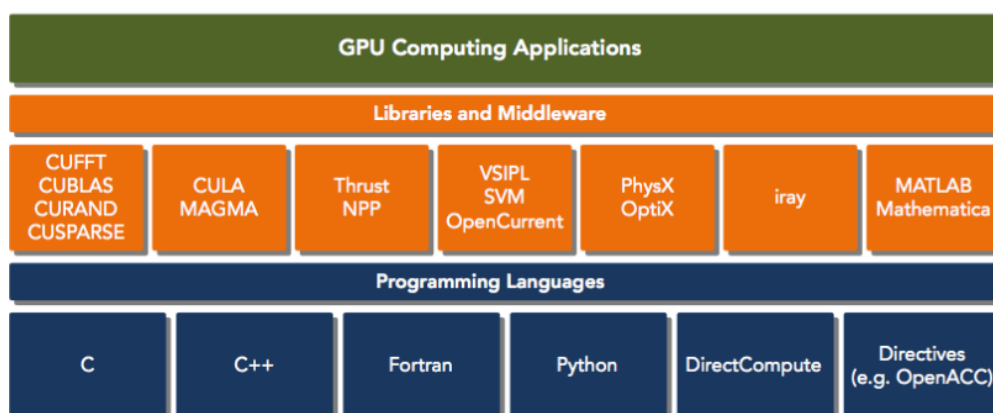
## Linguaggio CUDA C

CUDA sta per “*Compute Unified Device Architecture*”; CUDA è specializzata per un tipo specifico di GPU, ossia quella general purpose (*GPGPU*), ossia un dispositivo che non si occupa solo di computer grafica, ma anche di elaborazione dei dati.

### NVIDIA Toolkit

Per programmare questi dispositivi ci sono degli strumenti come Nvidia toolkit, il quale consente di accedere alle memorie e ai vari CUDA core (*nota che per lavorare con una scheda integrata Intel serve un kit a parte di Intel in quanto CUDA è per Nvidia*).

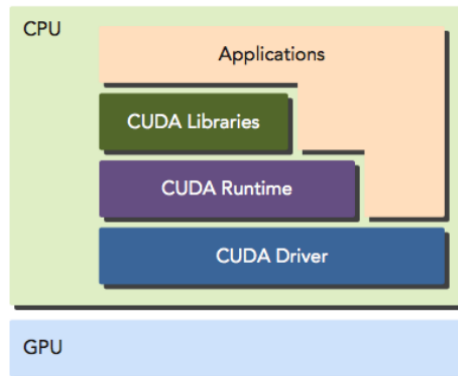
CUDA può essere un'estensione del C (*CUDA C*), in figura sotto vediamo i linguaggi di programmazione che consentono di utilizzare programmi per la GPU.



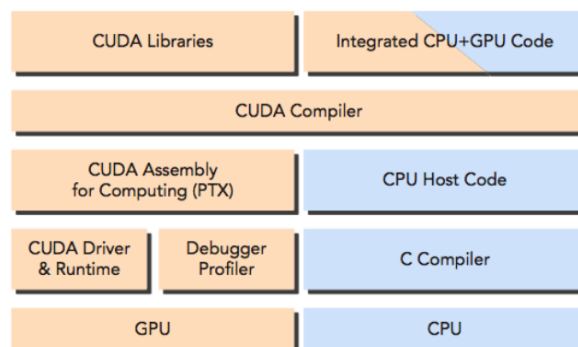
## CUDA API

CUDA fornisce due livelli di API per gestire la GPU:

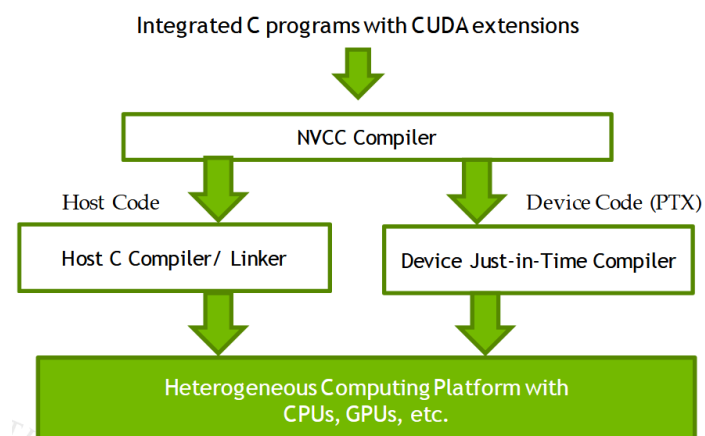
- **CUDA Runtime API**
- **CUDA Driver API**



Il codice ha una parte che viene eseguita su GPU e una parte eseguita su CPU.



La parte eseguita nella GPU è quella chiamata PTX in figura sotto, mentre l'Host Code è quello eseguito dalla CPU.



## Sintassi di CUDA C

Quando si esegue una funzione da eseguire sulla GPU bisogna mettere il nome della funzione seguita da <<< ? , ? >>>.

Per ora ignoriamo i numeri da inserire al posto dei ?.

```
int main(){
    foo<<<1, 1>>>(); // 1 thread
    return 0;
}
```

This red code is the grid. The grid is composed by the number of blocks and the number of threads per block: <<<nblocks, nthreads>>>

## Utilizzo della memoria del device

Notare che i puntatori a, b e c puntano a qualcosa nella memoria della GPU e non alla memoria host.

La memoria host e la memoria della GPU sono due dispositivi distinti: i puntatori utilizzati per comunicare con la memoria della GPU non possono essere dereferenziati come i puntatori alla memoria RAM.

Esistono delle funzioni per allocare, liberare e copiare la memoria della CPU, queste funzioni sono:

**cudaMalloc()**

Serve per allocare qualcosa nella GPU.

```
uint16_t *d_numbers;

cudaMalloc((void**) &d_numbers, N*sizeof(uint16_t));
```

**cudaFree()**

Serve per deallocare qualcosa dalla GPU.

```
cudaFree(d_numbers);
```

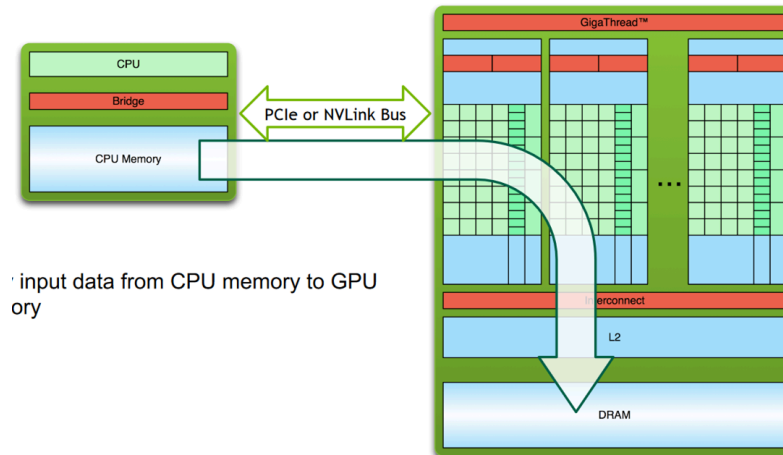
**cudaMemcpy()**

Serve per copiare qualcosa nella GPU dalla memoria Host e viceversa.

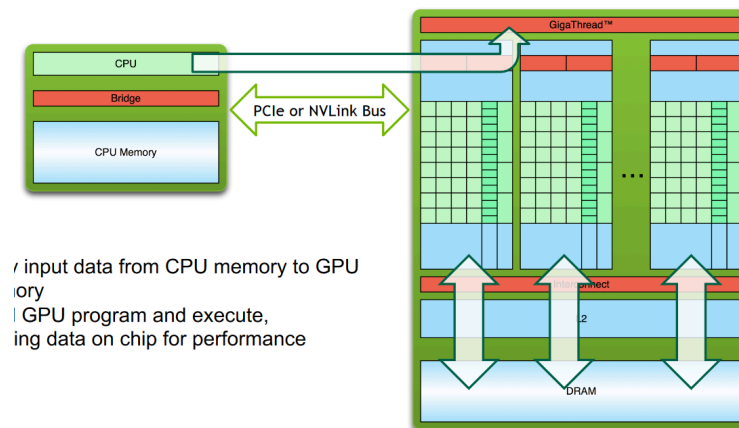
```
// HOST => GPU
cudaMemcpy(d_number, h_number, N*sizeof(uint16_t), cudaMemcpyHostToDevice);
// GPU => HOST
cudaMemcpy(h_number, d_number, N*sizeof(uint16_t), cudaMemcpyDeviceToHost);
```

## Trasferimento tra GPU e CPU

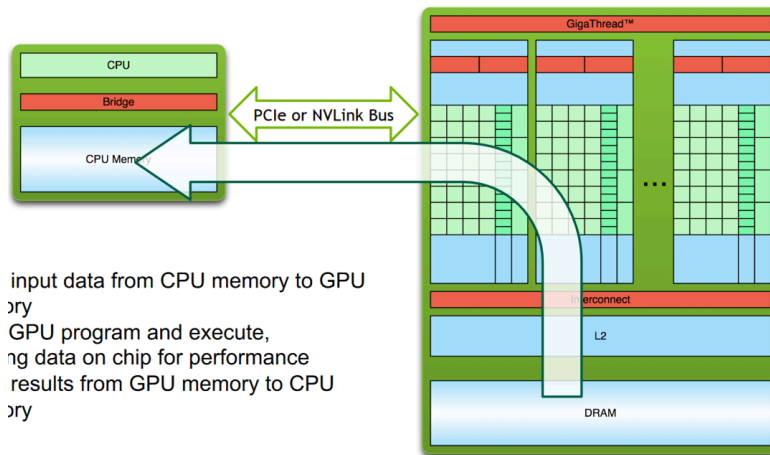
I dati sono copiati dalla memoria RAM alla memoria della CPU tramite una `cudaMemcpy()` con flag `cudaMemcpyHostToDevice`.



Il programma nella GPU viene eseguito e il risultato viene inserito nella cache.



Al termine del Kernel, i risultati della memoria della GPU vengono trasferiti nella memoria della CPU tramite una `cudaMemcpy()` con flag `cudaMemcpyDeviceToHost`.



input data from CPU memory to GPU  
 try  
 GPU program and execute,  
 ng data on chip for performance  
 results from GPU memory to CPU  
 try

## Kernel

Un kernel in CUDA è una funzione che viene chiamata all'interno della funzione main.

```
int main(void) {

    int a, b, c; // host copies of a, b, c
    int *d_a, *d_b, *d_c; // device copies of a, b, c
    int size = sizeof(int);

    // Allocate space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Setup input values
    a = 2;
    b = 7;
    ...

    ...
    // Copy inputs from host to device
    cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);

    add<<<1,1>>>>(d_a, d_b, d_c);    // Launch add() kernel on GPU

    // Copy the result back to the host
    cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);

    return 0;
}
```

## CUDA Blocks

Il numero **rosso** indica il numero di blocchi della GPU da utilizzare durante l'esecuzione della funzione `add()`.

Un blocco è un gruppo di thread: I thread all'interno di un blocco possono cooperare attraverso la memoria condivisa (*shared memory*) e possono essere sincronizzati. I blocchi in generale non possono comunicare tra loro e non possono essere sincronizzati.

```
add<<<N, 1>>>>();
```

Quindi nella chiamata in figura sotto, la funzione `add()` viene chiamata **N** volte in **N** blocchi diversi. In ogni blocco è presente 1 thread.

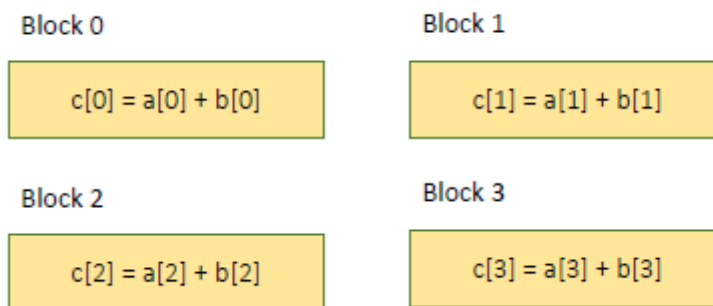


Ogni blocco ha un identificatore differente utilizzabile con `blockIdx.x`.

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

Quindi è possibile gestire un array di blocchi come avviene in figura specificando l'identificatore del blocco.

In questo esempio ogni blocco è gestito da un singolo thread; supponendo di avere 4 blocchi nella GPU avverrà questa cosa completamente in parallelo:



## CUDA Threads

Il thread è l'unità base di esecuzione e ogni thread esegue una copia del kernel. Il numero di thread per blocco si cambia cambiando il secondo numero.

```
add<<<1, N>>>>();
```

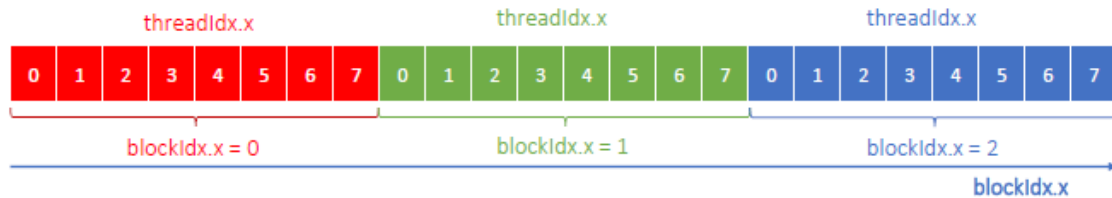
Nella funzione del caso precedente viene utilizzato l'indice del thread `threadIdx.x`.

```
__global__ void add(int *a, int *b, int *c) {  
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];  
}
```

## CUDA Grid Index

Una griglia è un insieme di blocchi: I blocchi all'interno di una griglia non possono comunicare direttamente tra loro e non possono essere sincronizzati.

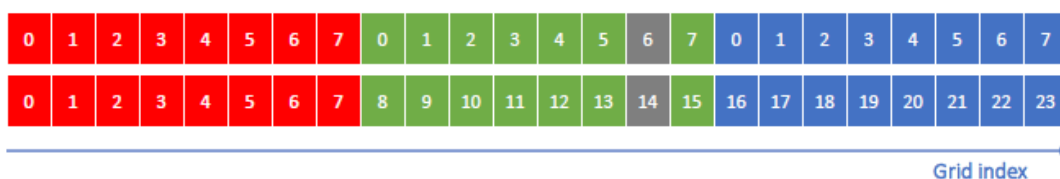
Consideriamo un array di 24 locazioni da suddividere in 3 blocchi; va da sé che ogni blocco avrà 8 thread.



```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

- `blockIdx.x` è l'indice del blocco corrente, è unico in tutta la GPU.
  - I primi 8 elementi sono dati al blocco con `blockIdx.x = 0`.
  - I secondi 8 elementi al block con `blockIdx.x = 1`.
  - Gli ultimi 8 al blocco con `blockIdx.x = 2`.
- `blockDim.x` è il numero di thread per blocco.
- `threadIdx.x` è l'indice del thread corrente all'interno del blocco, è unico solo all'interno del blocco.
- `index` (detto anche *grid id*) è l'indice globale del thread, è unico in tutta la GPU.

Con questa formula possiamo vedere che il grid id della cella **grigia** (la 6° del blocco verde) il suo indice si calcolerà come:  $6 + 1 * 8 = 14$



Possiamo calcolare la somma tra due vettori solo calcolando l'indice come nel riquadro sotto e utilizzarlo per accedere agli elementi corretti di `a`, `b` e `c`.

```
__global__ void add(int *a, int *b, int *c) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    c[index] = a[index] + b[index];  
}
```

Adesso proviamo a scrivere una versione più generica del codice: Notare che con la GPU non scriviamo dei cicli `for` o `while`, ma tramite una singola esecuzione del kernel risolviamo il processo che altrimenti sarebbe stato iterativo.

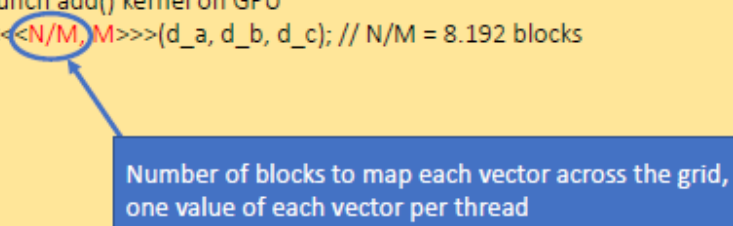
```
#define N (2048*2048)    // size of the vectors to be added
#define M 512            // number of threads per block

int main(void) {

    int *a, *b, *c;      // host copies of a, b, c
    int *d_a, *d_b, *d_c; // device copies of a, b, c
    ...

    // Launch add() kernel on GPU
    add<<N/M, M>>>(d_a, d_b, d_c); // N/M = 8.192 blocks
    ...

}
```



Number of blocks to map each vector across the grid, one value of each vector per thread

## Evitare un segmentation fault nella GPU

C'è anche la possibilità che il numero di valori da calcolare sia minore del numero di thread che vengono chiamati.

```
__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n)
        c[index] = a[index] + b[index];
}
```

Per gestire questo caso si usa un `if`: Se il grid index del thread è minore della dimensione degli array; in questo modo il thread non causerà un segmentation fault all'interno della GPU.

## Cuda Warps

Gruppo di 32 thread che vengono eseguiti in modo parallelo e sincrono (eseguono le stesse istruzioni simultaneamente) all'interno di un blocco.

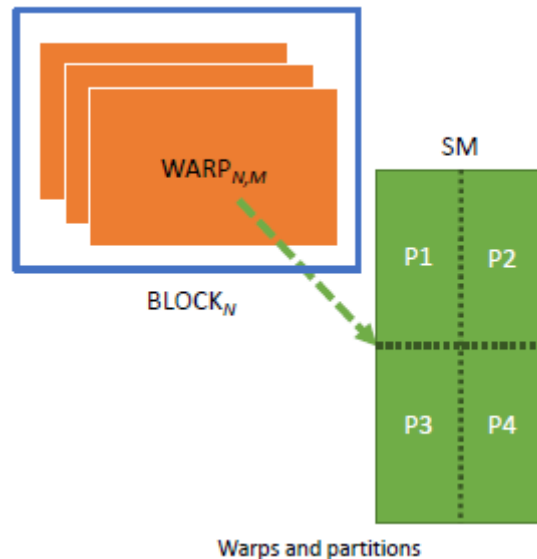
I warps sono l'unità fondamentale di esecuzione nella pipeline di elaborazione della GPU.

I blocchi vengono eseguiti in “*streaming multiprocessor*” dove il massimo numero di thread raggiungibile è di 2048 e il numero massimo di blocchi accessibili è 32.

I blocchi sono divisi in sotto-blocchi chiamati “*warps*”.

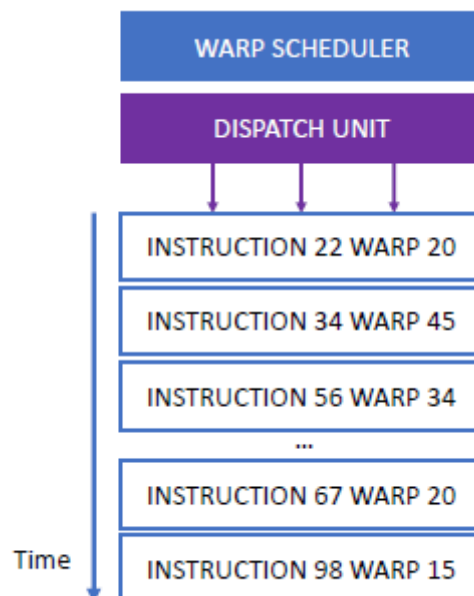
## Warp Scheduler

Ogni partizione ha dei componenti hardware specifici che sono il warp scheduler, la dispatch unit e i CUDA cores.



Che cosa fa lo warp scheduler?

1. Crea una coda di warp "leggibili", per leggibili si intende che sono pronti per essere eseguiti.
2. Ad ogni ciclo di clock, lo scheduler manda uno warp in esecuzione e aggiorna la coda dei warp.



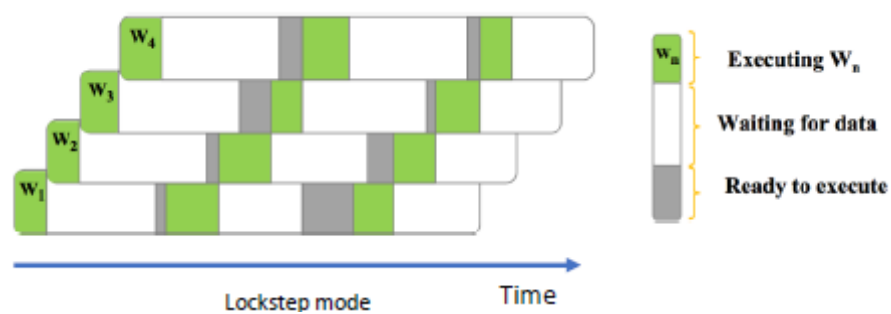
Ogni warp ha un `threadIdx.x` minimo e un `tthreadIdx.x` massimo, perchè gli warp sono sottoinsiemi di thread container dentro un blocco.

Da notare che non avremo mai il numero esatto di thread per accedere ad un array o ad una certa struttura dati, in alcuni casi i thread in più e di sicuro non utilizzati possono essere disabilitati per migliorare l'esecuzione e le sue prestazioni.

### Esecuzione degli Warp - Lockstep Mode

Con questa modalità gli warp ciclano in tre stati:

1. Esecuzione (*di solito eseguiti per un tempo prefissato e facendo a turno con altri warp*).
2. In attesa di dati
3. Pronti ad essere eseguiti.



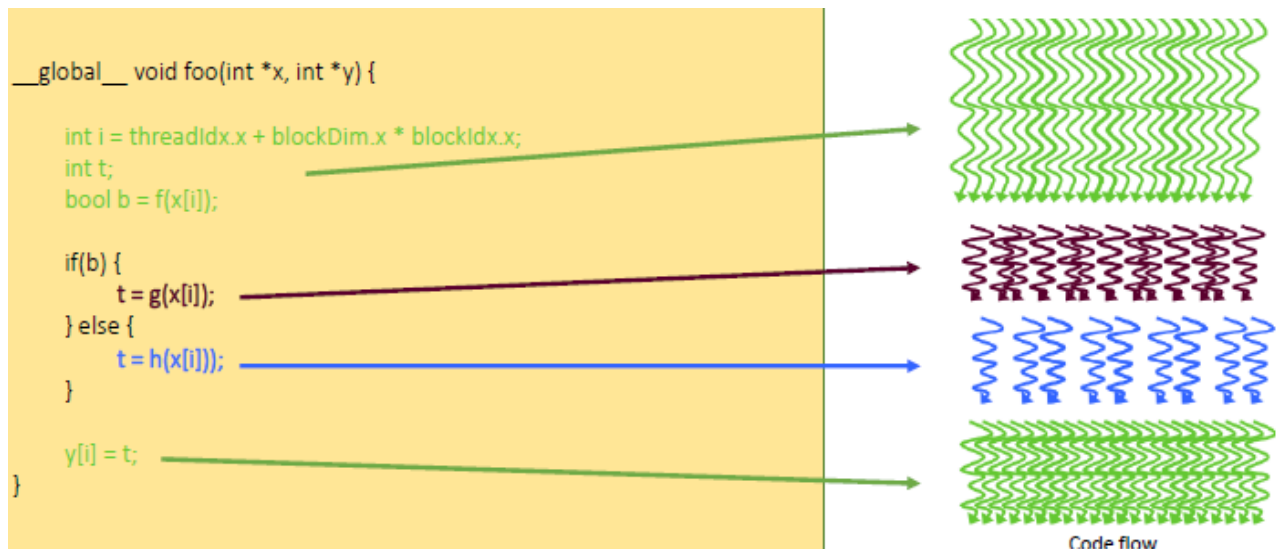
## Divergenza degli Warp

Se i thread in un warp devono eseguire istruzioni diverse a causa di un ramo condizionale (*ad esempio, un if-else*), si verifica un fenomeno chiamato "*divergenza del warp*".

In tal caso, il warp eseguirà le istruzioni di un ramo per tutti i thread che lo seguono, quindi eseguirà le istruzioni dell'altro ramo per i restanti thread.

L'esecuzione del warp viene di fatto spezzata a metà obbligando la GPU ad eseguire un ramo per volta.

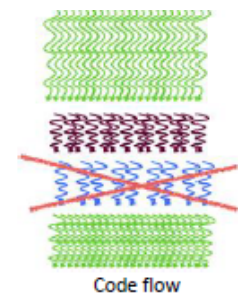
Per massimizzare l'efficienza, è importante che i thread all'interno di un warp eseguano percorsi di esecuzione simili. La divergenza può causare stalli e ridurre l'efficienza.



In questo caso non abbiamo parallelismo tra i thread del warp.

L'idea migliore per mitigare questo problema è quella di provare a sincronizzare i thread: si attende che tutti i thread siano sincronizzati, questi casi sono il caso peggiore e interrompono il programma.

```
if (threadIdx.x < 16) {  
    foo_then();  
    __syncthreads();  
} else if (threadIdx.x >= 16) {  
    foo_else();  
    __syncthreads();  
}
```



Le prestazioni diminuiscono con l'aumentare del grado di divergenza negli warp.

## Migliorare lo stalling dei warp

Gli stalli (o stall) si verificano quando i thread di un warp devono aspettare a causa di operazioni come:

- Accesso alla memoria globale.
- Divergenza nei percorsi di esecuzione.
- Sincronizzazione tra thread.

## Coalescing degli Accessi alla Memoria

Organizzare gli accessi alla memoria in modo che i thread all'interno di un warp accedano a posizioni di memoria consecutive.

Minimizza i tempi di latenza della memoria e sfrutta appieno la larghezza di banda della memoria.

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;

C[idx] = A[idx] + B[idx];
```

## Minimizzare la Divergenza del Warp

Ridurre al minimo l'uso di rami condizionali all'interno di un warp.

Evita che i thread all'interno di un warp seguano percorsi di esecuzione diversi.

```
__global__ void exampleKernel(int *data) {

    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx % 2 == 0) {

        data[idx] *= 2;

    } else {

        data[idx] *= 3;

    }

}
```

In questo caso, il warp può essere diviso. Se possibile, riorganizzare il codice per ridurre la divergenza.

## Uso della Shared Memory

Utilizzare la memoria condivisa per ridurre gli accessi alla memoria globale. La memoria condivisa ha una latenza molto inferiore rispetto alla memoria globale.

```

__global__ void exampleKernel(int *data) {
    __shared__ int sharedData[32];

    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sharedData[threadIdx.x] = data[idx];

    __syncthreads();

    // Ora i dati sono pronti nella memoria condivisa

    data[idx] = sharedData[threadIdx.x] * 2;
}

```

## Occupancy Elevata

Aumentare il numero di thread attivi per nascondere la latenza della memoria.

Più thread attivi permettono alla GPU di eseguire altri thread mentre alcuni sono in attesa.

Ottimizzare il numero di thread per blocco e il numero di blocchi per garantire che ci siano abbastanza thread attivi.

- **worst:** kernel<<<N, 1>>> (...)
- **okay:** kernel<<<(N+31) / 32, 32>>>(...)
- **better:** kernel<<<(N+127) / 128, 128>>>(...)

## Prefetching

Caricare i dati necessari in anticipo nella memoria condivisa per ridurre la latenza dell'accesso ai dati.

```

__global__ void exampleKernel(int *data) {
    __shared__ int sharedData[32];

    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sharedData[threadIdx.x] = data[idx];

    __syncthreads();

    // Ora i dati sono pronti nella memoria condivisa

    data[idx] = sharedData[threadIdx.x] * 2;
}

```



## CUDA Grid

A livello architetturale abbiamo i CUDA core i quali contengono una serie di processi che possono essere eseguiti.

I thread possono essere raggruppati in un blocchi di una, due e tre dimensioni.

### Dimensionalità della griglia

Le dimensioni della griglia sono definite da tre parametri:

**Dim3 gridDim:** Specifica il numero di blocchi in ciascuna dimensione della griglia. È un vettore con tre componenti:

- **gridDim.x:** Numero di blocchi lungo l'asse x.
- **gridDim.y:** Numero di blocchi lungo l'asse y.
- **gridDim.z:** Numero di blocchi lungo l'asse z.

### Dimensionalità di un blocco

Le dimensioni di un blocco sono definite da tre parametri:

**Dim3 blockDim:** Specifica il numero di thread in ciascun blocco. È un vettore con tre componenti:

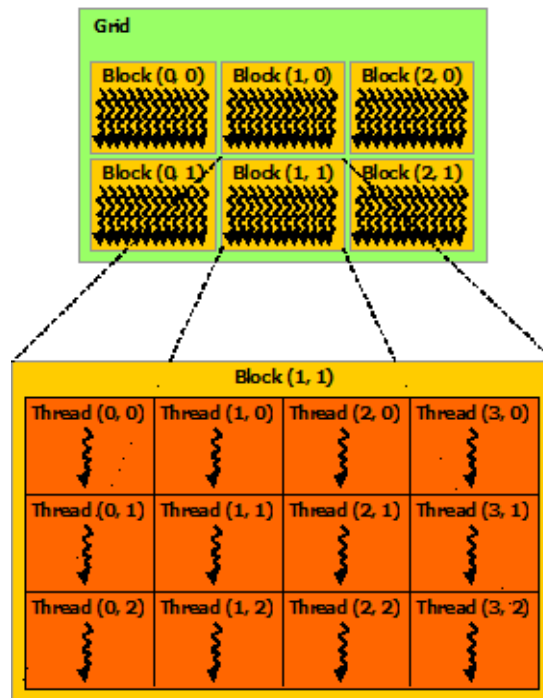
- **blockDim.x:** Numero di thread lungo l'asse x.
- **blockDim.y:** Numero di thread lungo l'asse y.
- **blockDim.z:** Numero di thread lungo l'asse z.

### Funzionamento della griglia

La griglia organizza i blocchi in un array tridimensionale. Ogni blocco, a sua volta, organizza i thread in un array tridimensionale.

La configurazione tridimensionale permette una scalabilità flessibile, adattandosi a problemi di diverse dimensioni e complessità.

L'uso degli indici di blocchi e thread permette di suddividere e accedere ai dati in parallelo, ottimizzando il calcolo e riducendo i tempi di esecuzione.



Sia una griglia di 16x16 blocchi; e ogni blocco è definito come una griglia di 32x32 thread.

Le variabili degli assi avranno le seguenti dimensioni:

```
grid.x = 16 grid.y = 16
```

```
block.x = 32 block.y = 32 block.z = 1
```

```
#define N 32 // Array size
```

```
__global__ void add(int *a, int *b, int *c) {
    int col = threadIdx.x + blockIdx.x * blockDim.x;
    int row = threadIdx.y + blockIdx.y * blockDim.y;
    int index = col + row * N;

    if (col < N && row < N)
        c[index] = a[index] + b[index];
}
```

```
int main() {
    dim3 block(16,16);
    dim3 grid(N/block.x, N/block.y);
    add<<<grid, block>>>>(d_a, d_b, d_c);
}
```

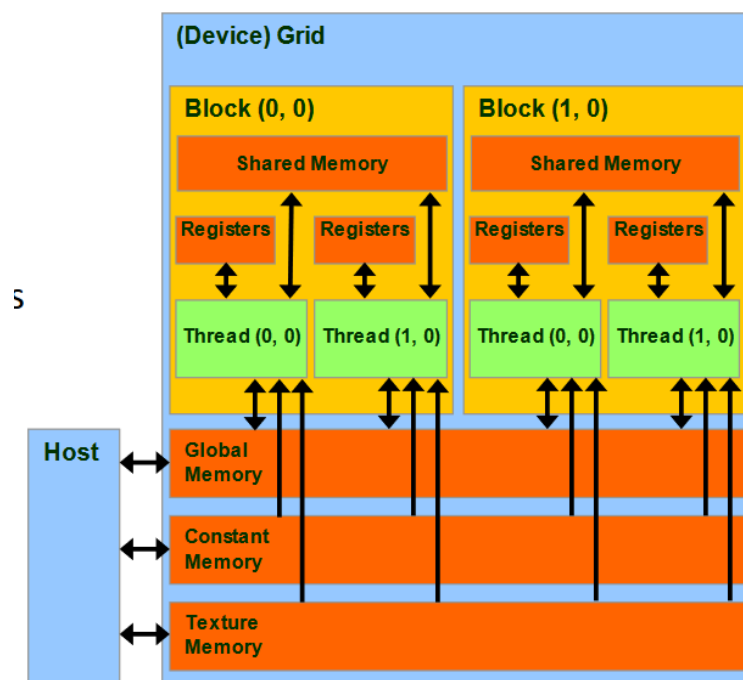
```
}
```

Esempio di come tradurre il seguente blocco di codice su GPU:

```
for(int i = 0; i < N; i++)  
    for(int j = 0; j < N; j++){  
        c[i][j] = 0;  
        for(int k = 0; k < N; k++)  
            c[i][j] = c[i][j] + a[i][k] * b[k][j];  
    }  
}
```

## Gestione della memoria della GPU

Su una GPU abbiamo diversi tipi di memorie, i registri, le cache tra cui la shared memory e anche la memoria globale.



### Texture memory

Inoltre abbiamo la texture memory dedicata esclusivamente alle texture da mostrare in video in display.

In questa memoria i thread possono fare operazioni di lettura e scrittura.

## Shared memory

Nelle shared memory si può accedere per blocco in lettura/scrittura.

## Registri

I registri sono effettivamente il tipo di memoria più veloce della memoria video e principalmente contengono gli operandi utilizzati per le operazioni nei CUDA core.

I registri sono memoria privata del thread (*ossia ad accesso esclusivo*).

## Local memory

Regione della global memory visibile ed accessibile al thread che ne fa richiesta.

Le informazioni che possono essere contenute al suo interno possono risiedere in una memoria cache di livello 1.

## Shared memory

Piccola quantità presente sulla regione hardware dedicata allo streaming multiprocessor.

Questo spazio è utile per una comunicazione inter-thread, ovvero tra thread che appartengono allo stesso blocco.

## Cache della GPU

Ovviamente consentono un accesso molto più rapido, è completamente trasparente, si trova tra la memoria globale e i CUDA core.

Le cache anche in GPU sono suddivise in livelli:

### Cache L0

Cache solo per istruzioni, è utilizzata per ciascun blocco di thread.

### Cache L1

Utilizzata per ciascun streaming di multiprocessor, una parte di questa cache è la **texture cache** contenente le texture che devono essere elaborate. Le dimensioni di queste cache possono essere decise a runtime. Texture cache è di sola lettura.

### Cache L2

Condivisa tra tutti gli streaming processor, gestisce operazioni di load e store, e anche di texture.

## Global memory

Memoria molto grande gestita insieme al dispositivo host, ci consente di trasferire informazioni dalla RAM centrale alla RAM della GPU e viceversa.

## Constant memory

Memoria sola lettura ed è porzione della memoria globale.

Per passare le informazioni esclusivamente alla memoria costante è `cudaMemcpyToSymbol()`.

Per ottenere le massime prestazioni tutti i thread dello stesso warp devono accedere allo stesso indirizzo.

## Qualifier

Il qualifier è una keyword che modifica il tipo di variabile, deve essere specificata prima del tipo, CUDA fornisce differenti qualificatori.

### **Variabile automatica**

Può essere definita senza specificare nessun qualificatore, può essere utilizzata all'interno del singolo registro.

### **Variabile array**

Può essere memorizzato nei vari registri se consentito, oppure in memoria globale. Può essere utilizzata senza qualificatore.

### **Variabile Shared**

Se vogliamo che la variabile sia utilizzata nella memoria shared serve il qualificatore `__shared__`

### **Variabile device**

Serve il qualificatore `__device__`

### **Variabile costante**

Serve il qualificatore `__constant__`

## Race condition

Un esempio è il seguente: vogliamo gestire l'incremento di una variabile che inizialmente vale 7.

Se abbiamo due thread non è detto che il risultato venga 9; potrebbe accadere che entrambi accedono prima di modificare il valore, entrambi leggono 7 ed il risultato ottenuto è 8 nonostante ho incrementato due volte.

## Operazioni atomiche

Per prevenire questo comportamento è possibile utilizzare operazioni atomiche.

Le operazioni atomiche hanno l'unico difetto di creare un collo di bottiglia, poiché fermano il parallelismo.

Il set di funzioni **CAS** è molto utile per gestire i la race condition.

## GPU in Python

In Python per ogni funzione bisogna utilizzare il decoratore **@cuda.jit**.

La funzione kernel non prevede di restituire nulla all'host, il passaggio del risultato avviene solo tramite puntatore.

In questo caso per chiamare la funzione sul device si utilizzano le parentesi quadrate al posto delle parentesi angolari `[blocks, threads]`.

Per dichiarare una funzione come device utilizziamo questa sintassi:  
`@cuda.jit(device=True)`.

Per calcolare l'indice di griglia per identificare il thread si calcola come in slide a pagina 9.

La funzione `cuda.grid()` consente ad un thread di recuperare gli indici di un determinato thread.

In Python per allocare e trasferire i dati in memoria il meccanismo è molto simile che in C.

Anche in Python tramite `cuda.shared.array()` è possibile allocare la memoria condivisa. Utilizzando la funzione `cuda.syncthreads()` si ottiene lo stesso comportamento della `syncthreads()` in C.

## Speedup

Che cosa sono i **FLOPS**? SONO IL NUMERO DI OPERAZIONI IN VIRGOLA MOBILE ESEGUITI IN un secondo. Possono essere osservati dalla documentazione fornita dal produttori o calcolati manualmente, però in questo caso non è facile.

Come si può calcolare questo valore? Facendo il prodotto tra il clock del dispositivo, il numero di core, il parametro **alfa** e il **fattore di scala**. Questi due ultimi valori dipendono dal dispositivo, calcolando il prodotto è possibile approssimare la quantità di GFlops.

Ovviamente bisogna tenere conto anche del tempo, quindi i FLOPS possono essere calcolati con questa formula:

$$FLOPS_{eff} = \left( \left( \sum N_{ops} \times N_{elem} \right) \div 10^9 \right) \div time$$

In questo caso vediamo il calcolo della banda di accesso alla memoria globale della GPU.

$$TR = 1,5 \times 2 \times 4 \times 2 = 24 \text{ GTps} = 24 \text{ Gbps}$$

$$BW = TR \times (384 \div 8) = 1152 \text{ GBps} = 1,152 \text{ TBps}$$

Il calcolo del throughput può essere effettuato con il seguente calcolo.

$$THR = BW_{eff} = \left( (B_r + B_w) \div 10^9 \right) \div time$$

Come misurare il tempo di esecuzione?

Vogliamo misurato lo start e l'end time dell'esecuzione del nostro frammento di codice.

```
start = timer();
ret = foo();
end = timer();
printf("Response time: %f", end-start);
```

Ripetendo le misurazioni almeno 30 volte è possibile calcolare la media armonica o aritmetica in base alla grandezza che misuriamo:

- **Harmonic mean** for index inversely proportional to time, e.g.:

$$THR = N \times \sum_{i=0}^n \frac{1}{THR_i}$$

- **Arithmetic mean** for index proportional to time, e.g.:

$$ACET = \frac{1}{N} \times \sum_{i=0}^n ET_i$$

Per calcolare il tempo possiamo utilizzare i timer usati per la CPU oppure usare i **CUDA event**, si creano due oggetti, uno “tic” ed uno “toc”, il clock utilizzato è quello della CPU e non quello della GPU.

```
cudaEvent_t tic, toc; float elapsed;
cudaEventCreate(&tic);
cudaEventCreate(&toc);

cudaEventRecord(tic, 0);
cudaKernel<<<nblocks, nthreads>>>(...);
cudaEventRecord(toc, 0);
cudaEventSynchronize(toc);

cudaEventElapsedTime(&elapsed, tic, toc);
cudaEventDestroy(tic); cudaEventDestroy(toc);
printf("The elapsed time in GPU was: %.2f ms\n", elapsed);
```

Ovviamente i trasferimenti non vanno inclusi nel calcolo del tempo, quindi prima si fanno i trasferimenti, poi il calcolo dello start e dell’end li facciamo subito prima e subito dopo della funzione.



## Profiler

Un tipo di profiler su GPU è **Nsight Compute**, fornisce informazioni abbastanza dettagliate sulla GPU, è già incluso con l'installazione di CUDA (a regola).

Tra le informazioni utili troviamo il throughput della GPU, l'occupazione, l'utilizzo della pipe, dimensione dei trasferimenti della memoria, come sono schedulati gli warp.

Un altro profiler utile per studiare in modo specifico l'esecuzione del nostro codice è **NVIDIA Nsight Systems**, anche questo dovrebbe essere già installato con CUDA.

## Debugger

Nvidia consente di registrare alcune impostazioni di debug, scrivendo `-g` viene attivato il debug per CPU, mentre `-G` attiva il debug per GPU.

**`nvcc -g -G kernel.cu -o kernel`**

Ci sono due tipologie di debugger, uno è **cuda-memcheck** che identifica le fonti di problema in memoria, quindi overflow, memory leak, accessi fuori limite in memoria ecc.

Invece **cuda-gdb** consente il debug dei dispositivi esterni.

In figura vediamo uno workflow su come effettuare il debug.

## 1. Set a breakpoint or enable a tool

```
(cuda-gdb) break kernel.cu:27  
(cuda-gdb) set cuda memcheck on
```

## 2. Run the debugger

```
(cuda-gdb) run  
(cuda-gdb) backtrace
```

## 3. Inspect the current state, e.g.:

```
(cuda-gdb) print variable_name
```

## 4. Figure out what went wrong

# Progetto

## Parte CPU

## Parte GPU

### Cosa fare?

Vedere e segnarsi le specifiche della nostra GPU

### Strumenti

Nvidia Nsight Compute è il profiler da usare quando si lavora con la GPU.

Le valutazioni vanno fatte al variare della dimensione delle griglie e del numero di thread, se si riesce si aumentano i thread, altrimenti bisogna aumentare anche la dimensione del problema.

Valutare anche il trasferimento, ovvero da device alla GPU.

### Consigli

Grafico dello speedup con nell'asse delle x la varie dimensioni delle griglie con gli warp e nell'asse delle y lo speedup. Gli assi non devono essere in scala logaritmica.

### Come osservare le caratteristiche dei dispositivi Nvidia?

Nvidia fornisce un plugin open source scaricabile da github, questo programma mostra in shell le informazioni della GPU, tra quelle importanti abbiamo una quantità globale di RAM, abbiamo anche la quantità globale della memoria condivisa, il numero di registri disponibili per ciascun blocco di memoria.

# Domande d'esame

## Pipeline

### Cos'è e come funziona la pipeline?

La pipeline serve ad aumentare il throughput misurato sulle unità funzionali. Le fasi dell'esecuzione di una singola istruzione comprende 5 fasi:

1. Fetch dell'istruzione dalla memoria
2. Decodifica
3. Esecuzione
4. Eventuale accesso in memoria
  - In questa fase il processore accede alla memoria per leggere/scrivere un operando.
5. Scrittura in registro

Ogni fase utilizza una sola unità funzionale della CPU. Quindi possiamo permettere che più istruzioni siano in esecuzione contemporaneamente. In particolare, ad ogni ciclo di clock il processore:

1. **IF**: Al ciclo di clock  $i$  il processore preleva una nuova istruzione.
2. **ID**: Decodifica dell'istruzione prelevata al ciclo  $i-1$ .
3. **EX**: Esegue l'istruzione prelevata al ciclo  $i-2$ .
4. Compie l'accesso in memoria per conto dell'istruzione prelevata al ciclo  $i-3$ .
5. Scrive il risultato dell'istruzione prelevata al ciclo  $i-4$  nel registro di destinazione specificato.

## Dal punto di vista delle performance, quali sono i fattori che possono incidere sul throughput delle unità funzionali?

Misurando il throughput come numero di istruzioni gestite dalle unità funzionali in un ciclo di clock.

Nel caso ideale  $\text{throughput} = 1$ : ad ogni ciclo di clock si esegue un'istruzione e ad ogni ciclo di clock ogni unità funzionale esegue una fase di un'istruzione.

Non sempre si avranno queste prestazioni poiché **la pipeline è costretta a stallare per consentire la risoluzione degli hazard**. In questo caso throughput non è 1 perchè durante gli stalli, le unità funzionali restano ferme in attesa che l'hazard si risolva.

La presenza di questi hazard dipendono dal programma da eseguire.

### Structural Hazards

**Quando un'istruzione cerca di accedere ad un'unità funzionale già occupata da un'altra istruzione.**

Visto che le istruzioni sono eseguite in parziale sovrapposizione, può accadere che una determinata combinazione di istruzioni acceda alla stessa unità funzionale ma in due stage diversi.

Questa tipologia di hazard nasce specialmente nei casi in cui non tutte le unità funzionali non sono duplicate o pipelined.

Vediamo un esempio: supponiamo che la `instruction memory` e la `data memory` siano unificate in una sola unità funzionale (*indicheremo la memoria unificata con Mem*).

Avendo una memoria unificata, avremo (*sempre*) un conflitto tra l'istruzione in fase di fetch e l'istruzione che si trova nello stage di accesso in memoria .

| Instruction    | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ | $C_7$ | $C_8$ |
|----------------|-------|-------|-------|-------|-------|-------|-------|-------|
| LD R2, \$0[R1] | Mem   | Reg   | ALU   | Mem   | Reg   |       |       |       |
| ADD R5, R4, R3 |       | Mem   | Reg   | ALU   | Mem   | Reg   |       |       |
| SUB R8, R7, R6 |       |       | Mem   | Reg   | ALU   | Mem   | Reg   |       |
| ST R3, \$4[R1] |       |       |       | Mem   | Reg   | ALU   | Mem   | Reg   |

**Questo tipo di hazard può essere risolto duplicando (o rendendo pipelined, quindi dividendo in fasi anche le singole unità funzionali) tutte le unità funzionali soggette a possibili conflitti.**

## Data hazard

Quando un'istruzione necessita del risultato di un'altra istruzione per poter proseguire l'esecuzione.

**ADD** R1, R2, R3

**SUB** R5, R4, R1

La **SUB** necessita in lettura del risultato prodotto dalla **ADD**; ma quell'operando è prodotto dall'istruzione precedente.

| Instruction    | C <sub>1</sub> | C <sub>2</sub> | C <sub>3</sub> | C <sub>4</sub> | C <sub>5</sub> | C <sub>6</sub> |
|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| ADD R1, R2, R3 | IF             | ID             | EX             | MEM            | WB             |                |
| SUB R5, R4, R1 |                | IF             | ID             | EX             | MEM            | WB             |

L'istruzione **ADD** scrive il risultato dell'addizione nel file register nella fase **WB** (che si svolge durante il ciclo di clock C<sub>5</sub>). Mentre il prelievo dell'operando contenuto in R1 da parte dell'istruzione **SUB** avviene nella fase **ID** (che si svolge durante il ciclo di clock C<sub>3</sub>).

Per garantire che la **SUB** utilizzi l'operando corretto, è necessario bloccarla nello stage **ID**: ciò implica stallare la pipeline fino a quando l'istruzione **ADD** scrive il risultato sul register file.

## Forwarding

Una soluzione al data hazard è dell'hardware aggiuntivo che permette di propagare il risultato in uscita da un'unità funzionale verso tutte le altre presenti all'interno del processore.

**ADD** R1, R2, R3

**SUB** R5, R2, R1

**AND** R5, R3, R1

**OR** R7, R6, R1

Tutte le istruzioni dopo la **ADD** utilizzano il suo risultato in R1 in lettura.

Implementiamo il forwarding:

- Dall'output dell'ALU verso uno dei suoi input.
  - Questo risolve l'hazard sollevato dall'istruzione **SUB**.
- Dall'output dello stage **MEM** all'input dell'ALU.
  - Questo risolve l'hazard sollevato dall'istruzione **AND**.
- Il risultato è già stato scritto nel file register e non serve il forwarding.

- Questo risolve l'hazard sollevato dall'istruzione *OR*.

| Instruction    | C <sub>1</sub> | C <sub>2</sub> | C <sub>3</sub> | C <sub>4</sub> | C <sub>5</sub> | C <sub>6</sub> | C <sub>7</sub> | C <sub>8</sub> |
|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| ADD R1, R2, R3 | IM             | Reg            | ALU            | DM             | Reg            |                |                |                |
| SUB R5, R2, R1 |                | IM             | Reg            | ALU            | DM             | Reg            |                |                |
| AND R5, R3, R1 |                |                | IM             | Reg            | ALU            | DM             | Reg            |                |
| OR R7, R6, R1  |                |                |                | IM             | Reg            | ALU            | DM             | Reg            |

## Control Hazards

**JE** R1, taken

**SUB** R2, R3, R4

**JMP** cont

taken:

**ADD** R2, R3, R4

cont:

...

Se la condizione è falsa: Tutto procede normalmente.

Se la condizione è vera: L'esecuzione parallela non può continuare perché il risultato della **JE** si saprà al termine della fase di decodifica.

Nel mentre la pipeline ha seguito il PC e ha eseguito la fetch della **SUB**. Ma l'istruzione che la pipeline dovrebbe eseguire sarebbe la **ADD**.

Per garantire la corretta esecuzione del programma, la pipeline deve stallare fino all'arrivo del risultato dell'istruzione di salto.

Questo hazard si mitiga con il branch prediction, che ci permette in sostanza di ignorare il problema eseguendo entrambi i rami; all'arrivo del verdetto tengo solo il ramo giusto.

## Cache

### Come sono gestite le scritture dalla cache?

Una scrittura deve sempre essere eseguita dopo che il controllore della cache ci ha dato conferma di hit o miss. Questo perché non possiamo rischiare di modificare un blocco in cache che non è quello giusto.

In caso di miss:

- **Write-Allocate:**
  - Si alloca in cache e si scrive sul blocco in cache.
- **No-Write-Allocate:**
  - Si scrive in memoria ma non si alloca in cache.

In caso di hit:

- **Write-back:**
  - Si scrive in cache tutte le scritture e al momento in cui questo blocco viene eliminato dalla cache si aggiorna anche il blocco in memoria.
  - Si ha un bit `dirty` che se a 1 indica che la cacheline è stata modificata e la memoria non è aggiornata.
  - Più complessa ma molto più prestante, perché la RAM non diventa un collo di bottiglia per le scritture.
- **Write-through:**
  - Ogni aggiornamento viene riflettuto sia sulla cache che sulla memoria.
  - Più semplice ma anche più lenta.
  - Usata spesso in caso di multiprocessori poiché rende molto più semplice mantenere la coerenza.
  - Spesso scelta rispetto alla write-back perché si preferisce avvantaggiare le letture invece che le scritture.

### Come sono gestite le cache miss da un processore pipelined?

Una cache miss è un load-use hazard (*attesa di un risultato*); quindi provoca uno stallo della pipeline, poiché il thread rimane bloccato fino a che la miss non viene risolta.

Quando il blocco viene ottenuto, la pipeline:

- Ripete il ciclo di fetch (*andando sulla cache stavolta*) se la miss è stata fatta su una istruzione.
- Ripete l'accesso in memoria (*andando sulla cache stavolta*) se la miss è stata fatta su un operando.

## Come funziona la cache in presenza di indirizzi virtuali?

Il TLB e la cache sono entrambi memorie utilizzate per ottimizzare qualcosa; nello specifico un TLB è una cache fully-associative

Mettere prima l'una o l'altra nel chip ha conseguenze non indifferenti:

### Physically Indexed Physically Tagged [ CPU → TLB → CACHE ]

La cache vede solo indirizzi fisici, poiché in ingresso ha l'output del TLB.

- **Molto più semplice**, poiché tutti i livelli di cache lavorano solo con indirizzi fisici (*come prima*) quindi il circuito delle cache non viene cambiato rispetto a quando non c'era il TLB..
- **Maggiore overhead**, in quanto la traduzione è abbastanza lenta.

### Virtually Indexed Virtually Tagged [ CPU → CACHE L1 → TLB → CACHE ]

La cache vede solo indirizzi virtuali, poiché invia l'uscita a l'input del TLB.

- **Molto più complesso**: poiché alcuni livelli di cache lavorano solo con indirizzi virtuali.
- **Meno overhead**: La cache L1 è priva dello overhead del TLB.
- **Cambio di contesto**: In quanto la L1 contiene solo indirizzi virtuali, deve essere flushata insieme al TLB in caso di cambio di processo.
- **Aliasing**: Può capitare che due processi condividano la stessa pagina di memoria fisica anche se si riferiscono alla pagina fisica con indirizzi virtuali diversi.
  - Se due processi condividono un blocco di memoria fisico e quest'ultimo è presente in cache su due blocchi diversi e se un processo scrive su uno dei suoi blocchi in cache, allora nasce un problema di incosistenza visto che l'altro processo non vede la modifica nella sua copia di blocco.
  - Si risolve complicando la cache L1.

Un modo per implementare questa tecnica è quella di accedere alla cache L1 utilizzando l'offset dell'indirizzo virtuale come index e nel mentre tradurre l'indirizzo con il TLB. Se il campo restituito dalla L1 è uguale al campo restituito dal TLB è hit.



## Impatto della cache sulle performance

Prendiamo in esame il CPU execution time, ovvero il tempo che il programma spende in esecuzione (*quindi escludiamo tutto il tempo introdotto dall'overhead dello scheduling, migrazione, sospensioni, ecc.*):

$$\text{CPU exec time} = (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{CPU cycle time}$$

La tecnologia DRAM richiede un tempo d'accesso nell'ordine dei 100 ns.

Un moderno processore lavora con un periodo di clock nell'ordine dei 0.1 ns.

La conseguenza si manifesta nel tempo di esecuzione di un programma: i memory stall cycles saranno di diversi ordini di grandezza più grandi rispetto ai CPU clock cycles!

$\text{Clock}_{\text{Rate}} = 2,5\text{Ghz}$  e  $\text{CPI} = 1$

Per ogni istruzione che accede in memoria, la CPU è costretta ad aspettare una penalty chiamata  $\text{Memory}_{\text{Stall}}$ , ossia il tempo necessario per accedere in memoria (*il quale è circa 250 volte il periodo di clock*).

## In quale blocco di memoria cache può essere memorizzato un blocco di memoria?

Ci sono 3 approcci disponibili:

### Direct Mapped

La posizione in cache (*index*) è ottenuta staticamente dall'indirizzo del blocco. Quindi un blocco può essere memorizzato in una e una sola cache line.

Molto semplice ma affetta dalle miss di conflitto.

### Fully Associative

La posizione in cache (*index*) non dipende dall'indirizzo del blocco. Quindi un blocco può essere memorizzato in qualsiasi cache line.

Molto complicate da realizzare ma immune alle miss di conflitto.

Può subire solo miss di capacità o compulsory.

### Set Associative

Un set è un gruppo di cacheline.

Ogni blocco viene memorizzato in un cache secondo una politica direct mapped; quindi un blocco può appartenere a un solo set.

Ma la cacheline all'interno del set viene decisa in maniera fully associative.

Questa soluzione va a mitigare il problema delle miss di conflitto (*due blocchi con lo stesso index che si scacciano via a vicenda*).

### **Come viene trovata una locazione di memoria se quest'ultima è presente nella cache?**

#### **Direct mapped**

Nelle cache direct mapped si accede al blocco di memoria cache dell'indice; ottenuto tramite una funzione deterministica partendo dall'indirizzo del blocco.

Le cache hanno un tag su ogni blocco di cache che fornisce l'indirizzo (*di memoria principale*) del blocco.

Una volta acceduti alla cacheline occorre accertarsi che:

- Sia il blocco che cerchiamo, allo scopo si usa il campo tag.
- Sia valida, allo scopo si utilizza un bit V, che indica la validità.

Se entrambe le condizioni sono vere si ha un hit.

#### **Fully associative**

Non ci sono tag o index ma l'indirizzo completo: all'interno si ha un comparatore di per ogni possibile cache line. La cacheline che ha quell'indirizzo e V a 1 dà hit.

#### **Set associative**

Nelle cache direct mapped o set associative si accede al blocco di memoria cache o al set in funzione dell'indice.

Tale indice viene calcolato a partire dall'indirizzo di memoria prodotto dal processore. Una volta acceduti al blocco di memoria cache, è necessario accertarsi che il blocco (set) acceduto contiene la locazione di memoria richiesta dal processore.

Le cache hanno un tag su ogni blocco di cache che fornisce l'indirizzo (*di memoria principale*) del blocco.

All'interno di un set la ricerca è di tipo fully associative, quindi si ha un comparatore di tag per ogni possibile cacheline del set.

Tutti i possibili tag vengono confrontati in parallelo per minimizzare il tempo d'accesso (hit time).

### **Quale blocco dovrebbe essere rimpiazzato in caso di cache miss?**

A deciderlo sarà un algoritmo implementato in hardware dal controllore cache.

In una cache direct mapped questo problema non si pone, ma in una cache fully associative o set associative è fondamentale che riesca ad individuare il working-set (*ossia le pagine che verranno utilizzate a breve*).

- **LRU:** Si elimina la pagina che non viene usata da più tempo; molto complicato da realizzare in hardware e per questo spesso evitato.
- **Pseudo-LRU (FIFO):** Imita il funzionamento di LRU ma cercando di semplificarne l'implementazione.
- **Random:** La pagina da eliminare è scelta randomicamente in modo uniforme.

### Quali sono i modi con cui un gruppo di thread possono essere schedulati?

Un gruppo di thread può essere schedulato in 3 modi:

#### Fine-Grained MT

Si passa da un thread all'altro molto velocemente e spesso in modo round-robin.

Questa rapidità di cambi di thread permette di nascondere eventuali stalli o comunque attese.

Di contro aumenta la complessità hardware (*il cambio di thread deve essere velocissimo*) e diminuisce le prestazioni del singolo thread.

#### Coarse-Grained MT

Si cambia thread solo a fronte di una lunga attesa, come ad esempio:

- Miss in L2 o L3.
- Lock bloccante su una risorsa.
- ...

Questa tecnica permette di avere hardware meno prestante e quindi meno complesso e inoltre le prestazioni del singolo thread non vengono deteriorate.

#### Simultaneous MT

Si eseguono più thread contemporaneamente, quindi le unità funzionali del processore vengono spartite tra i diversi thread.

La ridenominazione dei registri (*per evitare hazards*) e il dynamic scheduling consentono l'esecuzione in contemporanea di più istruzioni provenienti da thread indipendenti senza tener conto delle dipendenze tra di loro.

Questo permette di nascondere gli stalli della pipeline, aumentando così l'utilizzo delle unità funzionali.